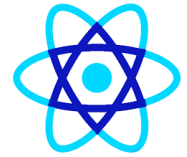


React js

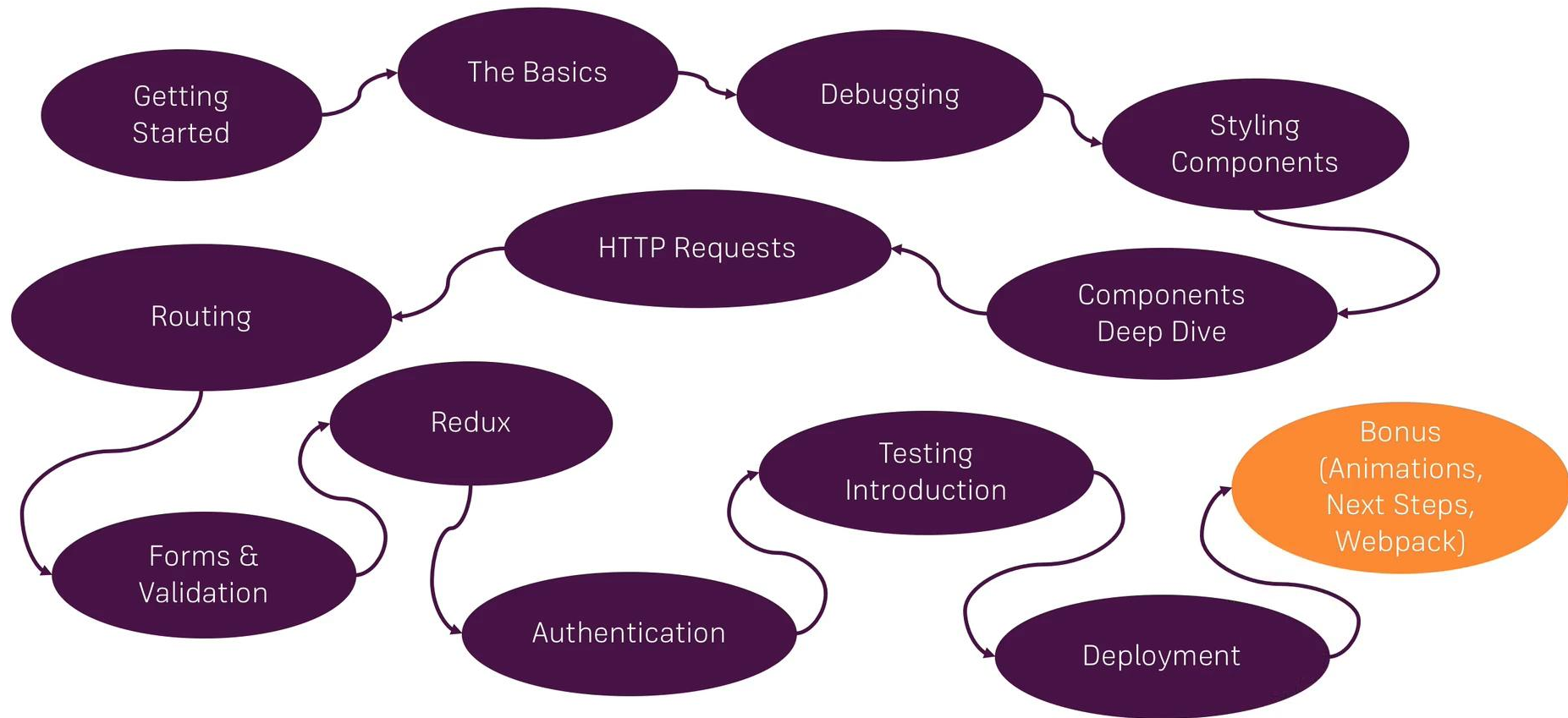
Parameswari E

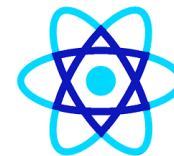
High performance. Delivered.

consulting | technology | outsourcing



Course Outline

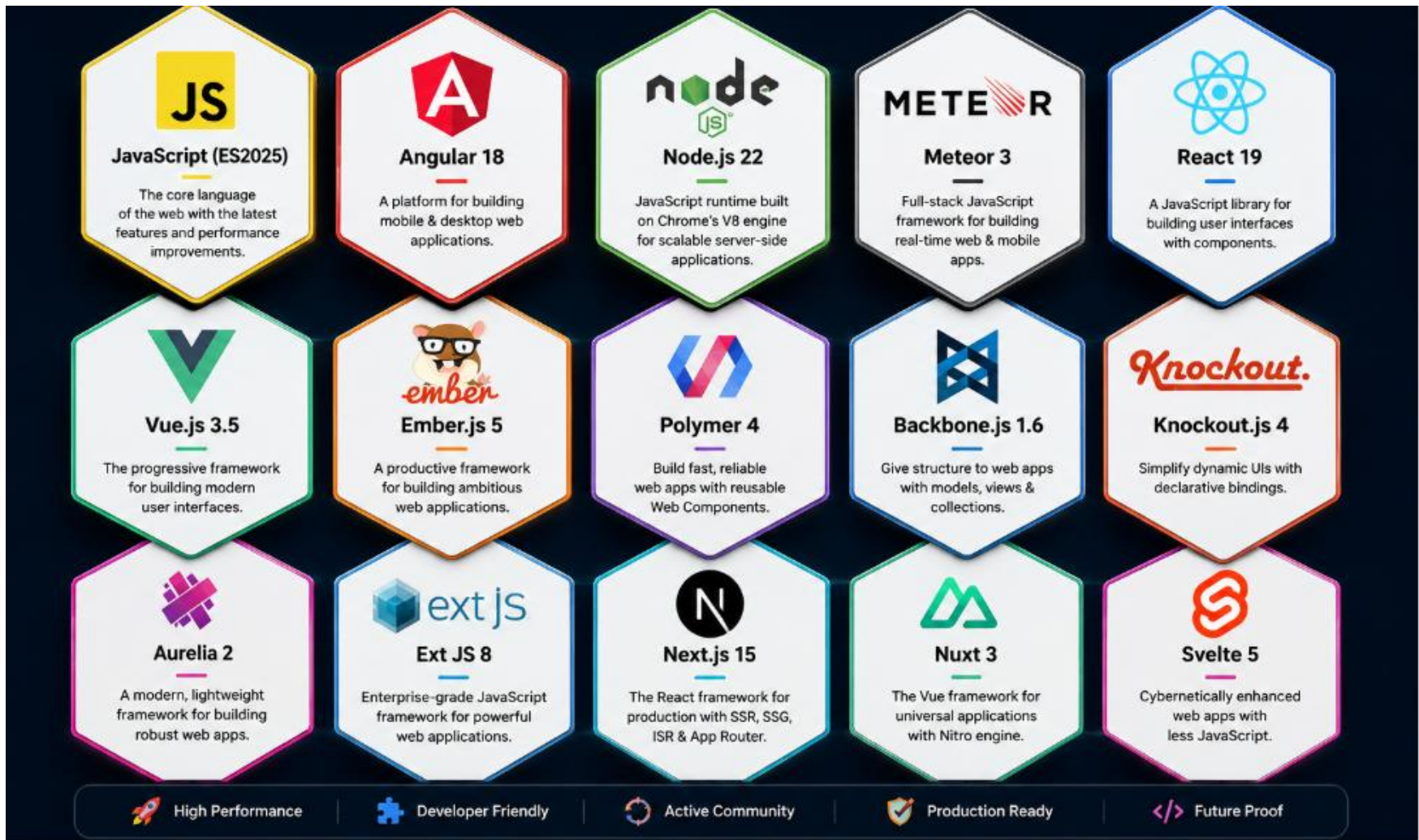
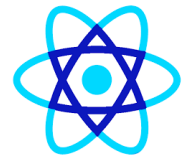




Getting Started

What? Why? How?

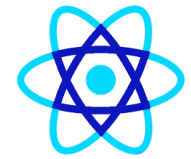
Modern Java script Eco System





What is React

- **React** is an **open-source JavaScript library** developed by **Meta (Facebook)** for building **fast, interactive, and reusable user interfaces (UI)**, especially for Single Page Applications (SPAs).
- React is a component-based JavaScript library that converts application data (state) into a user interface and efficiently updates only the parts of the page that change.

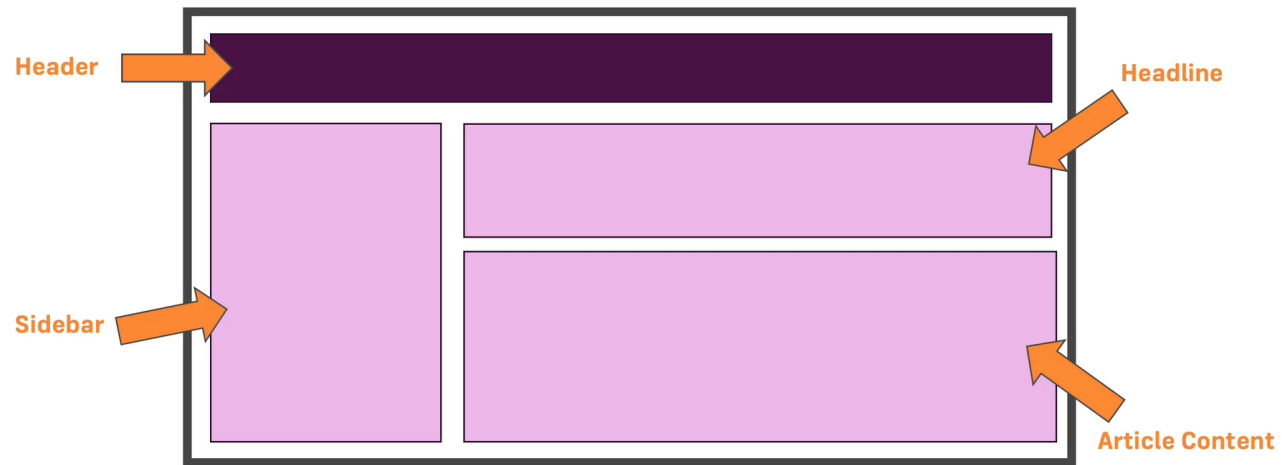


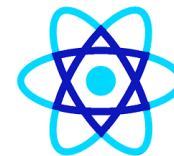
What is React

"A JavaScript Library for building User Interfaces"



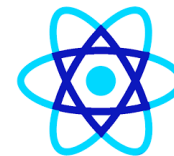
Components





What is React

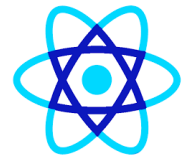
- Real-world Example
- Imagine a banking application displaying an account balance.
- Initial UI:
 - Account Balance
 - ₹25,000
- After the user transfers ₹2,000:
 - Account Balance
 - ₹23,000



What is React

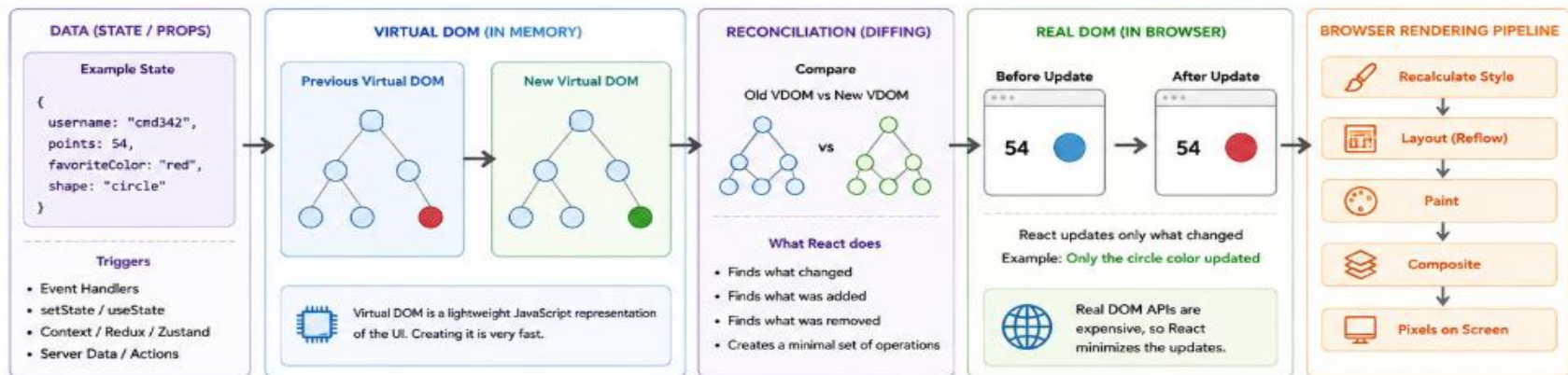
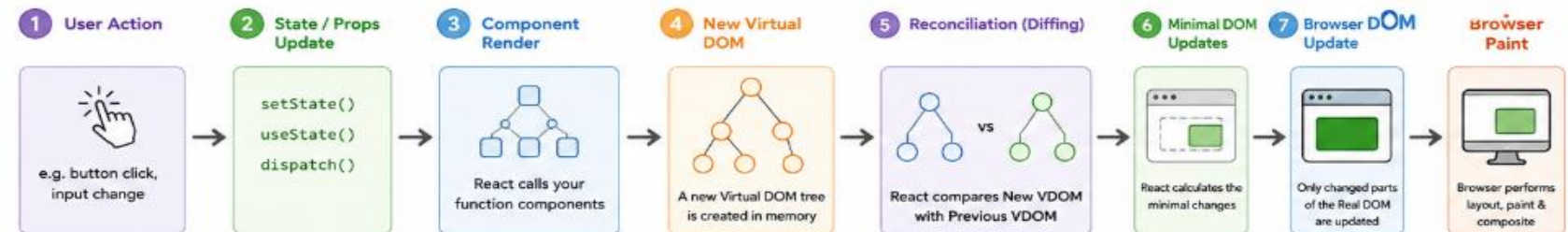
- Real-world Example
- **Without React**
- The browser might recreate much of the page.
- **With React**
- React notices that **only the balance changed.**
- It updates only:
- ₹25,000
- ↓
- ₹23,000
- Everything else stays exactly the same.

What is React



HOW REACT WORKS – RENDERING FLOW (React 19)

From State Change to UI Update



KEY POINTS



React isolates complexity and updates only what needs to change.



Virtual DOM is fast to create & diff.



You describe the UI for each state.



React figures out the most efficient way to update the DOM.

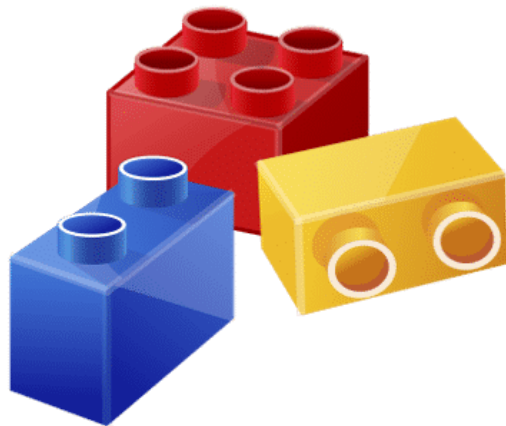
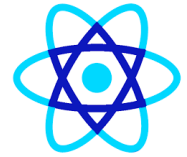


Result: Fast, predictable and maintainable UI.

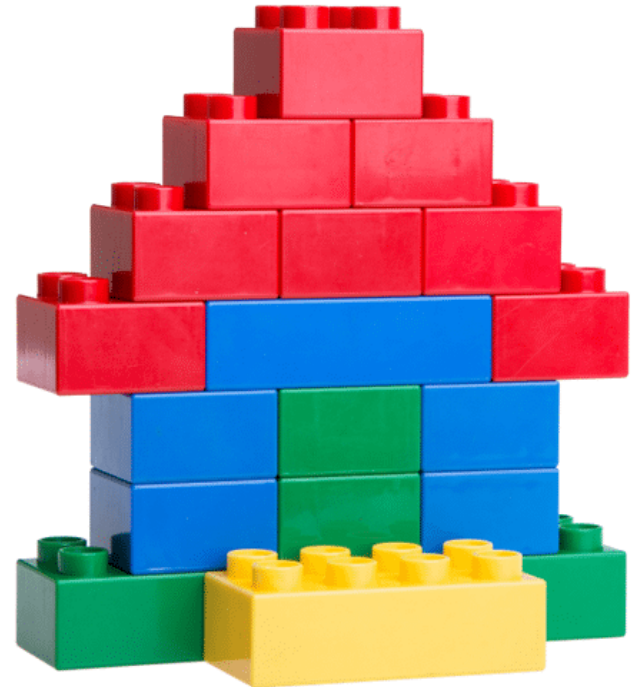
LEGEND

- Input / Trigger
- State / Data
- Rendering
- Browser
- Reconciliation

What Is React?



Building Blocks



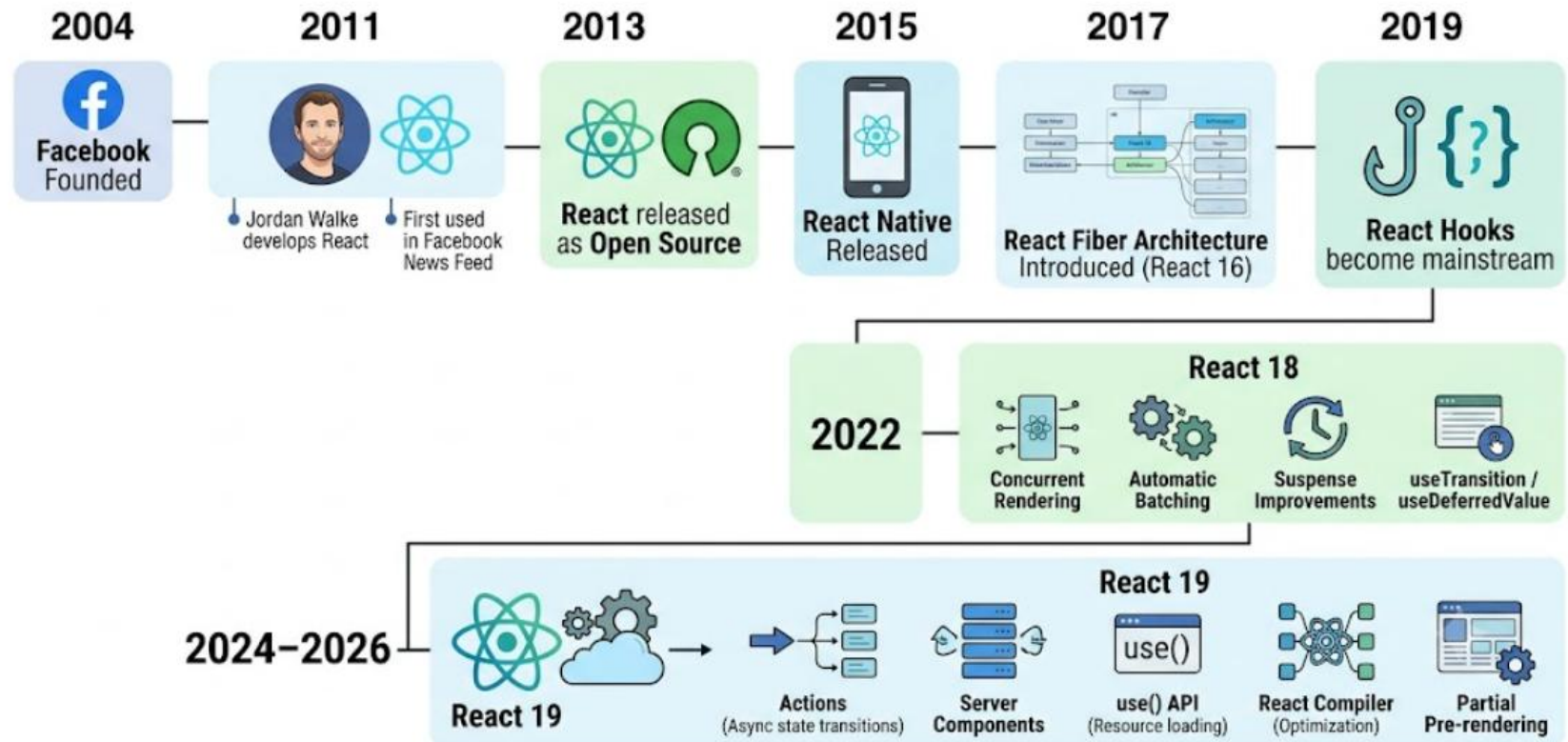
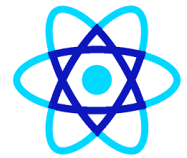
Final Product

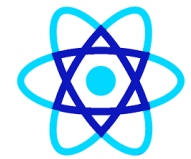


History of React

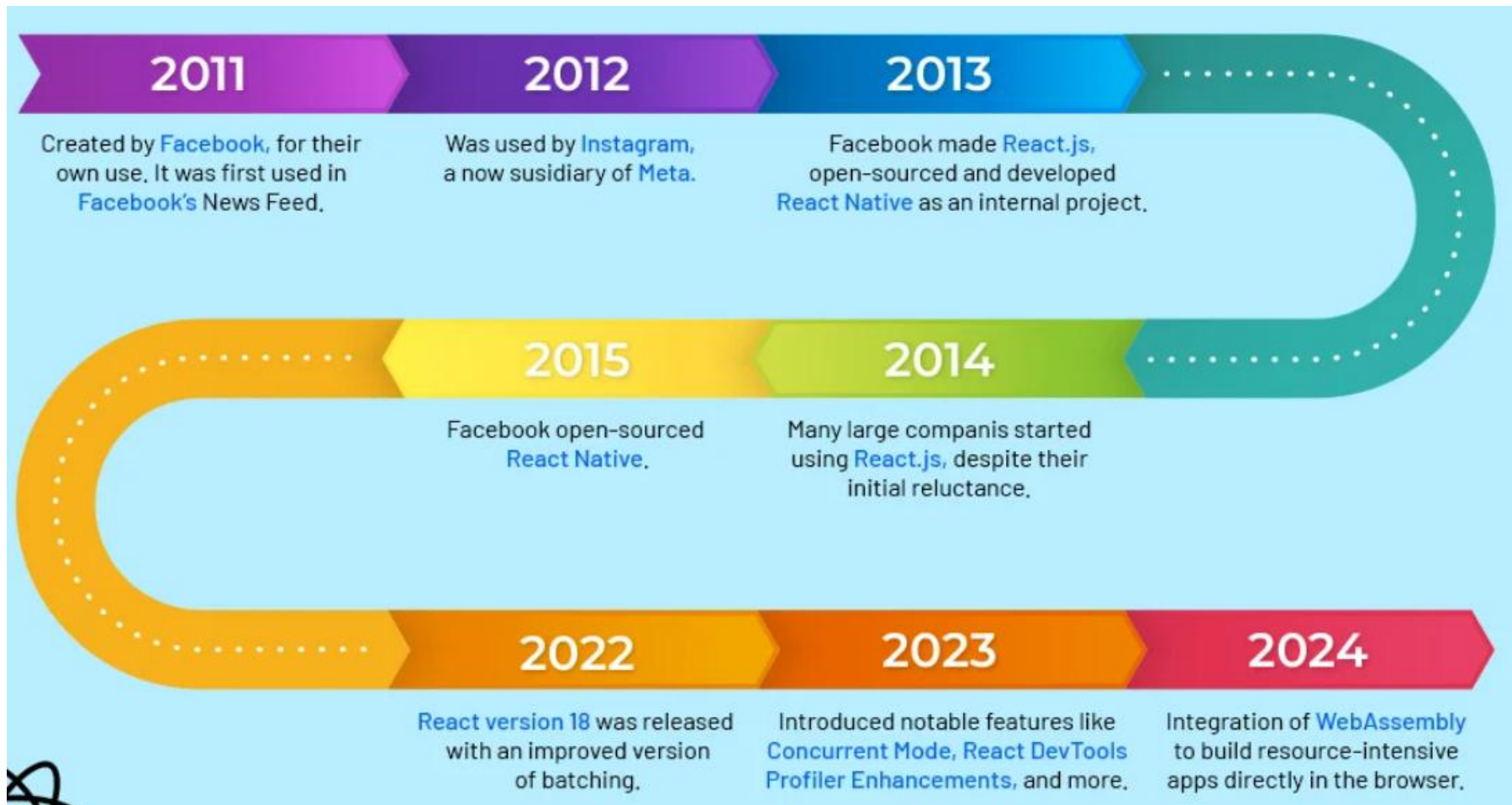
- ReactJS was developed by **Jordan Walke**, a software engineer at **Facebook (now Meta)**.
- **2011**: Facebook first used React internally to power its **News Feed**.
- **May 2013**: React was released as an **open-source JavaScript library**, allowing developers worldwide to build modern user interfaces.
- **2015**: Facebook released **React Native**, extending React's component model to mobile application development.
- The adoption of React significantly improved Facebook's user interface by making updates faster and more efficient.
- Its high performance, component-based architecture, and Virtual DOM quickly made it one of the most popular frontend libraries in the world.

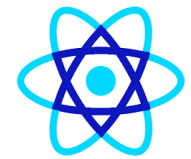
Evolution of Reactjs from 2004 to 2026



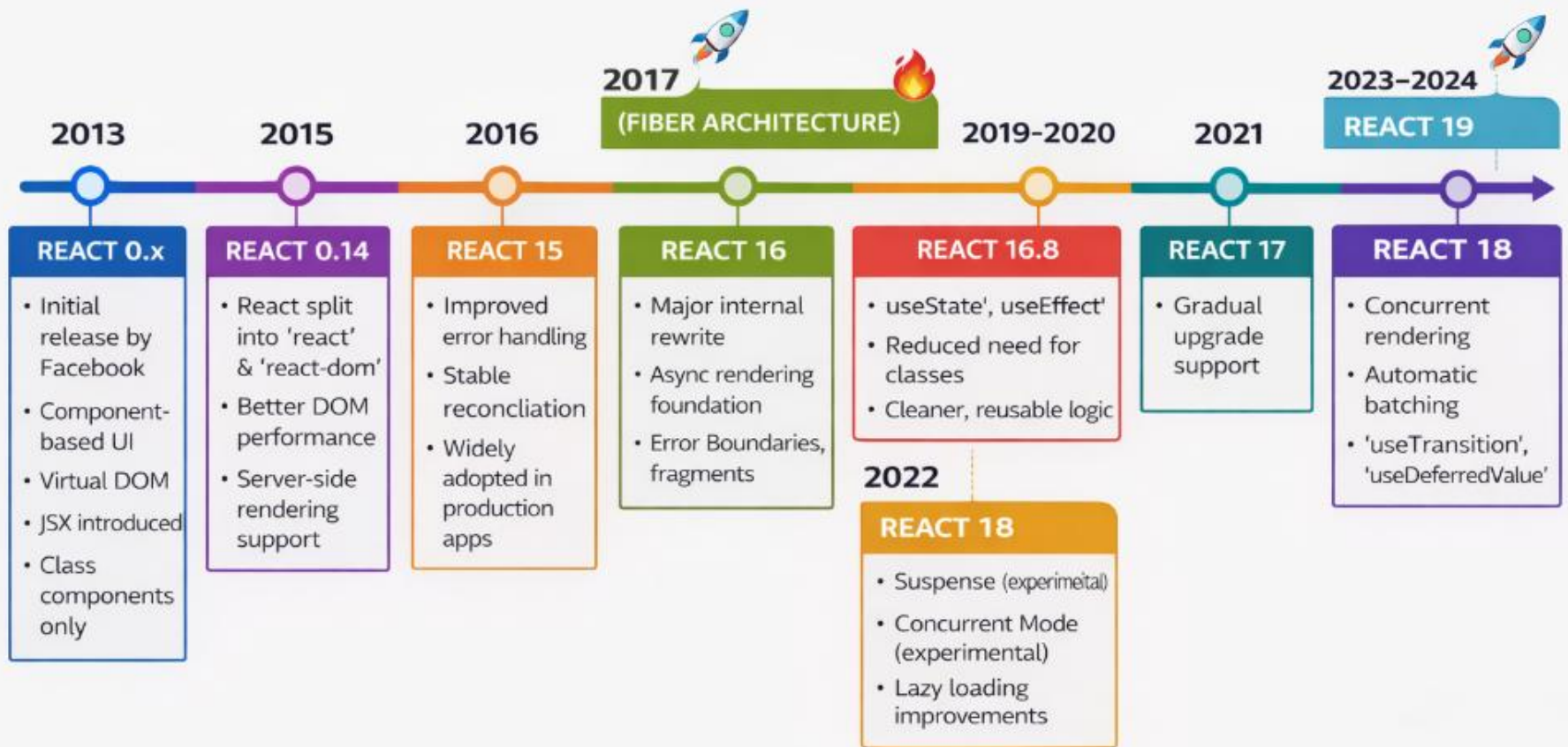


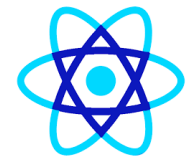
History and Evolution of Reactjs



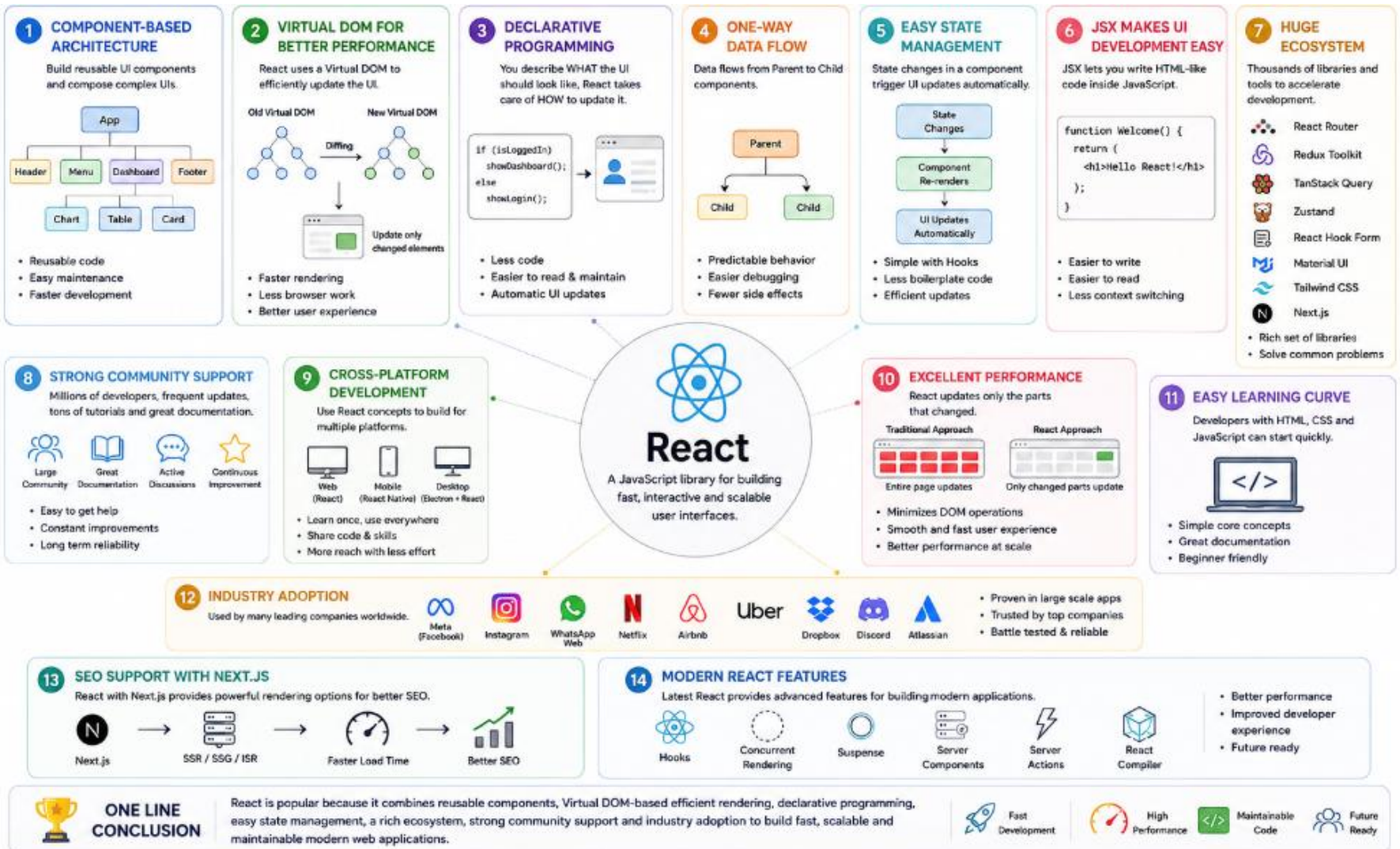


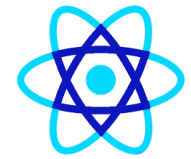
History and Evolution of Reactjs





Why React is so popular





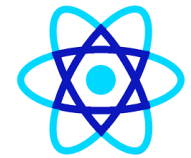
Why React is so popular

- **1) Component-based architecture**
 - Build UIs as reusable components (buttons, forms, dashboards).
 - Easier maintenance and scalability for large apps.
- **2) Fast performance (Virtual DOM)**
 - React updates a lightweight **Virtual DOM** first, then efficiently updates only what changed in the real DOM.
 - Results in smoother, faster UIs.



Why React is so popular

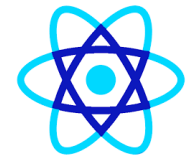
- **3) Declarative & predictable**
 - You describe *what* the UI should look like for a given state; React handles *how* to update it.
 - Less error-prone, easier to reason about.
- **4) Huge ecosystem & community**
 - Rich libraries: routing, state management, charts, forms, UI kits.
 - Massive community support, tutorials, and long-term stability.
- **5) Works everywhere**
 - **Web** (React)
 - **Mobile** (React Native)
 - **Desktop** (Electron + React)
 - **Server-side rendering** (Next.js) for SEO and performance.



Why React is so popular

- **6) Backed by Meta & industry adoption**
 - Used by companies like Facebook, Instagram, Netflix, Airbnb, Uber.
 - Strong job demand and long-term viability.
- **7) Easy integration**
 - Can be added to an existing project incrementally.
 - Plays well with REST APIs, GraphQL, and modern tooling.

React Alternatives



Modern JavaScript Frameworks & Libraries to Build UIs


Angular 18
Full Framework

- Complete solution with TypeScript
- Built-in routing, forms, HTTP client, state mgmt.
- Two-way data binding
- Backed by Google

★ Best for large enterprise apps


Vue 3.4
Progressive Framework

- Easy to learn & integrate
- Flexible and incrementally adoptable
- Composition API
- Great documentation

★ Best for fast development


Svelte 5
Compiler Framework

- No Virtual DOM
- Compiles to highly optimized vanilla JS
- Very small bundle size
- Excellent performance

★ Best for performance & small bundles


SolidJS 1.8
Reactive Library

- Fine-grained reactivity
- No Virtual DOM
- High performance
- Similar developer experience to React

★ Best for high performance web apps


Qwik 1.6
Resumable Framework

- Resumable (no hydration)
- Ultra-fast loading
- Fine-grained lazy loading
- Built for the edge

★ Best for speed & edge apps


Preact 10.20
Lightweight Library

- Small (~3KB) React compatible
- Same API as React
- Fast & minimal
- Huge ecosystem support

★ Best lightweight React alternative


Lit 3
Library

- Builds Web Components
- Fast, lightweight (no framework overhead)
- Reactive properties
- Works with any stack

★ Best for Web Components


Alpine.js 3.13
Micro Framework

- Tiny and simple
- Add interactivity to HTML directly
- Great with Tailwind CSS
- No build step needed

★ Best for simple interactivity


HTMX 2.0
Library

- Use HTML attributes to make dynamic apps
- No JavaScript framework needed
- Works with any backend

★ Best for server-driven UIs


Ember.js 5.7
Full Framework

- Convention over configuration
- Strong tooling & structure
- Long-term stability

★ Best for ambitious web apps


Nuxt 3.11
Vue Framework

- Full-stack Vue framework
- SSR, SSG, ISR, API routes
- File-based routing
- Great DX out of the box

★ Best full-stack Vue apps


Next.js 15
React Framework

- Full-stack React framework
- SSR, SSG, ISR, Server Components
- File-based routing
- Excellent performance

★ Best full-stack React apps

Quick Comparison

Framework / Library	Type	Learning Curve	Performance	Bundle Size	Best Use Case
 Angular	Full Framework	★★★☆☆	★★★☆☆	Larger	Enterprise applications
 Vue	Progressive Framework	★★★☆☆	★★★★☆	Medium	Flexible web apps
 Svelte	Compiler Framework	★★★★☆	★★★★★	Smallest	Performance-critical apps
 SolidJS	Reactive Library	★★★★☆	★★★★★	Small	High performance apps
 Preact	Lightweight Library	★★★★☆	★★★★☆	Smallest	Lightweight / React apps

When to Choose What?


Choose Angular

When building large-scale enterprise applications that need a complete solution.


Choose Vue

When you want flexibility, ease of learning and fast development.


Choose Svelte

When performance and small bundle size are your highest priorities.


Choose SolidJS

When you want React-like experience with maximum performance.

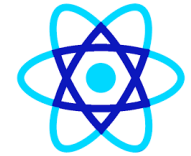

Choose Next.js

When building full-stack React applications with SSR/SSG and production-ready features.



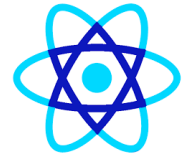
Note: There is no single best framework. Choose the one that fits your project requirements, team skills, and long-term goals.

SPA vs Traditional (Multi-Page) Applications



- **1 Traditional Web Applications (MPA)**
- **MPA = Multi-Page Application**
- **How it works**
 - Every user action (click, submit) sends a request to the server
 - Server returns a **new HTML page**
 - Browser reloads the entire page
- **Example stack**
 - JSP / Servlet
 - PHP
 - ASP.NET MVC
 - Django / Rails (server-rendered)
- **Characteristics**
 - Full page reload on each request
 - UI logic mostly on the server
 - Slower user experience
 - Easier SEO by default

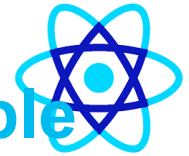
SPA vs Traditional (Multi-Page) Applications



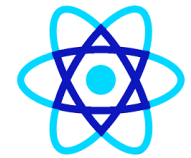
- **[2] Single Page Applications (SPA)**
- **SPA loads once, updates dynamically**
- **How it works**
 - Browser loads **one HTML page**
 - JavaScript (React / Angular / Vue) updates content dynamically
 - Backend exposes **APIs (REST / GraphQL)**
 - No full page reloads
- **Example stack**
 - React / Angular / Vue
 - Backend: Spring Boot, Node.js, .NET Web API
- **Characteristics**
 - Smooth, app-like experience
 - Faster navigation
 - Client-side routing
 - UI logic on the browser



SPA vs Traditional Apps – Comparison Table



Feature	Traditional App (MPA)	SPA
Page Reload	Every request	Only once
Speed	Slower	Faster
User Experience	Basic	App-like
Backend Role	HTML + Data	APIs only
Frontend Logic	Minimal JS	Heavy JS
SEO	Easy	Needs SSR (Next.js)
Scalability	Limited	High
Mobile-friendly	Average	Excellent



When to use Traditional Apps

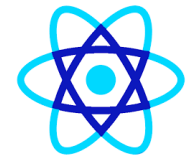
- Simple websites
- Content-heavy sites (blogs, news)
- Minimal interactivity
- SEO-critical pages



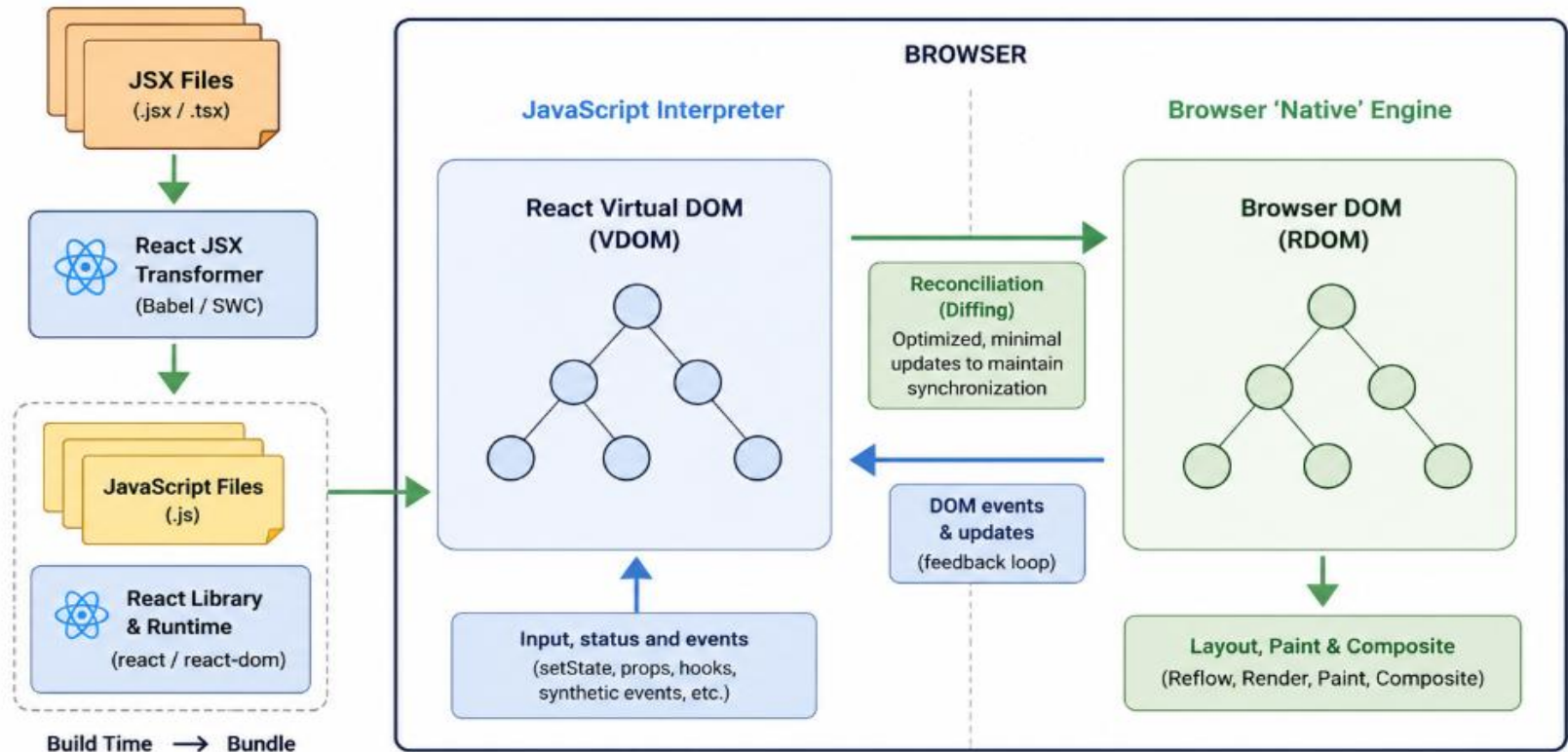
When to use SPA

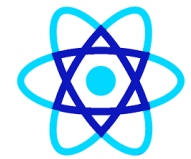
- Dashboards & admin portals
- Banking, insurance, enterprise apps
- Real-time apps (chat, analytics)
- Mobile-first applications

Reactjs Architecture

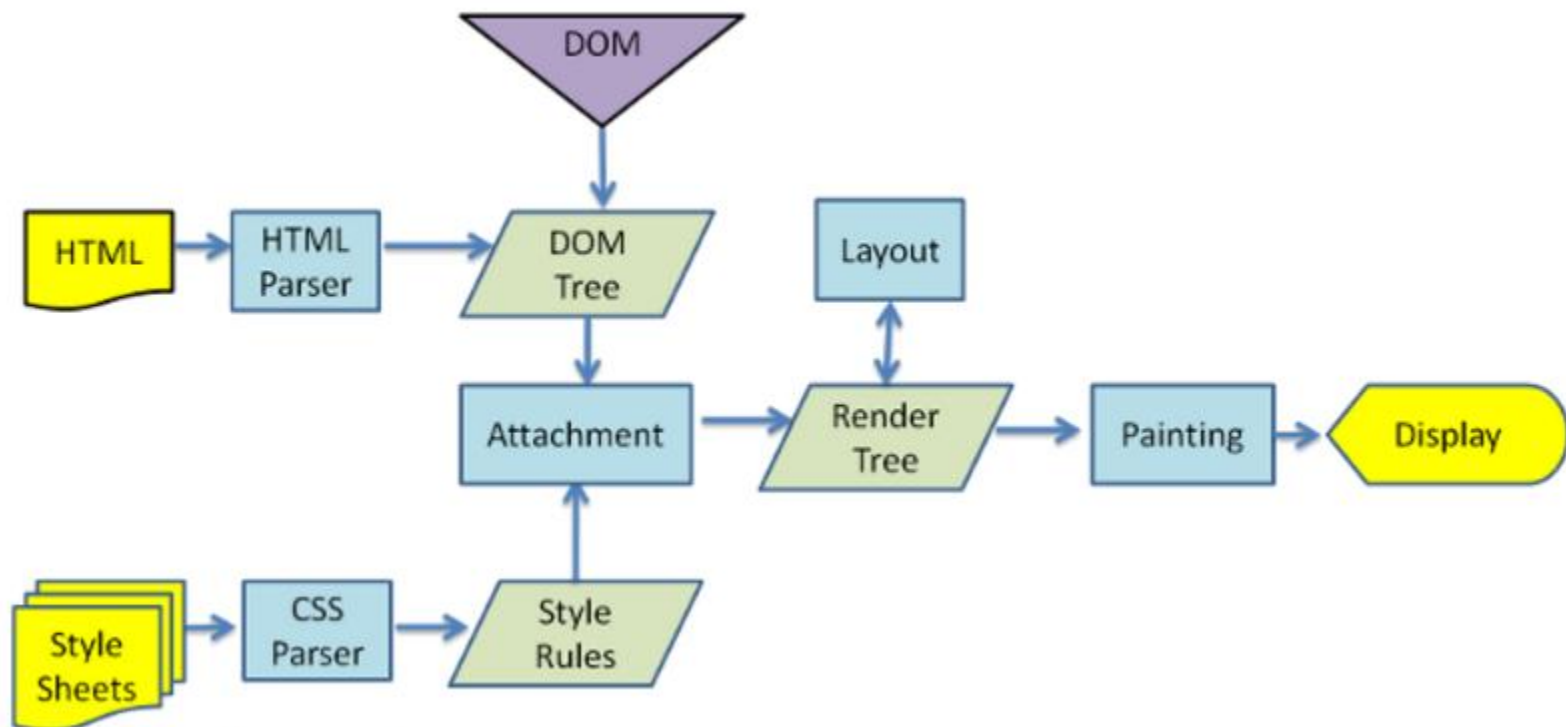


VDOM TO RDOM (React Rendering Flow – Latest)





Why updating Real DOM is slow:





Why updating Real DOM is slow:

-  **Why the Real DOM is Slow**
 - DOM manipulation involves **layout, repaint, reflow**
 - Frequent updates cause performance bottlenecks
 - Direct DOM access blocks the browser rendering pipeline

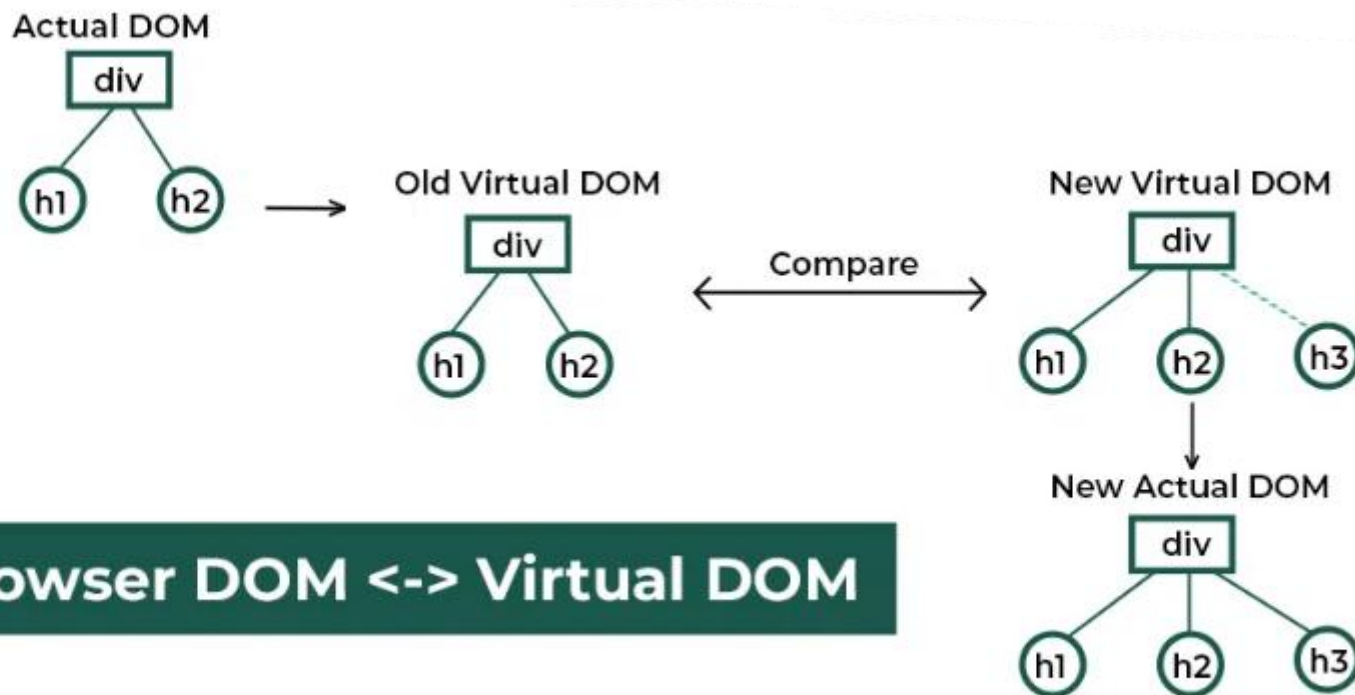


Virtual DOM

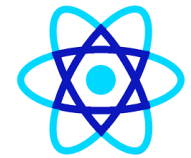
- The **Virtual DOM** is a **lightweight, in-memory copy of the real browser DOM** used by libraries like **React** to make UI updates **faster and more efficient**.
- Instead of updating the real DOM directly every time data changes (which is slow), React:
- Updates the **Virtual DOM**
- Compares it with the **previous Virtual DOM**
- Updates **only the changed parts** in the real DOM



Virtual DOM

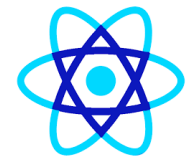


Browser DOM <-> Virtual DOM



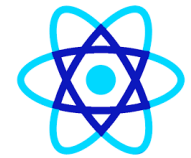
How Virtual DOM Works (step by step)

-  **How Virtual DOM Works (Step-by-Step)**
- **Initial Render**
 - React creates a Virtual DOM tree from your components
 - It renders this to the real DOM
- **State / Props Change**
 - Component state or props change
 - A **new Virtual DOM** is created
- **Diffing (Reconciliation)**
 - React compares **old VDOM vs new VDOM**
 - Finds the **minimal set of changes**
- **Batch Update**
 - Only the changed nodes are updated in the real DOM
- 👉 **Result: Fast UI updates with minimal DOM operations**



Virtual DOM vs Real DOM

Virtual DOM	Real DOM
It is a lightweight copy of the original DOM	It is a tree representation of HTML elements
It is maintained by JavaScript libraries	It is maintained by the browser after parsing HTML elements
After manipulation it only re-renders changed elements	After manipulation, it re-render the entire DOM
Updates are lightweight	Updates are heavyweight
Performance is fast and UX is optimised	Performance is slow and the UX quality is low
Highly efficient as it performs batch updates	Less efficient due to re-rendering of DOM after each update



⚡ Why Virtual DOM is Faster

Feature	Real DOM	Virtual DOM
Update cost	Expensive	Cheap (JS object)
DOM operations	Many	Minimal
Performance	Slower	Faster
Re-render scope	Whole subtree	Only changed nodes

Without Virtual DOM

```
js  
  
document.getElementById("count").innerText = count;
```

➡ Direct DOM update every time

With Virtual DOM (React)

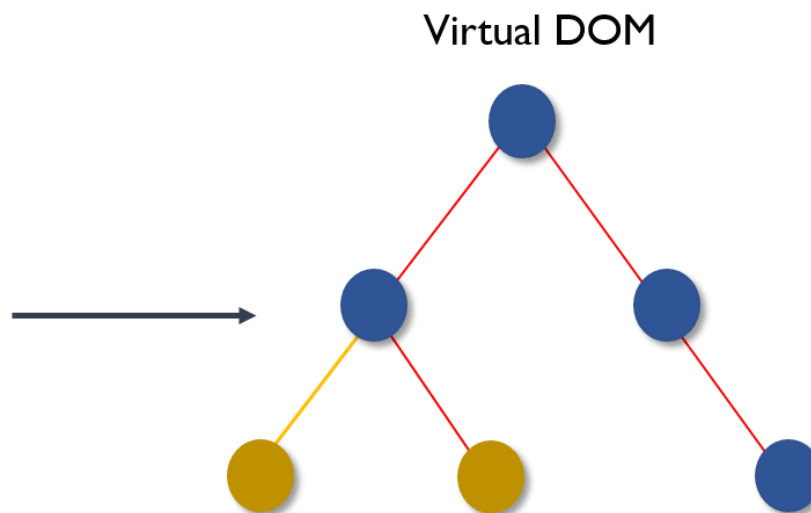
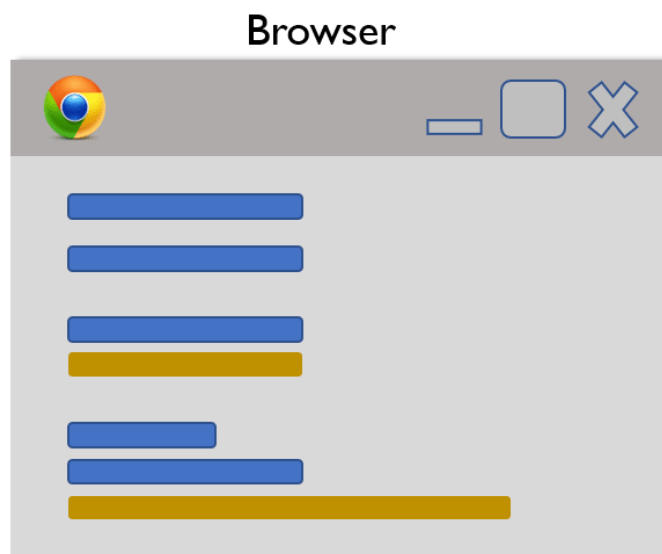
```
jsx  
  
setCount(count + 1);
```

➡ React figures out what actually changed



Virtual DOM

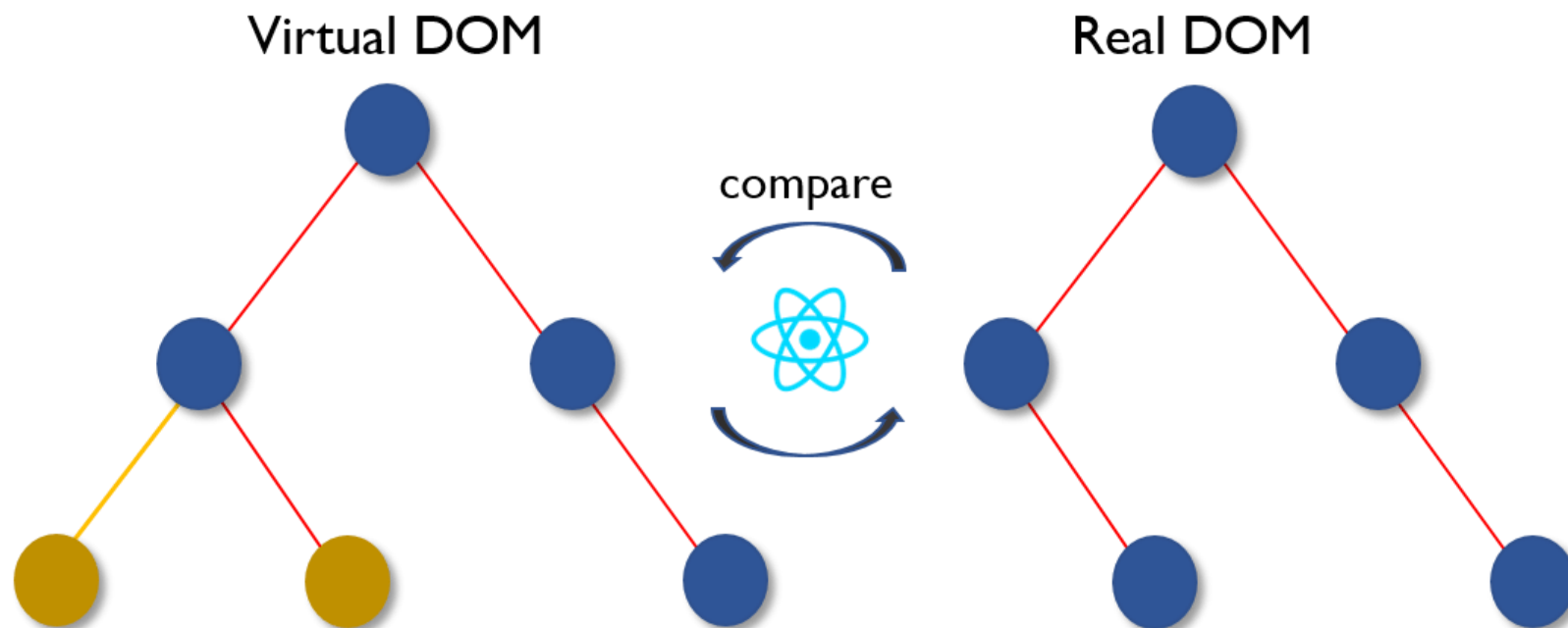
- Whenever any underlying data changes, the entire UI is re-rendered in Virtual DOM representation.

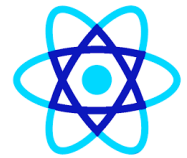




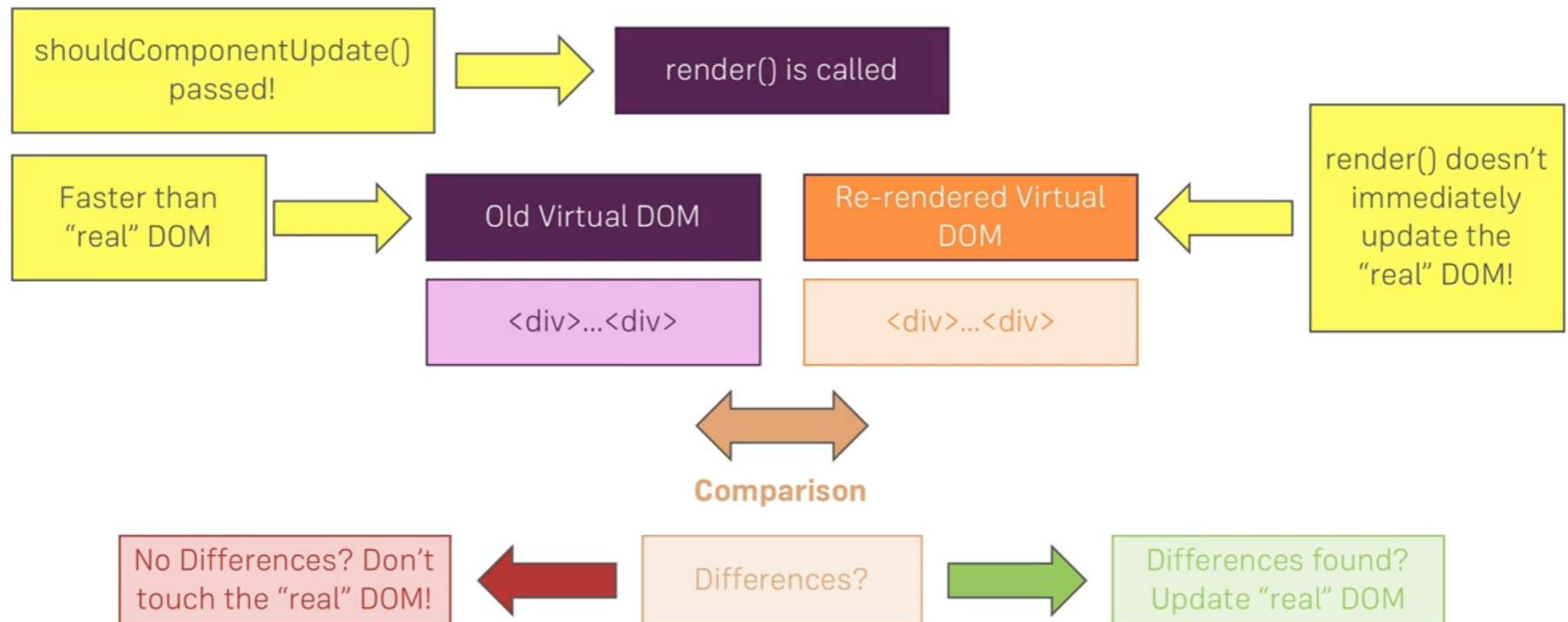
Virtual DOM

- Then the difference between the previous DOM representation and the new one is calculated..





How React Updates The DOM

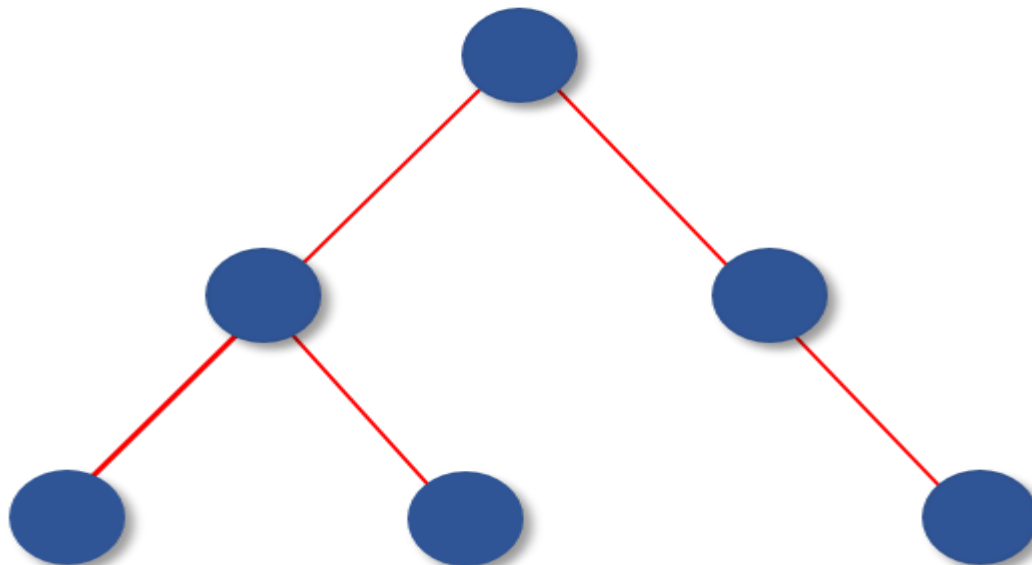




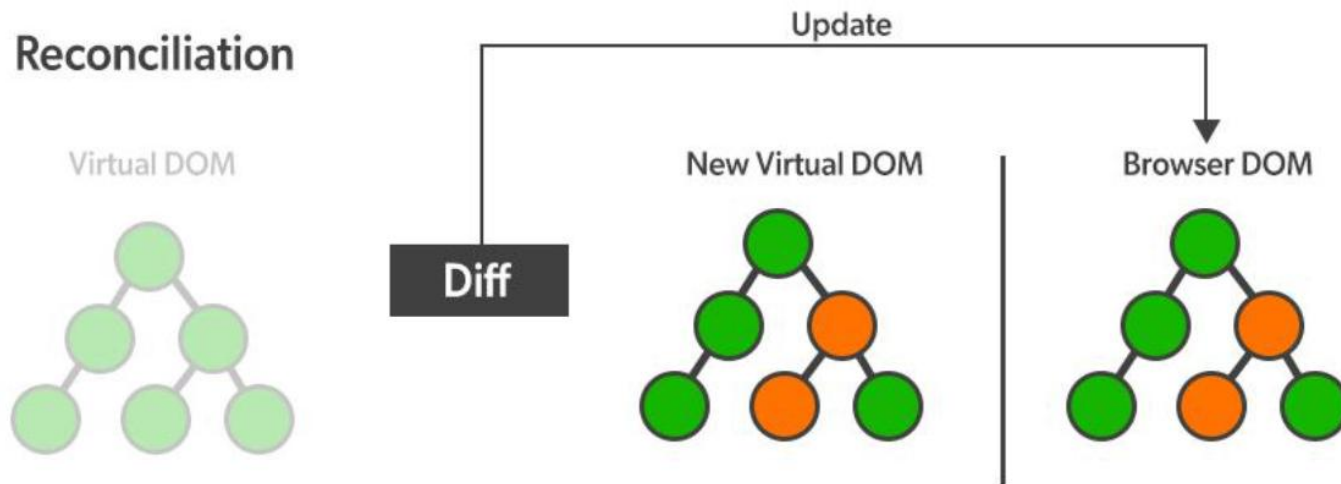
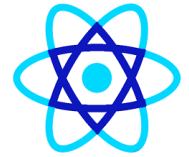
Virtual DOM

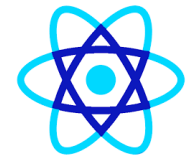
- Once the calculations are done, the real DOM will be updated with only the things that have actually changed.
- You can think of it as a patch.
- As patches are applied only to the affected area, similarly, the virtual DOM acts as patches and are applied to the elements which are updated or changed, in the real DOM.

Real DOM (updated)



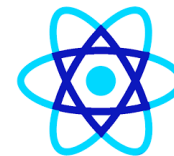
Virtual DOM





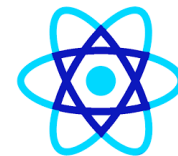
Reactjs Architecture

Feature	Real DOM	Virtual DOM
Nature	Browser DOM	JS object (in memory)
Update speed	Slow	Fast
Re-render	Whole DOM / large parts	Only changed nodes
Memory	No extra memory	Uses additional memory
UI performance	Poor for frequent updates	Optimized
Direct access	Yes (document.getElementById)	No direct access
Used by	Vanilla JS, jQuery	React, Vue



Reactjs Architecture

- **1 JSX Files (Top-Left)**
- **What it shows:**
 -  JSX files
- **Meaning:**
- Developers write UI using **JSX** (HTML-like syntax in JavaScript)
- Example:
- `<button>Save</button>`
- **JSX is not understood by browsers directly.**



Reactjs Architecture

- **[2] React JSX Transformer (Build Time)**
- **What it shows:**
 -  React JSX transformer
- **Meaning:**
- JSX is converted into **plain JavaScript**
- Done by tools like:
 - **Babel**
 - **Vite**
 - **Webpack**
- JSX → `React.createElement()`
- `React.createElement("button", null, "Save")`
- ✓ Happens **before runtime**



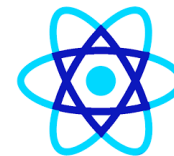
Reactjs Architecture

- **3 JavaScript Files + React Runtime**
- **What it shows:**
 - 📁 JavaScript files
 - 📦 React Library / runtime
- **Meaning:**
- Transformed JS + React runtime are loaded into the browser
- React runtime handles:
 - Component lifecycle
 - State updates
 - Virtual DOM creation



Reactjs Architecture

- **4 Browser – JavaScript Interpreter**
- **What it shows:**
 -  Browser
 -  JavaScript interpreter
- **Meaning:**
- Browser executes JavaScript using:
 - Chrome → V8
 - Firefox → SpiderMonkey
- React code now runs inside browser memory



Reactjs Architecture

- **5 React Virtual DOM (Core Concept)**
- **What it shows:**
 - React virtual DOM
- **Meaning:**
- Virtual DOM is a **lightweight JavaScript object tree**
- It represents **UI structure in memory**, NOT the real browser DOM



Reactjs Architecture

- **5 React Virtual DOM (Core Concept)**

Example:

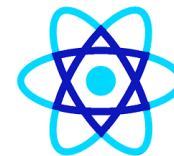
```
{  
  type: "div",  
  children: [  
    { type: "h1", text: "Hello" }  
  ]  
}
```

- ✓ Very fast to create and compare
- ✓ No browser repaint yet



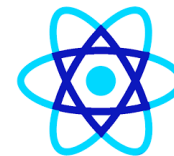
Reactjs Architecture

- **6 Input, State & Events (Bottom Arrow)**
- **What it shows:**
 -  Input, status and events
- **Meaning:**
- User actions:
 - Click
 - Input
 - Scroll



Reactjs Architecture

- **[7] Diffing & Reconciliation**
- **What it shows:**
 - ➔ Optimized minimal updates to maintain synchronization
- **Meaning:**
- React compares:
 - Old Virtual DOM
 - New Virtual DOM
- Finds **only the differences**
- This process is called:
 - **Diffing**
 - **Reconciliation**
- ✓ Avoids full DOM re-render
- ✓ Improves performance



Reactjs Architecture

- **8 Browser Native Engine**
- **What it shows:**
 -  Browser 'native' engine
- **Meaning:**
- React now instructs the browser:
 - What exactly to update
 - Where to update
- Browser executes minimal DOM operations



Reactjs Architecture

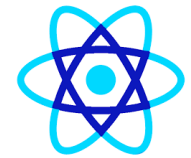
-  **Real DOM (RDOM)**
- **What it shows:**
 -  Browser DOM
- **Meaning:**
- Actual HTML DOM tree
- Only **changed nodes** are updated
- Browser repaints UI
- ✓ Faster than traditional full reload
- ✓ Smooth SPA experience



Reactjs Compilation

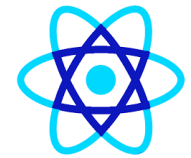


ESM (ES Modules)
HMR Hot Module Replacement



Why This Architecture is Powerful

Feature	Benefit
Virtual DOM	High performance
Diffing	Minimal updates
One-way data flow	Predictable UI
Component tree	Reusability
Browser-agnostic	Works everywhere



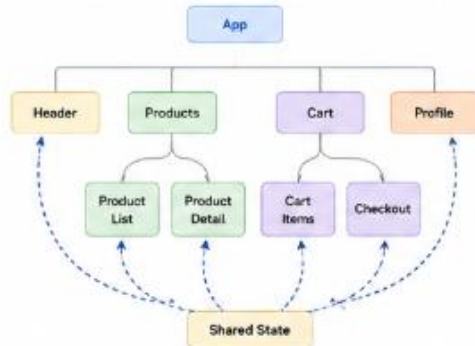
Why State Management

Managing state properly makes your application predictable, scalable, maintainable and high-performance.



1. COMPLEX APPLICATIONS HAVE COMPLEX STATE

As your app grows, state is shared and used in many components at different levels.

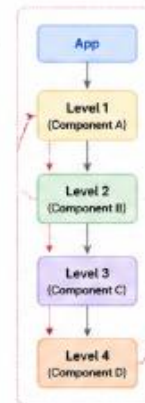


Prop drilling becomes hard to manage.



2. PROBLEMS WITHOUT PROPER STATE MANAGEMENT

- ❌ **Prop Drilling**
Passing state through many components is tedious and error-prone.
- ❌ **Hard to Maintain**
State logic scattered across components.
- ❌ **Unpredictable State**
Multiple updates from different places can cause bugs.
- ❌ **Hard to Debug**
Difficult to track where state is changed.
- ❌ **Poor Performance**
Unnecessary re-renders and duplicate state.
- ❌ **Not Scalable**
Difficult to add new features as the app grows.



Prop Drilling



3. STATE MANAGEMENT SOLVES THESE PROBLEMS

- ✅ **Centralized State**
Store state in one place, access anywhere.
- ✅ **Predictable**
State changes follow a clear flow.
- ✅ **Easier Maintenance**
Update logic in one place.
- ✅ **Easier Debugging**
Track actions and state changes easily.
- ✅ **Better Performance**
Optimized updates with minimal re-renders.
- ✅ **Scalable**
Easier to manage as the application grows.



All components subscribe to the store and get the state they need.



4. HOW STATE MANAGEMENT WORKS (EXAMPLE FLOW)



5. EXAMPLES OF STATE MANAGEMENT LIBRARIES



6. REAL-WORLD EXAMPLE

E-commerce App

- User adds item to cart
- Quantity changes
- Applies coupon
- Checkout
- Payment success

→ All these features update and read the same state. →

Without State Management

- ❌ State duplicated in many components
- ❌ Hard to keep data in sync
- ❌ High chance of errors

With State Management

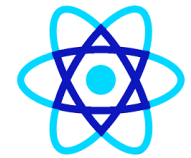
- ✅ Single source of truth
- ✅ Always in sync
- ✅ Reliable and easy to extend



KEY TAKEAWAY

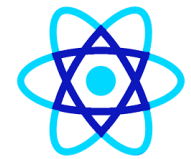
State Management is not about adding a library. It's about managing the state of your application in a predictable, efficient and scalable way.

Better State Management = Better Application

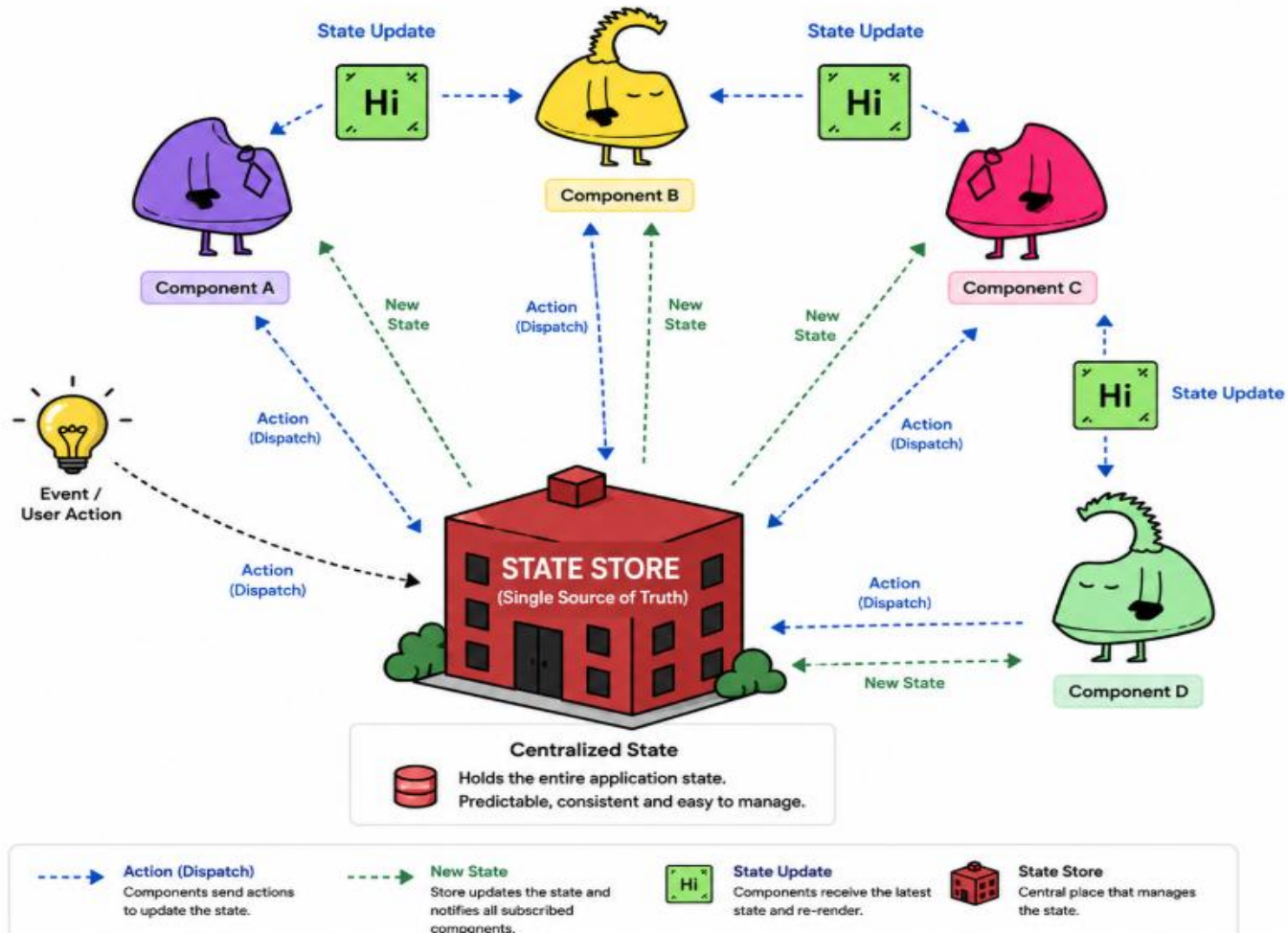


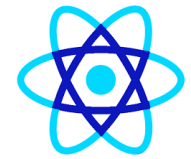
Why State Management

-  Problem shown in the image: *Without Redux (or centralized state)*
- What's happening here
- Components **share state directly**
- Data flows:
 - Parent → child
 - Child → sibling
 - Sibling → unrelated component
- Communication becomes **unpredictable and hard to track**
- This is called:
-  **Prop Drilling + Tight Coupling**
- Component A → B → C → D → E



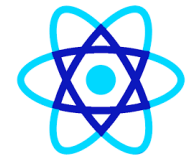
Why State Management





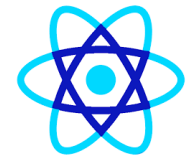
Why State Management

-  Red building (bottom-left) → **Redux Store**
- **What it represents**
- The **single source of truth**
- Holds the **entire application state**
 - store = {
 - user,
 - cart,
 - orders,
 - theme
 - }
- ✓ Only one store
- ✓ Centralized
- ✓ Predictable



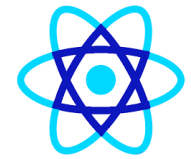
Why State Management

-  **Green blocks (Hi) → State data**
- **What they represent**
- Pieces of global state
- Examples:
 - Logged-in user
 - Notifications
 - Cart count
 - Dashboard data
- These values **live in the store**, not inside components.



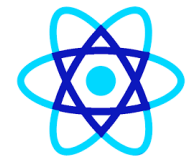
Why State Management

-  **Colored characters** → **React Components**
- Each character:
 - Needs some data
 - May trigger state changes
 - **Is independent of other components**
- ✓ No direct coupling
 - ✓ Easy to reuse
 - ✓ Easy to test



Why State Management

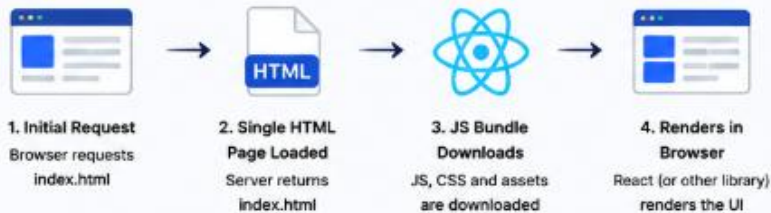
-  **Colored characters → React Components**
- Each character:
 - Needs some data
 - May trigger state changes
 - **Is independent of other components**
- ✓ No direct coupling
 - ✓ Easy to reuse
 - ✓ Easy to test



Two Kinds of Applications

SPA (Single Page Application) Single HTML Page, Content is rendered on Client

How it works



Navigation Flow



Key Points

- ✓ Only ONE HTML file is loaded
- ✓ Fast navigation after initial load
- ✓ Content is rendered in the browser
- ✓ Uses client-side routing (e.g., React Router)
- ✓ Better user experience (no full page reload)

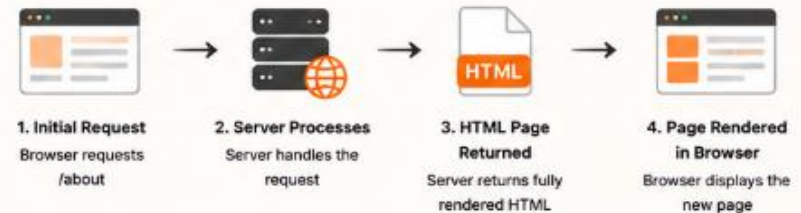
Example Tech Stack



Best For: Dashboards, SaaS apps, Social platforms, Real-time apps, Highly interactive applications

MPA (Multi Page Application) Multiple HTML Pages, Content is rendered on Server

How it works



Navigation Flow



Key Points

- ✓ Multiple HTML pages are loaded
- ✓ Slower navigation (full page reload)
- ✓ Content is rendered on the server
- ✓ Each navigation is a new request
- ✓ Better for SEO and security

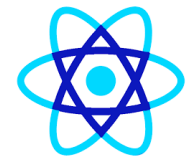
Example Tech Stack



Best For: Content websites, Blogs, E-commerce, SEO-friendly sites, Traditional web applications

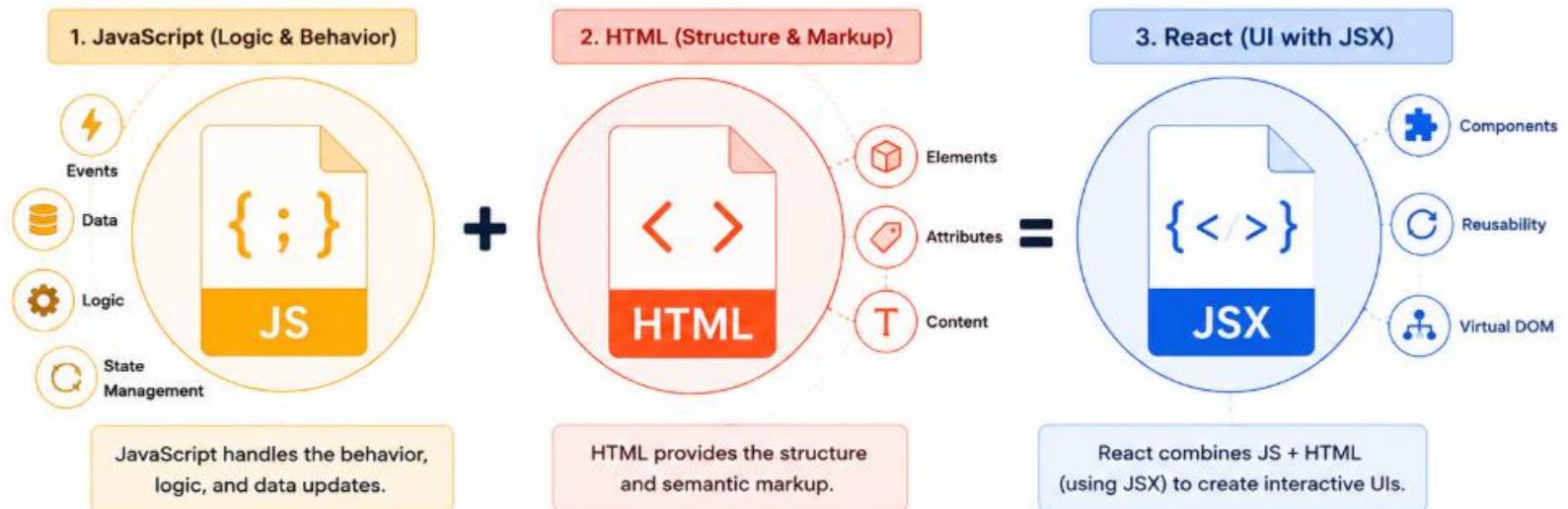


Summary: SPA loads one page and updates content dynamically using JavaScript. MPA loads new pages from the server for each navigation. Choose SPA for rich user experience and MPA for better SEO, performance on low-end devices, and security.

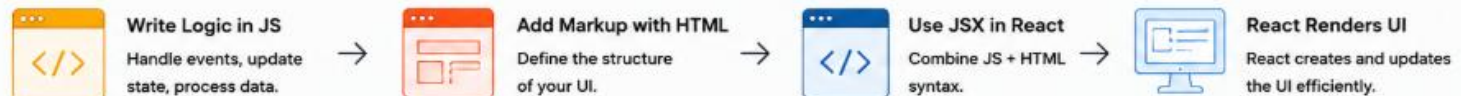


Features Of React

React is a JavaScript library for building user interfaces.



What Happens?



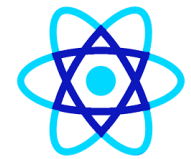
In Short

React = JavaScript (logic) + HTML (structure) → **JSX** (React Syntax) → Interactive UI

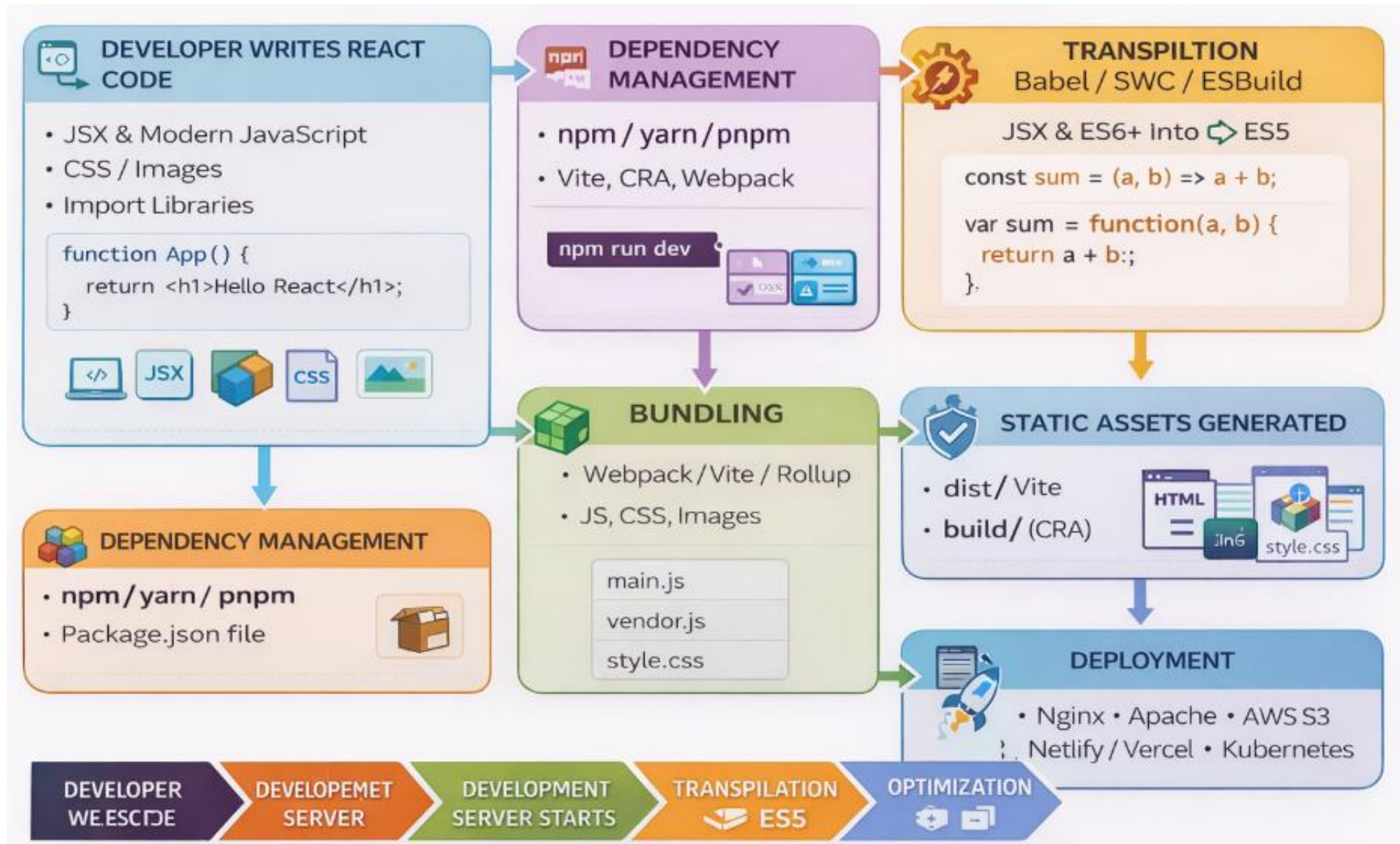


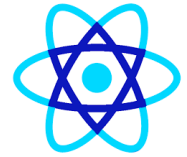
Quick Overview

Era	Version	Key Highlight
Early	0.x – 15	Virtual DOM, JSX
Major Shift	16	Fiber architecture
DX Revolution	16.8	Hooks
Stability	17	Gradual upgrades
Modern React	18	Concurrent rendering
Future	19	Server-first React

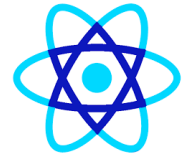


Using a Build Workflow

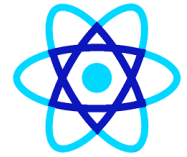




- **❶ Prerequisites (Must Have)**
- **✓ Operating System**
- Windows / macOS / Linux
- **✓ Node.js (Mandatory)**
- React runs on **Node.js**
- Includes **npm** (Node Package Manager)
- **◆ Recommended:**
- **Node.js LTS (18.x or 20.x)**
- Check installation:
- `node -v`
- `npm -v`

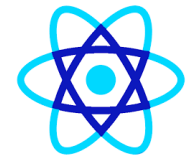


- **2 Code Editor**
- **✓ Recommended Editors**
- **VS Code** (most popular)
- WebStorm
- IntelliJ IDEA
- **🔌 Useful VS Code Extensions**
- ES7+ React Snippets
- Prettier
- ESLint
- Auto Rename Tag



- **3 Browser & Dev Tools**
- **✓ Browser**
- Google Chrome (recommended)
- **🔧 React Developer Tools**
- Install browser extension:
- React Developer Tools (Chrome / Firefox)
- Used for:
- Inspecting components
- Checking props & state

React Environment Setup



React + Vite

Modern, Lightning-Fast React Development ⚡

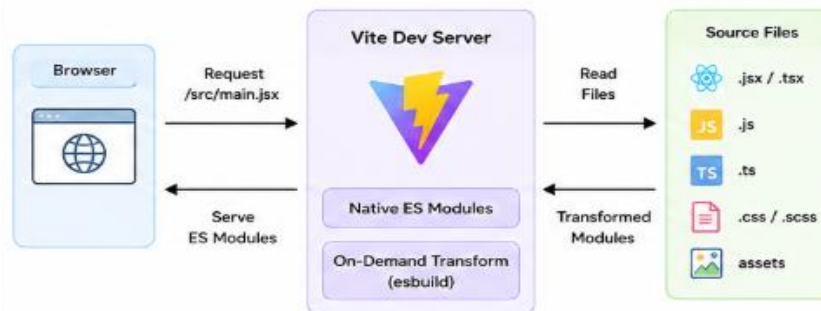


Vite is a next-generation frontend build tool that provides an extremely fast development experience for React. It uses native ES Modules in development and bundles for production.

How It Works

- 1. Start Dev Server**
Vite starts a dev server using native ES modules.
- 2. On-Demand Compilation**
Files are transformed and served only when requested.
- 3. Hot Module Replacement**
Updates are pushed to the browser instantly without full reload.
- 4. Production Build**
Vite uses Rollup to bundle the app into optimized static assets.

Development Flow (vite dev)



Production Build Flow (vite build)



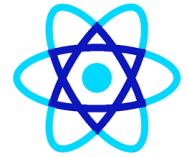
Key Benefits

- Extremely Fast**
Instant server start and HMR.
- Native ES Modules**
No bundling in dev, only on-demand.
- Optimized Build**
Uses Rollup for smaller, faster production builds.
- Out of the Box**
Zero config for React with fast refresh.

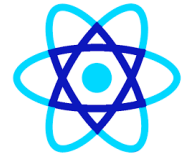


In Short

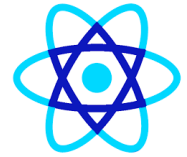
Vite (Dev Server) + **React** (UI Library) = ⚡ **Lightning Fast Development** + **Optimized Production Build**



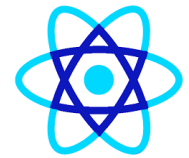
- **4 Create React Application (3 Ways)**
- **Option 1: Vite (Recommended – Modern)**
- Fastest & industry preferred
- `npm create vite@latest my-react-app`
- `cd my-react-app`
- `npm install`
- `npm run dev`
- Runs on:
- `http://localhost:5173`



- **4 Create React Application (3 Ways)**
- **Option 2: Create React App (CRA – Older)**
 - `npx create-react-app my-app`
 - `cd my-app`
 - `npm start`
 - Runs on:
 - `http://localhost:3000`
 - **⚠ Slower than Vite, but still common in legacy projects**



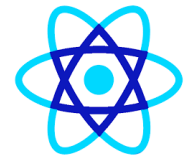
- **4 Create React Application (3 Ways)**
- **Option 3: Next.js (Framework-based)**
 - Used for SSR, SEO, production apps
 - `npx create-next-app my-app`
 - `cd my-app`
 - `npm run dev`



5 Project Structure (Vite Example)

pgsql

```
my-react-app/  
├─ node_modules/  
├─ public/  
├─ src/  
│   ├─ App.jsx  
│   ├─ main.jsx  
│   └─ index.css  
├─ package.json  
└─ vite.config.js
```



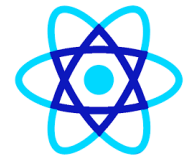
6 Verify React Is Working

Edit `App.jsx`:

jsx

```
function App() {  
  return <h1>React Setup Successful 🚀</h1>;  
}  
  
export default App;
```

Save → browser auto refreshes ✓



7 Build for Production

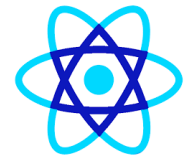
```
bash
```

```
npm run build
```

Output:

- `dist/` (Vite)
- `build/` (CRA)

👉 This folder is **deployment-ready**



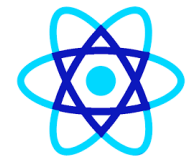
8 Optional (Professional Setup)

- ESLint + Prettier
- TypeScript
- Git & GitHub
- Docker (optional)
- CI/CD (GitHub Actions)



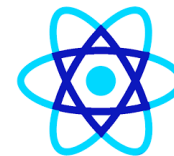
What SWC does

- SWC can:
- ☒ Compile **ES6+ → ES5**
- ☒ Transform **JSX / TSX**
- ☒ Compile **TypeScript → JavaScript**
- ☒ Enable **React Fast Refresh**
- ☒ Minify JavaScript



SWC(Speedy Web Compiler) vs Babel

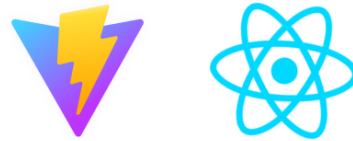
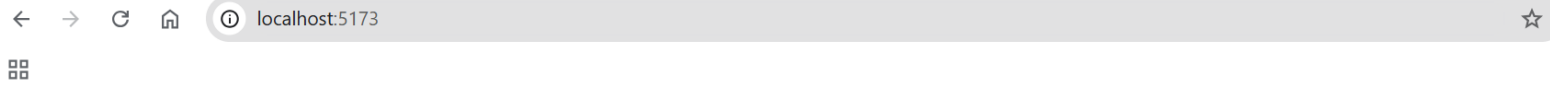
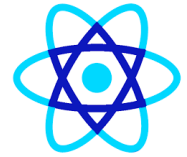
Feature	SWC	Babel
Language	Rust	JavaScript
Speed	⚡ Very Fast	🐢 Slower
JSX	✓	✓
TypeScript	✓	✓
Plugin ecosystem	Limited	Very rich
Configuration	Minimal	Complex



.npmrc settings

- npm config set strict-ssl=false
- npm config set proxy
http://username:password@proxyname:port
- npm config set https-proxy
http://username:password@proxyname:port
- npm config set registry <http://registry.npmjs.org/>
- Or
- Npm config edit
- proxy = http://username:password@proxyname:port
- https-proxy = http://username:password@proxyname:port
- Registry= <http://registry.npmjs.org/>

Loading Reactjs in Browser

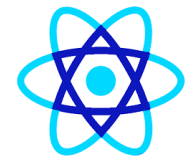


Vite + React

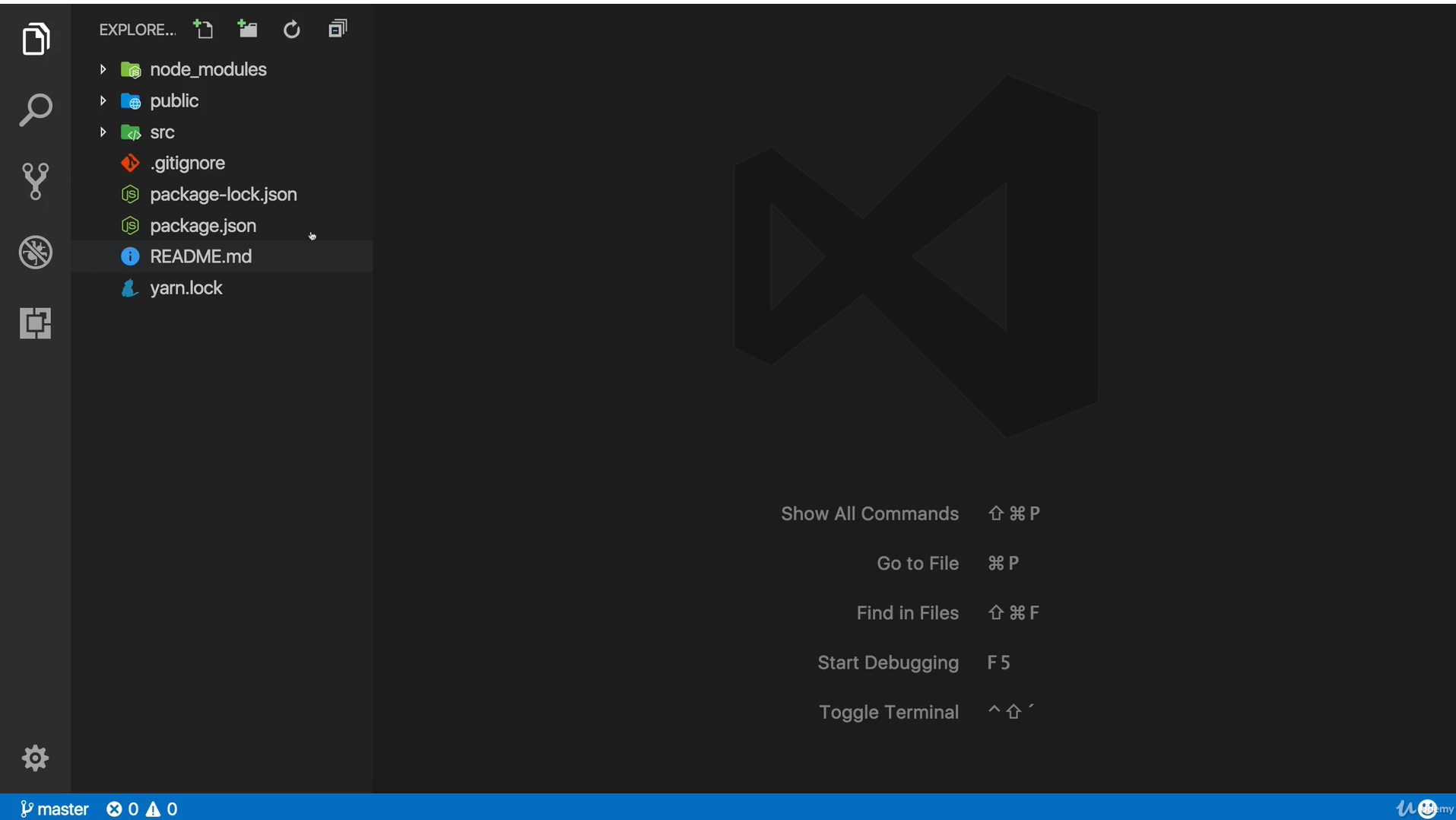
count is 0

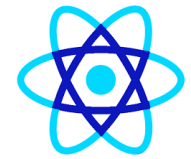
Edit `src/App.jsx` and save to test HMR

Click on the Vite and React logos to learn more



Open Project in Editor

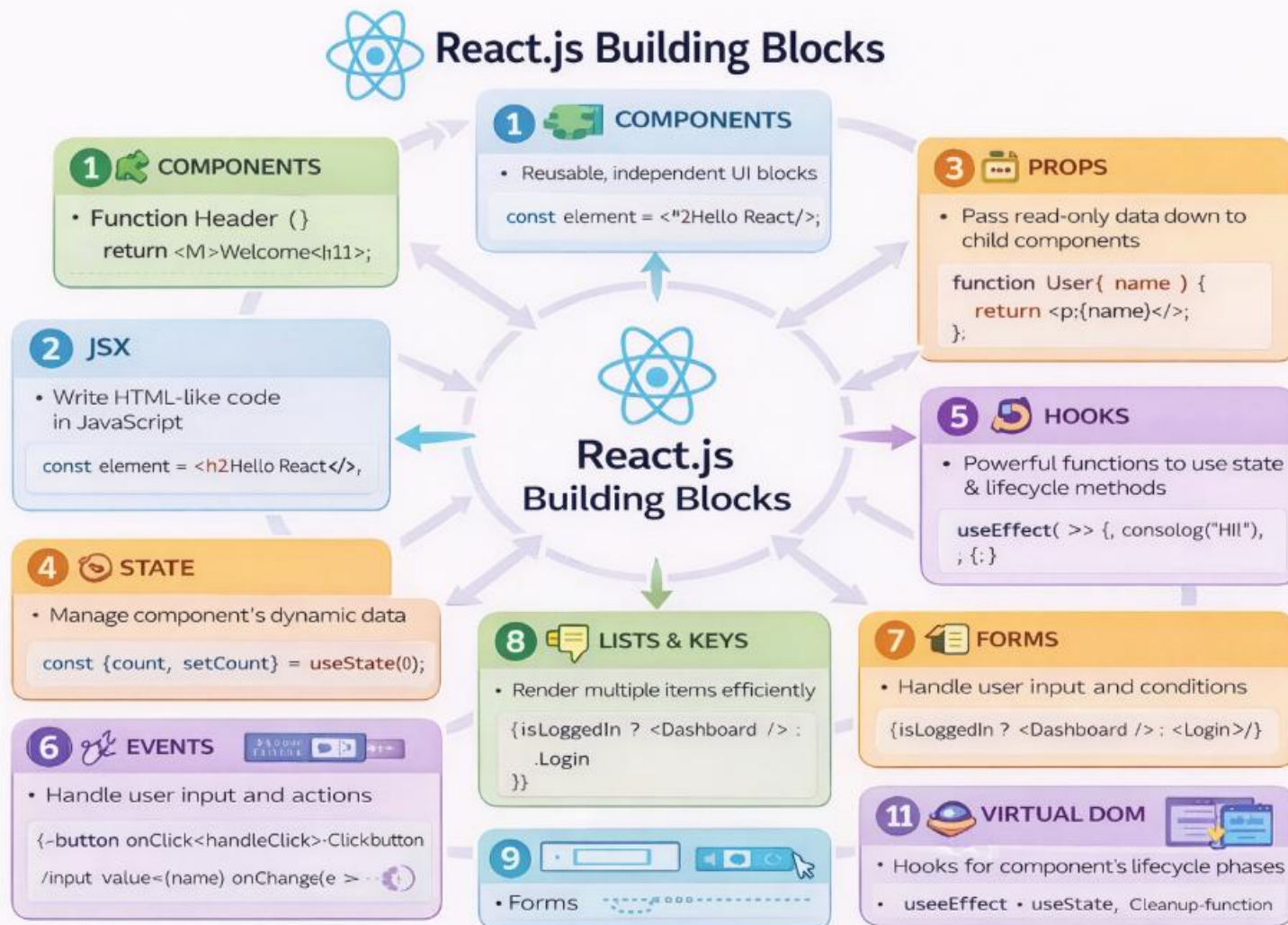
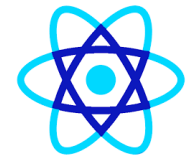




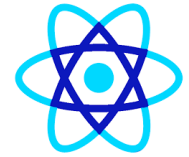
Building Blocks

- Components
- Props
- State
- State Lifecycle
- Event handling
- Keys

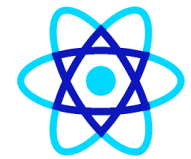
Building Blocks



Planning React App



- 1 Component Tree / Component Structure
- 2 Application State (Data)
- 3 Components vs Containers



Planning React App

```
{
  "name": "react-complete-guide",
  "version": "0.1.0",
  "private": true,
  "dependencies": {
    "react": "^16.0.0",
    "react-dom": "^16.0.0",
    "react-scripts": "1.0.13"
  },
  "scripts": {
    "start": "react-scripts start",
    "build": "react-scripts build",
    "test": "react-scripts test --env=jsdom",
    "eject": "react-scripts eject"
  }
}
```

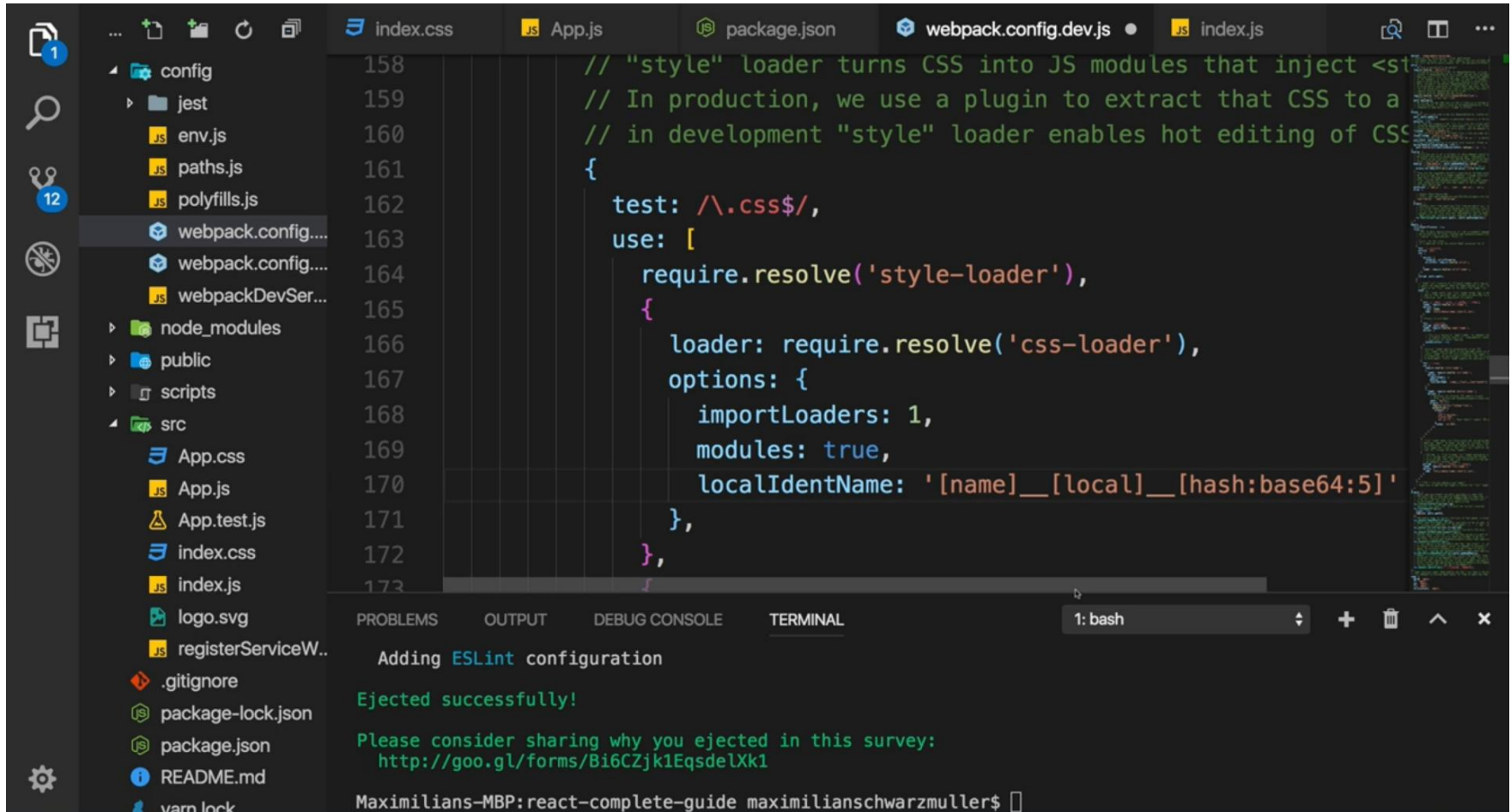
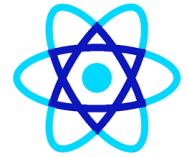
Maximilians-MBP:react-complete-guide maximilianschwarzmueller\$ npm run eject

> react-complete-guide@0.1.0 eject /Users/maximilianschwarzmueller/development/reactjs/tutorials/udemy/react-complete-guide

> react-scripts eject

? Are you sure you want to eject? This action is permanent. (y/N)

Planning React App

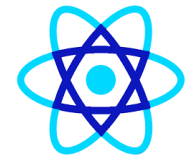


The screenshot shows a VS Code editor with the following components:

- File Explorer (Left):** Displays a project structure with folders like `config`, `jest`, `node_modules`, `public`, `scripts`, and `src`. The `src` folder contains files like `App.css`, `App.js`, `App.test.js`, `index.css`, `index.js`, `logo.svg`, and `registerServiceWorker.js`.
- Editor (Center):** Shows the `webpack.config.dev.js` file. The visible code includes comments about the "style" loader and a configuration for the `test` rule using the `style-loader` and `css-loader`.

```
// "style" loader turns CSS into JS modules that inject <style> tags into the document
// In production, we use a plugin to extract that CSS to a separate file
// in development "style" loader enables hot editing of CSS
{
  test: /\.css$/,
  use: [
    require.resolve('style-loader'),
    {
      loader: require.resolve('css-loader'),
      options: {
        importLoaders: 1,
        modules: true,
        localIdentName: '[name]__[local]__[hash:base64:5]'
      }
    }
  ],
}
```
- Terminal (Bottom):** Shows the output of the `npm eject` command. The messages include "Adding ESLint configuration", "Ejected successfully!", and a survey link: <http://goo.gl/forms/Bi6CZjk1EqsdeIXk1>. The prompt shows the user is in the `react-complete-guide` directory.

Planning React App



Google Fonts

Viewing 846 of 846 font families

Roboto
Christian Robertson (12 styles)

Encode Sans Expanded
Impallari Type, Andres Torresi, Jacques Le Bailly (9 styles)

Asap Condensed
Omnibus-Type (8 styles)

Open Sans
Steve Matteson (10 styles)

Lato
Łukasz Dziedzic (10 styles)

1 Family Selected

Your Selection [Clear All](#)

Open Sans

EMBED CUSTOMIZE [Load Time: Fast](#)

Embed Font

To embed your selected fonts into a webpage, copy this code into the <head> of your HTML document.

STANDARD @IMPORT

```
<link href="https://fonts.googleapis.com/css?family=Open+Sans:400,700" rel="stylesheet">
```

Specify in CSS

Use the following CSS rules to specify these families:

```
font-family: 'Open Sans', sans-serif;
```

For examples of how fonts can be added to webpages, see the [getting started guide](#).

Categories

- ☒ Serif
- ☒ Sans Serif
- ☒ Display
- ☒ Handwriting
- ☒ Monospace

Sorting

Trending

Languages

All Languages

Number of styles

Thickness

Slant

Width

udemy



Atoms, Molecules and Organisms

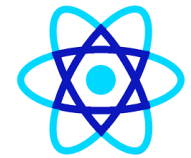
- Examples of Atoms:
 - A button component (Button)
 - A label (Label)
 - An input field (TextField)
 - Icons (Icon)
 - Text elements like headings (Heading, Text)

jsx

```
import React from 'react';

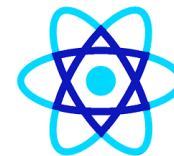
const Button = ({ label, onClick }) => {
  return <button onClick={onClick}>{label}</button>;
};

export default Button;
```



Atoms, Molecules and Organisms

- Molecules are formed by combining two or more atoms.
- They are slightly more complex than atoms but are still simple and often represent small, reusable components that encapsulate specific functionality.
- Examples of Molecules:
 - A form input field with a label and error message (combining atoms like Label, Input, ErrorMessage)
 - A search bar (Input + Button)
 - A card with text and an image (Image + Text)



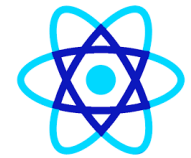
Atoms, Molecules and Organisms

jsx

```
import React from 'react';
import Button from './Button'; // An Atom
import TextField from './TextField'; // An Atom

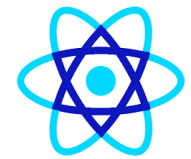
const SearchBar = ({ placeholder, buttonText, onSearch }) => {
  return (
    <div>
      <TextField placeholder={placeholder} />
      <Button label={buttonText} onClick={onSearch} />
    </div>
  );
};

export default SearchBar;
```



Atoms, Molecules and Organisms

- Organisms are more complex UI components that consist of molecules and/or atoms working together to form a relatively distinct section of the interface.
- They represent larger and more complex UI sections, like headers, forms, or cards with multiple functionalities.
- Examples of Organisms:
 - A navbar that includes multiple links, search bars, and user profile components (composed of molecules like SearchBar, NavLink, UserAvatar)
 - A user profile section that includes an image, name, and contact information
 - A complete form with various input fields, buttons, and labels



Atoms, Molecules and Organisms

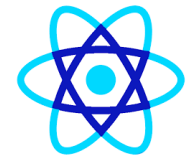
jsx

Copy code

```
import React from 'react';
import SearchBar from './SearchBar'; // A Molecule
import UserProfile from './UserProfile'; // Another Molecule

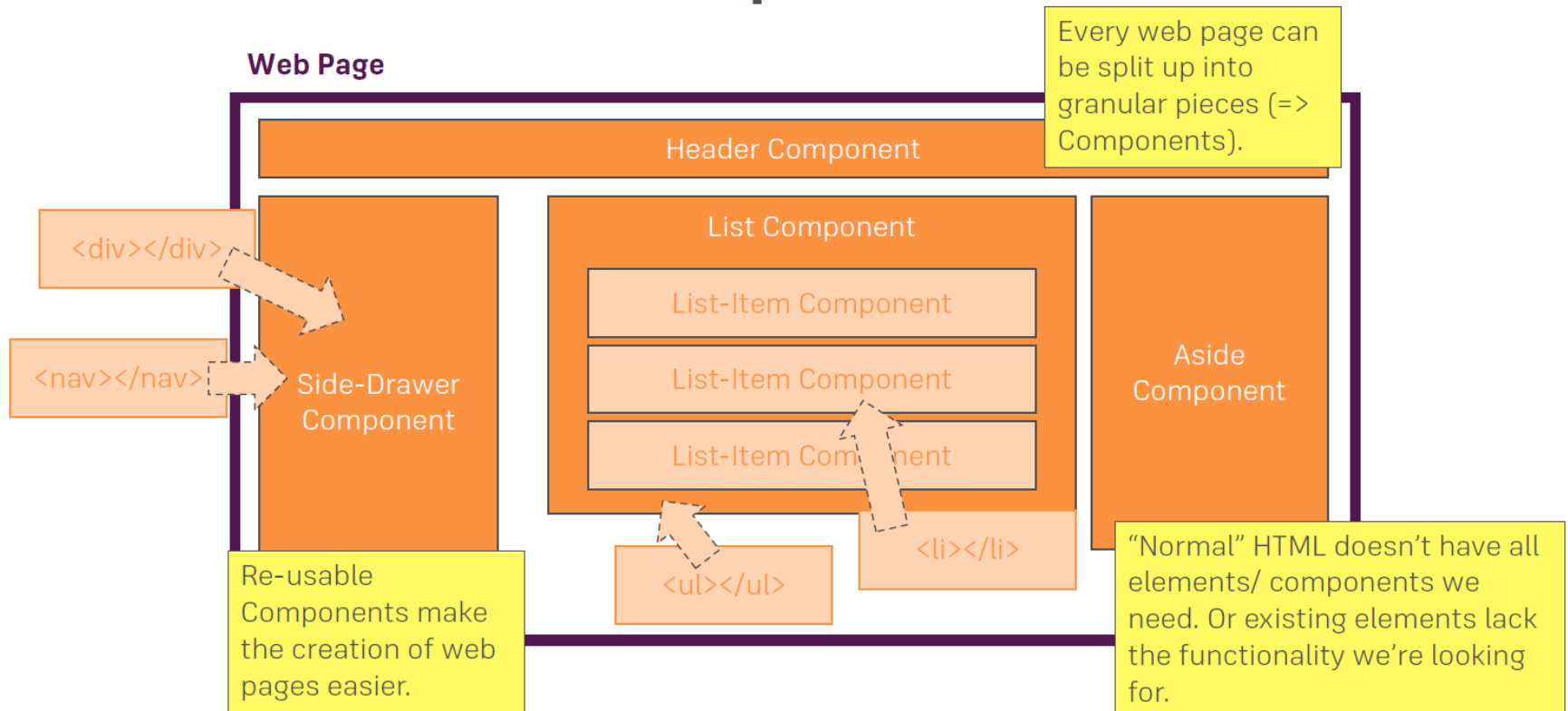
const Header = () => {
  return (
    <header>
      <h1>My App</h1>
      <SearchBar placeholder="Search..." buttonText="Go" onSearch={() => {}} />
      <UserProfile name="John Doe" avatarUrl="/avatar.png" />
    </header>
  );
};

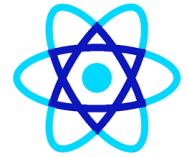
export default Header;
```



Application Architecture

Components

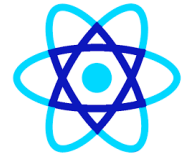




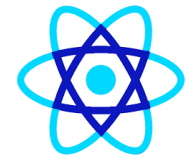
- Components are independent and reusable bits of code.
- They serve the same purpose as JavaScript functions.
- It works in isolation and returns HTML.
- In other words, we can say that every application we develop in React is made up of pieces called components.

Functional Components

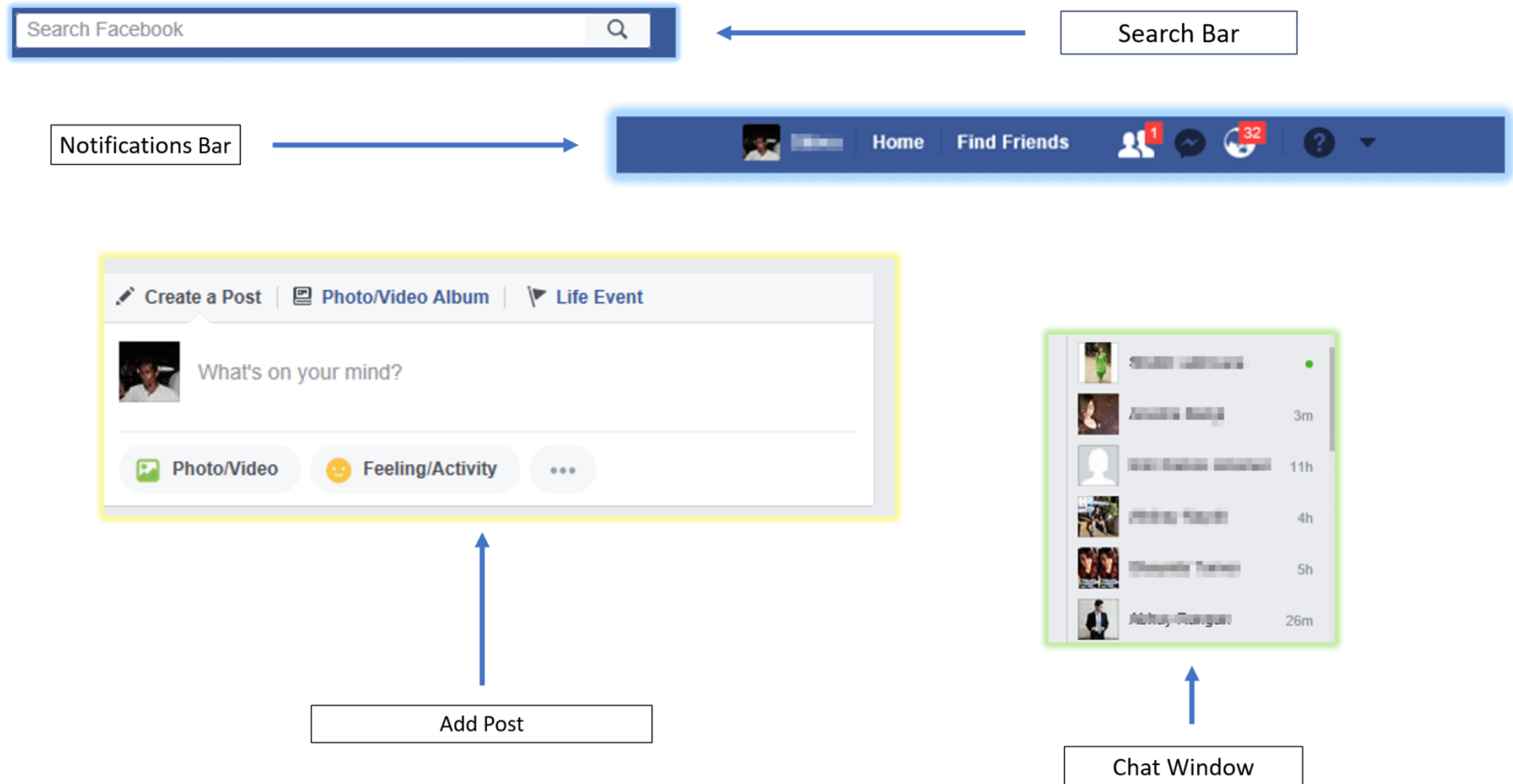
Class Components

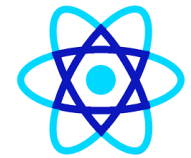


- The entire application can be modeled as a set of independent components.
- Different components are used to serve different purposes.
- This enables us to keep logic and views separate.
- React renders multiple components simultaneously.
- Components can be either stateful or stateless.



Facebook Components





Two types of components

Stateless vs Stateful Components

Stateful (Containers)

class XY extends Component



Access to State



Lifecycle Hooks

Access State and Props via **"this"**

`this.state.XY` & `this.props.XY`

Use only if you need to manage State or
access to Lifecycle Hooks!

Stateless

`const XY = (props) => { ... }`



Access to State

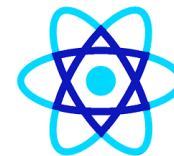


Lifecycle Hooks

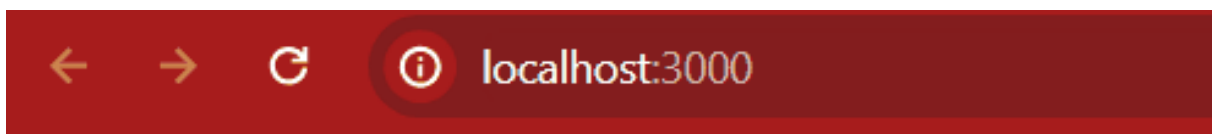
Access Props via **"props"**

`props.XY`

Use in all other Cases



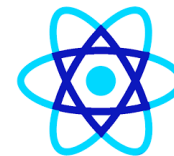
State Management



You've clicked 0 times!

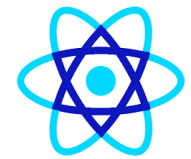
The number of times you have clicked is even!

Click me



Component

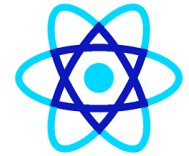
- Components can be generated using cli tool.
- Go to root of the project and execute the following command
- `npx generate-react-cli component Logo`



Component

```
export class BankApp extends Component{
  no usages
  constructor(props, context) {
    super(props, context);
    //console.log("Entering  constructor");
    //step 1 create state
    this.state={
      currentTime:new Date(),
    }
  }
  //step2
  1+ usages
  timerEvent=() : void =>{
    this.setState( state: {
      currentTime:new Date()
    })
  }
}
```

Component Lifecycle



Only available in Stateful Components!

`constructor()`

`componentWillMount()`

`componentWillReceiveProps()`

`shouldComponentUpdate()`

`componentWillUpdate()`

`componentDidUpdate()`

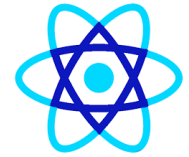
`componentDidCatch()`

`componentDidMount()`

`componentWillUnmount()`

`render()`

Component Lifecycle



`constructor()`

`componentWillReceiveProps()`

`componentWillUpdate()`

`componentDidCatch()`

`componentWillUnmount()`

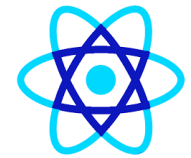
`componentWillMount()`

`shouldComponentUpdate()`

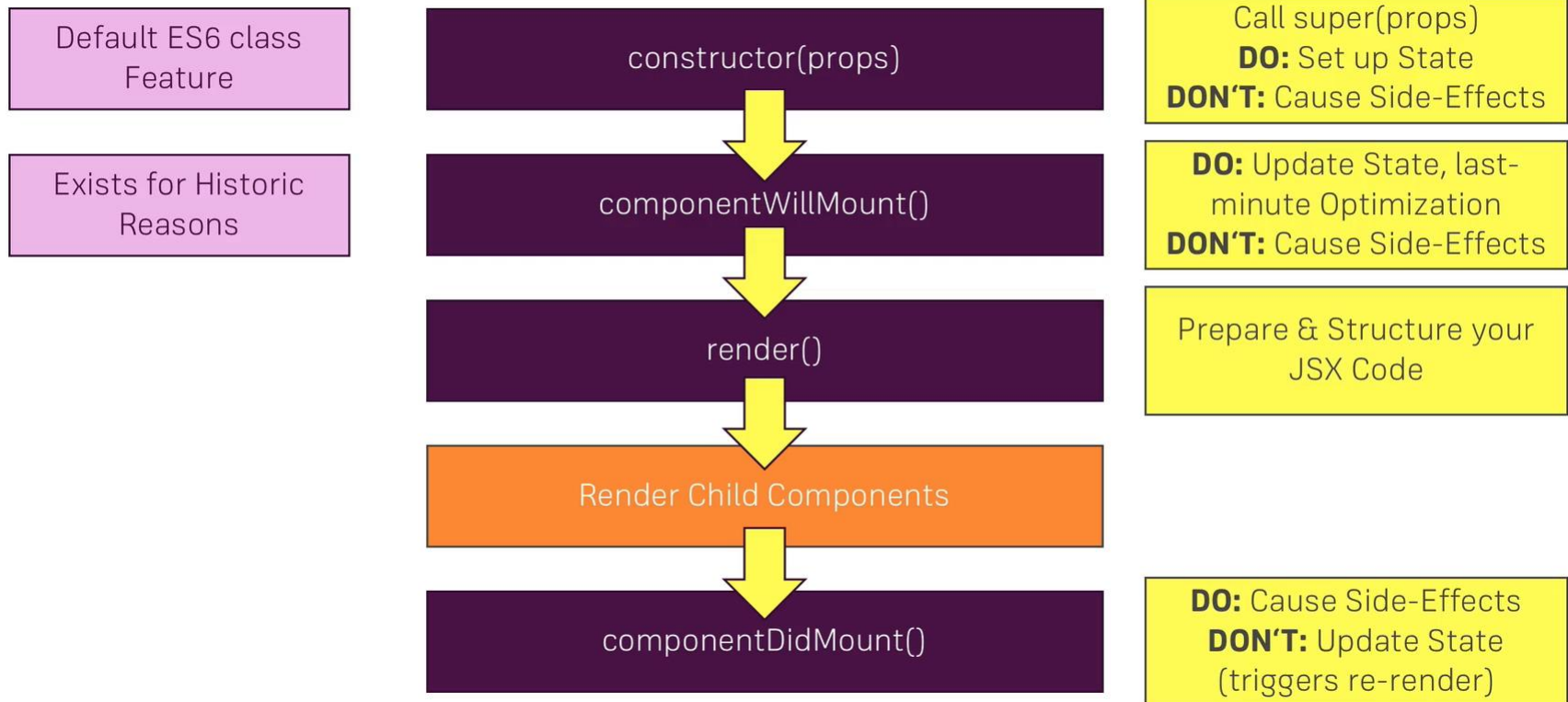
`componentDidUpdate()`

`componentDidMount()`

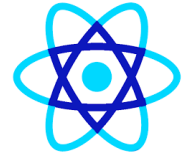
`render()`



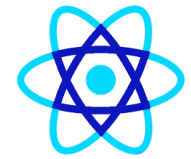
Component Lifecycle



React Component Lifecycle



-
- Initial Phase
 - Updating Phase
 - Props change Phase
 - Unmounting Phase

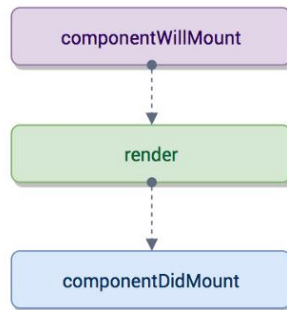


React Component Lifecycle

Initialization

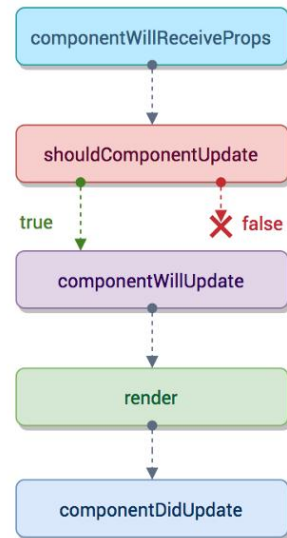
setup props and state

Mounting

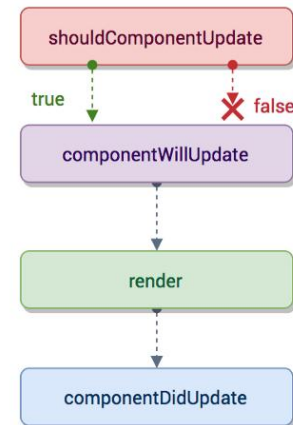


Updation

props

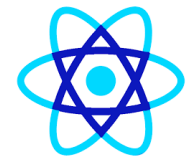


states

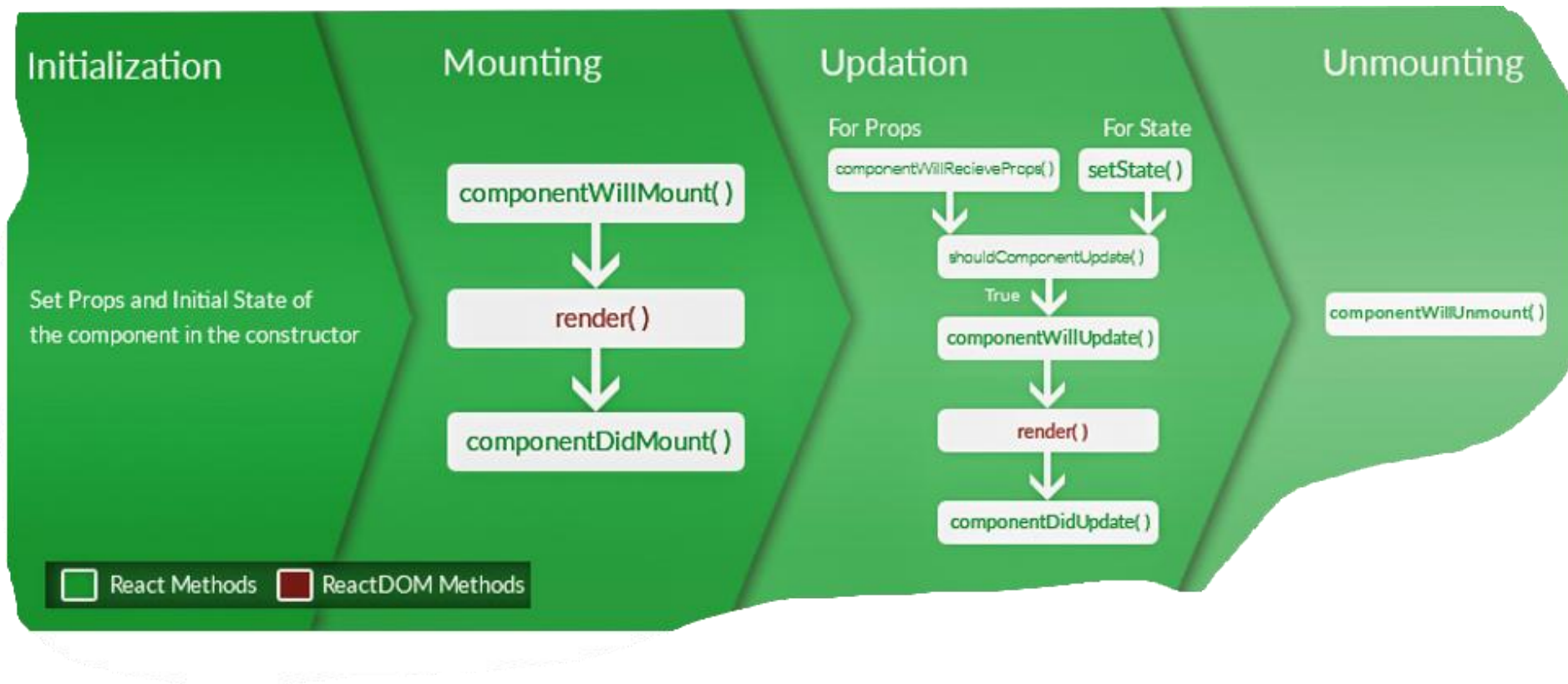


Unmounting

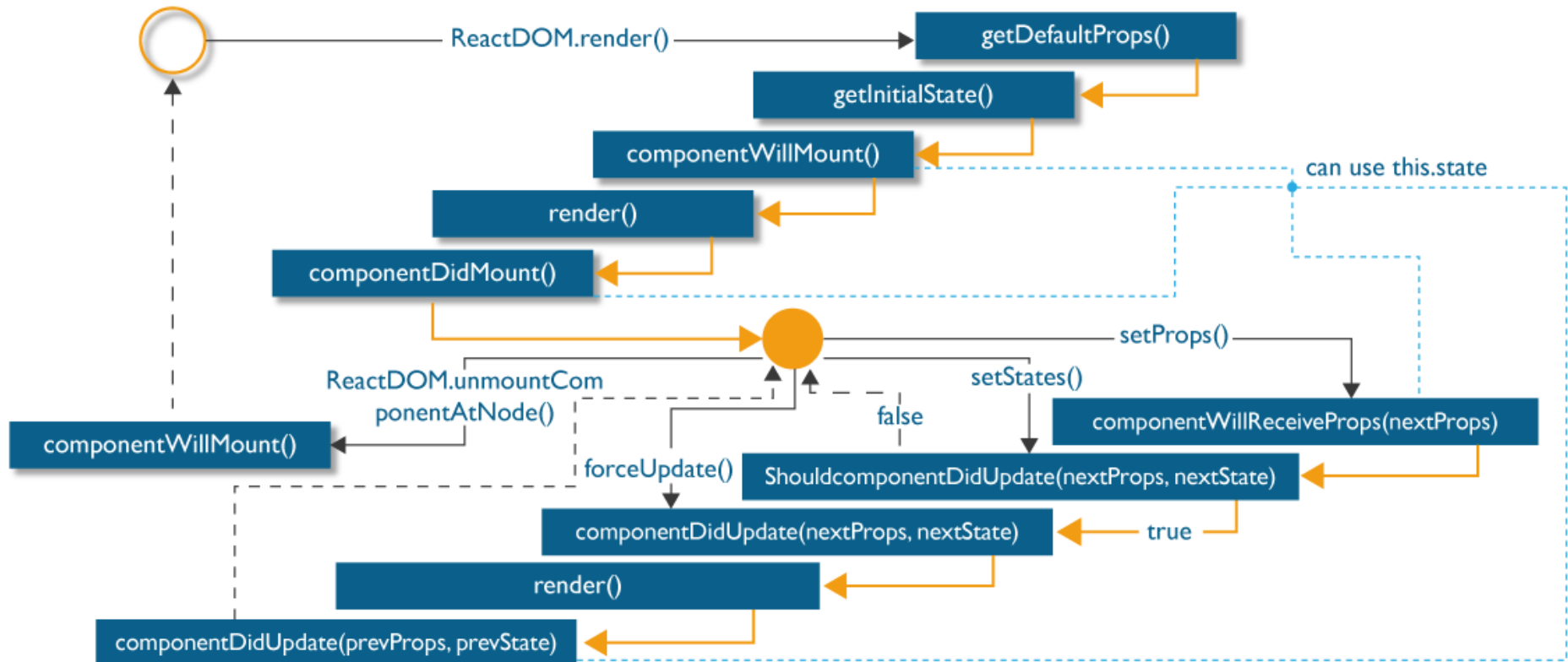
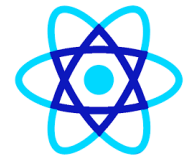


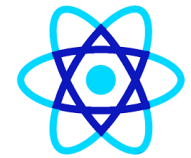


React Component Lifecycle



React Component Lifecycle





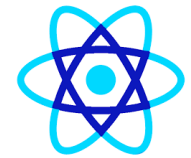
Initial Phase

- ***getDefaultProps():***
 - This method is used to specify the default value of **this.props**.
 - It gets called before your component is even created or any props from the parent are passed into it.
- ***getInitialState():***
 - This method is used to specify the default value of **this.state** before your component is created.
- ***componentWillMount():***
 - This is the last method that you can call before your component gets rendered into the DOM.
 - But if you call **setState()** inside this method your component will not re-render.
- ***render():***
 - This method is responsible for returning a single root HTML node and must be defined in each and every component.
 - You can return **null** or **false** in case you don't want to render anything.
- ***componentDidMount():***
 - Once the component is rendered and placed on the DOM, this method is called. Here you can perform any DOM querying operations.



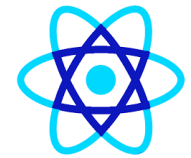
Updating Phase

- ***shouldComponentUpdate():***
 - Using this method you can control your component's behavior of updating itself.
 - If you return a true from this method, the component will update. Else if this method returns a false, the component will skip the updating.
- ***componentWillUpdate():***
 - This method is called just before your component is about to update. In this method, you can't change your component state by calling **this.setState**.
- ***render():***
 - If you are returning false via **shouldComponentUpdate()**, the code inside **render()** will be invoked again to ensure that your component displays itself properly.
- ***componentDidUpdate():***
 - Once the component is updated and rendered, then this method is invoked. You can put any code inside this method, which you want to execute once the component is updated.



Props Change Phase

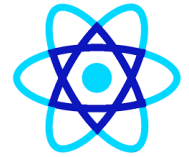
- ***componentWillReceiveProps():***
 - This method returns one argument which contains the new prop value that is about to be assigned to the component.
 - *Rest of the lifecycle methods behave identically to the methods which we saw in the previous phase.*
- ***shouldComponentUpdate()***
- ***componentWillUpdate()***
- ***render()***
- ***componentDidUpdate()***



The Unmounting Phase

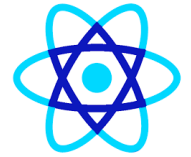
- ***componentWillUnmount():***
 - Once this method is invoked, your component is removed from the DOM permanently.
 - In this method, you can perform any clean-up related tasks like removing event listeners, stopping timers, etc.

2 Functional Components (Modern & Recommended)



-  **Definition**
 - A **functional component** is a **JavaScript function** that:
 - Returns JSX
 - Uses **Hooks** for state and lifecycle
 - No `this` keyword
 - Simpler and cleaner

2 Functional Components (Modern & Recommended)



Syntax with Hooks

jsx

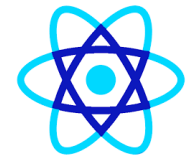
```
import { useState } from "react";

function Counter() {
  const [count, setCount] = useState(0);

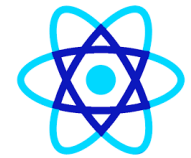
  return (
    <div>
      <h2>Count: {count}</h2>
      <button onClick={() => setCount(count + 1)}></button>
    </div>
  );
}

export default Counter;
```

2 Functional Components (Modern & Recommended)



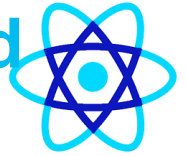
Lifecycle	Hook
Mount	<code>useEffect(() => {}, [])</code>
Update	<code>useEffect(() => {}, [deps])</code>
Unmount	<code>useEffect(() => { return () => {} }, [])</code>



Key Differences(Class vs Functional)

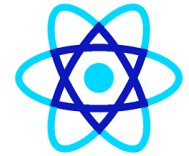
Feature	Class Component	Functional Component
Syntax	Verbose	Simple
State	this.state	useState()
Lifecycle	Methods	Hooks
this keyword	Yes	No
Performance	Slightly slower	Slightly faster
Code reuse	HOCs, Render props	Custom Hooks
Future support	Legacy	Recommended

⚡ Why Functional Components Are Preferred Today



- ☒ Less boilerplate
- ☒ Easier to test
- ☒ Better performance optimizations
- ☒ Cleaner lifecycle handling
- ☒ Works better with modern tools (Vite, SWC)

When to Use What?



Scenario	Choice
New project	☒ Functional
Legacy codebase	Class
Simple UI	Functional
Heavy lifecycle logic	Functional + Hooks



Example Same Component Both Types

Class

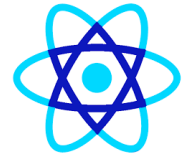
jsx

```
class Hello extends React.Component {  
  render() {  
    return <h1>Hello {this.props.name}</h1>;  
  }  
}
```

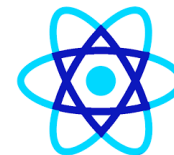
Functional

jsx

```
const Hello = ({ name }) => <h1>Hello {name}</h1>;
```



- ♦ **What is an Expression?**
- An **expression** is anything that produces a value. In JSX, expressions are written inside `{ }`.
-  **Allowed**
- Variables
- Function calls
- Ternary operators
- Mathematical operations



jsx

```
const name = "React";
```

```
<h1>Hello {name}</h1>
```

```
<h2>{10 + 5}</h2>
```

```
<p>{isLoggedIn ? "Welcome" : "Please Login"}</p>
```

✗ Not Allowed

- `if`, `for`, `while` statements
- `switch` statements

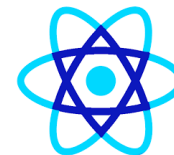
! Use ternary or logical operators instead.



2 JSX Attributes

- ◆ What are JSX Attributes?
- Attributes are used to **pass values to elements or components**.

HTML	JSX
class	className
for	htmlFor
onclick	onClick



2 JSX Attributes

Attribute Examples

jsx

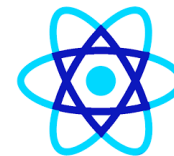
```

<button disabled={true}>Submit</button>
<input type="text" maxLength={10} />
```

Dynamic Attributes

jsx

```
<button className={isActive ? "active" : "inactive"}>
  Save
</button>
```



3 JSX Styles

- ◆ **Inline Styles in JSX**
- Styles are written as a **JavaScript object**.
- ✓ camelCase property names
- ✓ Values as strings or numbers

jsx

```
const style = {  
  color: "blue",  
  fontSize: "20px",  
  backgroundColor: "lightgray"  
};  
  
<h2 style={style}>Hello JSX</h2>
```



Inline Style (Direct)

jsx

```
<h1 style={{ color: "red", marginTop: 20 }}>  
  React Style  
</h1>
```



◆ Styling with className (Recommended)

jsx

```
<h2 className="title">Welcome</h2>
```

CSS

```
.title {  
  color: green;  
  font-size: 24px;  
}
```

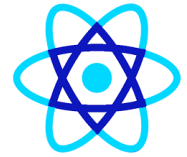


Common JSX Rules (Very Important)

-  **Must Follow**
-  Single parent element
-  Self-closing tags
- `<img />`
- `<input />`
-  JavaScript inside `{}`

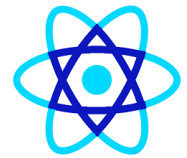


JSX Cheat Sheet (Interview Ready)

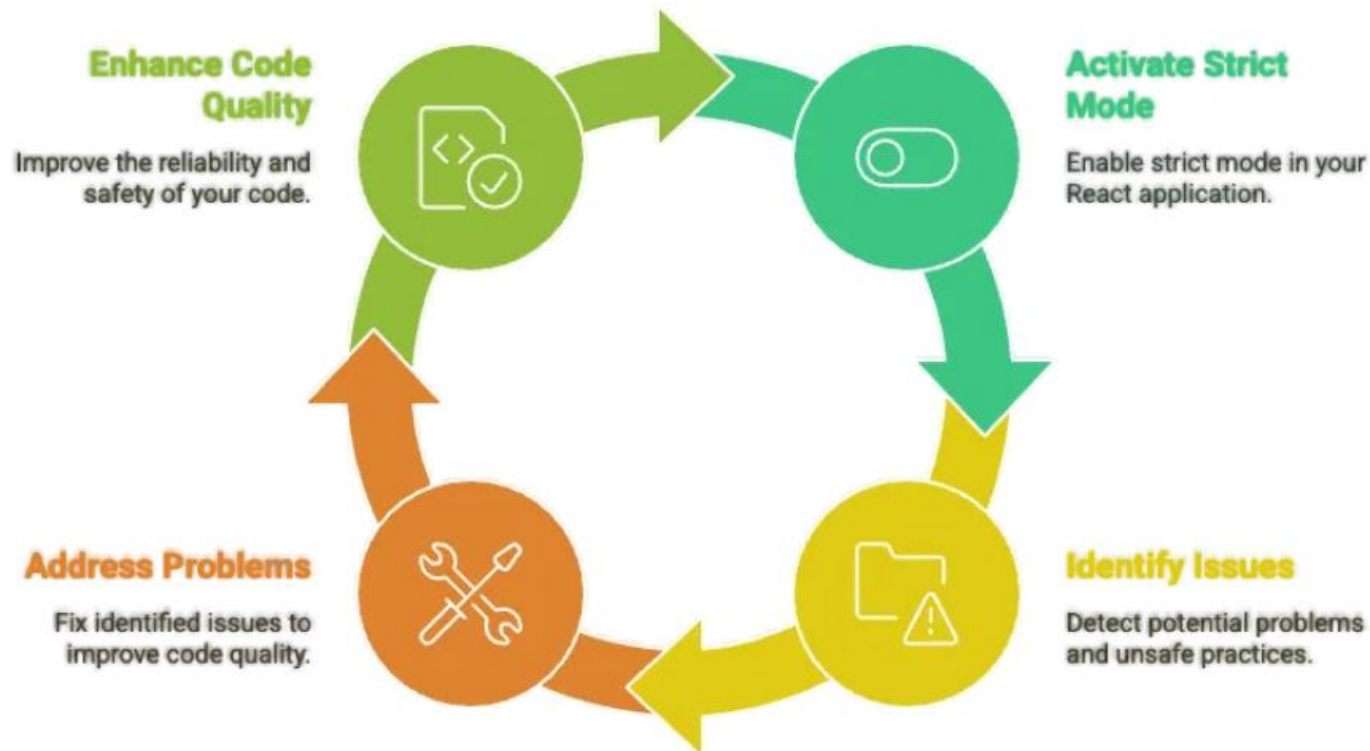


Topic	Key Point
Expressions	Use {}
Attributes	camelCase
Styles	JS object
Classes	className
Conditions	Ternary / &&

React.StrictMode



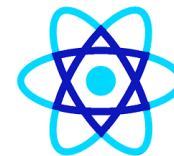
React Strict Mode Cycle





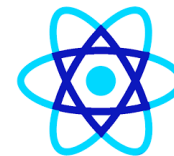
React.StrictMode

- **StrictMode** is a **development-only tool** that:
 - Finds unsafe side effects
 - Warns about deprecated APIs
 - Helps you write future-proof React code
- 👉 **It does NOT affect production builds**



React.StrictMode

- **[2] Where StrictMode is in a Vite App**
- **main.jsx (default Vite React template)**
- `import { StrictMode } from 'react'`
- `import { createRoot } from 'react-dom/client'`
- `import './index.css'`
- `import App from './App.jsx'`
- `createRoot(document.getElementById('root')).render(`
- `<StrictMode>`
- `<App />`
- `</StrictMode>,`
- `)`
-



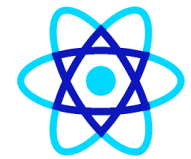
React.StrictMode

- **③ What StrictMode Actually Does**
- **↺ Double Invokes (DEV only)**
- StrictMode intentionally runs certain things **twice** to detect side effects:
- Component render
- useEffect (setup + cleanup + setup)
- useState initializer
- useReducer initializer
- **!** This is **expected behavior**, not a bug.



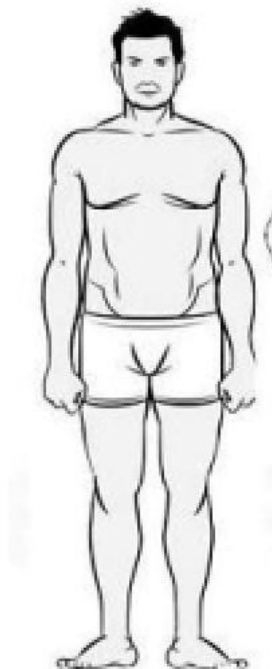
React.StrictMode

Feature	Vite	CRA
StrictMode default	✓ Yes	✓ Yes
Fast refresh	Faster	Slower
Dev behavior	Same	Same
Production impact	None	None



Require vs Import

require

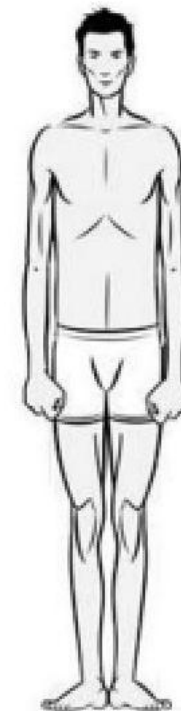


```
//----- lib.js -----  
function add(x, y) {  
  return x + y  
}  
module.exports = {  
  add: add,  
};
```

```
//----- main.js -----  
var add = require('lib').add;  
console.log(add(1,2));  
|
```



import



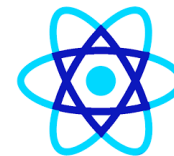
```
//----- lib.js -----  
export function add(x, y) {  
  return x + y  
}
```

```
//----- main.js -----  
import { add } from 'lib';  
console.log(add(1,2));
```



Different types of export

- 1— Named exports (several per module)
- 2— Default exports (one per module)
- 3— Mixed named & default exports
- 4— Cyclical Dependencies



Named Export

```
//----- lib.js -----  
export const sqrt = Math.sqrt;  
export function square(x) {  
  return x * x;  
}  
export function diag(x, y) {  
  return sqrt(square(x) + square(y));  
}
```

```
//----- main.js -----  
import { square, diag } from 'lib';  
console.log(square(11)); // 121  
console.log(diag(4, 3)); // 5
```




Named Export

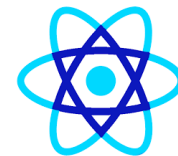
or

//----- main.js -----

import * as lib from 'lib';

console.log(lib.square(11)); // 121

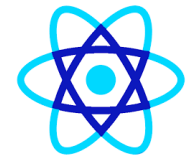
console.log(lib.diag(4, 3)); // 5



Default exports (one per module)

- `//----- myFunc.js -----`
- `export default function () { ... };`

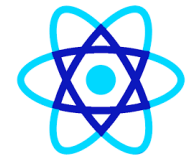
- `//----- main1.js -----`
- `import myFunc from 'myFunc';`
- `myFunc();`



Mixed Default and Named Exports

- `//----- underscore.js -----`
- `export default function (obj) {`
- `...`
- `};`
- `export function each(obj, iterator, context) {`
- `...`
- `}`
- `export { each as forEach };`

- `//----- main.js -----`
- `import _, { each } from 'underscore';`
- `...`



Cyclical Dependency

- `// lib.js`
- `import Main from 'main';`
- `var lib = {message: "This Is A Lib"};`
- `export { lib as Lib };`

- `// main.js`
- `import { Lib } from 'lib';`
- `export default class Main {`
- `// ....`
- `}`



React Form Design

- **1. What Formik actually does**
- Formik helps you manage:
 - **form state** – values, errors, touched fields, isSubmitting, etc.
 - **validation** – required fields, patterns, custom rules (often with Yup)
 - **submit** – central onSubmit handler
 - **UX helpers** – dirty, isValid, resetForm(), etc.



React Form Design

- **2. Setup**
 - npm install formik yup
 - # or
 - yarn add formik yup
- We'll use:
 - **Formik** – form state + helpers
 - **Yup** – schema-based validation

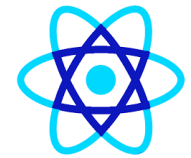


React Form Design

- **3. Basic Formik usage (hook style)**

First, the simplest possible form using useFormik:

```
import React from "react";
import { useFormik } from "formik";
function SimpleLoginForm() {
  const formik = useFormik({
    initialValues: {
      email: "",
      password: "",
    },
    onSubmit: (values) => {
      console.log("Form submitted:", values);
    },
  });
}
```



React Form Design

- **3. Basic Formik usage (hook style)**

```
return (  
  <form onSubmit={formik.handleSubmit}>  
    <label>  
      Email  
      <input  
        type="email"  
        name="email"  
        value={formik.values.email}  
        onChange={formik.handleChange}  
        onBlur={formik.handleBlur}  
      />  
    </label>  
  
    <button type="submit">Login</button>  
  </form>
```



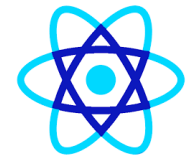

React Form Design

- Core ideas:
 - `initialValues` defines form fields.
 - `formik.values` holds current values.
 - `formik.handleChange` and `formik.handleBlur` are wired to each input.
 - `formik.handleSubmit` is attached to `<form onSubmit>`.



React Form Design

- **4. Adding validation with Yup**
 - Now let's design a **User Registration form** with:
 - name
 - email
 - password
 - confirmPassword
 - role (select)
 - acceptTerms (checkbox)

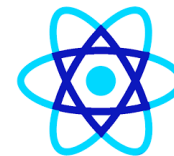


React Form Design

- **4. Adding validation with Yup**

- 4.1 Define validation schema**

```
import * as Yup from "yup";  
const registrationSchema = Yup.object({  
  name: Yup.string()  
    .trim()  
    .required("Name is required"),  
  email: Yup.string()  
    .email("Invalid email format")  
    .required("Email is required"),  
  password: Yup.string()  
    .min(8, "Password must be at least 8 characters")  
    .required("Password is required"),
```



React Form Design

- **4. Adding validation with Yup**

```
confirmPassword: Yup.string()
  .oneOf([Yup.ref("password")], "Passwords must match")
  .required("Please confirm your password"),
role: Yup.string()
  .oneOf(["user", "admin"], "Invalid role")
  .required("Role is required"),
acceptTerms: Yup.boolean()
  .oneOf([true], "You must accept the terms & conditions"),
});
```



React Form Design

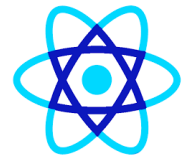
- **UX best practices with Formik**
- **Disable submit** when:
 - `isSubmitting` is true
 - or `!dirty` (nothing changed)
 - or `!isValid` (validation errors)
- **Show errors only after blur/change**
Formik tracks touched. Using `meta.touched` && `meta.error` prevents errors flashing as soon as the form renders.



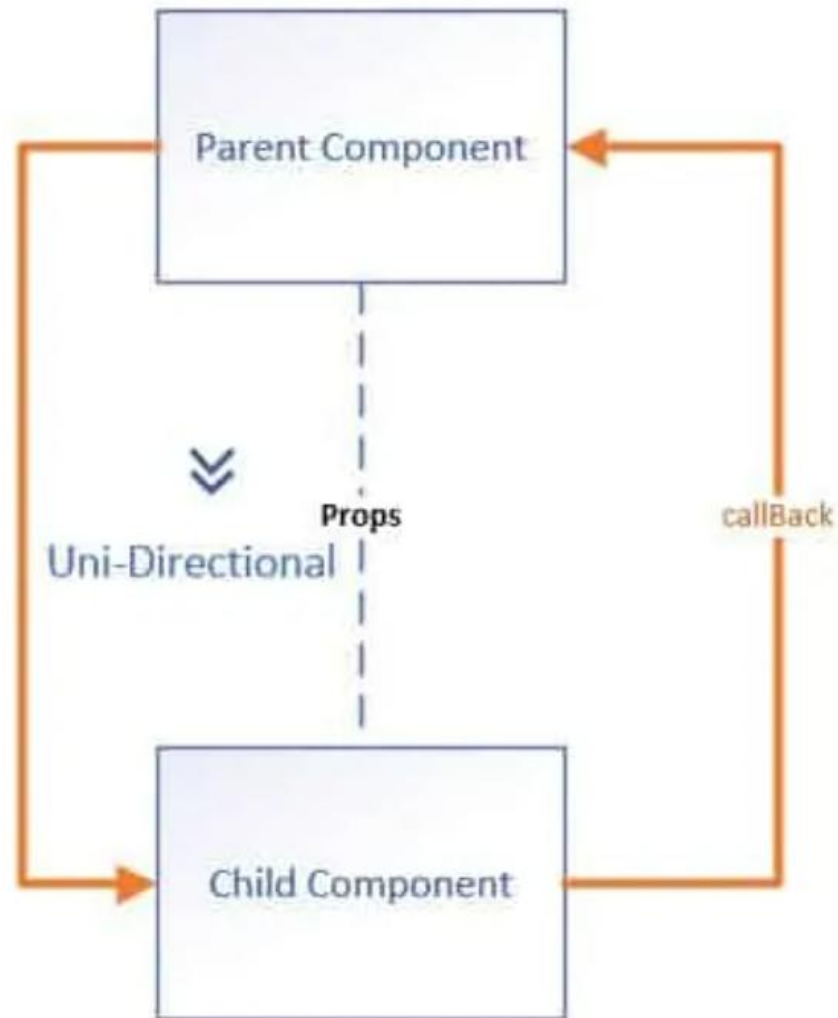
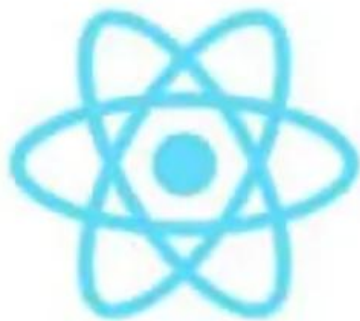
React Form Design

- **UX best practices with Formik**
- **Inline vs global errors**
 - Use `<ErrorMessage>` or `useField` to show inline errors.
 - You can also show a global error (e.g., API error) in `onSubmit` using `setStatus` or `setErrors`.
- **Reset after submit**
 - `resetForm()` inside `onSubmit` once API succeeds.
- **Loading indicator**
 - Use `isSubmitting` to show loading text/spinner on the button

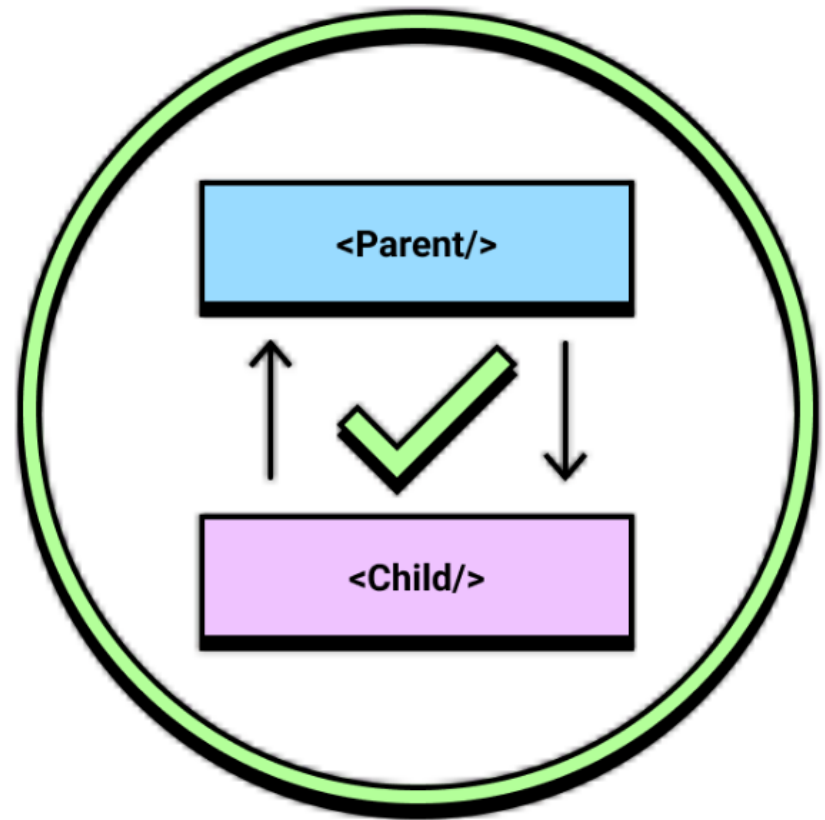
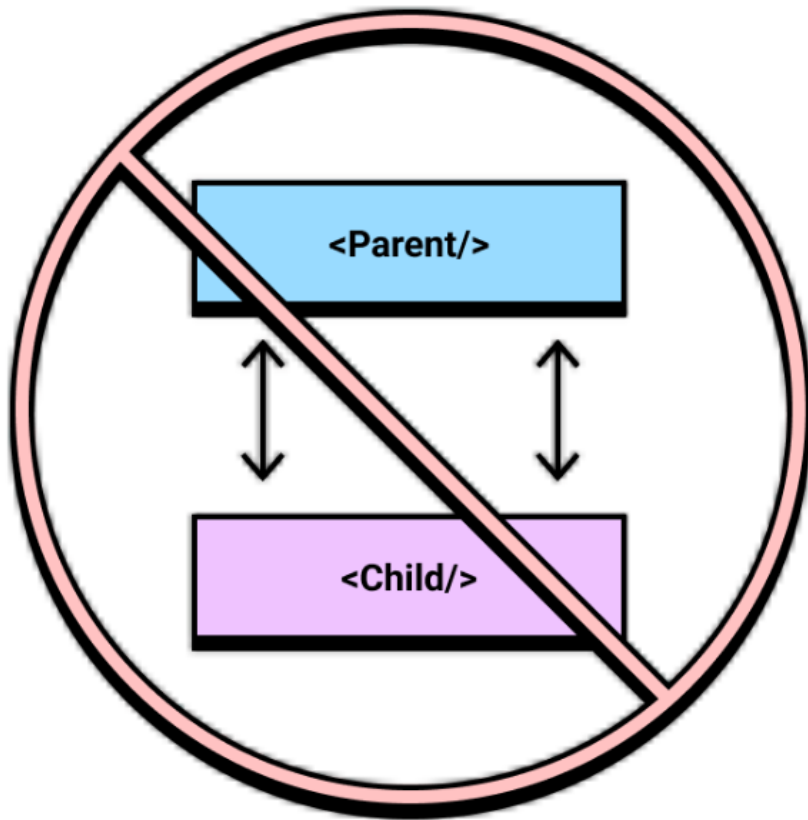
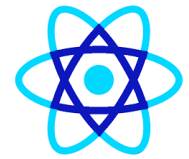
Props



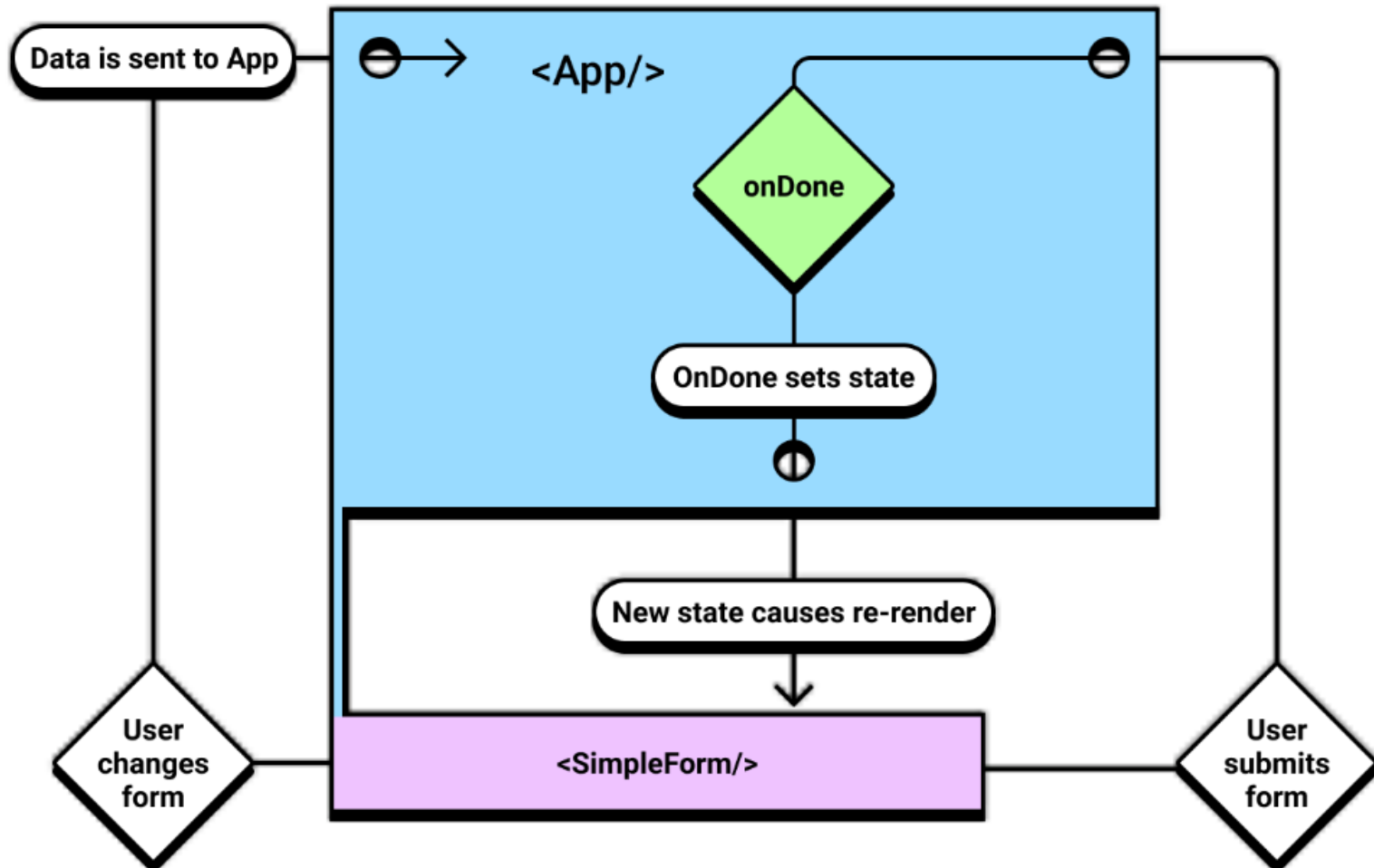
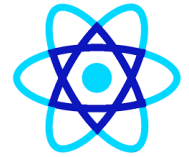
React



Props



Props

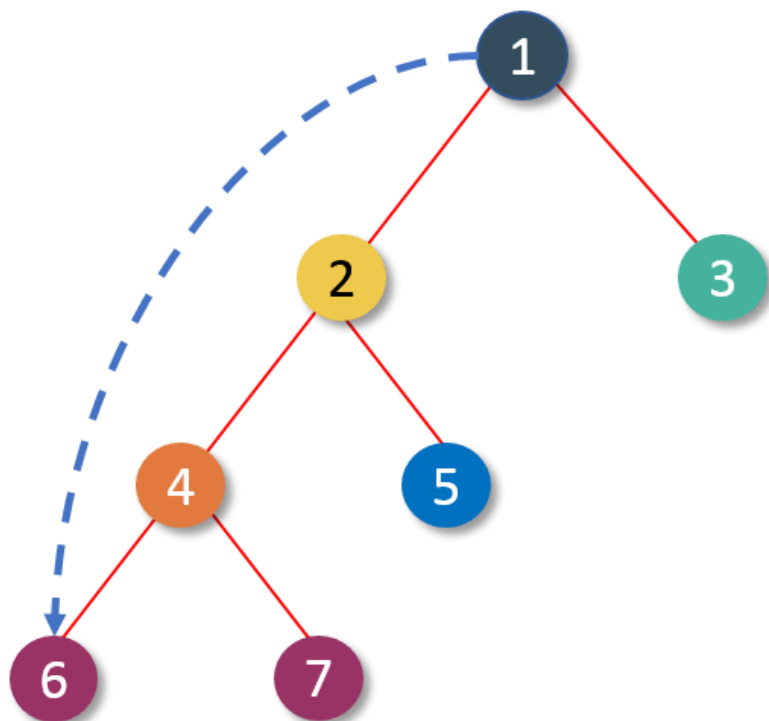




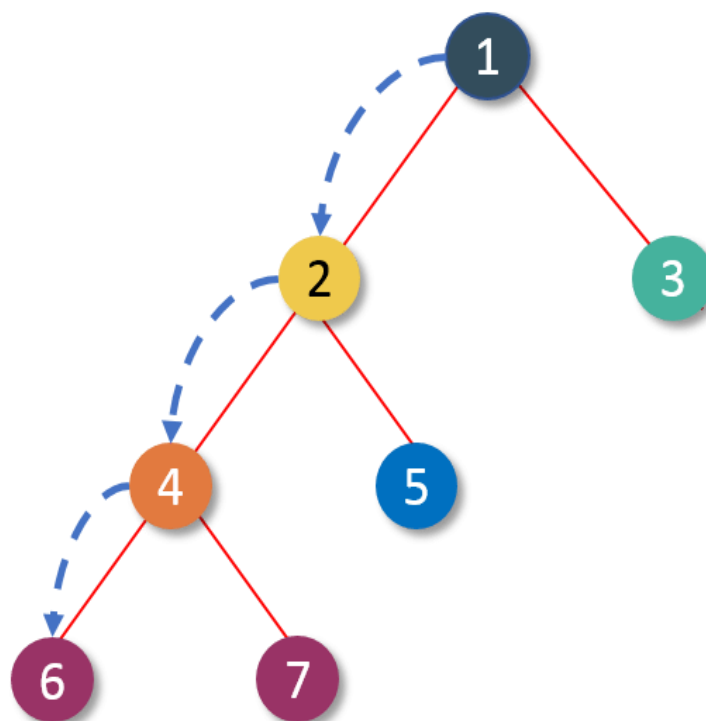
Props

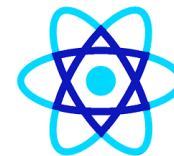
Prop is a way of passing data from parent to child component.

UI Tree



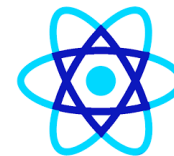
UI Tree





Props

- ♦ **What are Props?**
- **Props (properties) are read-only data passed from a parent component to a child component.**
- 👉 **Child cannot modify props.**



Props

Example: Passing Props

```
jsx

function Parent() {
  return <Child name="React" />;
}

function Child(props) {
  return <h2>Hello {props.name}</h2>;
}
```

Key Points about Props

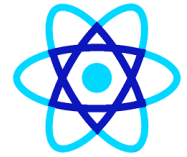
- ✓ Passed from parent to child
- ✓ Immutable (read-only)
- ✓ Used for configuration & display

Destructuring Props

```
jsx

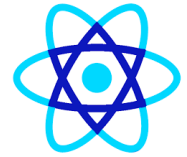
function Child({ name }) {
  return <h2>Hello {name}</h2>;
}
```

React Props Validation



SN	PropsType	Description
1.	<code>PropTypes.any</code>	The props can be of any data type.
2.	<code>PropTypes.array</code>	The props should be an array.
3.	<code>PropTypes.bool</code>	The props should be a boolean.
4.	<code>PropTypes.func</code>	The props should be a function.
5.	<code>PropTypes.number</code>	The props should be a number.
6.	<code>PropTypes.object</code>	The props should be an object.
7.	<code>PropTypes.string</code>	The props should be a string.

React Props Validation

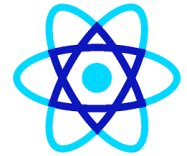


8.	<code>PropTypes.symbol</code>	The props should be a symbol.
9.	<code>PropTypes.instanceOf</code>	The props should be an instance of a particular JavaScript class.
10.	<code>PropTypes.isRequired</code>	The props must be provided.
11.	<code>PropTypes.element</code>	The props must be an element.
12.	<code>PropTypes.node</code>	The props can render anything: numbers, strings, elements or an array (or fragment) containing these types.
13.	<code>PropTypes.oneOf()</code>	The props should be one of several types of specific values.
14.	<code>PropTypes.oneOfType([PropTypes.string, PropTypes.number])</code>	The props should be an object that could be one of many types.



State

- **[2] State — Managing Dynamic Data with useState**
- **◆ What is State?**
- **State is data owned by a component that can change over time.**
- **👉 When state changes, React re-renders the component.**



Example: useState

jsx

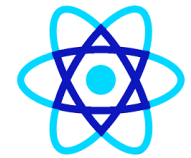
```
import { useState } from "react";

function Counter() {
  const [count, setCount] = useState(0);

  return (
    <button onClick={() => setCount(count + 1)}>
      Count: {count}
    </button>
  );
}
```

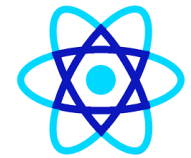
Key Points about State

- ✓ Local to component
- ✓ Mutable using setter function
- ✓ Triggers re-render



Props vs State

Feature	Props	State
Who owns it	Parent	Component itself
Mutable	✗ No	✓ Yes
Purpose	Pass data	Manage data
Triggers re-render	✗	✓



Props vs State

SN	Props	State
1.	Props are read-only.	State changes can be asynchronous.
2.	Props are immutable.	State is mutable.
3.	Props allow you to pass data from one component to other components as an argument.	State holds information about the components.
4.	Props can be accessed by the child component.	State cannot be accessed by child components.
5.	Props are used to communicate between components.	States can be used for rendering dynamic changes with the component.
6.	Stateless component can have Props.	Stateless components cannot have State.
7.	Props make components reusable.	State cannot make components reusable.
8.	Props are external and controlled by whatever renders the component.	The State is internal and controlled by the React Component itself.



4 One-Way Data Flow (Very Important Concept)

◆ What is One-Way Data Flow?

React follows **unidirectional data flow**:

nginx

Parent → Child → UI

- Data flows **downward only**
- Child cannot change parent state directly



4 One-Way Data Flow (Very Important Concept)

🔧 Example: Child Updating Parent (Correct Way)

```
jsx

function Parent() {
  const [count, setCount] = useState(0);

  return <Child count={count} onIncrement={() => setCount(count + 1)} />;
}

function Child({ count, onIncrement }) {
  return (
    <>
      <p>{count}</p>
      <button onClick={onIncrement}>+</button>
    </>
  );
}
```

- ✓ Parent owns state
- ✓ Child triggers update via callback



4 One-Way Data Flow (Very Important Concept)

-  **Why One-Way Data Flow Matters**
-  Predictable behavior
-  Easier debugging
-  Better performance
-  Clear ownership of data



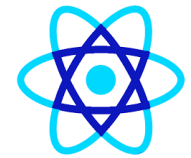
4 One-Way Data Flow (Very Important Concept)

-  **Common Beginner Mistakes**
-  Trying to modify props
-  Sharing state unnecessarily
-  Updating state directly (state++)



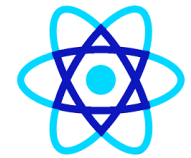
Index.js

- npm start
- Below adding individual links for above commands for node @8.9.4
- @node_modules/react-scripts/bin/react-scripts.js >> line number 35
- @node_modules/react-scripts/scripts/start.js >> line number 46
- @node_modules/react-scripts/config/webpack.config.dev.js >> line number 21, 102
- @node_modules/react-scripts/config/paths.js >> line number 56



Atoms, Molecules and Organisms

- In ReactJS and Atomic Design, components can be organized into three main categories: Atoms, Molecules, and Organisms.
- This pattern helps in building reusable and scalable UI components in a structured and hierarchical way.
- 1. Atoms are the simplest, most fundamental building blocks in our application.
- They represent the smallest and most basic UI components that cannot be broken down any further.
- They often correspond to basic HTML elements like buttons, inputs, labels, etc.



Recaptcha Admin Console

Google reCAPTCHA

← Register a new site



Get started with reCAPTCHA

Add advanced features like MFA, spam/fraud protection & Google Cloud integration.

- ✓ Up to 10,000 assessments/month at no cost
- ✓ No Credit Card required

<https://www.google.com/recaptcha/admin/create>

Label ⓘ

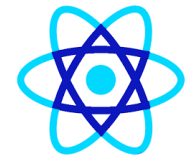
e.g. example.com

0 / 50

reCAPTCHA type ⓘ

- ☐ Score based (v3) Verify requests with a score
- ☒ Challenge (v2) Verify requests with a challenge

Recaptcha Admin Console



reCAPTCHA type

- ☐ Score based (v3) Verify requests with a score
- ☒ Challenge (v2) Verify requests with a challenge
 - ☒ "I'm not a robot" Checkbox Validate requests with the "I'm not a robot" checkbox
 - ☐ Invisible reCAPTCHA badge Validate requests in the background

Domains

+ localhost

Google Cloud Platform

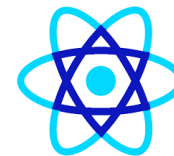
It looks like you've used Google Cloud before. To get started, we'll create a new project and enable the necessary APIs.

Project Name*

 vebdentalcare

13 / 30

Recaptcha Admin Console



Label ⓘ

chitapp

7 / 50

reCAPTCHA type: v2 Checkbox

[VIEW IN CLOUD CONSOLE](#) ↗

reCAPTCHA keys ^

Use this site key in the HTML code your site serves to users. [See client side integration](#)

🔑 COPY SITE KEY

6LeE5U4qAAAAABdXr-9Lb53OA-nTJdrEnvRprFAf

Use this secret key for communication between your site and reCAPTCHA. [See server side integration](#)

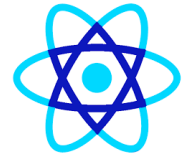
🔑 COPY SECRET KEY

6LeE5U4qAAAAANUJjZYVI5NB8vgg1efdF54WYRs8

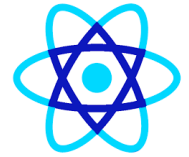
Domains ⓘ

✕ localhost

Recaptcha HTML Content Security Policy



```
<head>
<meta charset="utf-8" />
<link rel="icon" href="%PUBLIC_URL%/favicon.ico" />
<meta name="viewport" content="width=device-width, initial-scale=1" />
<meta http-equiv="Content-Security-Policy" content="script-src 'self' 'nonce-abc123' https://apis.google.com;">
<meta http-equiv="Content-Security-Policy" content="script-src 'self' 'nonce-abc123' https://apis.google.com https://www.google.c
<meta name="theme-color" content="#000000" />
<meta
  name="description"
  content="Web site created using create-react-app"
/>
<link rel="apple-touch-icon" href="%PUBLIC_URL%/logo192.png" />
<!--
  manifest.json provides metadata used when your web app is installed on a
  user's mobile device or desktop. See https://developers.google.com/web/fundamentals/web-app-manifest/
-->
<script nonce="abc123" src="https://www.google.com/recaptcha/api.js?onload=onloadcallback&render=explicit" async defer></script>
```



- Before Hooks (React <16.8), only **class components** could:
 - ✓ store state
 - ✓ use lifecycle methods
 - ✓ access context
- Now function components can do all thes



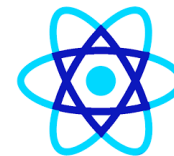
React Hook

- In React, Hooks gives functional components access to use the states and manage side effects.
- They were introduced to React 16.8.
- They let developers use state and other React features without writing a class
- For example, The state of a component It is important to note that hooks are not used inside the classes.



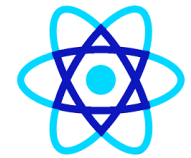
When to use React Hooks

- If you have created a function component in React and later need to add state to it, you used to have to convert it to a class component.
- However, you can now use a Hook within the existing function component to add a state.
- This way, you can avoid having to rewrite your code as a class component.



Types of React Hooks

- The Built-in React Hooks are:
 - State Hooks
 - Context Hooks
 - Ref Hooks
 - Effect Hooks
 - Performance Hooks
 - Resource Hooks
 - Other Hooks



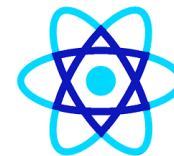
Types of React Hooks

Hook	Purpose
useState	Store component state
useEffect	Perform side effects (API calls, timers, subscriptions)
useRef	Access DOM elements or persist mutable values
useMemo	Cache expensive calculations
useCallback	Cache function references
useContext	Consume shared context
useReducer	Manage complex state logic
Custom Hook	Reuse stateful logic across components



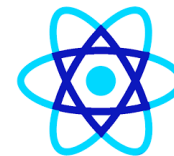
React State Hooks

- State Hooks stores and provide access to the information.
- To add state in Components we use:
 - useState Hook: useState Hooks provides state variable with direct update access.
 - useReducer Hook: useReducer Hook provides a state variable along with the update logic in reducer function.



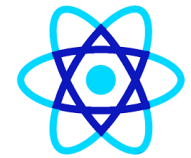
React Context Hooks

- Context hooks make it possible to access the information without being passed as a prop.
- useContext Hooks:
 - shares the data as a global data with passing down props in component tree.
 - `const context = useContext(myContext);`



React Ref Hooks

- Refs creates the variable containing the information not used for rendering e.g. DOM nodes.
- useRef:
 - Declares a reference to the DOM elements mostly a DOM Node.
 - useImperativeHandle: It is an additional hook that declares a customizable reference
 - `const textRef = useRef(null);`



Effect Hooks:

- Effects connect the components and make it sync with the system.
- It includes changes in browser DOM, networks and other libraries.
- Runs side-effects (API calls, timers, subscriptions).
- `useEffect`:
 - `useEffect` Hook connects the components to external system
 - `useLayoutEffect`: used to measure the layout, fires when the screen rerenders.
 - `useInsertionEffect`: used to insert the CSS dynamically, fires before the changes made to DOM.



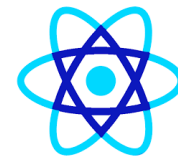
Effect Hooks:

```
useEffect(()->{  
  // Code to be executed  
}, [dependencies] )
```



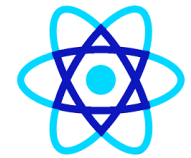
React Performance Hooks

- Performance hooks are a way to skip the unnecessary work and optimise the rendering performance.
 - `useMemo`: return a memoized value reducing unnecessary computations.
 - `useCallback`: returns a memoized callback that changes if the dependencies are changed.
 - `const memoizedValue = useMemo(functionThatReturnsValue, arrayDependencies)`



React Resource Hooks:

- It allows component to access the resource without being a part of their state e.g., accessing a styling or reading a promise.
 - use: used to read the value of resources e.g., context and promises.
 - `const data = use(dataPromise);`



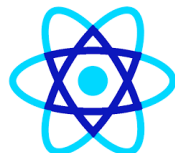
Benefits of using React Hooks

- Hooks can improve code reusability and make it easier to split complex components into smaller functions.
 - Simpler, cleaner code: Functional components with hooks are often more concise and easier to understand than class components.
 - Better for complex UIs: Hooks make it easier to manage state and side effects in components with intricate logic.
 - Improved maintainability: Code using hooks is often easier to test and debug.



Why the need for ReactJs Hooks ?

- There are multiple reasons responsible for the introduction of the Hooks which may vary depending upon the experience of developers in developing React application.
- Needs for react hooks are:
 - Use of 'this' keyword
 - Reusable stateful logics
 - Simplifying complex scenarios



Why the need for ReactJs Hooks ?

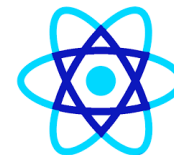
```
// Filename - index.js

import React, { useState } from "react";
import ReactDOM from "react-dom/client";

function App() {
  const [click, setClick] = useState(0);

  // Using array destructuring here
  // to assign initial value 0
  // to click and a reference to the function
  // that updates click to setClick
  return (
    <div>
      <p>You clicked {click} times</p>
      <button onClick={() => setClick(click + 1)}>
        Click me
      </button>
    </div>
  );
}

const root = ReactDOM.createRoot(
  document.getElementById("root")
);
root.render(
  <React.StrictMode>
    <App />
  </React.StrictMode>
);
```



useCallback and Memo

useCallback



What it does:

Memorizes (remembers) a function so React can reuse the same function reference between renders.

```
const handleDelete = useCallback(() => {  
  dispatch(deleteProduct(id));  
}, [dispatch, id]);
```

React.memo



What it does:

Prevents a component from re-rendering when its props have not changed.

```
const ProductRow = memo(function ProductRow(props) {  
  return <tr>...</tr>;  
});
```



Why use them?

1 Without useCallback:

- A new function is created on every render.
- Child components may re-render even if props didn't change.

2 With useCallback + React.memo:

- The same function reference is reused.
- Memoized child components skip unnecessary re-renders.

Example

Parent Component

```
const handleDelete = useCallback((id) => {  
  dispatch(deleteProduct(id));  
}, [dispatch]);
```

Child Component

```
const ProductRow = memo(({ product, onDelete }) => {  
  return (  
    <button onClick={() => onDelete(product._id)}>  
      Delete  
    </button>  
  );  
});
```



When to use?

- ✓ Pass callback functions to child components.
- ✓ Child component uses React.memo.
- ✓ Callback is a dependency in useEffect or another hook.
- ✓ List or table with many items (e.g., products).



Not usually needed:

Small components with a few elements or functions used only inside the same component.

How useCallback + React.memo work together

1 Parent renders



Parent component re-renders.

2 useCallback



React reuses the same function reference (unless dependencies change).

3 ProductRow



Child component receives the same props (including the same function reference).

4 React.memo



React skips re-rendering the child component if props haven't changed.



Result:

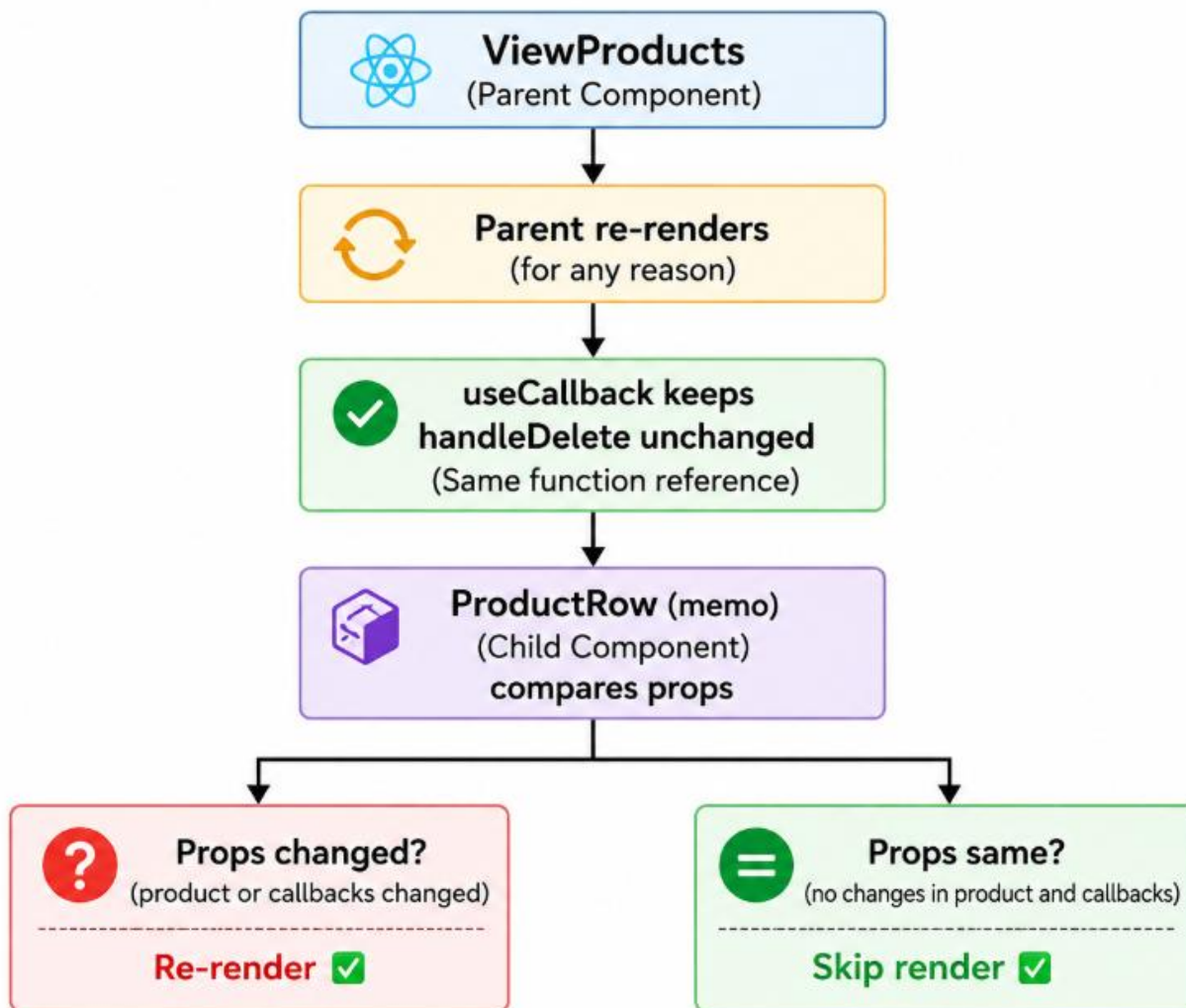
Fewer unnecessary renders and better performance — especially for large lists or tables.



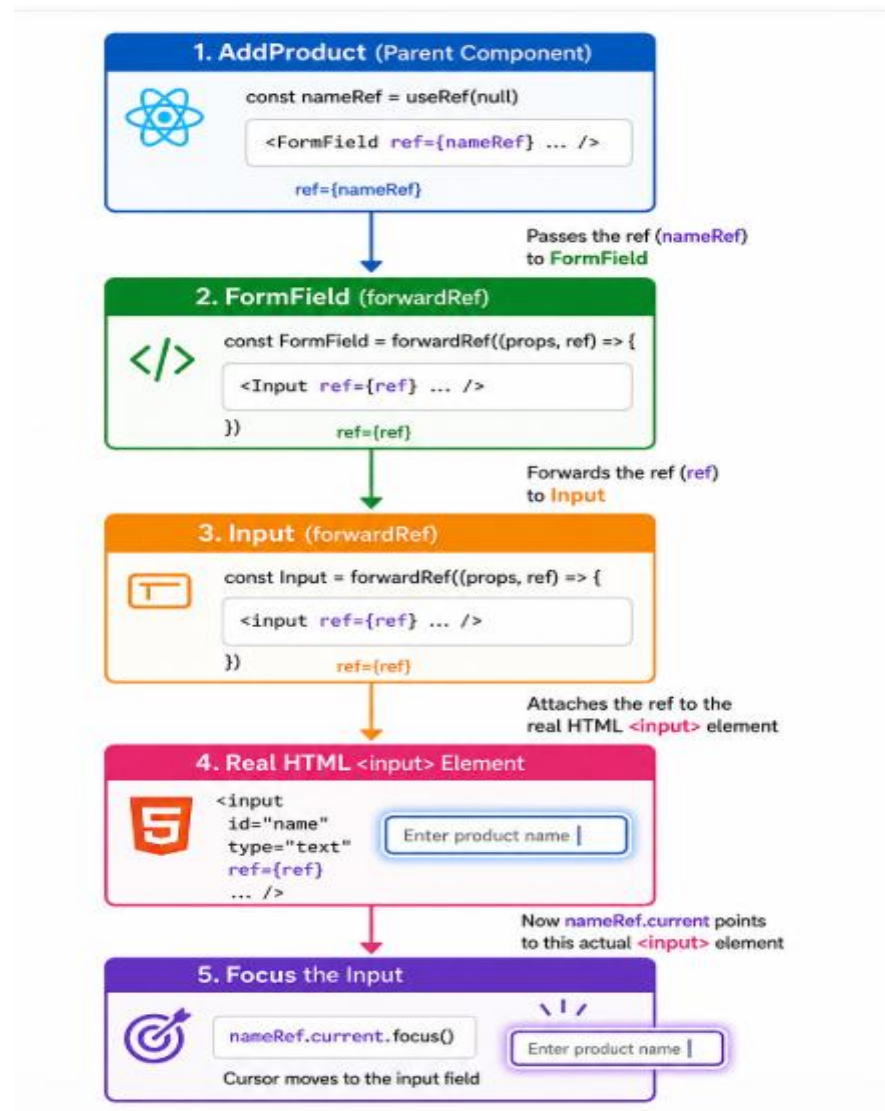
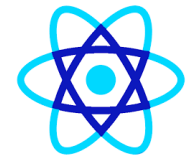
Rule of thumb: Use **useCallback** to keep function references stable, and **React.memo** to skip unnecessary component renders.



useCallback and Memo



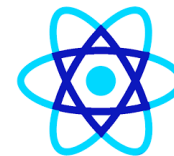
UseRef





Custom Hooks Using Reuse

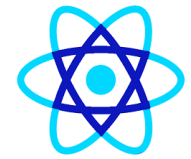
```
function useSimpleCategory( title, items) {  
  const [search, setSearch] = useState("");  
  
  const filteredItems = useMemo(() => {  
    const q = search.trim().toLowerCase();  
    if (!q) return items;  
  
    return items.filter((item) => {  
      const name = String(item?.name ?? "").toLowerCase();  
      const category = String(item?.category ?? "").toLowerCase();  
      const price = String(item?.price ?? "").toLowerCase(); // optional  
  
      return (  
        name.includes(q) ||  
        category.includes(q) ||  
        price.includes(q)  
      );  
    });  
  }, [items, search]);  
  
  return { title, search, setSearch, items: filteredItems };  
}
```



Custom Hooks Using Reuse

- ✓ Custom hooks **must**:
 - start with use
 - be called **unconditionally**
 - not be treated as a component
- useMemo is a React hook that remembers (memoizes) the result of a calculation and recomputes it only when its dependencies change.

Differences between React Hook Vs Class Component



Feature	Class Components	React Hooks
State Management	<code>this.state</code> and lifecycle methods	<code>useState</code> and <code>useEffect</code>
Code Structure	Spread across methods, can be complex	Smaller, focused functions
Reusability	Difficult to reuse logic	Easy to create and reuse custom hooks
Learning Curve	Familiar to OOP developers	Requires different mindset than classes
Error Boundaries	Supported	Not currently supported
Third-party Libraries	Some libraries rely on them	May not all be compatible yet



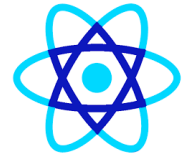
React Router

Npm install react-router-dom

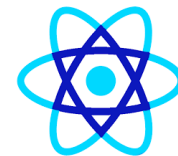
Index.js

```
const app=(  
  <BrowserRouter>  
    <App/>  
  </BrowserRouter>  
)  
  
ReactDOM.render(app,  
document.getElementById('root'));
```

Different types of routers in react router



- Static Routing: For simple page navigation.
- Nested Routing: For hierarchically structured routes.
- Dynamic Routing: Handling variable URLs.
- Protected Routes: Routes restricted to authenticated users.
- Redirects: Automatically navigating to other routes.
- Lazy Loading: Improving performance by loading components on demand.
- Hash Router: URL-based routing using hashes.
- Memory Router: In-memory routing for testing or environments without a URL.
- Query Params: Handling query strings in URLs.



Static Routing

- Use Case: The most common routing type used in SPAs for navigating between static pages (e.g., home, about, contact).

```
jsx Copy code  
  
import { BrowserRouter as Router, Routes, Route } from 'react-router-dom';  
import Home from './Home';  
import About from './About';  
import Contact from './Contact';  
  
const App = () => (  
  <Router>  
    <Routes>  
      <Route path="/" element={<Home />} />  
      <Route path="/about" element={<About />} />  
      <Route path="/contact" element={<Contact />} />  
    </Routes>  
  </Router>  
>);  
  
export default App;
```



Nested Routing

- Use Case: When you want to have child routes (nested pages) inside a parent route.
- For example, a blog page can have multiple blog posts.



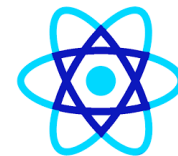
Nested Routing

```
import { BrowserRouter as Router, Routes, Route } from 'react-router-dom';
```

[Copy code](#)

```
const Blog = () => (  
  <div>  
    <h2>Blog</h2>  
    <Outlet /> { /* This will render the nested routes */ }  
  </div>  
);  
  
const BlogPost = ({ postId }) => <div>Blog Post {postId}</div>;  
  
const App = () => (  
  <Router>  
    <Routes>  
      <Route path="/blog" element={<Blog />}>  
        <Route path="post1" element={<BlogPost postId="1" />} />  
        <Route path="post2" element={<BlogPost postId="2" />} />  
      </Route>  
    </Routes>  
  </Router>  
);
```



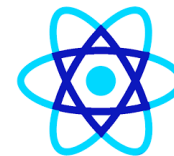


Dynamic Routing

- Use Case: Dynamic routes are useful for handling variable parts of the URL, such as showing different pages or posts based on dynamic IDs or slugs (e.g., viewing a specific product, user profile, or blog post).

```
jsx Copy code  
  
import { BrowserRouter as Router, Routes, Route, useParams } from 'react-router-dom';  
  
const Product = () => {  
  const { productId } = useParams();  
  return <div>Product ID: {productId}</div>;  
};  
  
const App = () => (  
  <Router>  
    <Routes>  
      <Route path="/product/:productId" element={<Product />} />  
    </Routes>  
  </Router>  
>;  
  
export default App;
```





Protected Routes (Private Routes)

- Use Case: When you want to restrict certain routes to authenticated users only. For example, a dashboard or profile page should only be accessible after logging in.

```
jsx Copy code  
  
import { BrowserRouter as Router, Routes, Route, Navigate } from 'react-router-dom';  
  
const isAuthenticated = false; // Set this to true if the user is authenticated  
  
const ProtectedRoute = ({ children }) => {  
  return isAuthenticated ? children : <Navigate to="/login" />;  
};  
  
const Dashboard = () => <div>Dashboard: Protected Content</div>;  
const Login = () => <div>Login Page</div>;  
  
const App = () => (  
  <Router>  
    <Routes>  
      <Route path="/dashboard" element={<ProtectedRoute><Dashboard /></ProtectedRoute>} />  
      <Route path="/login" element={<Login />} />  
    </Routes>  
  </Router>  
)  
);
```




Redirects

- Use Case: When you need to automatically redirect the user to another route. This is often used for login redirects, handling 404 pages, or other cases where navigation needs to be automated.

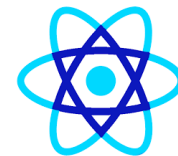
jsx

Copy code

```
import { BrowserRouter as Router, Routes, Route, Navigate } from 'react-router-dom';

const App = () => (
  <Router>
    <Routes>
      <Route path="/old-path" element={<Navigate to="/new-path" />} />
      <Route path="/new-path" element={<div>New Path Content</div>} />
      <Route path="*" element={<div>404 Not Found</div>} /> {/* Catch-all for undefined routes */}
    </Routes>
  </Router>
);

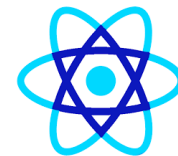
export default App;
```



Lazy Loading (Code-Splitting Routes)

- Use Case: For large applications where you want to load components lazily to improve performance. Pages are loaded only when needed.

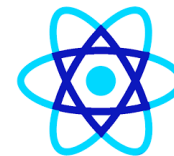
```
jsx Copy code  
  
import React, { Suspense, lazy } from 'react';  
import { BrowserRouter as Router, Routes, Route } from 'react-router-dom';  
  
const Home = lazy(() => import('./Home'));  
const About = lazy(() => import('./About'));  
const Contact = lazy(() => import('./Contact'));  
  
const App = () => (  
  <Router>  
    <Suspense fallback={<div>Loading...</div>}>  
      <Routes>  
        <Route path="/" element={<Home />} />  
        <Route path="/about" element={<About />} />  
        <Route path="/contact" element={<Contact />} />  
      </Routes>  
    </Suspense>  
  </Router>  
>);
```



Lazy Loading (Code-Splitting Routes)

- Use Case: For large applications where you want to load components lazily to improve performance. Pages are loaded only when needed.

```
jsx Copy code  
  
import React, { Suspense, lazy } from 'react';  
import { BrowserRouter as Router, Routes, Route } from 'react-router-dom';  
  
const Home = lazy(() => import('./Home'));  
const About = lazy(() => import('./About'));  
const Contact = lazy(() => import('./Contact'));  
  
const App = () => (  
  <Router>  
    <Suspense fallback={<div>Loading...</div>}>  
      <Routes>  
        <Route path="/" element={<Home />} />  
        <Route path="/about" element={<About />} />  
        <Route path="/contact" element={<Contact />} />  
      </Routes>  
    </Suspense>  
  </Router>  
>);
```



Memory Router

- A router which doesn't change the URL in your browser instead it keeps the URL changes in memory
- It is very useful for testing and non browser environments 
- But in browser, It doesn't have history. So you can't go back or forward using browser history 



Memory Router

- Use Case: Primarily used for testing or non-browser environments. It keeps the history of your "navigation" in memory (does not interact with the browser URL).

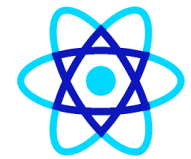
jsx

Copy code

```
import { MemoryRouter, Routes, Route } from 'react-router-dom';

const App = () => (
  <MemoryRouter>
    <Routes>
      <Route path="/" element={<div>Home</div>} />
      <Route path="/about" element={<div>About</div>} />
    </Routes>
  </MemoryRouter>
);

export default App;
```



Query Params Handling

- Use Case: Useful when your route needs to handle query parameters (e.g., /search?query=react).

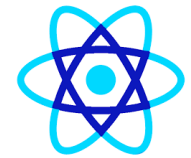
```
jsx
import { BrowserRouter as Router, Routes, Route, useSearchParams } from 'react-router-dom'

const Search = () => {
  const [searchParams] = useSearchParams();
  const query = searchParams.get('query');

  return <div>Search query: {query}</div>;
};

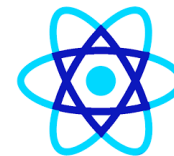
const App = () => (
  <Router>
    <Routes>
      <Route path="/search" element={<Search />} />
    </Routes>
  </Router>
);

export default App;
```



Hash Router

- A router which uses client side hash routing.
- Whenever, there is a new route get rendered, it updated the browser URL with hash routes. (eg., `/#/about`)
- Hash portion of the URL won't be handled by server, server will always send the `index.html` for every request and ignore hash value. Hash value will be handled by react router.
- It is used to support legacy browsers which usually doesn't support HTML pushState API 
- It doesn't need any configuration in server to handle routes 
- This route isn't recommended by the team who created react router package. Use it only if you need to support legacy browsers or don't have server logic to handle the client side routes 



Hash Router

- Use Case: Useful for applications that are hosted on static file servers where the server doesn't handle requests for all URLs (e.g., GitHub Pages).

jsx

Copy code

```
import { HashRouter as Router, Routes, Route } from 'react-router-dom';
import Home from './Home';
import About from './About';

const App = () => (
  <Router>
    <Routes>
      <Route path="/" element={<Home />} />
      <Route path="/about" element={<About />} />
    </Routes>
  </Router>
);

export default App;
```




Browser Router

- The widely popular router and a router for modern browsers which use HTML5 pushState API. (i.e., pushState, replaceState and popState API).
- It routes as normal URL in browser, you can't differentiate whether it is server rendered page or client rendered page through the URL.
- It assumes, your server handles all the request URL (eg., /, /about) and points to root index.html. From there, BrowserRouter take care of routing the relevant page.
- It accepts forceRefresh props to support legacy browsers which doesn't support HTML5 pushState API 

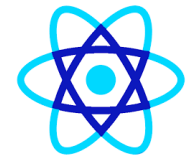


React Router

Npm install react-router-dom

Index.js

```
const app=(  
  <BrowserRouter>  
    <App/>  
  </BrowserRouter>  
)  
  
ReactDOM.render(app,  
document.getElementById('root'));
```



Uncontrolled Element

```
import React, {useRef, useState} from 'react';
```

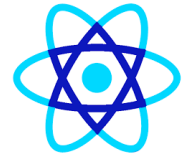
1+ usages

```
function NewVacancy(){  
  const [age : string , setAge] = useState( initialState: '' );  
  const nameInputRef : MutableRefObject<null> = useRef( initialValue: null );
```

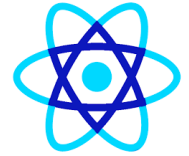
1+ usages

```
  const handleSubmit = (event) : void => {  
    event.preventDefault();  
    //uncontrolled element  
    const name = nameInputRef.current.value;  
    alert(`Name: ${name}, Age: ${age}`);  
  };
```

```
  return (  
    <form onSubmit={handleSubmit}>  
      <div>  
        <label>Name: </label>  
        <input type="text" ref={nameInputRef} />  
      </div>  
      <div>  
        <label>Age: </label>  
        <input  
          type="number"  
          value={age}  
          onChange={(e : ChangeEvent<HTMLInputElement> ) : void => setAge(e.target.value)} // Controlled input  
        />  
      </div>  
      <button type="submit">Submit</button>
```



- Apollo: Easy to set up and use GraphQL client.
- Axios: Promise based HTTP client for the browser and node.js.
- Relay Modern - A JavaScript framework for building data-driven React applications.
- Request: Simplified HTTP request client.
- Superagent: A lightweight “isomorphic” library for AJAX requests.

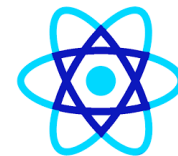


-
- `npm install axios --save`



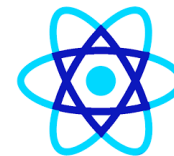
HOC (High Order Components)

- Higher-Order Components (HOCs) in React are a pattern used for reusing component logic.
- An HOC is a function that takes a component as an argument and returns a new component with enhanced behavior.
- This allows developers to share common functionality between components without duplicating code.



HOC (High Order Components)

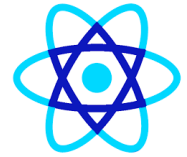
- What is a Higher-Order Component?
 - A Higher-Order Component is essentially a function that Receives a component as an input.
 - Returns a new component that is a wrapped version of the original component with added or modified behavior.
- HOCs are often used for tasks such as:
 - Code reuse and logic abstraction.
 - Managing side effects like data fetching.
 - Handling permissions or authentication.
 - Modifying props



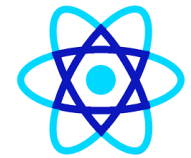
Why use HOCs?

- HOCs help you:
 - Share logic between components
 - Keep components **clean & focused**
 - Avoid code duplication
 - Implement **cross-cutting concerns**
- Common use cases:
 - Authentication / Authorization
 - Logging & analytics
 - Data fetching
 - Permission checks
 - Error handling
 - Theming

vs HOC vs Hooks

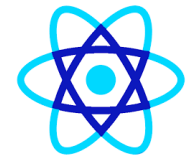


Feature	HOC	Hooks
Logic reuse	✓	✓
Wrapper components	✗ Adds extra layers	✓ No wrapper
Readability	⚠ Can be complex	✓ Cleaner
Modern React	⚠ Legacy pattern	✓ Preferred



Context API vs Redux

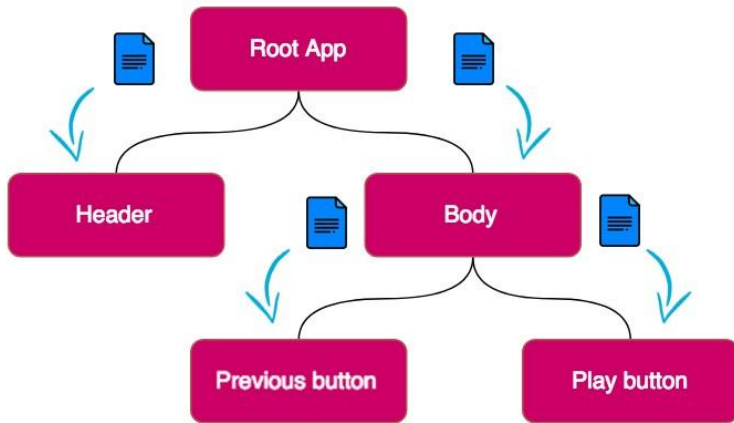
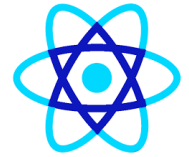
Context API	Redux
Built-in tool	Install the libraries
Requires minimal setup	Extensive setup
It is meant for static data which is not often refreshed or updated	Works for both static and Dynamic
Adding new contexts requires creation from scratch	Easily extendible due to ease of adding new data/actions after the initial setup



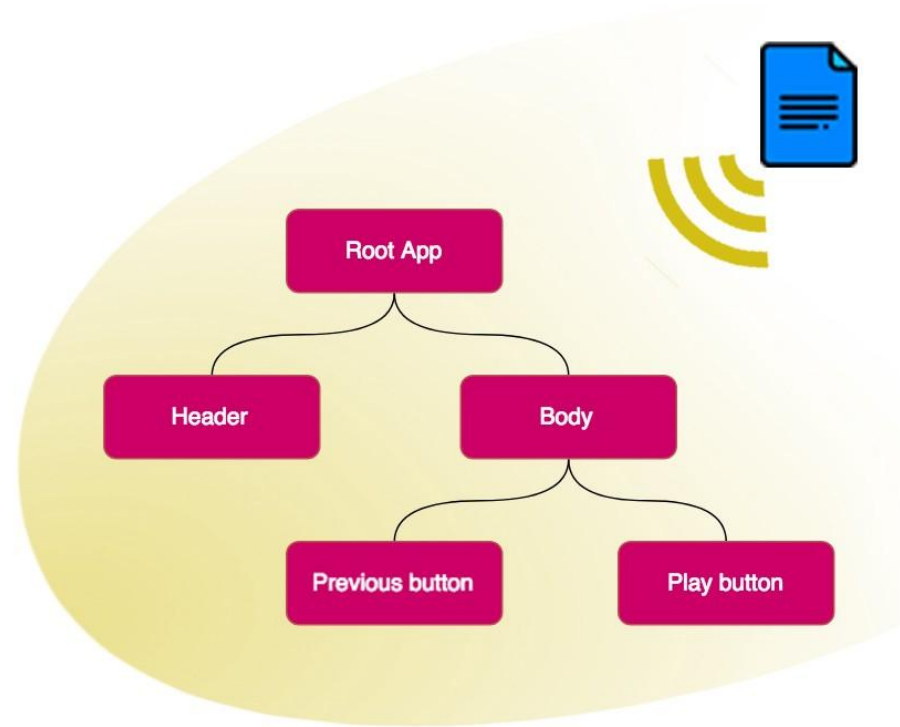
Context API vs Redux

Context API	Redux
Debugging is hard because it is recommended for nested react components	Dev tool for redux
UI logic and state are in the same component	UI logic and state separated from component

Context API

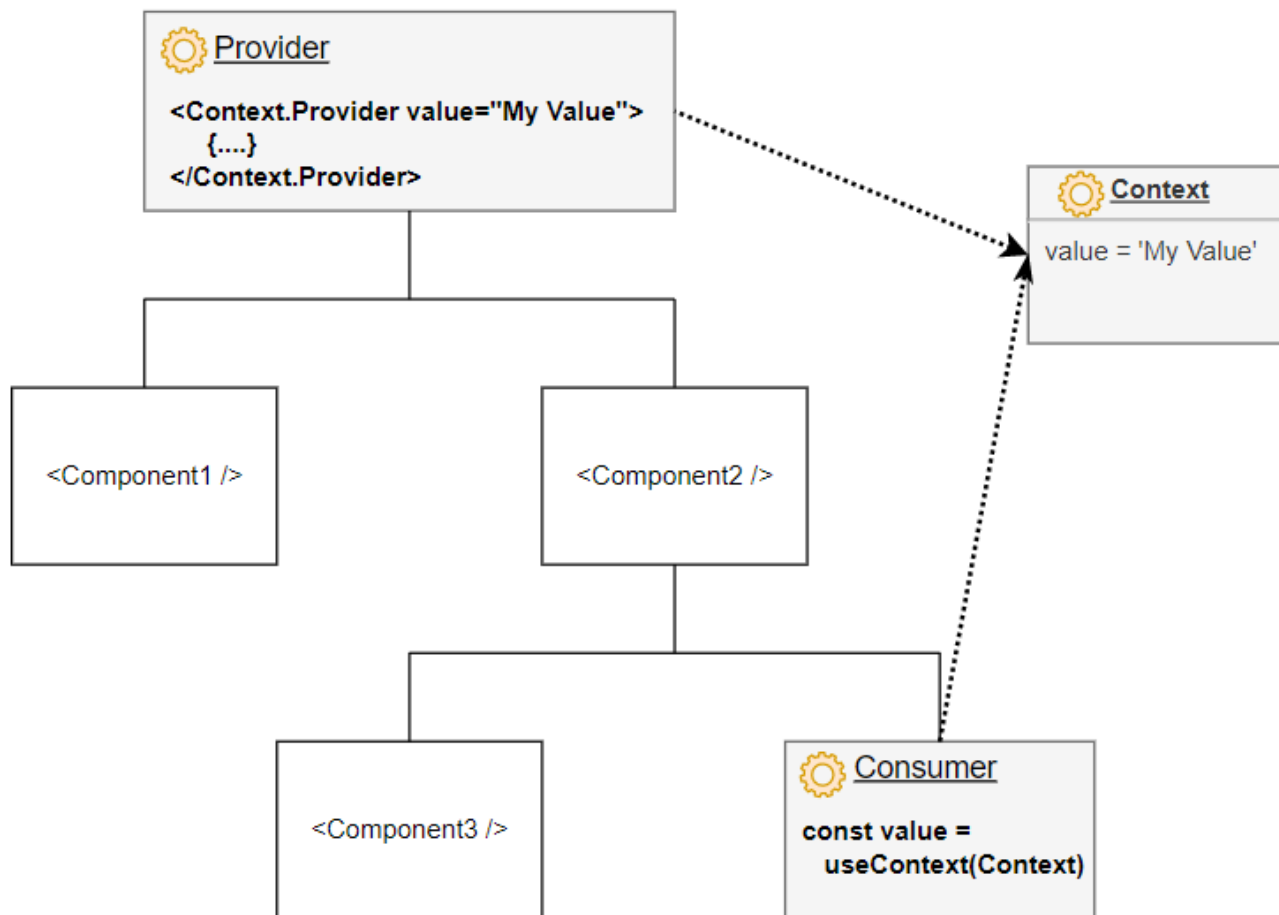
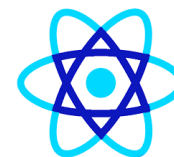


Passing prop to each level



The data in Context available to every component

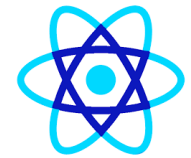
Context API





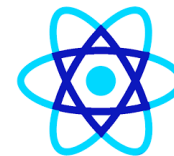
Context API Use cases

- **Authentication**
- AuthProvider
 - ├ Login
 - ├ Header
 - ├ Profile
 - └ ProtectedRoute
- Store:
 - Logged-in user
 - Login status
 - User role
 - Logout function



Context API Usecases

- **Theme**
- ThemeContext
 - ├─ Header
 - ├─ Sidebar
 - └─ Settings
- Store:
 - Light or dark mode
 - Theme-switching function
- **Language**
Store:
 - Selected language
 - Translation settings



Context API Usecases

- **Small shopping application**

Store:

- Current user
- Currency
- Basic cart count

- **Application configuration**

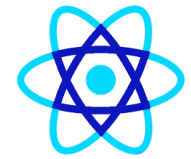
Store:

- API base URL
- Feature flags
- User preferences



What is Redux

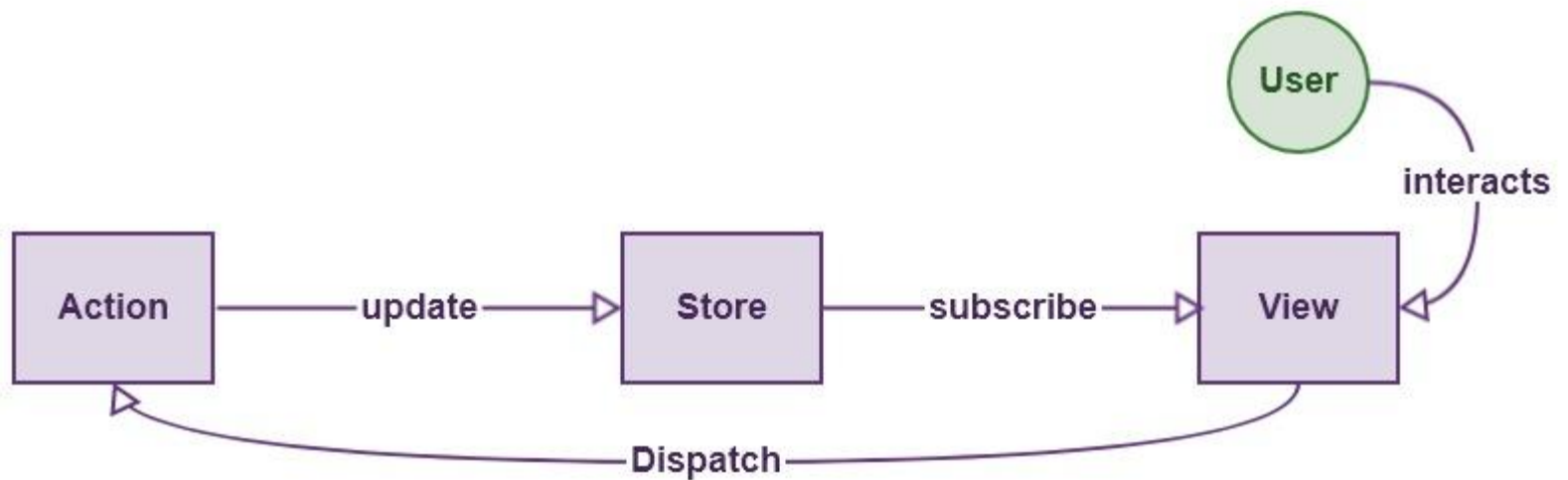
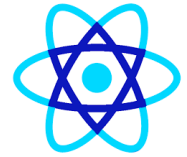
- Redux is a **state management library** for JavaScript apps (very common with React).
- It solves: **many components need the same data + predictable updates.**

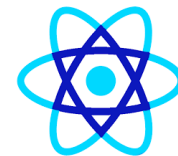


Why Redux is needed

- In React, state is usually:
 - local (useState)
 - shared via props (prop drilling)
 - shared via Context
- Redux is useful when:
 - many screens need same state (user, cart, products)
 - you need predictable updates, debugging, time-travel, logs
 - state changes are complex (cart, checkout, auth)
- Example in your app:
 - cartItems needed in Header + Cart page + Checkout page
 - selectedCategory needed in sidebar + product list
 - userLogin needed everywhere

Core concepts of Redux





Core concepts of Redux

- **1 Store**
- A **single object** that holds the entire application state
- Acts as the **single source of truth**
- {
 - auth: {...},
 - cart: {...},
 - user: {...}
- }



Core concepts of Redux

- **2 Action**
- A **plain JavaScript object**
- Describes *what happened*
- { type: "ADD_ITEM", payload: item }



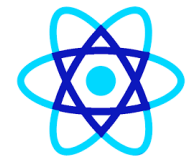
Core concepts of Redux

- **3 Reducer**
- A **pure function**
- Decides *how the state changes*
- $(state, action) \Rightarrow newState$



Core concepts of Redux

- **4 Dispatch**
- The method used to send actions to the store
- `dispatch({ type: "LOGOUT" })`



Why Redux

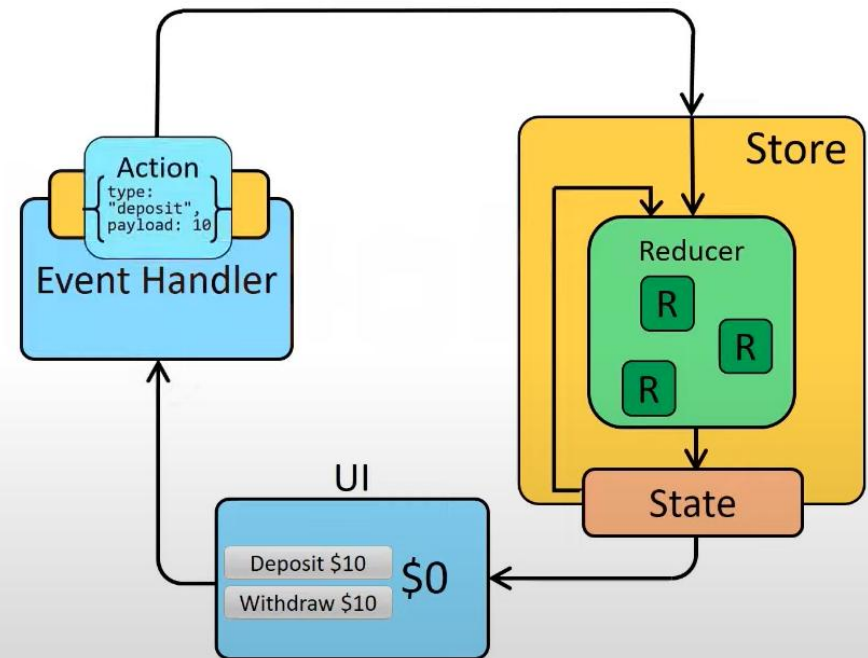
Centralized State Management

Predictable State Changes

Debugging

Developer Experience

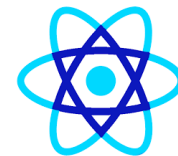
Scalability





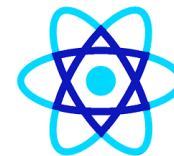
Why Redux

- **1 Centralized State Management**
- All application state lives in **one place (Store)**
- No scattered state across components
- 👉 Easy to access, update, and reason about



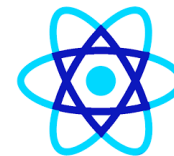
Why Redux

- **2 Predictable State Changes**
- State changes happen **only through actions & reducers**
- No hidden or accidental updates
- 👉 Same input (action) → same output (state)



Why Redux

- **3 Debugging**
- Every action is logged
- You can inspect:
 - Previous state
 - Action
 - Next state
- 👉 Enables **Redux DevTools & time-travel debugging**



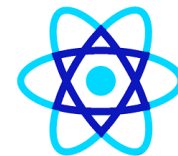
Why Redux

- **4 Developer Experience**
 - Clear structure: **Action** → **Reducer** → **Store**
 - Easier teamwork in large projects
 - Cleaner, readable code (especially with Redux Toolkit)
- **5 Scalability**
 - Works well for **large and complex applications**
 - Easy to add new features without breaking existing ones



Why Redux

-  **UI (Bottom – “Deposit / Withdraw”)**
- This is your **React UI component**.
- Example:
 - Deposit \$10
 - Withdraw \$10
 - Balance: \$0
- **UI does NOT change state directly.**



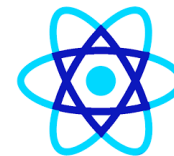
Why Redux

-  **Event Handler**
- Triggered by user actions (button click).
- Example:
- `dispatch({`
- `type: "deposit",`
- `payload: 10`
- `});`
- ✓ Converts UI events into Redux **actions**



Why Redux

-  **Action**
- A **plain JavaScript object** describing what happened:
 - {
 - type: "deposit",
 - payload: 10
 - }
- ✓ No logic
- ✓ Just data



Why Redux

-  **Store (Yellow Box)**
- **The Redux Store:**
 - Holds the entire application state
 - Single source of truth
 - Inside the store:
- **Reducers**
- **State**



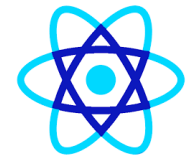
Why Redux

-  **Reducer (Green “R” blocks)**
- Reducers:
- Receive (state, action)
- Return **new state**
- Example:
 - `function balanceReducer(state = 0, action) {`
 - `if (action.type === "deposit") {`
 - `return state + action.payload;`
 - `}`
 - `return state;`
 - `}`
- ✓ Pure function
- ✓ No mutation

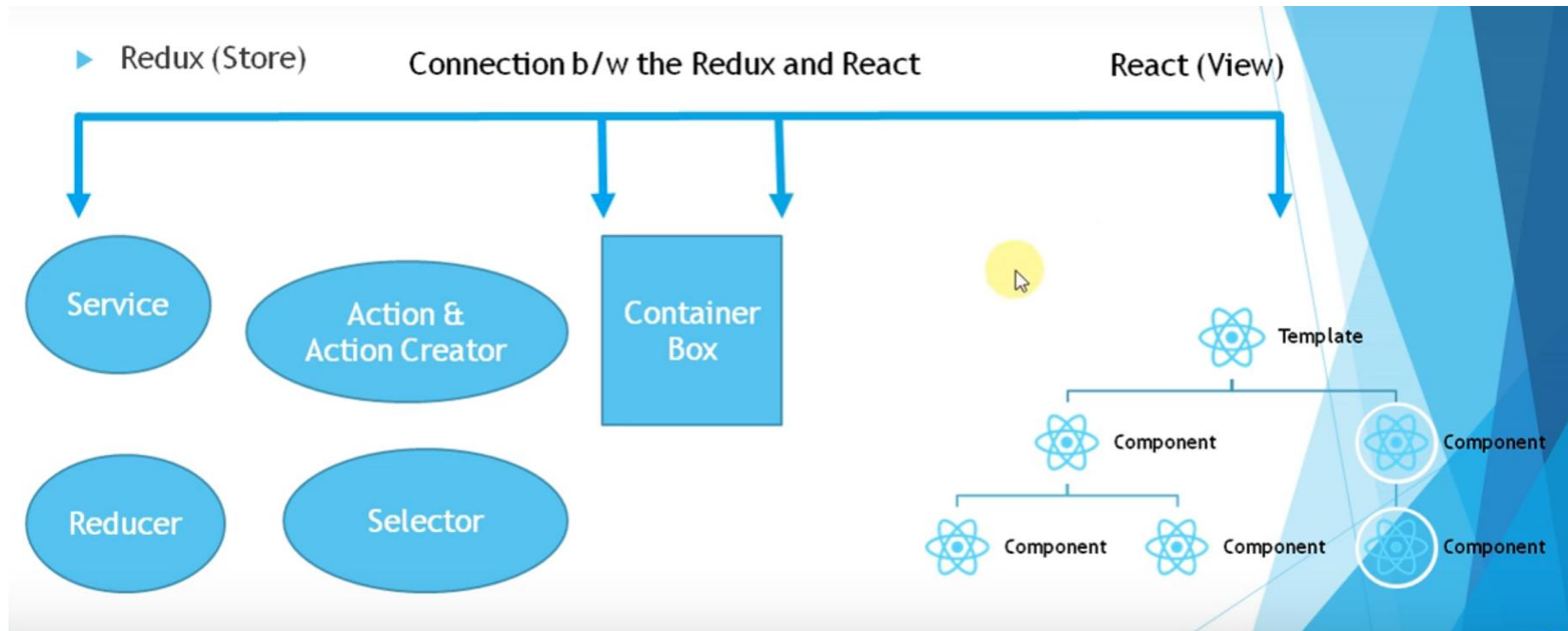


Why Redux

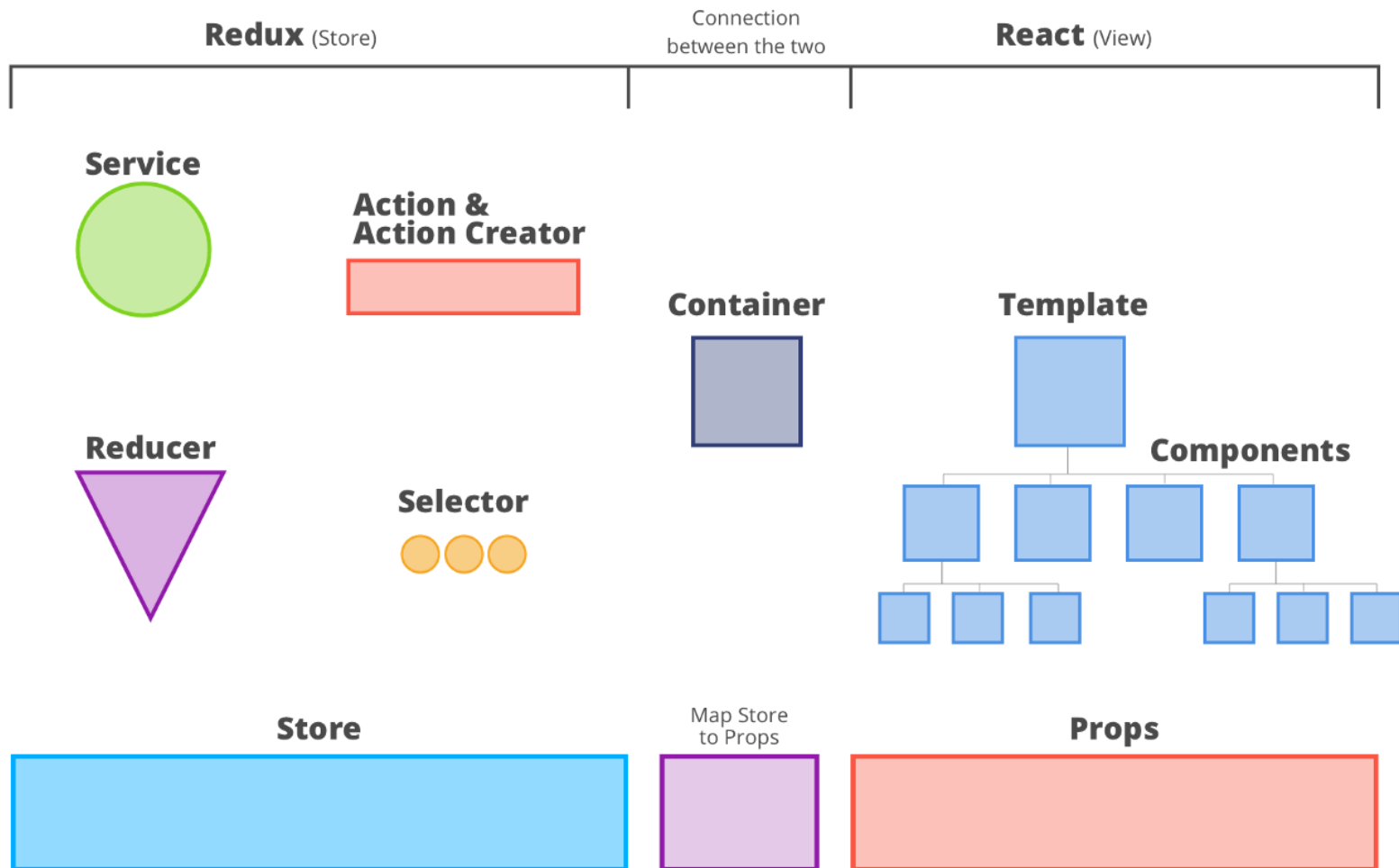
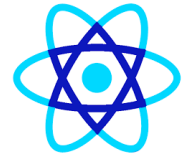
-  **State**
- Updated result from reducers
- Stored in Redux Store
- Example:
- `state = { balance: 10 }`

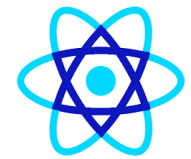


Redux With React Architecture

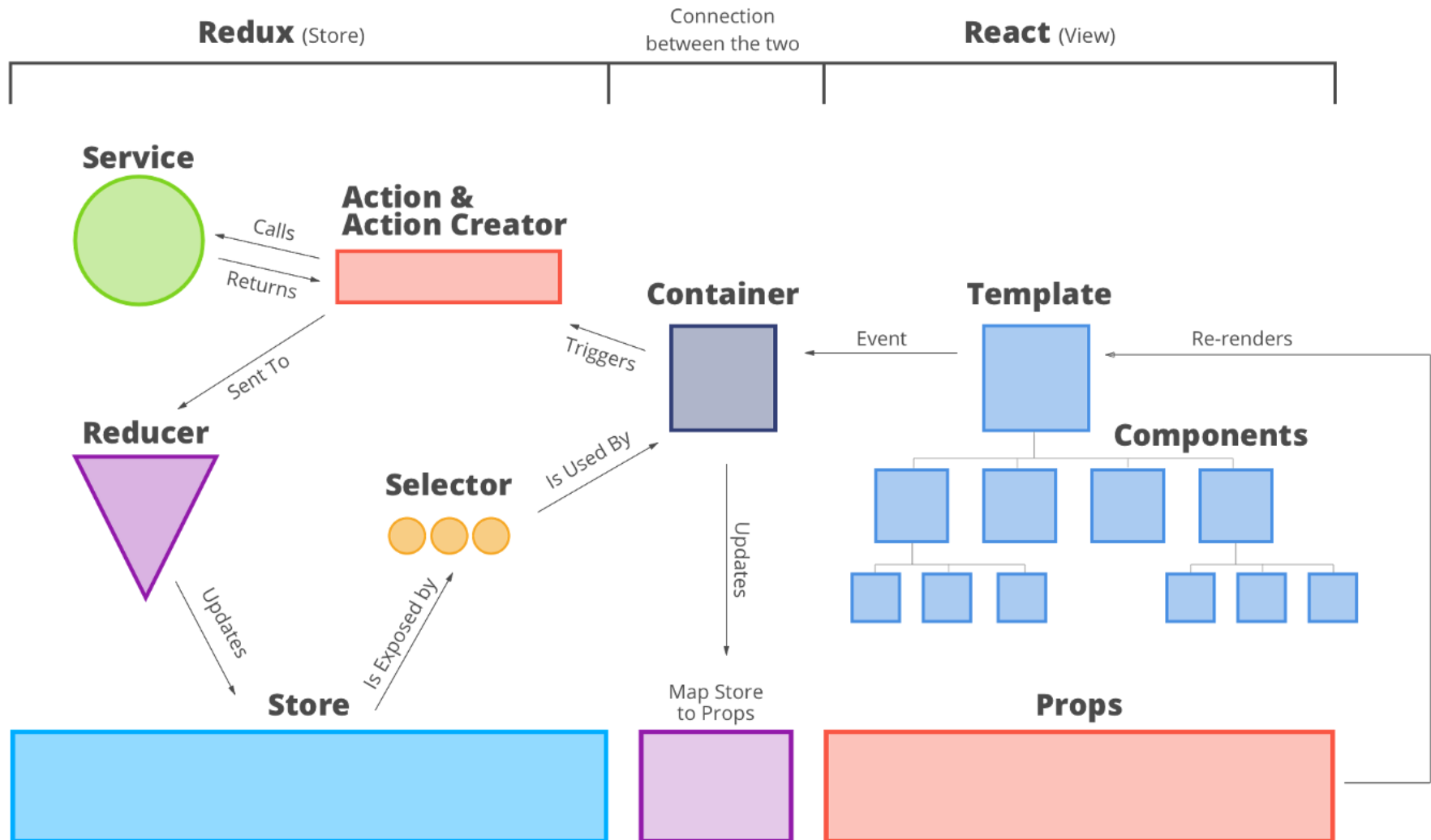


Why Redux

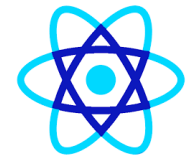




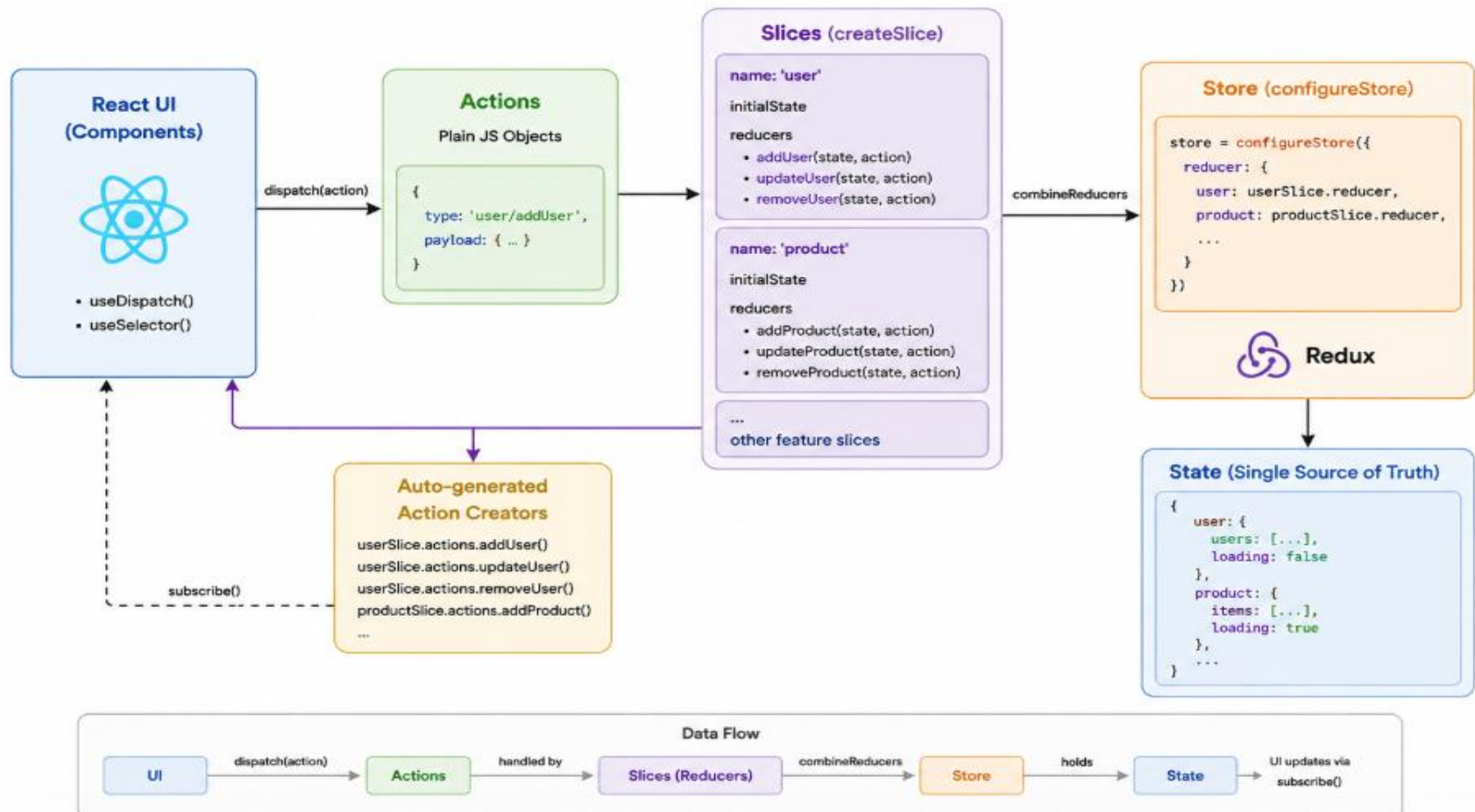
Why Redux



Redux Architecture



Redux with Redux Toolkit (Redux Slices) – Architecture





Redux Usecases

- **E-commerce application**

- Redux Store

- ├── authSlice

- ├── productSlice

- ├── orderSlice

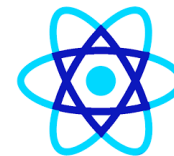
- ├── cartSlice

- ├── customerSlice

- └── inventorySlice

- Redux can manage:

- Product listing
 - Add-to-cart operations
 - Order creation
 - Inventory updates
 - Customer information
 - API loading and errors



Redux Usecases

- **Banking application**
- Redux Store
 - ├─ accountSlice
 - ├─ transactionSlice
 - ├─ beneficiarySlice
 - └─ notificationSlice
- Use Redux for:
 - Account balance
 - Transaction history
 - Fund transfer status
 - Beneficiary management
 - Server-side errors



Redux Usecases

- **Food-delivery application**

Store:

- Restaurants
- Menu items
- Cart
- Current order
- Delivery status
- Driver location

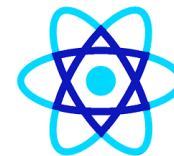


Redux Usecases

- **Project-management application**

Store:

- Projects
- Tasks
- Team members
- Filters
- Notifications
- API status

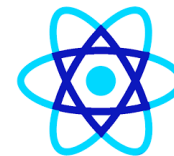


Redux Usecases

- **Real-time dashboard**

Store:

- Sales data
- Inventory count
- Alerts
- Filters
- WebSocket messages
- Refresh status



Redux Usecases

- **Real-time dashboard**

Store:

- Sales data
- Inventory count
- Alerts
- Filters
- WebSocket messages
- Refresh status



Redux Usecases

- **Large admin application**

- Redux Store

- └─ dashboardSlice

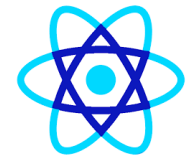
- └─ ordersSlice

- └─ productsSlice

- └─ customersSlice

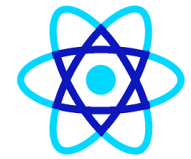
- └─ discountsSlice

- └─ settingsSlice



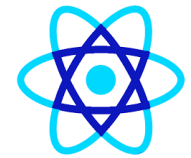
Practical Difference

Requirement	Context API	Redux Toolkit
Logged-in user	Good	Good
Light/dark theme	Best choice	Usually unnecessary
Language selection	Good	Good
Product CRUD	Possible, but difficult as it grows	Best choice
Order CRUD with API	Not ideal	Best choice
Loading and error states	Manual handling	Structured handling
Complex state updates	Difficult	Easier
Debugging history	Limited	Redux DevTools
Frequent state changes	May cause unnecessary renders	Better control
Large application	Hard to maintain	Easier to organize



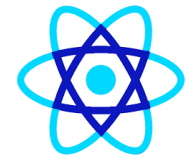
End to End Flow

- **5 Full Data Flow (End-to-End)**
-  **Step-by-step flow**
- User clicks a button in **Component**
- Event goes to **Container**
- Container **dispatches Action**
- Action calls **Service** (API)
- Service returns data
- Action is sent to **Reducer**



End to End Flow

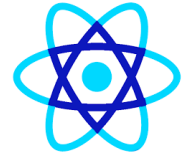
- **5 Full Data Flow (End-to-End)**
- Reducer creates **new state**
- Store is updated
- Selector extracts required data
- Container maps state → props
- React components **re-render**



Why This Architecture Is Powerful

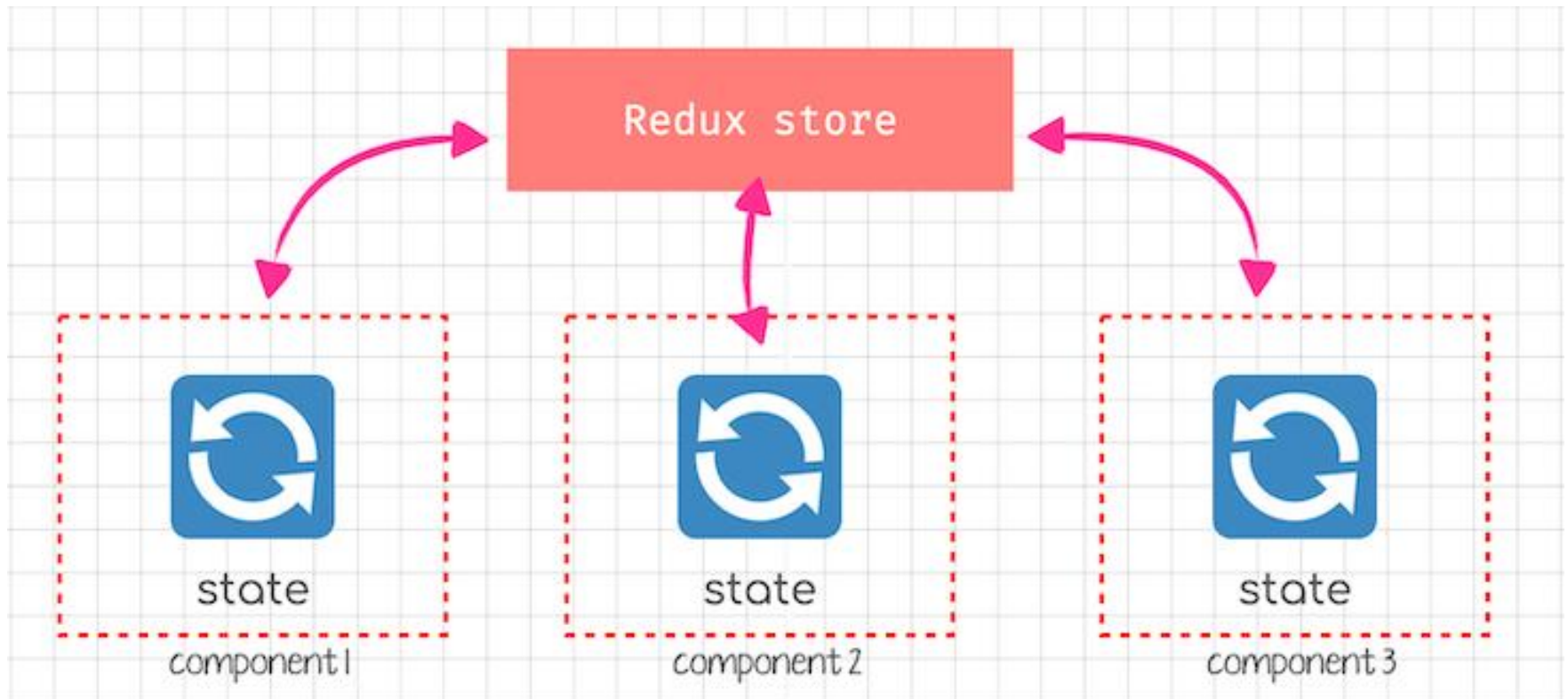
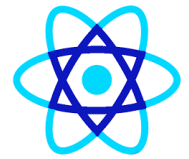
- ☒ Clear separation of concerns
- ☒ Predictable state updates
- ☒ Easy debugging (Redux DevTools)
- ☒ Scales well for large applications
- ☒ Ideal for enterprise apps (like banking, insurance, dashboards)

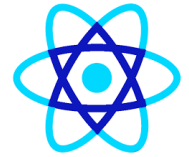
THREE PRINCIPLES OF REDUX



-
- Single source of truth.
 - The state is read-only.
 - Changes are made with pure functions.

Why Redux



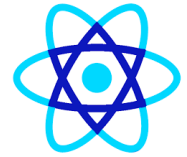


- A store is an object that brings them together. A store has the following responsibilities:
- Holds application state;
- Allows access to state via `getState()`;
- Allows state to be updated via `dispatch(action)`;
- Registers listeners via `subscribe(listener)`;
- It handles unregistering of listeners via the function returned by `subscribe(listener)`



Redux Toolkit (Recommended)

- Today we use **Redux Toolkit (RTK)** because it:
 - reduces boilerplate
 - supports immutable updates easily (Immer)
 - includes dev tools friendly patterns
 - provides async helpers (createAsyncThunk)
 - supports RTK Query (data fetching)
 - Install:
 - `npm i @reduxjs/toolkit react-redux`

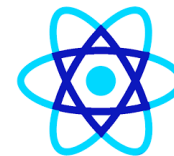


- Dumb Components
- Smart Component(Containers)
- Dumb components render the data, and they did not care about any logic. They have nothing to do with a redux store.
- Smart components are concern about logic and directly connected to the store.



Redux with mongodb

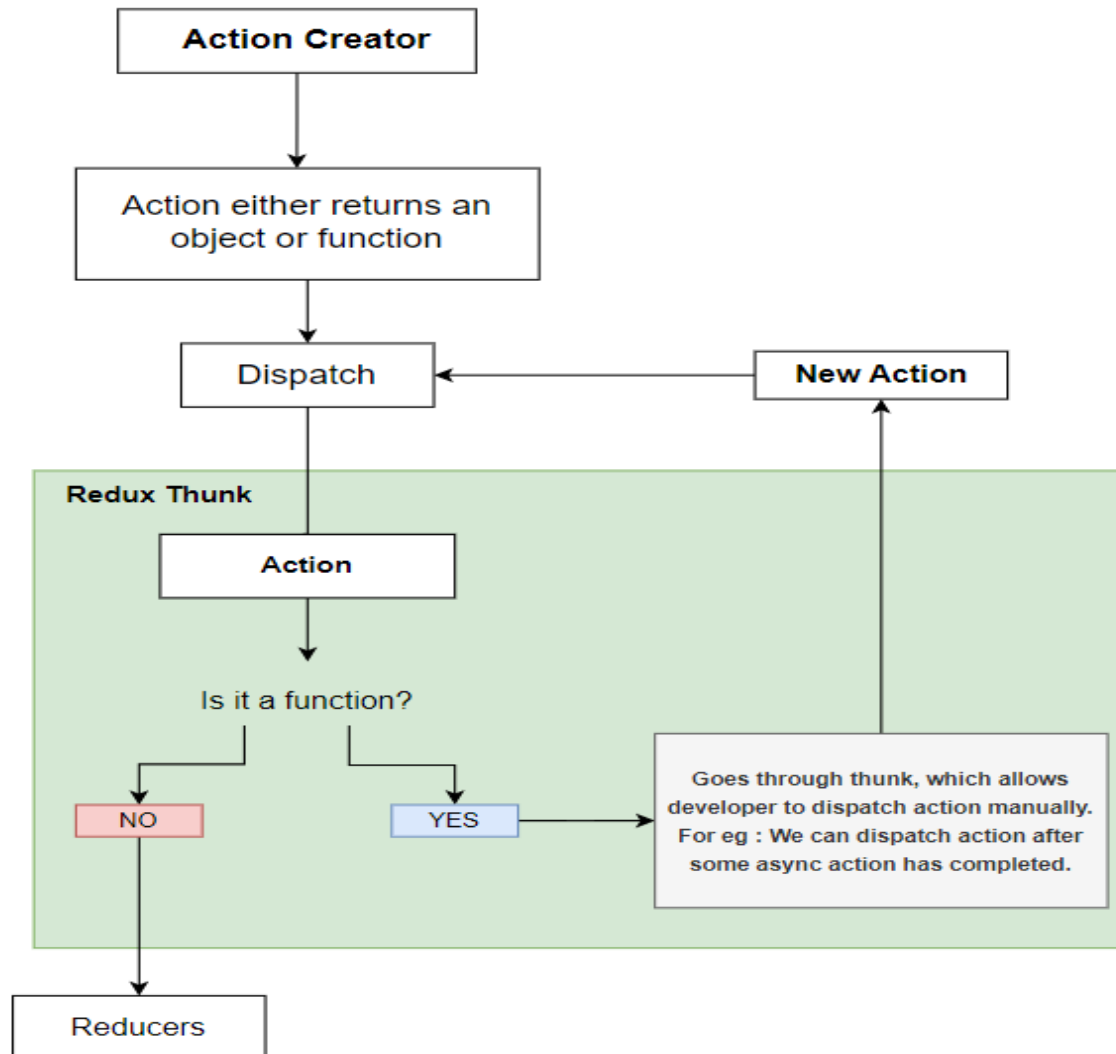
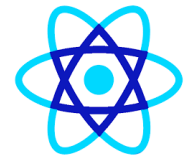
- Go to the project folder and install dependencies using this command: **npm install**
- **yarn add** redux react-redux
- **yarn add** @reduxjs/toolkit
- Start the mongodb server using this command: **mongod**
- Start the Node.js server using this command: **nodemon server/server**
- Start the Reacyt.js dev server using this command: **yarn start**



Redux Async

- `npm install redux-thunk --save`
- *For middleware*
- `npm install express body-parser mongoose --save`
- `npm install nodemon --save-dev`

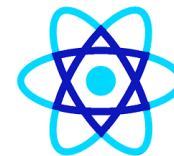
Redux Async





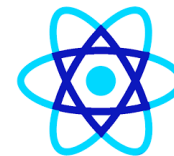
Redux Steps

- Step 1
 - `npm i redux --save-dev`
- Step 2
 - `mkdir -p src/js/store`
 - Create `index.js`
 - `createStore` takes a reducer as the first argument
- Step 3
 - A Redux reducer is just a JavaScript function. It takes two parameters: the current state and action



Redux Steps

- Step 3
 - In a typical React component the local state might be mutated in place.
 - In Redux we're not allowed to do that.
 - The third principle of Redux prescribes that the state is immutable and cannot change in place.
 - The reducer must be pure. A pure function is one that returns the exact same output for the given input.



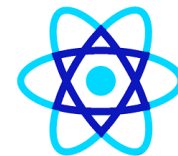
Redux Steps

- Step 4
 - mkdir -p src/js/reducers
 - create a new file, **src/js/reducers/index.js**
- Step 5
 - mkdir -p src/js/actions/index.js
- Step 6
 - Strings are prone to typos and duplicates and for this reason it's better to declare actions as constants.
 - Create a new folder for them
 - mkdir -p src/js/constants



Redux Steps

- `src/js/constants/action-types.js`
- `src/js/actions/index.js`
- Refactoring the reducer
 - how does a reducer know when to generate the next state? The key here is the Redux store.
 - When an action is dispatched, the store forwards a message (the action object) to the reducer.
 - At this point the reducer says "oh, let's look at the type property of this action".



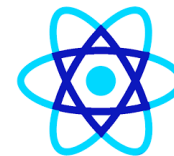
Redux Steps

- Then depending on the action type, the reducer produces the next state, eventually merging the action payload into the new state.
- Open up `src/js/reducers/index.js` and update the reducer with an if statement for checking the action type



Redux store methods

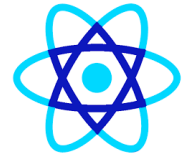
- Redux itself is a small library (2KB) and the most important methods are just three:
 - getState for reading the current state of the application
 - dispatch for dispatching an action
 - subscribe for listening to state changes
- Step 7
 - ***connecting a React component with Redux***
 - ***Create a new file named src/Book/BookList.js.***



what is a Redux middleware?

- The building blocks of Redux:
- **store**, the traffic policeman. **Reducers**, which makes the state in Redux.
- Then there are actions, plain JavaScript objects, acting as messengers in your application.
- Finally, we have action creators, functions for creating those messages.
- A **Redux middleware** is a function that is able to intercept, and act accordingly, our actions, before they reach the reducer.

Json-server



```
Administrator: C:\WINDOWS\system32\cmd.exe - "node" "C:\Users\Balasubramaniam\AppData\Roaming\npm\node_modules\json-server\lib\cli\bin.js" --watch products.json --port 3004
GET /insuranceProducts 404 5.015 ms - 2
GET /insuranceProducts 404 12.859 ms - 2
^C
F:\sapients2022\globalinsurancereactjs\globalinsuranceapp>cd db
F:\sapients2022\globalinsurancereactjs\globalinsuranceapp\db>json-server --watch products.json --port 3004

\{^_^}/ hi!

Loading products.json
Done

Resources
http://localhost:3004/insuranceProducts

Home
http://localhost:3004

Type s + enter at any time to create a snapshot of the database
Watching...

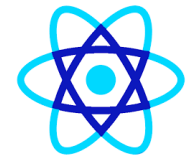
GET /insuranceProducts 200 13.649 ms - -
GET /insuranceProducts 304 30.340 ms - -
GET /insuranceproducts 200 31.847 ms - -
GET /insuranceproducts 304 30.107 ms - -
GET /insuranceproducts 304 28.985 ms - -
GET /insuranceproducts 304 31.366 ms - -
GET /insuranceproducts 304 31.850 ms - -
GET /insuranceproducts 304 14.128 ms - -
```



Asynchronous actions in Redux with Redux Thunk

- `npm i redux-thunk --save-dev`

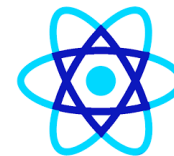
Docker Reactjs



```
Administrator: Command Prompt
=> => transferring dockerfile: 398B 0.2s
=> [internal] load .dockerignore 0.2s
=> => transferring context: 2B 0.1s
=> [internal] load metadata for docker.io/library/node:alpine 4.8s
=> [1/5] FROM docker.io/library/node:alpine@sha256:c59fb39150e4a7ae14dfd42d3f9874398c7941784b73049c2d274115f00d36c8 10.6s
=> => resolve docker.io/library/node:alpine@sha256:c59fb39150e4a7ae14dfd42d3f9874398c7941784b73049c2d274115f00d36c8 0.0s
=> => sha256:c59fb39150e4a7ae14dfd42d3f9874398c7941784b73049c2d274115f00d36c8 1.43kB / 1.43kB 0.0s
=> => sha256:3a718461938f351292be2cb77f7299cd4e3d8f28f29d548f0cf3c8551dc80e08 1.16kB / 1.16kB 0.0s
=> => sha256:41b5b4b810c1b5c99e59c6e97ab6e405081dce4438ec72015626c5b1c7d335c1 6.44kB / 6.44kB 0.0s
=> => sha256:ca7dd9ec2225f2385955c43b2379305acd51543c28cf1d4e94522b3d94cce3ce 2.81MB / 2.81MB 1.4s
=> => sha256:4487691952c066cb3964b94606825bc96c698377909c7d74c889fd12e24e36a7 47.41MB / 47.41MB 5.4s
=> => sha256:206c50ffab466a0ed68db742d6d2015abcedd0a0b2500babb1938ce2a272b425 2.35MB / 2.35MB 1.6s
=> => extracting sha256:ca7dd9ec2225f2385955c43b2379305acd51543c28cf1d4e94522b3d94cce3ce 1.3s
=> => sha256:f6d4361cf153f2e83958f504356eef6e3d041eb3c4d23da466480ee2dfe656ae 448B / 448B 1.8s
=> => extracting sha256:4487691952c066cb3964b94606825bc96c698377909c7d74c889fd12e24e36a7 4.3s
=> => extracting sha256:206c50ffab466a0ed68db742d6d2015abcedd0a0b2500babb1938ce2a272b425 0.3s
=> => extracting sha256:f6d4361cf153f2e83958f504356eef6e3d041eb3c4d23da466480ee2dfe656ae 0.0s
=> [internal] load build context 549.5s
=> => transferring context: 454.39MB 549.2s
=> [2/5] WORKDIR /react-app 0.9s
=> [3/5] COPY ./package.json /react-app 0.5s
=> [4/5] RUN npm install 93.2s
=> [5/5] COPY . . 11.5s
=> exporting to image 11.9s
=> => exporting layers 11.8s
=> => writing image sha256:8b0f636cfbe216471fdee6a24dec3d53191b49092abb10745b5efcc01d81561 0.0s
=> => naming to docker.io/library/amexapp 0.0s

F:\amexecma\amexstream>
F:\amexecma\amexstream>docker run --name amex-c1 -p 3001:3001 -d amexapp
0e6aa4daecf760a5ba0ee97ccf0e5d8358ba47b729c77b71e8e85d9954755512

F:\amexecma\amexstream>
```

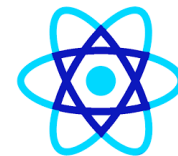


Docker Expressjs

```
F:\amexecma\amexapi>docker build -f dockerfile -t amexapi .
[+] Building 10.7s (10/10) FINISHED
=> [internal] load build definition from dockerfile                                0.1s
=> => transferring dockerfile: 32B                                                0.0s
=> [internal] load .dockerignore                                                  0.1s
=> => transferring context: 2B                                                    0.0s
=> [internal] load metadata for docker.io/library/node:16                      2.6s
=> [internal] load build context                                                 0.8s
=> => transferring context: 406.95kB                                             0.8s
=> [1/5] FROM docker.io/library/node:16@sha256:68fc9f749931453d5c8545521b021dd97267e0692471ce15bdec0814ed1f8fc3 0.0s
=> CACHED [2/5] WORKDIR /usr/src/app                                             0.0s
=> CACHED [3/5] COPY package*.json ./                                           0.0s
=> CACHED [4/5] RUN npm install                                                  0.0s
=> [5/5] COPY . .                                                                4.3s
=> exporting to image                                                            2.7s
=> => exporting layers                                                            2.7s
=> => writing image sha256:38382b7395add999e0f33ae464152758496e6b8d2198d60b721b26cfaa448891 0.0s
=> => naming to docker.io/library/amexapi                                       0.0s

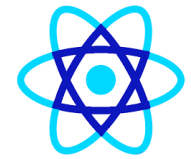
F:\amexecma\amexapi>docker run --name amexapi-c1 -p 4000:4000 --link amex-mongo:mongo -d amexapi
fa396f299f1225e1755c563d91d1c8626f5145ef9e28a2e77d1ee3f2c88ba04b

F:\amexecma\amexapi>
```



Introduction to React UI Testing

- **What is UI/Component Testing?**
UI testing checks whether a React component renders and behaves correctly from the user's point of view.
- For example, if you have a Login component with:
 - Email input
 - Password input
 - Login button
- A UI test verifies that these elements appear, accept input, and respond correctly when the user clicks the button.



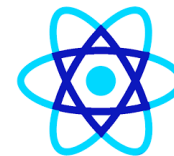
Unit Testing Structure in React

- **1. What is Unit Testing?**
- Unit testing verifies the **smallest testable part of an application independently**.
- In React, a unit can be:
 - Button component
 - Input component
 - FormField component
 - Utility function
 - Custom Hook
 - Reducer
 - Validation function



Unit Testing Structure in React

```
function Button({ children, onClick, disabled }) {  
  return (  
    <button  
      onClick={onClick}  
      disabled={disabled}  
    >  
      {children}  
    </button>  
  );  
}
```



Unit Testing Structure in React

Typical questions are:

Does the button render?

Does it display the correct text?

Does click work?

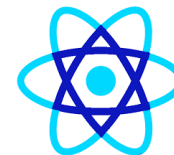
Is onClick called?

Does disabled=true disable the button?



2. Typical Unit Test Project Structure

```
src/  
├── components/  
│   ├── atoms/  
│   │   ├── Button/  
│   │   │   ├── Button.jsx  
│   │   │   └── Button.test.jsx  
│   │   ├── Input/  
│   │   │   ├── Input.jsx  
│   │   │   └── Input.test.jsx  
│   │   └── molecules/  
│   │       ├── FormField/  
│   │       │   ├── FormField.jsx  
│   │       │   └── FormField.test.jsx  
│   └── utils/  
│       ├── validation.js  
│       └── validation.test.js  
├── hooks/  
│   ├── useCart.js  
│   └── useCart.test.js  
├── store/  
│   ├── cartSlice.js  
│   └── cartSlice.test.js  
└── setupTests.js  
  
vitest.config.js
```



3. Basic Structure of a Test File

```
import { render, screen } from "@testing-library/react";
import { describe, test, expect } from "vitest";
import Button from "../Button";

describe("<Button />", () => {

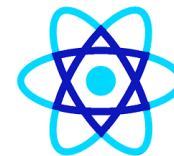
  test("renders button", () => {

    render(<Button>Save</Button>);

    expect(
      screen.getByRole("button", {
        name: "Save"
      })
    ).toBeInTheDocument();

  });

});
```



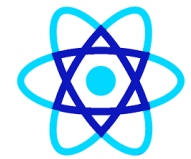
4. describe() – Group Related Tests

`describe()` groups tests belonging to the same component or functionality.

```
</> JavaScript  
  
describe("<Button />", () => {  
  
});
```

Inside it, we can have several test cases:

```
</> JavaScript  
  
describe("<Button />", () => {  
  
  test("renders button", () => {});  
  
  test("handles click", () => {});  
  
  test("supports disabled state", () => {});  
  
});
```



5. test() / it() – Define a Test Case

Each `test()` represents one behavior.

```
</> JavaScript  
  
test("renders Save button", () => {  
  
});
```

`it()` can also be used:

```
</> JavaScript  
  
it("renders Save button", () => {  
  
});
```

Both are equivalent.

A good test name should explain the expected behavior.



6. Arrange → Act → Assert

- A useful structure for unit tests is:

ARRANGE



ACT



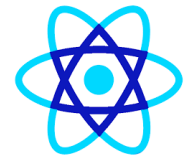
ASSERT



Arrange

```
const handleClick = vi.fn();

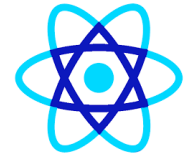
render(
  <Button onClick={handleClick}>
    Save
  </Button>
);
```



```
</> JavaScript
```

```
await user.click(  
  screen.getByRole("button")  
);
```

Assert



```
</> JavaScript
```

```
expect(handleClick)  
  .toHaveBeenCalledTimes();
```




Complete Test

```
test("calls onClick when clicked", async () => {

  // Arrange
  const user = userEvent.setup();
  const handleClick = vi.fn();

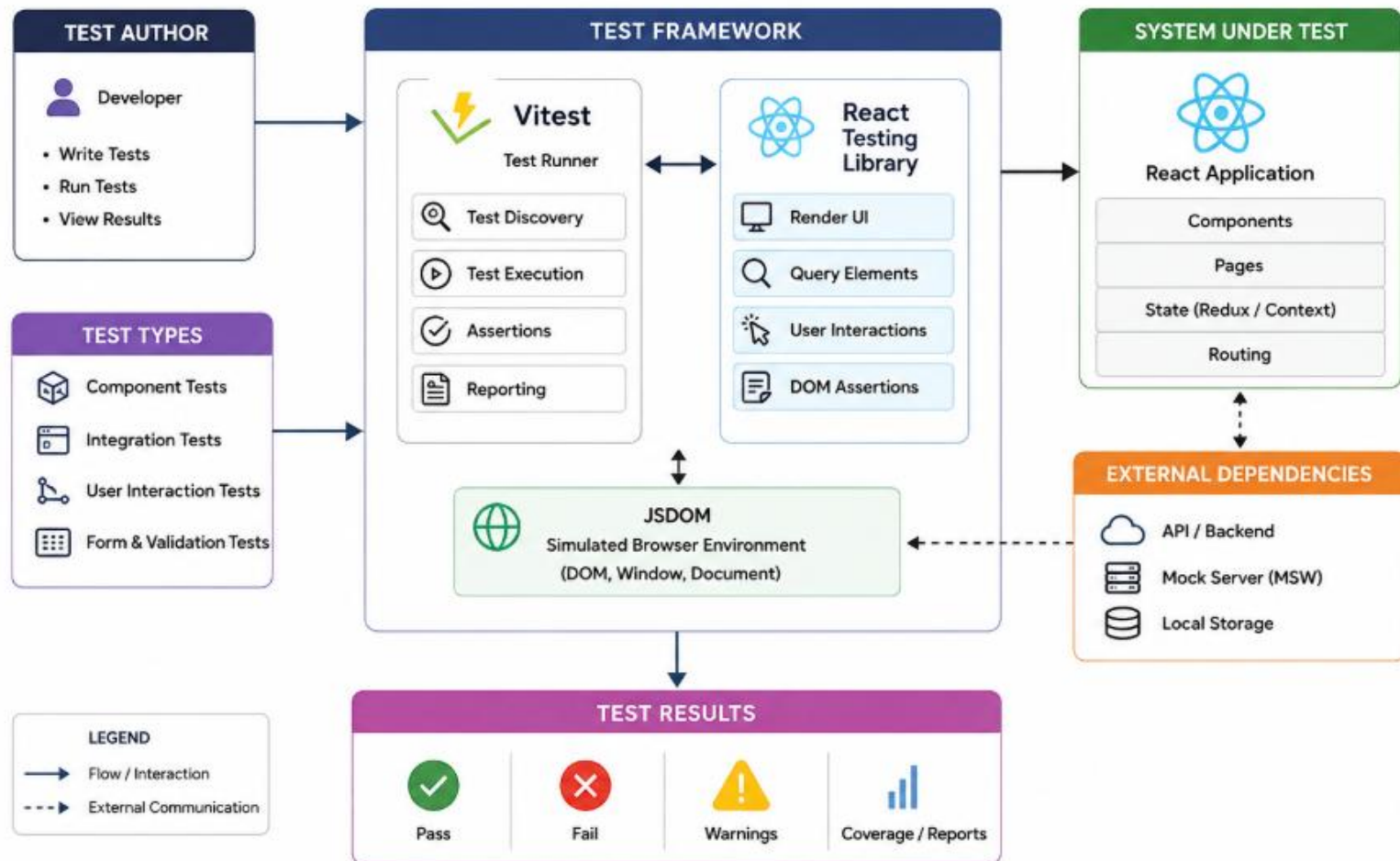
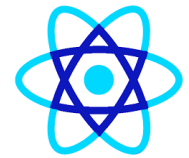
  render(
    <Button onClick={handleClick}>
      Save
    </Button>
  );

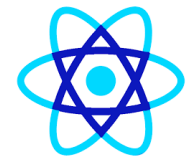
  // Act
  await user.click(
    screen.getByRole("button", {
      name: "Save"
    })
  );

  // Assert
  expect(handleClick)
    .toHaveBeenCalledTimes(1);

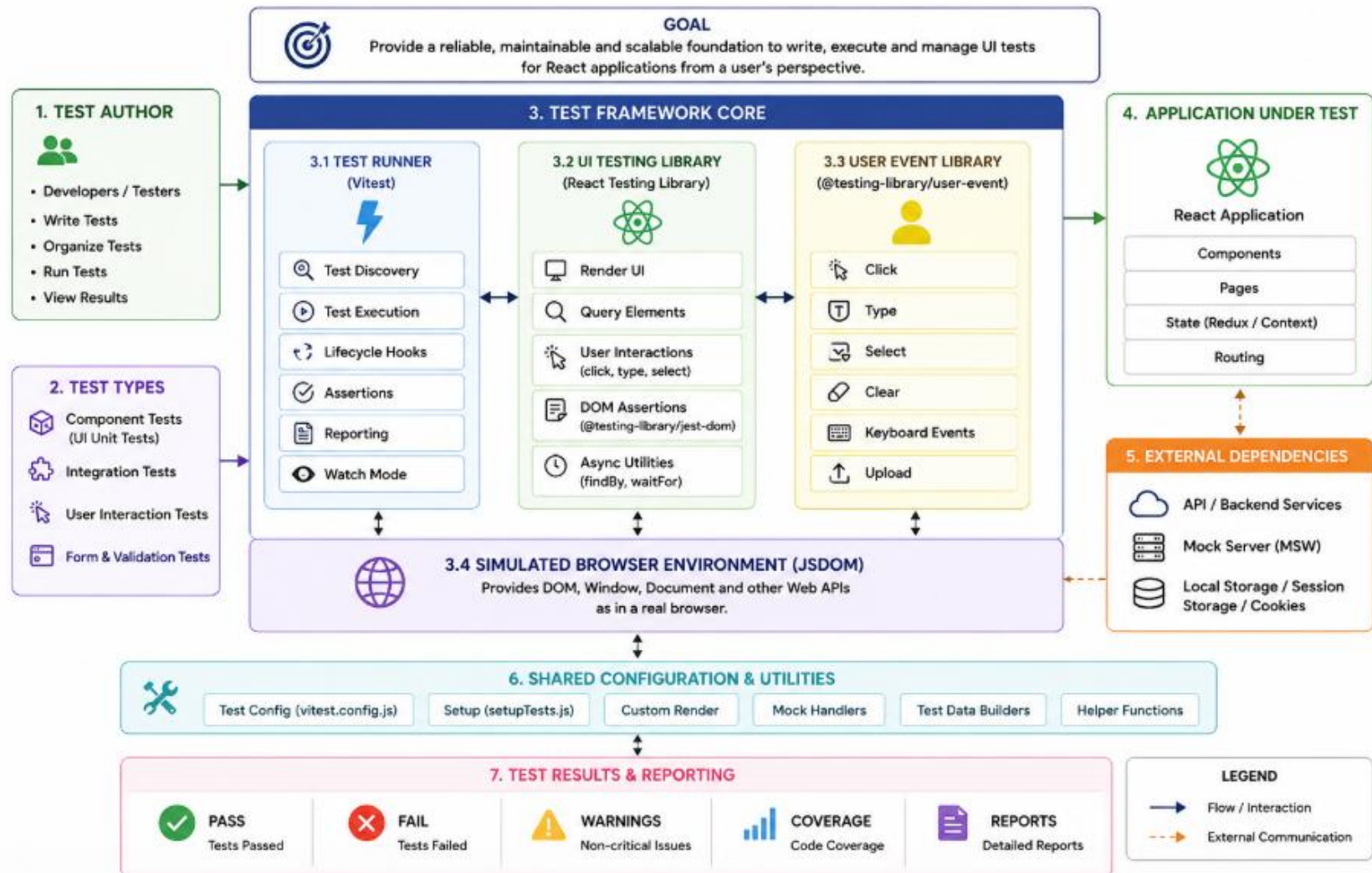
});
```

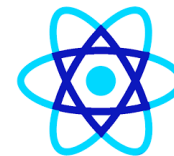
React UI Testing Architecture





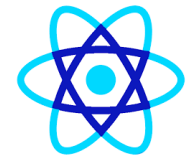
React UI Testing Architecture





Vitest Setup Steps for React UI Testing

- Install Vitest as a development dependency:
 - `npm install -D vitest`
- Vitest provides:
 - Test runner
 - `describe()`
 - `test()` / `it()`
 - `expect()`
 - Mocking using `vi`
 - Lifecycle hooks
 - Test reporting



Vitest Setup Steps for React UI Testing

- Install React Testing Library
 - `npm install -D @testing-library/react`
- React Testing Library provides:
 - `render()`
 - `screen`
 - `fireEvent()`
 - `waitFor()`
- Example:
 - `render(<Button />);`
 - `screen.getByRole("button");`



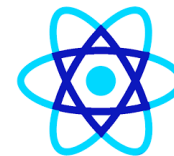
Vitest Setup Steps for React UI Testing

- **Install User Event**
 - `npm install -D @testing-library/user-event`
- Used to simulate realistic user interactions.
 - `const user = userEvent.setup();`
 - `await user.click(button);`
 - `await user.type(input, "React");`
- Prefer `userEvent` for normal user interactions.



Vitest Setup Steps for React UI Testing

- **Install JSDOM**
 - `npm install -D jsdom`
- Vitest runs in Node.js by default.
- React UI tests need browser-like objects such as:
 - `window`
 - `document`
 - `HTMLElement`
 - `DOM`
- JSDOM provides this simulated browser environment.



Vitest Setup Steps for React UI Testing

- **Install Everything Together**
- Instead of installing separately:
 - `npm install -D vitest jsdom @testing-library/react @testing-library/jest-dom @testing-library/user-event`
 - Testing Stack

```
Vitest
  +
React Testing Library
  +
User Event
  +
Jest DOM
  +
JSDOM
```




Vitest Setup Steps for React UI Testing

- **Configure Vitest**
- Open:
 - vite.config.js

Configure the test environment:

`</>` JavaScript

```
import { defineConfig } from "vite";
import react from "@vitejs/plugin-react";

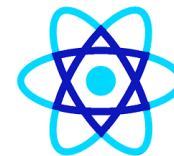
export default defineConfig({
  plugins: [react()],

  test: {
    environment: "jsdom",
    globals: true,
    setupFiles: "./src/setupTests.js",
  },
});
```



Vitest Setup Steps for React UI Testing

- Create `setupTests.js`
- Create:
 - `src/setupTests.js`
- Add:
 - `import "@testing-library/jest-dom/vitest";`
- This enables matchers such as:
 - `toBeInTheDocument()`
 - `toBeDisabled()`
 - `toBeEnabled()`
 - `toBeVisible()`
 - `toHaveValue()`
 - `toHaveTextContent()`



Vitest Setup Steps for React UI Testing

- **Add Test Scripts to package.json**

Open:

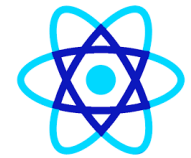
package.json

Add:

```
{  
  "scripts": {  
    "dev": "vite",  
    "test": "vitest",  
    "test:run": "vitest run",  
    "test:coverage": "vitest run --coverage"  
  }  
}
```

Now you can run:

npm test

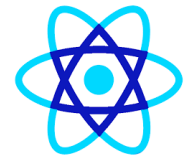


What is Code Coverage?

- **Code Coverage** shows how much of your source code is executed by tests.
- It answers:
 - Which lines ran?
 - Which branches were tested?
 - Which functions are unused?



Coverage Metrics in Jest



Metric	Meaning
Statements	Lines of code executed
Branches	if / else / switch paths
Functions	Functions called
Lines	Executed source lines

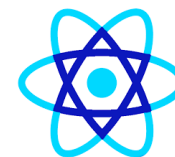
1 Generate Test Coverage Report (HTML – most common)



-  **Install coverage plugin**
- `npm install -D @vitest/coverage-v8`



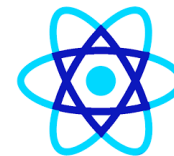
Update vite.config.js



```
import { defineConfig } from "vite";
import react from "@vitejs/plugin-react";

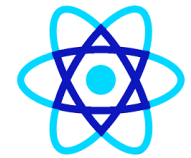
export default defineConfig({
  plugins: [react()],
  test: {
    environment: "jsdom",
    globals: true,
    setupFiles: "./src/setupTests.js",

    coverage: {
      provider: "v8",
      reporter: ["text", "html", "lcov"], // ✅ reports
      reportsDirectory: "./coverage",
    },
  },
});
```



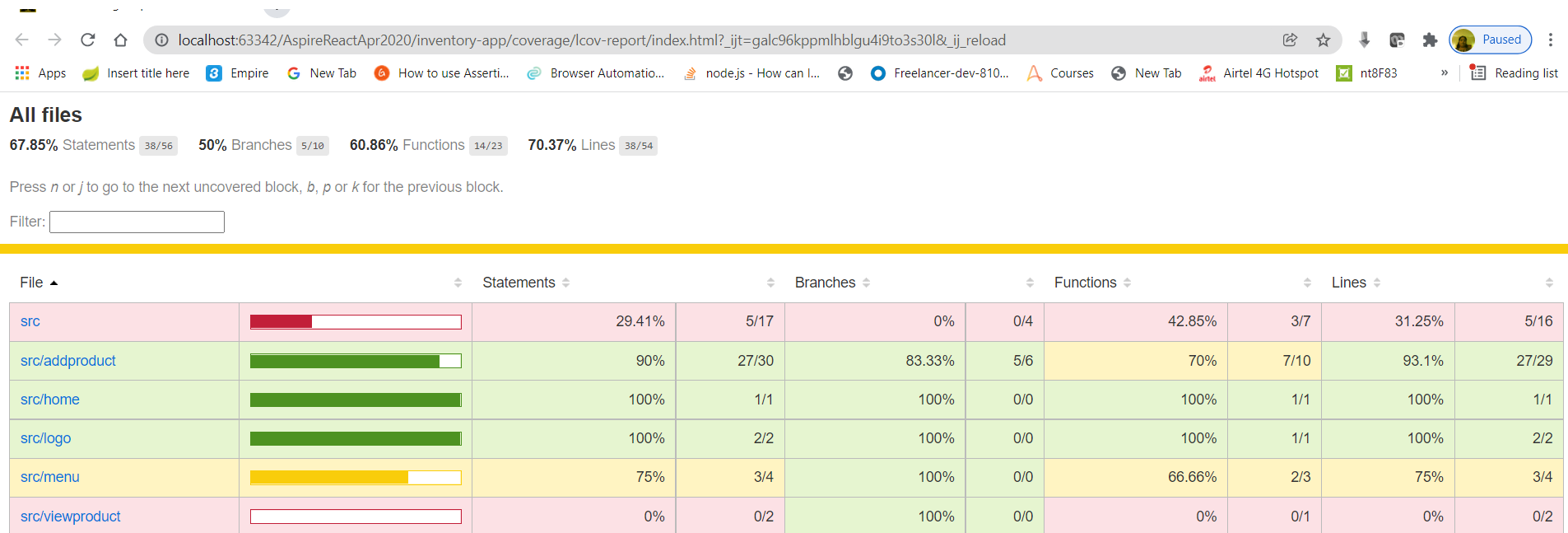
Run tests with coverage

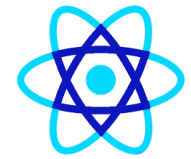
- `npx vitest run --coverage`



React Coverage Report

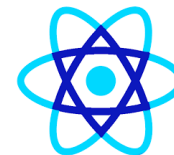
- `npm run test -- --coverage --watchAll=false`





Cypress Testing

- **1. Install Cypress**
- Go to your React project:
 - `cd ecommerce-app`
- Install Cypress:
 - `npm install --save-dev cypress`
 - `npm install -D eslint-plugin-cypress`
- Verify:
 - `npx cypress --version`
- **2. Open Cypress for the First Time**
- Run:
- `npx cypress open`
- Cypress opens its graphical interface.

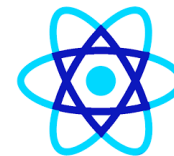


Parcel Bundler

Benchmarks

Based on a reasonably sized app, containing 1726 modules, 6.5M uncompressed. Built on a 2016 MacBook Pro with 4 physical CPUs.

Bundler	Time
browserify	22.98s
webpack	20.71s
parcel	9.98s
parcel - with cache	2.64s



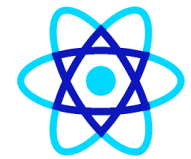
Parcel Bundler

- Assets Bundling - Parcel has out of the box support for JS, CSS, HTML, file assets.
- Automatic transforms - All your code are automatically transformed using Babel, PostCSS, and PostHTML.
- Code Splitting - Parcel allows you to split your output bundle by using the dynamic `import()` syntax.
- Hot module replacement (HMR) - Parcel automatically updates modules in the browser as you make changes during development, no configuration needed.
- Error Logging - Parcel prints syntax highlighted code frames when it encounters errors to help you pinpoint the problem.

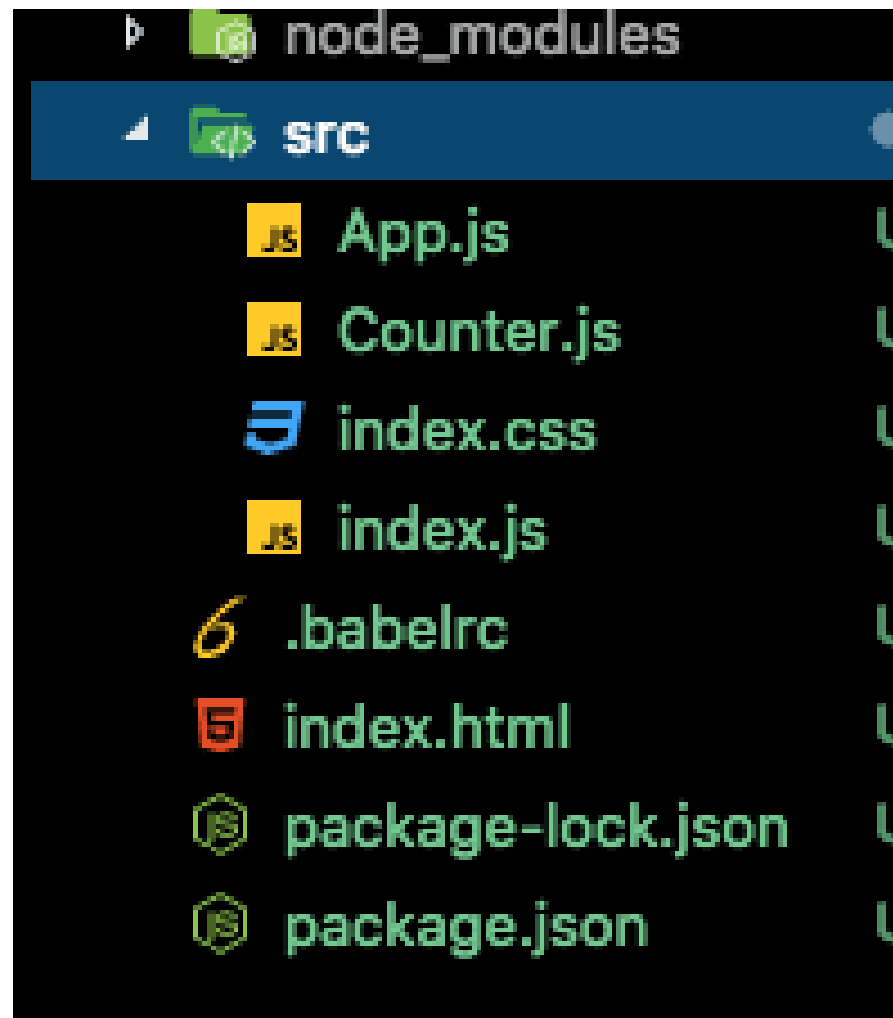


Parcel Bundler

- `mkdir react-parcel`
- `cd react-parcel`
- `npm init -y`
- `npm i --save-dev parcel-bundler`
- `npm i --save react react-dom`



Parcel Bundler

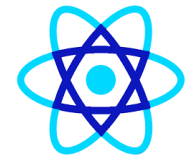




E2e testing

- `npm install nightwatch`
- `npm install chromedriver --save-dev`
- `npm install geckodriver --save-dev`
- Keep web driver in current folder

React Production Deployment



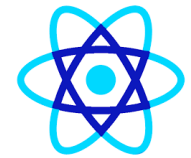
```
File Explorer Administrator: Node.js command prompt - serve -s build
F:\fiservpune2019\bankingapp>npm install netlify-cli -g
npm WARN deprecated flatten@1.0.2: I wrote this module a very long time ago; you should use something else.
C:\Users\Balasubramaniam\AppData\Roaming\npm\ntl -> C:\Users\Balasubramaniam\AppData\Roaming\npm\node_modules\netlify-cli\bin\run
C:\Users\Balasubramaniam\AppData\Roaming\npm\netlify -> C:\Users\Balasubramaniam\AppData\Roaming\npm\node_modules\netlify-cli\bin\run

> core-js@2.6.10 postinstall C:\Users\Balasubramaniam\AppData\Roaming\npm\node_modules\netlify-cli\node_modules\core-js
> node postinstall || echo "ignore"

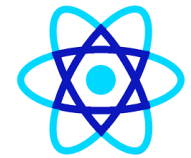
Thank you for using core-js ( https://github.com/zloirock/core-js ) for polyfilling JavaScript standard library!

The project needs your help! Please consider supporting of core-js on Open Collective or Patreon:
> https://opencollective.com/core-js
> https://www.patreon.com/zloirock
```


React Production Deployment



```
Reactjs_v1.pptx - PowerPoint
Administrator: Node.js command prompt - serve -s build
es/netlify-cli/node_modules/node-fetch/lib/index.js:1455:11)
F:\fiservpune2019\bankingapp>netlify deploy
Please provide a publish directory (e.g. "public" or "dist" or "."):
F:\fiservpune2019\bankingapp
? Publish directory F:\fiservpune2019\bankingapp\build
Deploy path: F:\fiservpune2019\bankingapp\build
Deploying to draft URL...
✓ Finished hashing 17 files
✓ CDN requesting 0 files
✓ Finished uploading 0 assets
✓ Draft deploy is live!
Logs: https://app.netlify.com/sites/banking-app-veb/deployments/5da743c6f4ee609bdb2b7deb
Live Draft URL: https://5da743c6f4ee609bdb2b7deb--banking-app-veb.netlify.com
If everything looks good on your draft URL, take it live with the --prod flag.
netlify deploy --prod
```



React Production Deployment

Administrator: Node.js command prompt - netlify deploy -d .

Site ID: 7f57284a-725d-487d-98b1-7c26e68c6248

Deploy path: G:\Local disk\vigneshpics\vebapplication\vebdentalcare\dist\vebdentalcare\netlify\build

Deploying to draft URL...

✓ Finished hashing 0 files

✓ CDN requesting 0 files

✓ Finished uploading 0 assets

/ Waiting for deploy to go live...Terminate batch job (Y/N)? y

G:\Local disk\vigneshpics\vebapplication\vebdentalcare\dist\vebdentalcare>netlify unlink

Folder is not linked to a Netlify site. Run 'netlify link' to link it

G:\Local disk\vigneshpics\vebapplication\vebdentalcare\dist\vebdentalcare>netlify deploy -d .

This folder isn't linked to a site yet

? What would you like to do? + Create & configure a new site

? Team: eswaribala's team

Choose a unique site name (e.g. isnt-eswaribala-awesome.netlify.com) or leave it blank for a random name. You can update the site name later.

? Site name (optional): vebdentalcare

» Warning: vebdentalcare.netlify.com already exists. Please try a different slug.

Choose a unique site name (e.g. super-cool-site-by-eswaribala.netlify.com) or leave it blank for a random name. You can update the site name later.

? Site name (optional): vebdentalcare

Site Created

Admin URL: https://app.netlify.com/sites/vebdentalcare

URL: https://vebdentalcare.netlify.app

Site ID: c4ced4d0-96ce-43b5-a821-c4f09c11973b

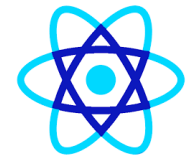
Deploy path: G:\Local disk\vigneshpics\vebapplication\vebdentalcare\dist\vebdentalcare

Deploying to draft URL...

✓ Finished hashing 482 files

✓ CDN requesting 369 files

\ (466/467) Uploading 3rdpartylicenses.txt...



React Production Deployment

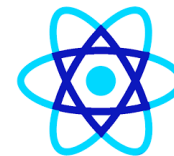
Yarn global add serve

```
Administrator: Node.js command prompt - serve -s build
F:\fiservpune2019\react-voting-app-master>serve -s build

Serving!

- Local: http://localhost:5000
- On Your Network: http://172.29.53.177:5000

Copied local address to clipboard!
```



React Production Layout

```
UI External: http://localhost:3003
-----
[Browsersync] Serving files from: ./
[Browsersync] Watching files...
19.11.12 15:05:22 404 GET /index.html
^CTerminate batch job (Y/N)? y

F:\daimlerreactoct2019\blog2019>lite-server --baseDir=build/
Did not detect a `bs-config.json` or `bs-config.js` override file. Using lite-server defaults...
** browser-sync config **
{
  injectChanges: false,
  files: [ './**/*.html,css,js' ],
  watchOptions: { ignored: 'node_modules' },
  server: { baseDir: 'build/', middleware: [ [Function], [Function] ] }
}
[Browsersync] Access URLs:
-----
    Local: http://localhost:3002
    External: http://172.29.53.177:3002
-----
    UI: http://localhost:3003
UI External: http://localhost:3003
-----
[Browsersync] Serving files from: build/
[Browsersync] Watching files...
19.11.12 15:05:53 200 GET /index.html
```



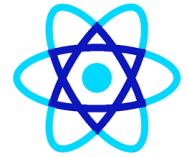
React Optimization

- **Using Immutable Data Structures**
- **Function/Stateless Components and `React.PureComponent`**
- **Multiple Chunk Files**
- **Using Production Mode Flag in Webpack**
- **Dependency optimization**
- **Use `React.Fragments` to Avoid Additional HTML Element Wrappers**

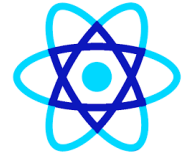


React Optimization

- **Avoid Inline Function Definition in the Render Function.**
- Throttling and Debouncing Event Action in JavaScript
- **Spreading props on DOM elements**
- **CSS Animations Instead of JS Animations**

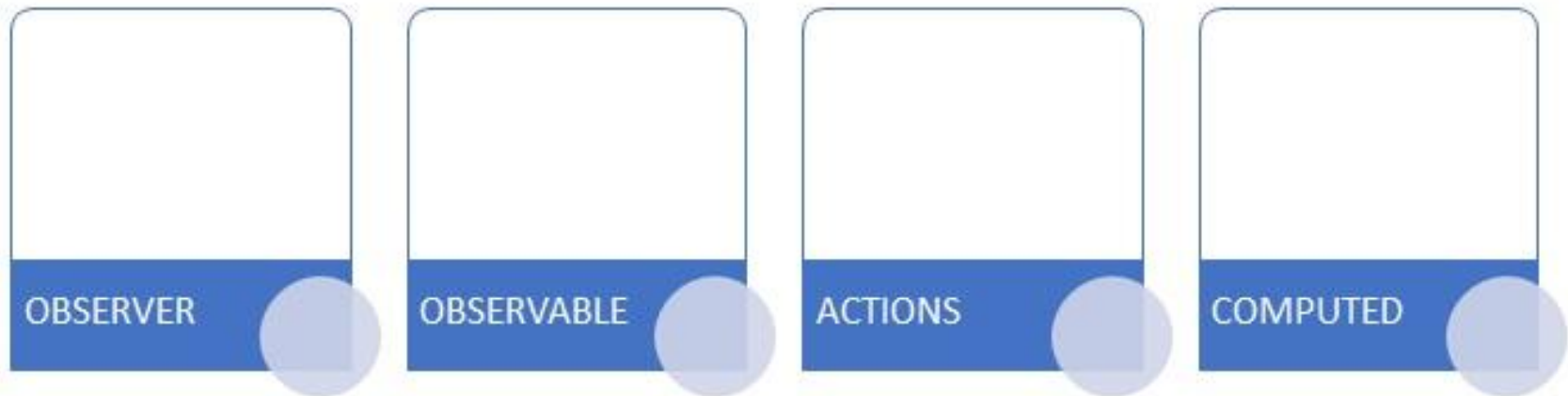
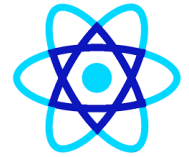


- MobX is a library that aims to make state management simple and scalable by transparently applying functional reactive programming (TFRP).
- According to the readme, the approach of MobX is thus: “Anything that can be derived from the application state, should be derived Automatically.”
- This includes the UI, data serialization, server communication, etc.

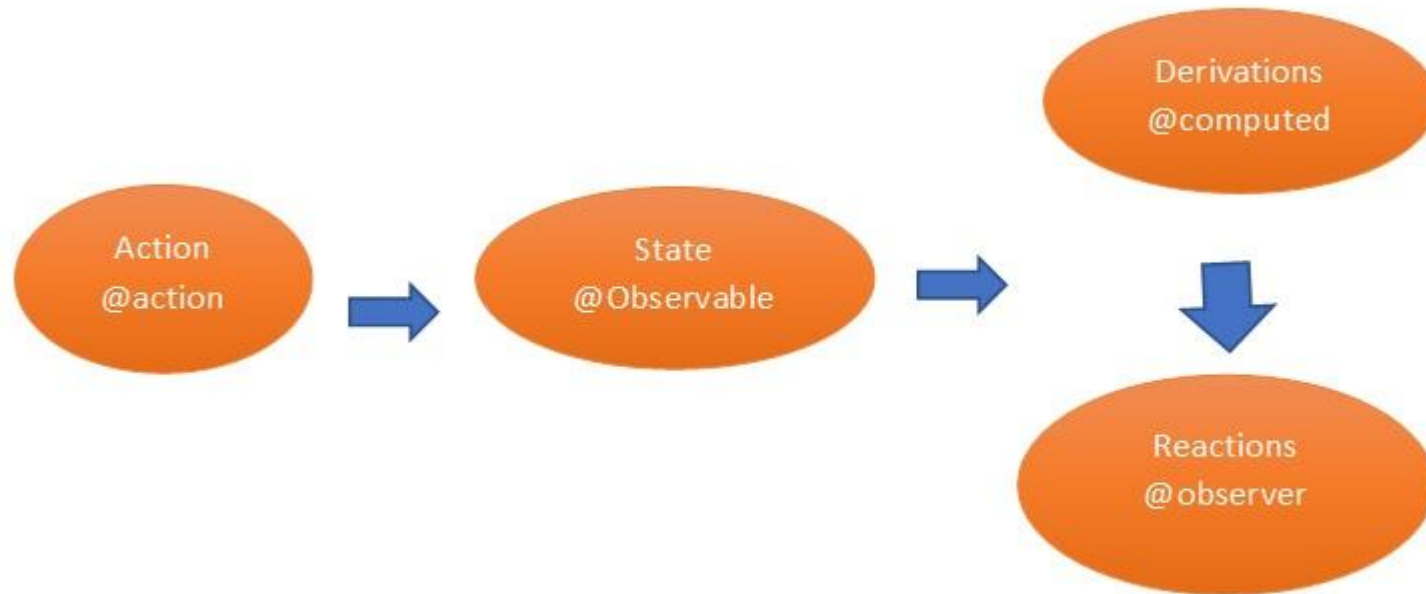
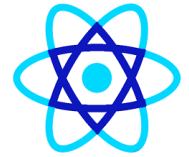


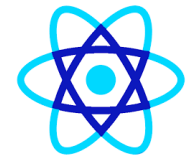
- In addition, some of the core principles of MobX include:
 - May have multiple stores to store the state of the application
 - The state of the entire application is stored in a single object tree
 - Action in any piece of code can change the state of the application
 - All derivations from the state are automatically and atomically updated when the state is changed

MOB X Core Concepts



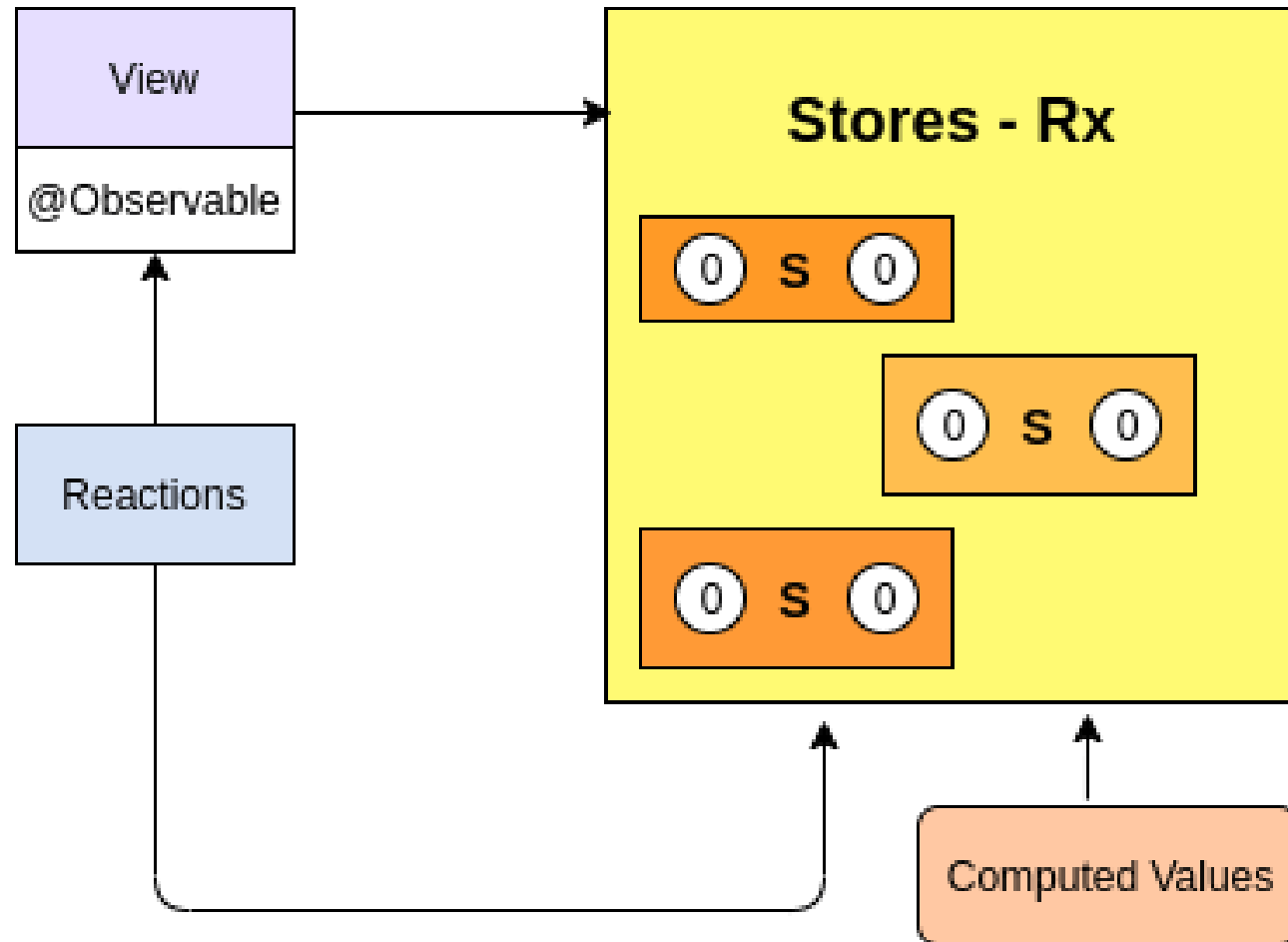
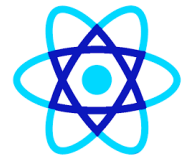
MOB X Core Concepts

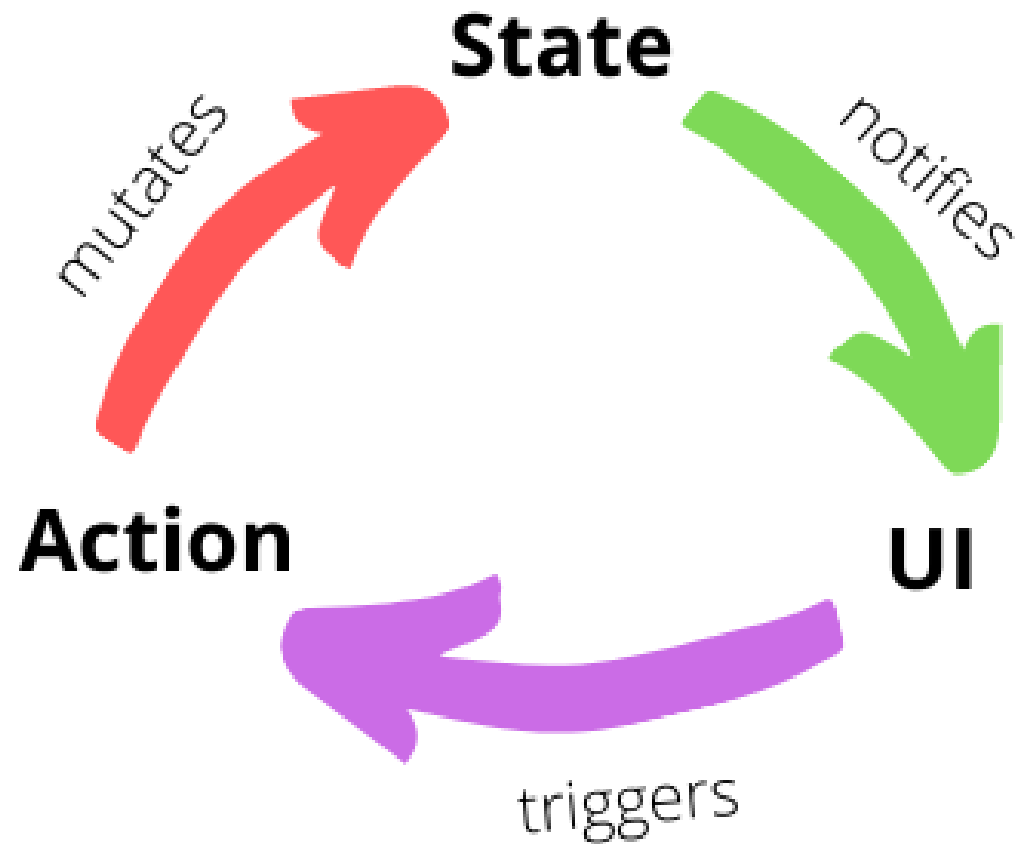
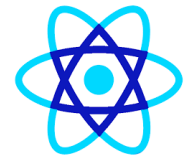


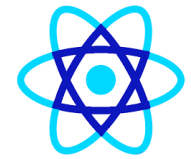


MOB X Core Concepts

- **@Observable**
 - Mobx listens through observables to identify when the values is changed. It can array objects and with @Observable key words we can decorate the object.
- **@Action**
 - Action is a method or function that is used to modify the state of a data. We can decorate with @Action keycard to declare as it is a Mob X Action method.
- **@Inject**
 - To use any store value we have to inject the store using @inject.
- **@Observer**
 - Any component that needs to update with state will need to be decorated with @observer.



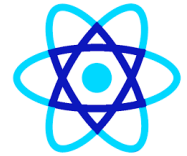




React MobX steps

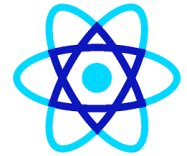
- 1. `npx create-react-app bankingapp`
- 2. `npm install mobx --save`
- 3. `npm install mobx-react --save`
- 4. `npm install react-router-dom --save`
- To install semantic UI(similar to bootstrap) please use the below command,
- `npm install semantic-ui-react --save`
- `npm install semantic-ui-css --save`

Redux vs MobX for React state management



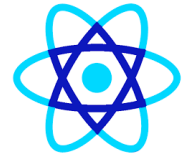
- Data store – single store vs. multiple stores
 - Redux – Single store
 - In Redux, there is only one store.
 - It serves as the single source of truth containing normalized data.
 - Redux state is immutable and for each new state, an ancestor state is cloned
 - MobX – Multiple stores
 - Unlike Redux, MobX usually maintains at least two stores – one for the UI state and one (or more) for the domain state, and they contain denormalized data.
 - The advantage of multiple stores is in the ability to reuse and query the domain state universally, including other applications.
 - All the while, the UI store would remain specific to the current application.

Redux vs MobX for React state management



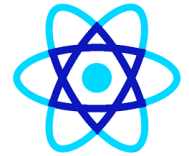
- Store data structure
 - Redux – JavaScript objects
 - Redux uses simple JavaScript objects as the data structure to store the state.
 - This requires that updates be tracked manually.
 - Which can add quite a bit of overhead when it comes to applications with complex states to manage and maintain.
 - Redux vs MobX in React
 - MobX uses observable (or noticeable) data to automatically track changes through subscriptions.
 - Quite clearly, automation makes for an easier life for a developer.
 - So it's no wonder many find MobX to be an obvious winner in this category. It's simply more comfortable to use.

Redux vs MobX for React state management



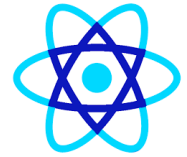
- Purity
- Redux – Pure
 - We've already established that Redux uses a single, immutable source of truth for the states stored.
 - This means that states are all read-only, and reducers can overwrite a state invoked by an action.
 - Reducers are pure functions, as they receive a state and action and return a new state.

Redux vs MobX for React state management



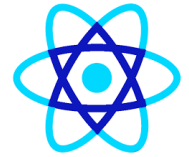
- Purity
 - MobX – Impure
 - On the flip side there's MobX that allows for states to be easily updated and overwritten with new values.
 - While it may be easy to implement, testing and maintaining can become a nightmare of unpredictable outputs.

Redux vs MobX for React state management



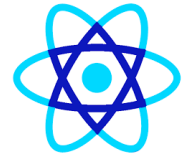
- Boilerplate code
 - Redux – Explicit
 - Perhaps one of the main shortcomings of Redux is the sheer volume of boilerplate code it brings.
 - This is especially true when it comes to React applications.
 - MobX – Implicit
 - Since MobX is a lot more implicit in nature, it packs a lot less boilerplate.
 - In this category, MobX is a clear winner.

Redux vs MobX for React state management



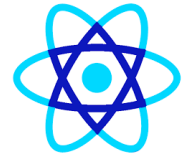
- Scalability
 - The purity and somewhat rigid approach of Redux is an advantage when it comes to scalability and debugging of applications.
 - The predictability of pure functions makes Redux much easier to test, maintain and scale than MobX.

Redux vs MobX for React state management

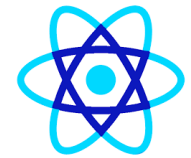


- Debugging
 - Redux – Predictable
 - As mentioned above, Redux is a lot more predictable than MobX as it comes with a lot less abstraction.
 - Add to that a superior set of developer tools (including time traveling) and you got yourself code that is a breeze to debug.
- MobX – Abstract
 - Unlike Redux, MobX relies a lot more on abstraction, which can produce unpredictable results and generally make debugging difficult.
 - Moreover, the lack of efficient tools for MobX debugging and testing add another hurdle for developers considering MobX for their state management needs.

Redux vs MobX for React state management

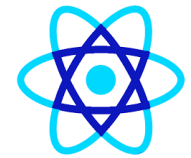


- Learning curve
 - Redux – Steep
 - MobX – Moderate



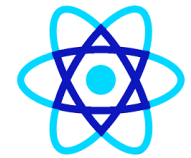
Micro Front End

- Extends the **Microservices** concept to the frontend.
- Breaks a **monolithic frontend** into **smaller, independent applications**.
- Each micro frontend represents a **business feature** (e.g., Products, Cart, Orders, Profile).
- Each feature is **owned by a dedicated team**.
- Teams can **develop, test, deploy, and maintain** their modules independently.
- All micro frontends are combined through a **Shell (Host/Container) application**.
- Users experience the application as **one seamless UI**.



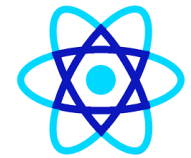
Micro Front End

- Supports **independent deployment** without affecting other modules.
- Enables **parallel development** by multiple teams.
- Improves **scalability** and **maintainability**.
- Allows **technology flexibility** (React, Angular, Vue, etc., if required).
- Reduces the impact of changes by **isolating features**.



Micro Front End

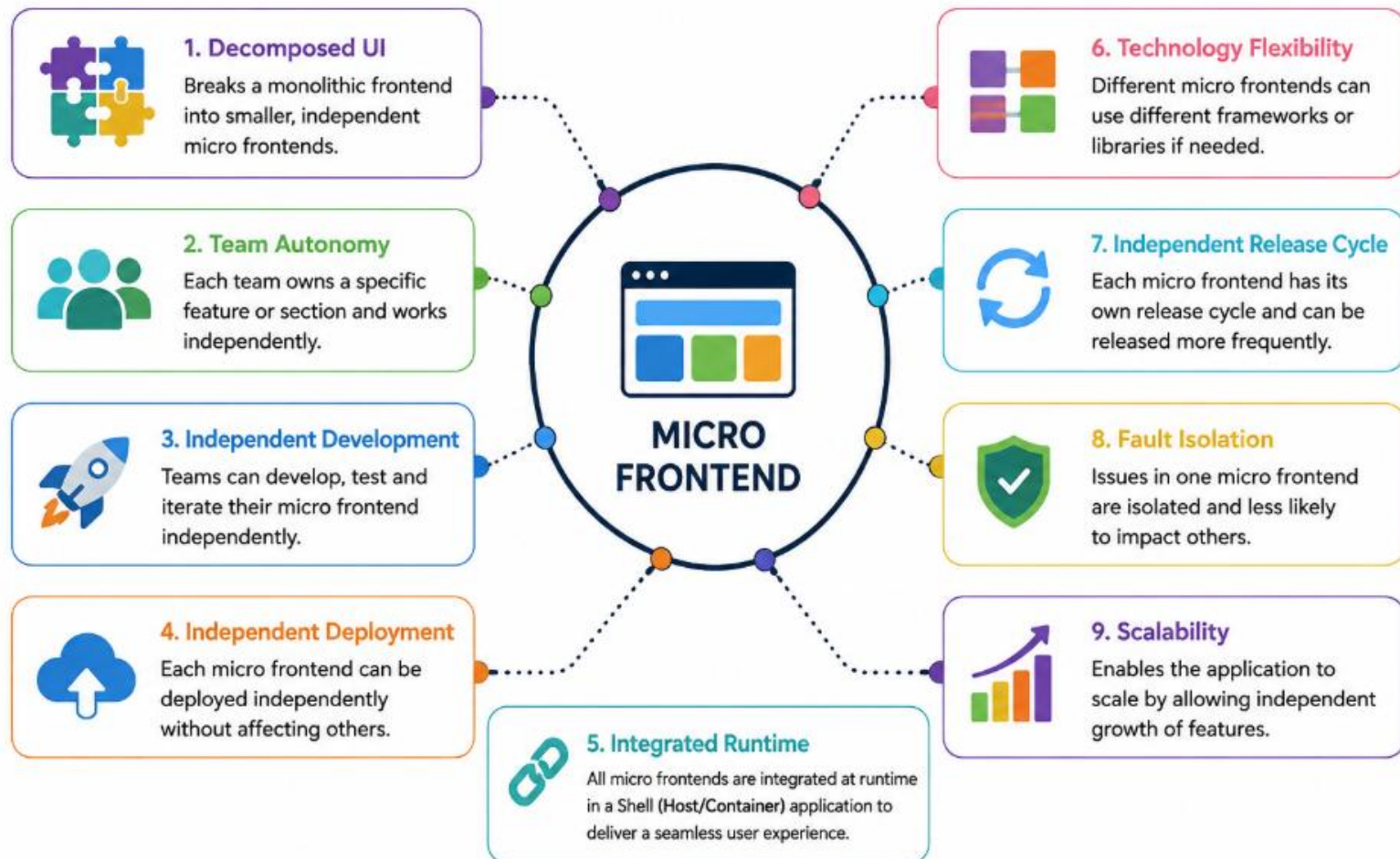
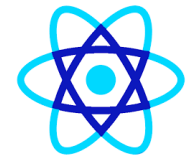
- Common integration approaches include:
 - **Module Federation**
 - **Single-SPA**
 - **Web Components**
 - **iFrames** (less preferred)
- Communication between micro frontends can be done using:
 - Custom Events
 - Shared State (Redux, Zustand, RxJS)
 - URL-based routing
 - Backend APIs

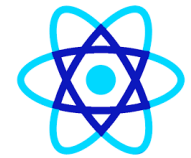


Monolithic vs Micro Front End

Feature	Monolith	Micro Frontend
Codebase	Single	Multiple
Deployment	One application	Independent deployments
Teams	Shared ownership	Domain ownership
Scalability	Moderate	High
Complexity	Lower	Higher
Technology	Usually one framework	Can vary by module

Key Characteristics

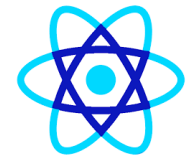




Micro Front End Advantages

- **Advantages**

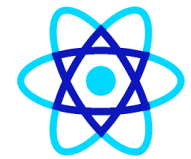
- Independent development
- Independent deployment
- Faster releases
- Better scalability
- Easier maintenance
- Team autonomy
- Fault isolation
- Reusable components



Micro Front End Challenges

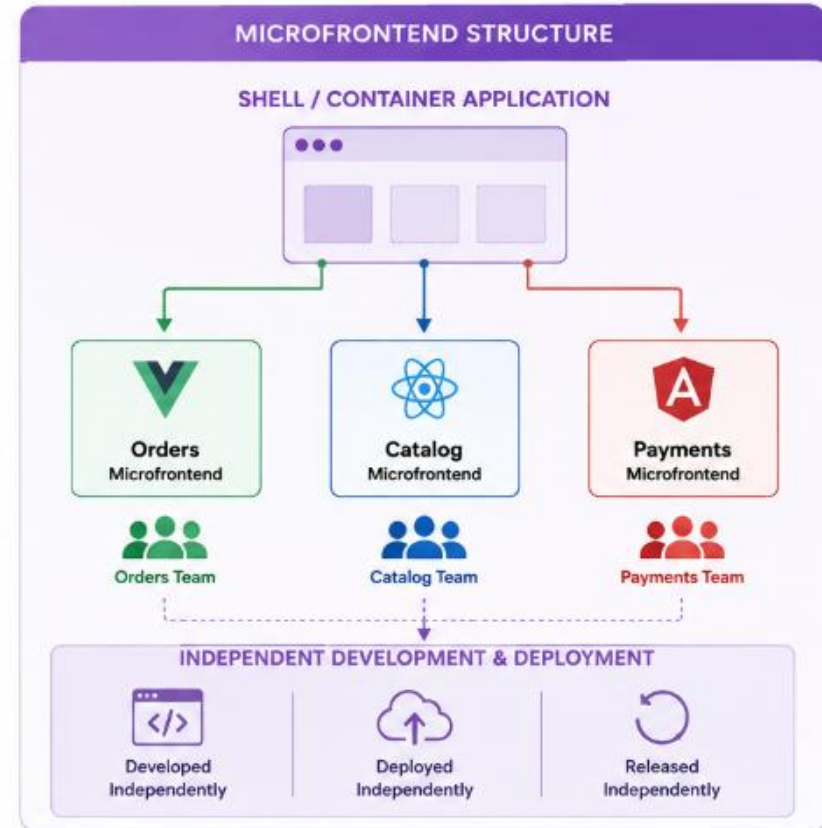
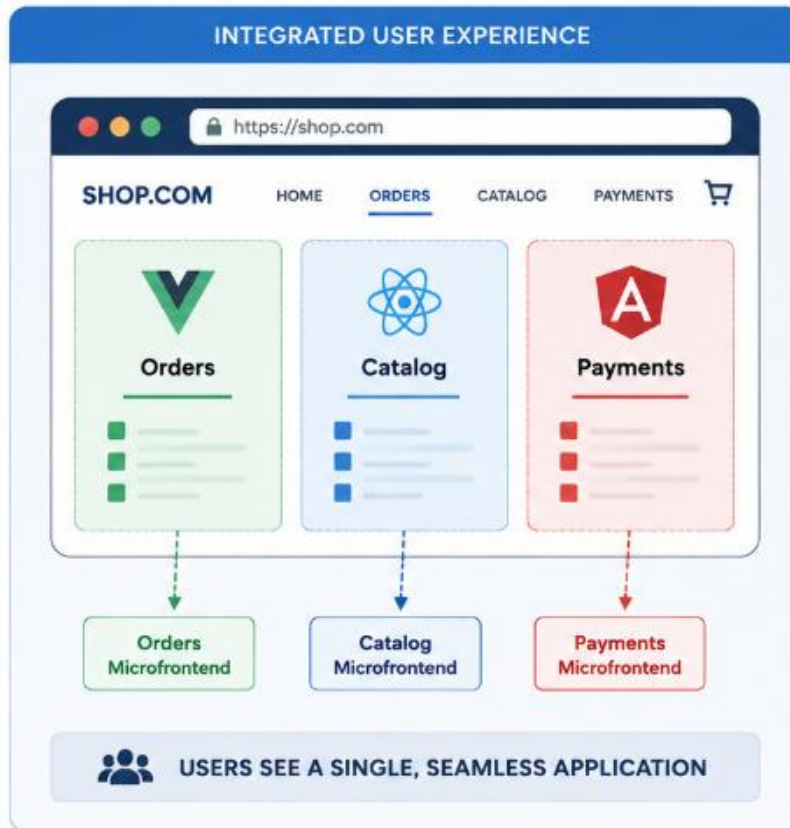
- **Challenges**

- More complex architecture
- Shared state management
- Routing coordination
- Authentication handling
- UI consistency across teams
- Dependency management
- Performance optimization



Micro Front End Architecture

Independent Development • Independent Deployment • Integrated User Experience



KEY BENEFITS



**Team
Autonomy**



**Faster
Delivery**



**Technology
Flexibility**

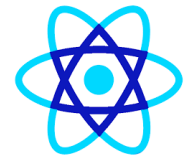


Scalability



**Fault
Isolation**

Micro Front Ends



WHAT ARE MICROFRONTENDS?



Divide a **monolithic** app into multiple, smaller apps



Each smaller app is responsible for a **distinct feature** of the product



WHY USE THEM?

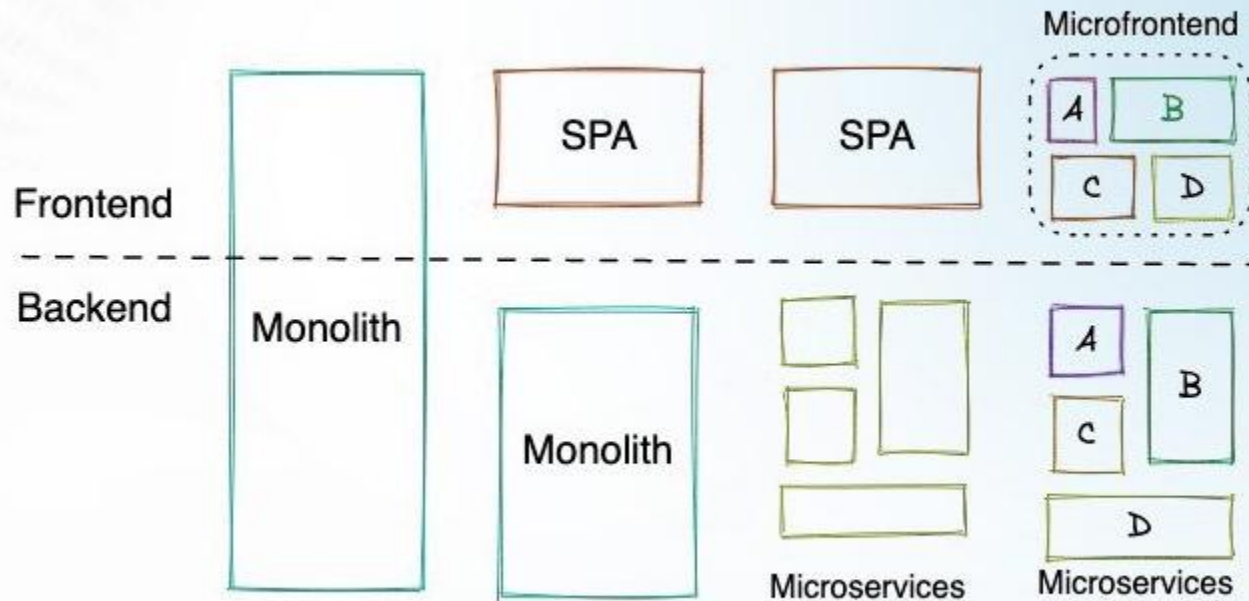
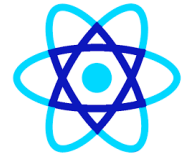


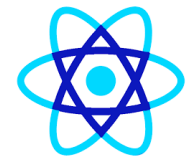
Multiple engineering teams can **work in isolation**



Each smaller app is **easier to understand** and make changes to

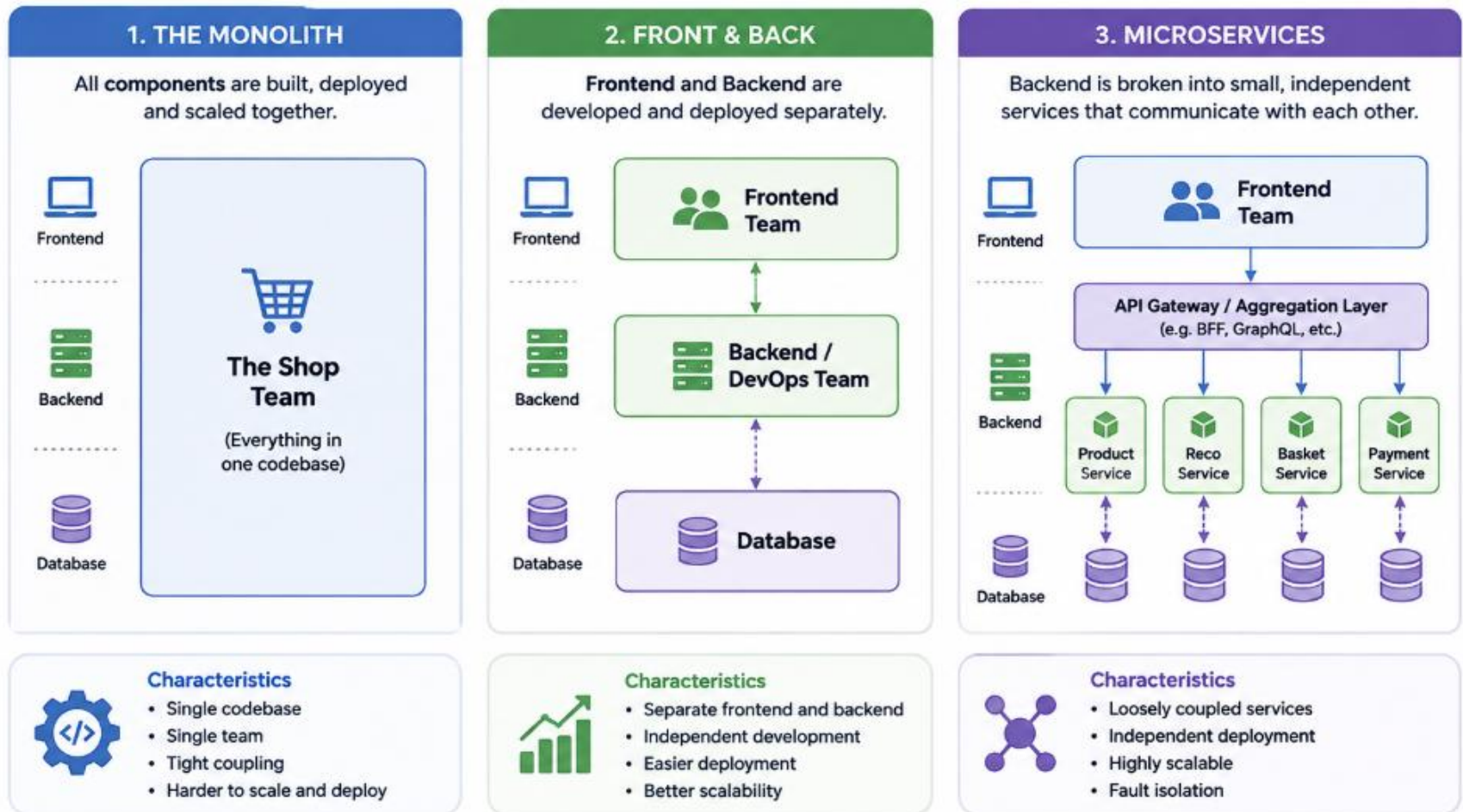
Micro Front Ends



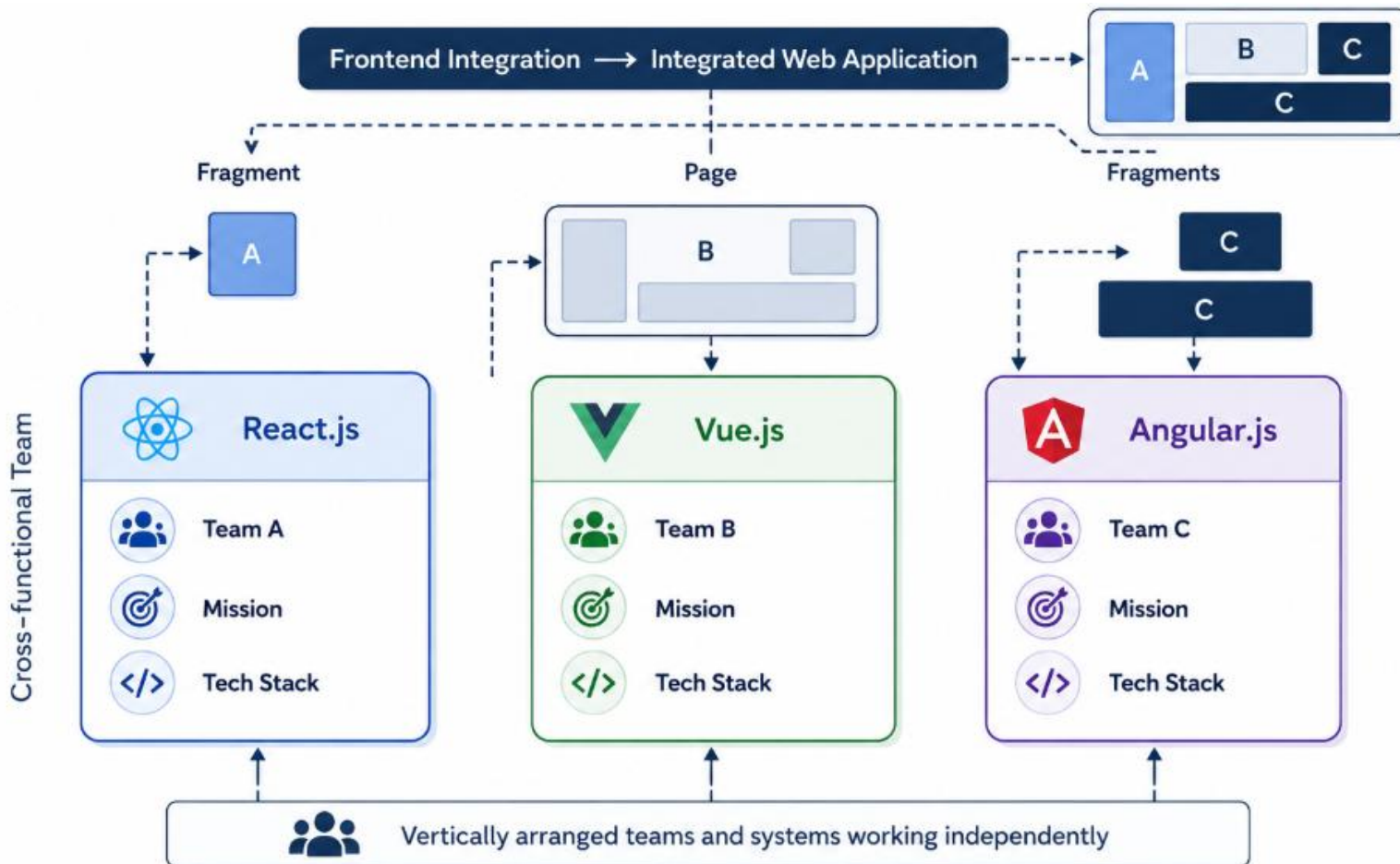
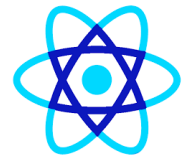


Different Architectural Approaches

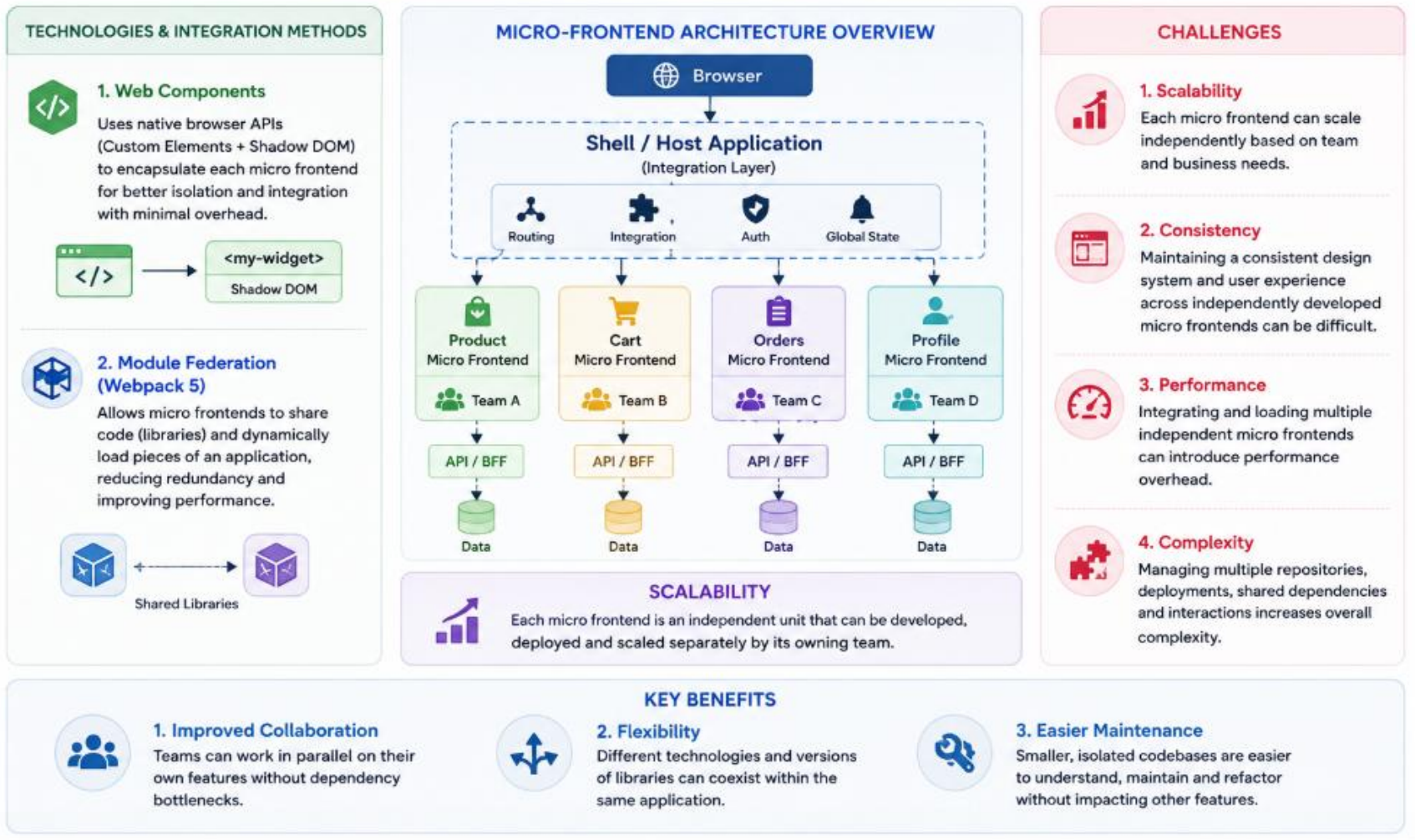
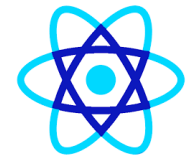
From Monolith to Microservices

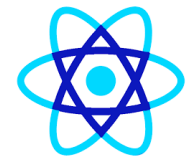


Different Architectural Approaches



MFE Architecture





Types of Micro front ends

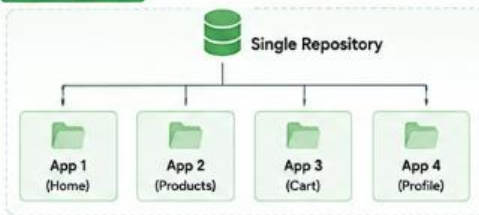
Different approaches to structure and manage microfrontend applications



1. MONOREPOSITORY

All microfrontends live in a single repository, sharing the same dependencies and tooling.

HOW IT WORKS



KEY FEATURES

- ✓ All code in one repository
- ✓ Shared dependencies and libraries
- ✓ Simpler setup and local development
- ✓ Single CI/CD pipeline
- ✓ Tighter coupling between teams



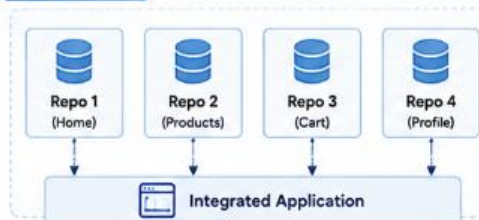
BEST FOR: Small to medium teams, early adoption of microfrontends



2. MULTIREPOSITORY

Each microfrontend has its own repository. Repositories are independent but collaborate together.

HOW IT WORKS



KEY FEATURES

- ✓ Each microfrontend in its own repository
- ✓ Independent development and deployment
- ✓ Clear ownership and boundaries
- ✓ More DevOps overhead
- ✓ Coordination between repositories



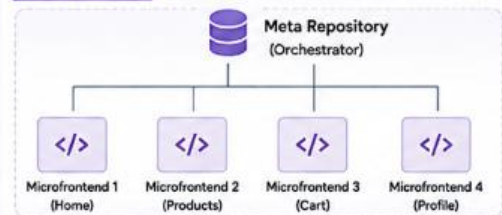
BEST FOR: Medium to large teams, growing applications



3. METAREPOSITORY

A single repository orchestrates multiple repositories as submodules or packages. Acts as a container for all microfrontends.

HOW IT WORKS



KEY FEATURES

- ✓ Single repository manages multiple repos
- ✓ Uses Git submodules / packages
- ✓ Centralized orchestration and configuration
- ✓ Simplifies dependency management
- ✓ Adds a layer of abstraction



BEST FOR: Large enterprise teams, complex dependency management



QUICK COMPARISON

MONOREPOSITORY

Simple to start, tightly coupled, best for small teams.

MULTIREPOSITORY

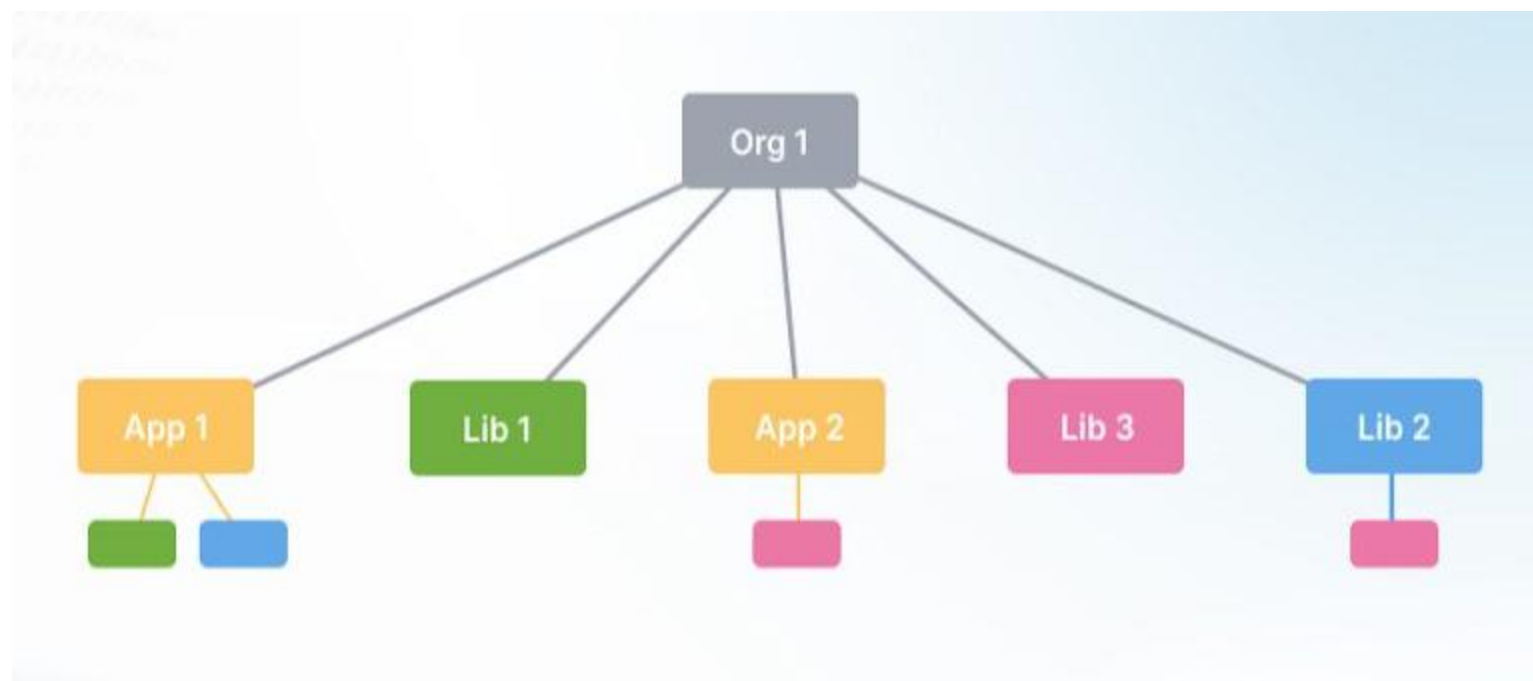
Independent and scalable, best for mid to large teams.

METAREPOSITORY

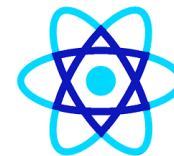
Centralized orchestration, best for large and complex ecosystems.



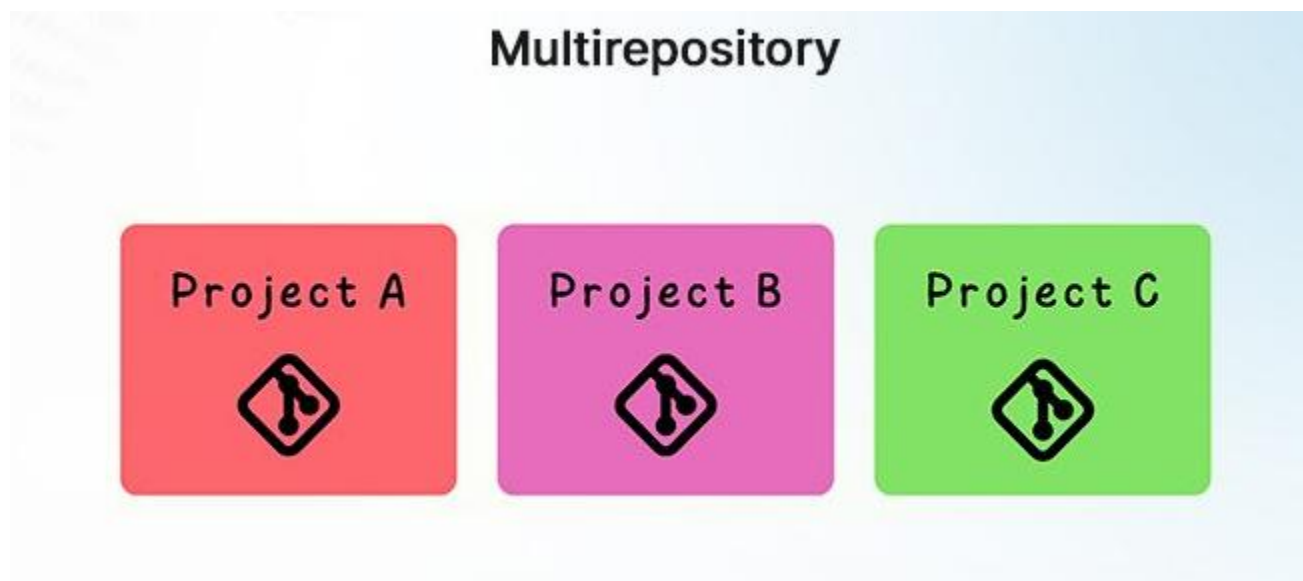
Mono Repository



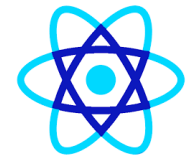
- **Example:** Imagine building an e-commerce website with multiple micro front ends, including product listings, a shopping cart, and a user account.
- **Instead of maintaining separate repositories for each micro frontend, you keep them all in a single mono repository.**



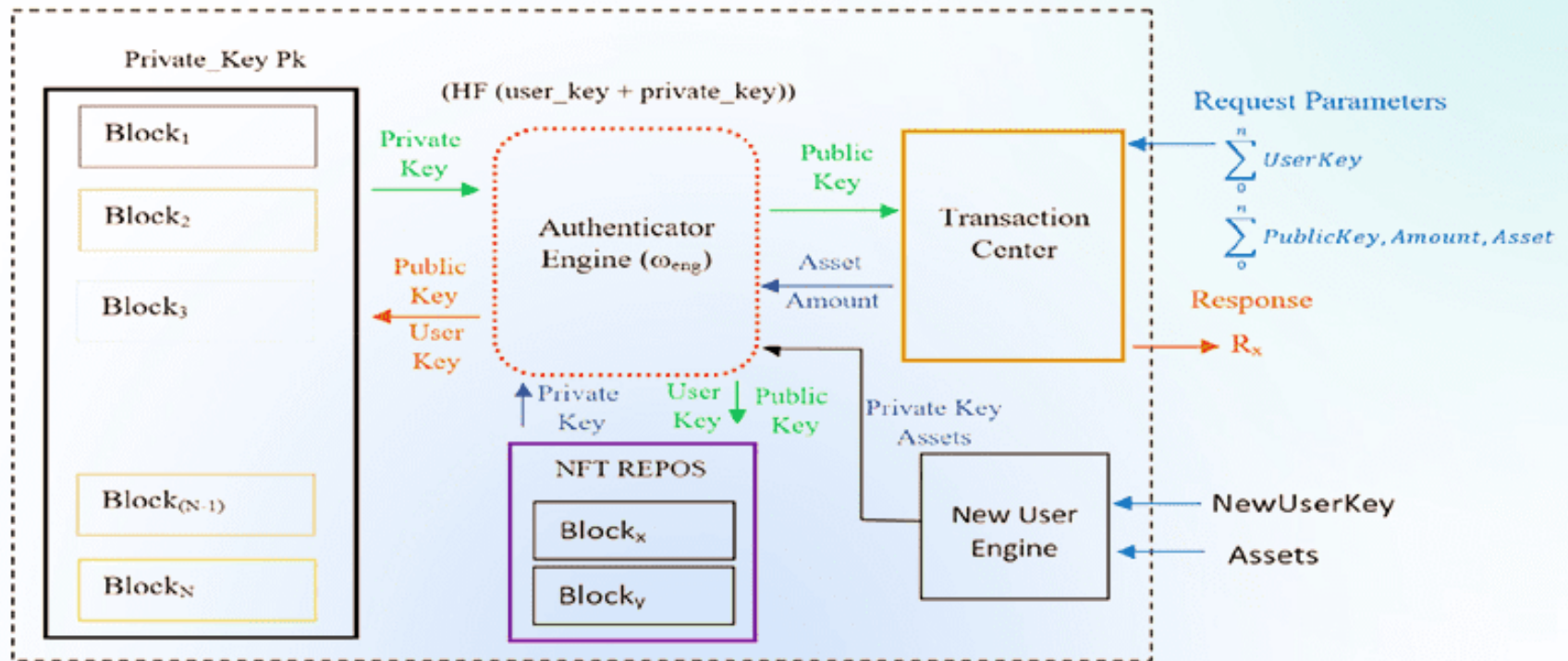
Multi Repository



- **Example: Consider an e-commerce website with micro frontends for product listings, shopping carts, and user accounts.**

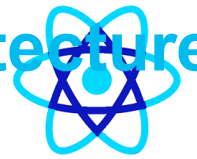


The working architecture of the MetaRepo system



Example: Imagine building a content management system (CMS) with various microfrontends, including a blog editor, user dashboard, and analytics dashboard.

Different Approaches of Micro frontend Architecture Integration



MICRO FRONTENDS ARCHITECTURE INTEGRATION APPROACHES

Different ways to integrate multiple micro frontends into a single web application



1. SERVER-SIDE COMPOSITION (SSR)

The server combines multiple micro frontends and sends the fully rendered HTML to the browser.

- ✓ Better SEO
- ✓ Faster First Contentful Paint (FCP)
- ✓ Consistent initial load
- ✓ Less JavaScript on client



2. RUN TIME INTEGRATION (CLIENT-SIDE)

Micro frontends are loaded dynamically in the browser and composed using JavaScript at runtime.

- ✓ More dynamic and interactive
- ✓ Independent deployment
- ✓ Technology flexibility
- ✓ Higher client-side overhead



3. BUILD TIME INTEGRATION (SSG / STATIC BUILD)

Micro frontends are integrated during the build process to generate a static site.

- ✓ High performance
- ✓ Better caching and CDN delivery
- ✓ Lower runtime overhead
- ✓ Less dynamic at runtime



SERVER-SIDE COMPOSITION (SSR)

Server renders and sends HTML.
Best for SEO and performance.



RUN TIME INTEGRATION (CLIENT-SIDE)

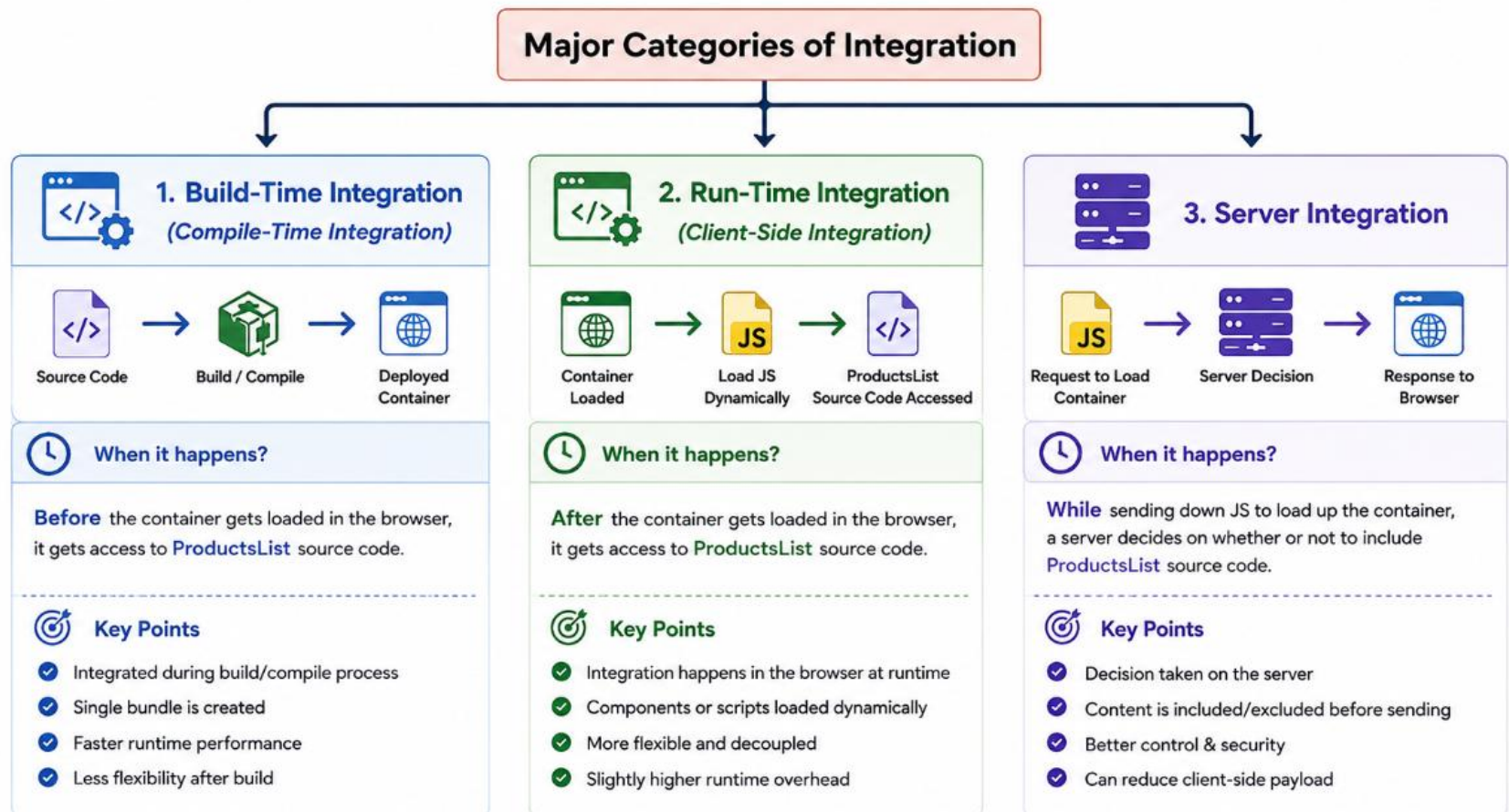
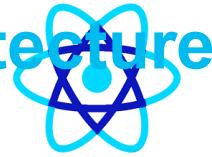
Dynamic loading in browser.
Best for flexibility and interactivity.

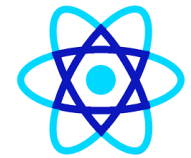


BUILD TIME INTEGRATION (SSG)

Pre-built and served as static files.
Best for speed and caching.

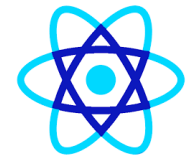
Different Approaches of Micro frontend Architecture Integration





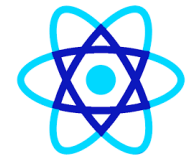
Server-Side Composition

- Micro frontends are **cached at Content Delivery Networks (CDNs)** for faster content delivery.
- The **server fetches** the required micro frontends from different sources.
- The server **combines (composes)** all micro frontends into a single web page.
- The **fully assembled page** is sent to the browser.
- Composition can occur:
 - **At build time** (Static Site Generation)
 - **At request time** (Server-Side Rendering)
- The browser receives a **complete page**, improving the initial user experience.
- Suitable for applications requiring **better SEO** and **faster first page load**.
- Reduces the amount of work performed by the browser.



Server-Side Composition

- The server is responsible for:
 - Gathering micro frontends
 - Arranging the page layout
 - Composing the final view
 - Delivering the finished page to users
- If pages are **highly personalized**, the server must handle many unique requests efficiently.
- Requires a **scalable server architecture** to support multiple concurrent users.
- CDNs improve performance by serving cached content from locations closer to users.
- Commonly used in **enterprise-scale micro frontend applications**.



Server-Side Composition

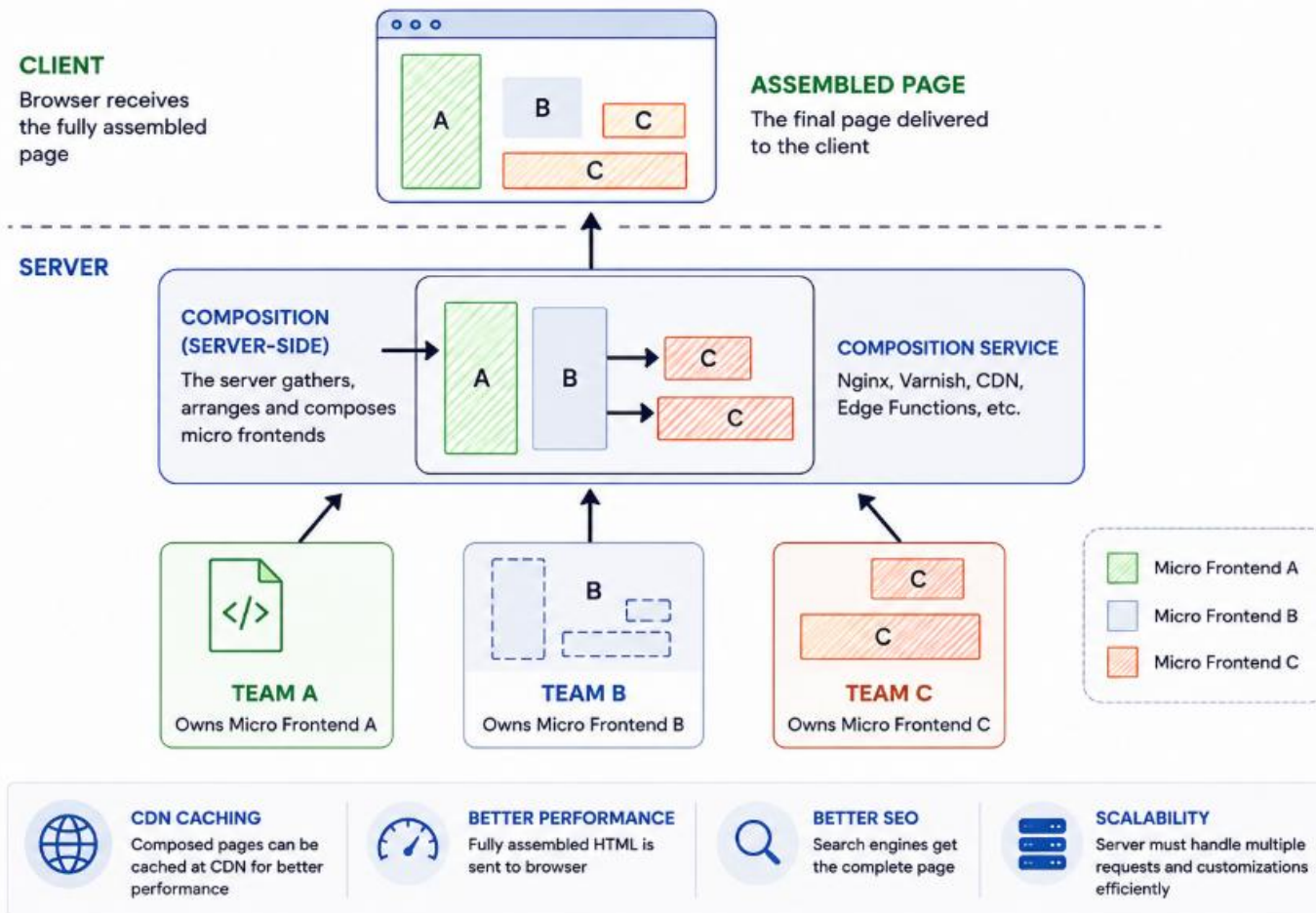
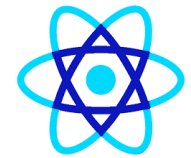
- **Key Advantages**

- Faster initial page load
- Better SEO
- Centralized page composition
- Reduced client-side processing
- CDN caching improves performance

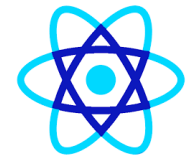
- **Challenges**

- Increased server complexity
- Scalability becomes critical
- Personalized pages reduce cache efficiency
- Higher server infrastructure cost

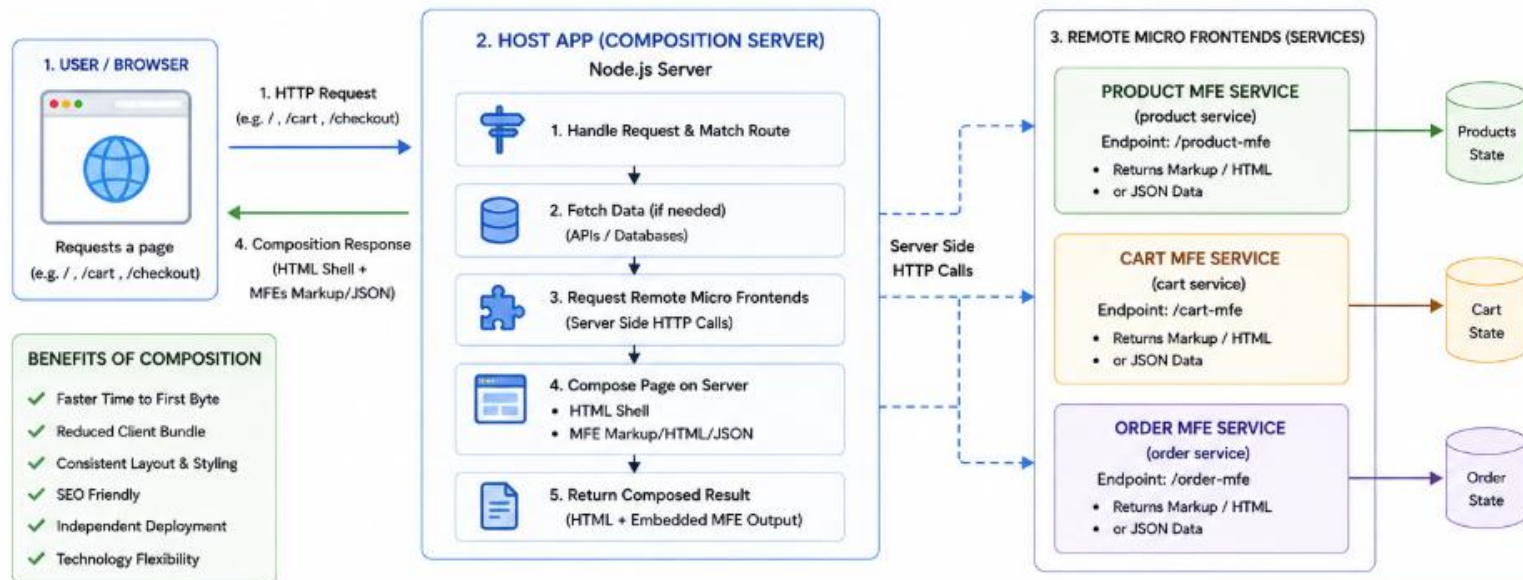
Server-Side Composition



Server-Side Composition



Server Side Composition (No Rendering) (with Host, Product, Cart and Order Micro Frontends)



ROUTING & COMPOSITION IN HOST



- / (Home)**
- Host matches route "/"
 - Calls Product MFE
 - Composes with Host layout
 - Returns Composed Result



- /cart**
- Host matches route "/cart"
 - Calls Cart MFE
 - Composes with Host layout
 - Returns Composed Result



- /checkout (Order)**
- Host matches route "/checkout"
 - Calls Order MFE
 - Composes with Host layout
 - Returns Composed Result

HOW SERVER SIDE COMPOSITION WORKS

1. User requests a route in the browser.
2. Request goes to Host Composition Server.
3. Host server calls required Micro Frontend services.
4. Host composes the final page (HTML Shell + MFE output).
5. Host sends the composed response to the browser.
6. Browser displays the page.

Note: No rendering is done on the server.

BUILD & DEPLOYMENT

Build Host App
(Composition Server)
Node.js

Deploy Product MFE
(Service)
e.g. Node.js / Vite SSR

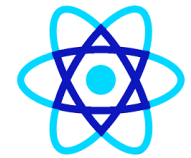
Deploy Cart MFE
(Service)
e.g. Node.js / Vite SSR

Deploy Order MFE
(Service)
e.g. Node.js / Vite SSR

Deploy All
(AWS / Azure / GCP / Docker)

TECH STACK (EXAMPLE)

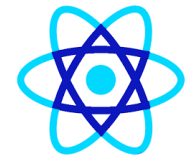
- Node.js (Host)
- React (UI)
- Axios / Fetch (Server Calls)
- Express.js (Server)
- Each MFE as Independent Service
- Returns HTML / JSON
- Any UI Library (React, Vue, etc.)



Server Side Rendering vs Composition

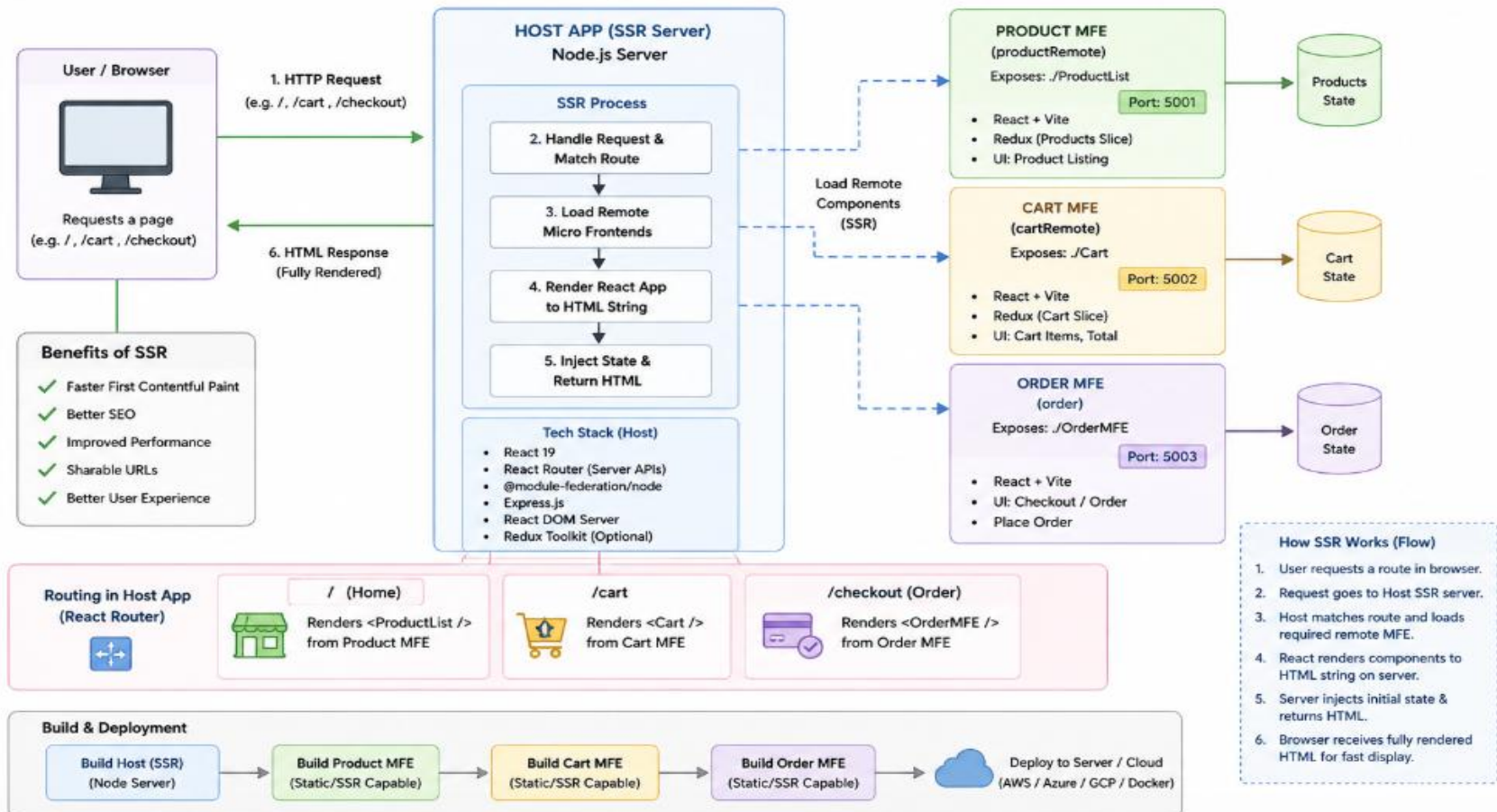
Concept	What it does
Server-Side Composition	Combines multiple MFEs on the server
Server-Side Rendering (SSR)	Converts React components into HTML on the server
Both together	Server composes the MFEs and renders the resulting application to HTML

Server-Side Rendering

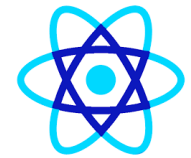


Server Side Rendering (SSR) with Host, Product, Cart and Order (Micro Frontends)

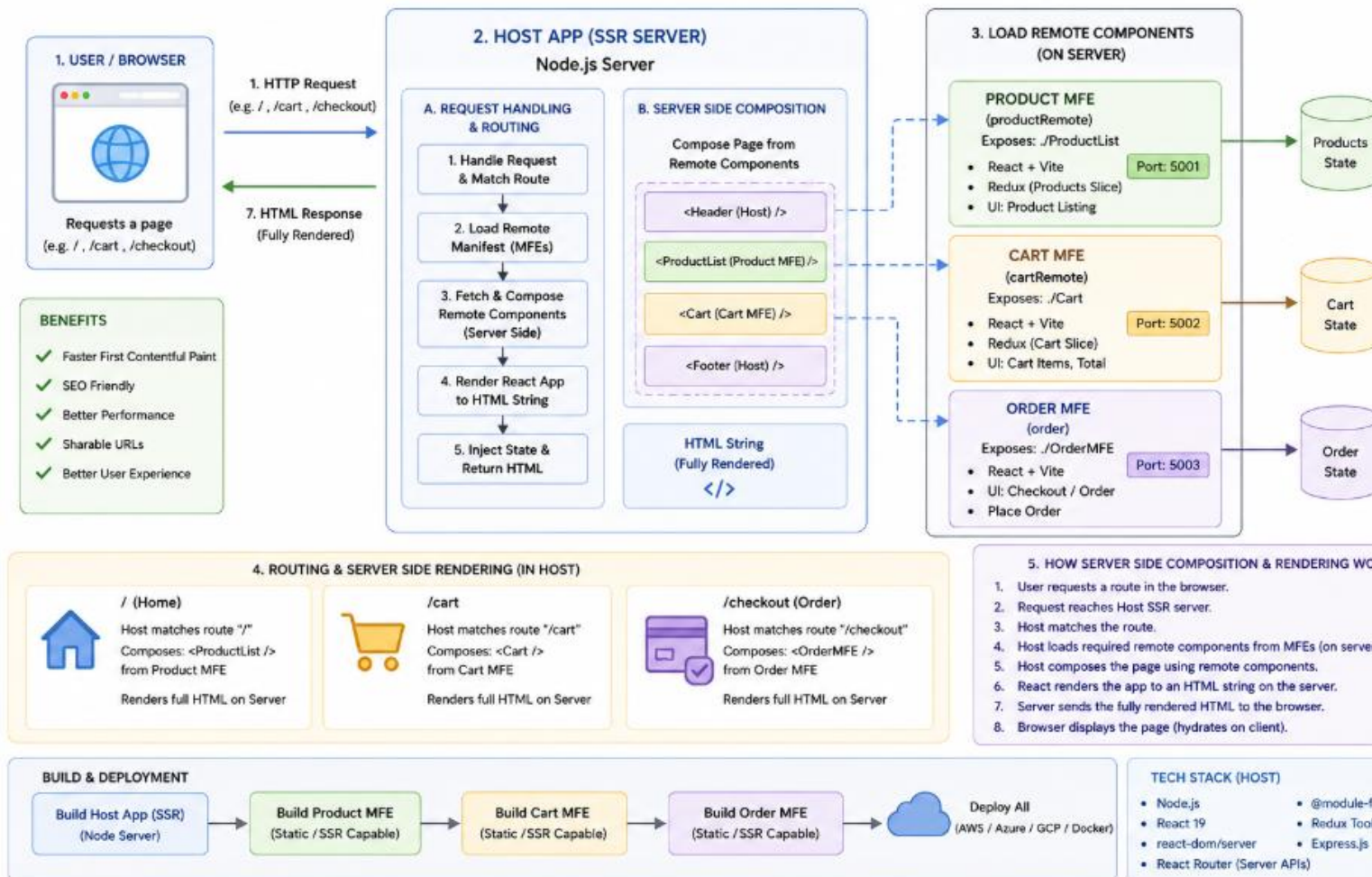
High Level Architecture & Flow



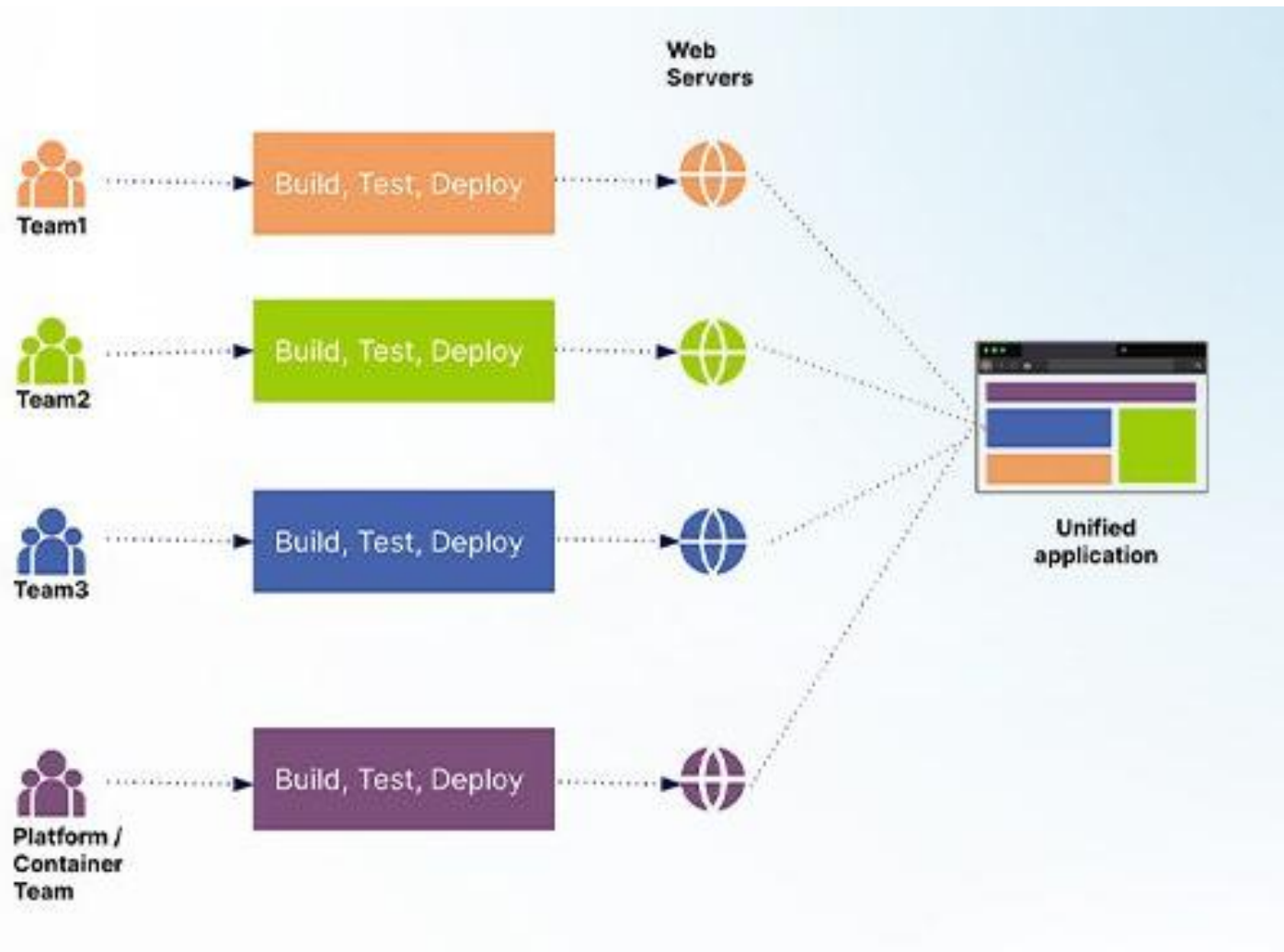
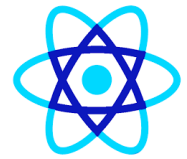
Server-Side Composition and Rendering



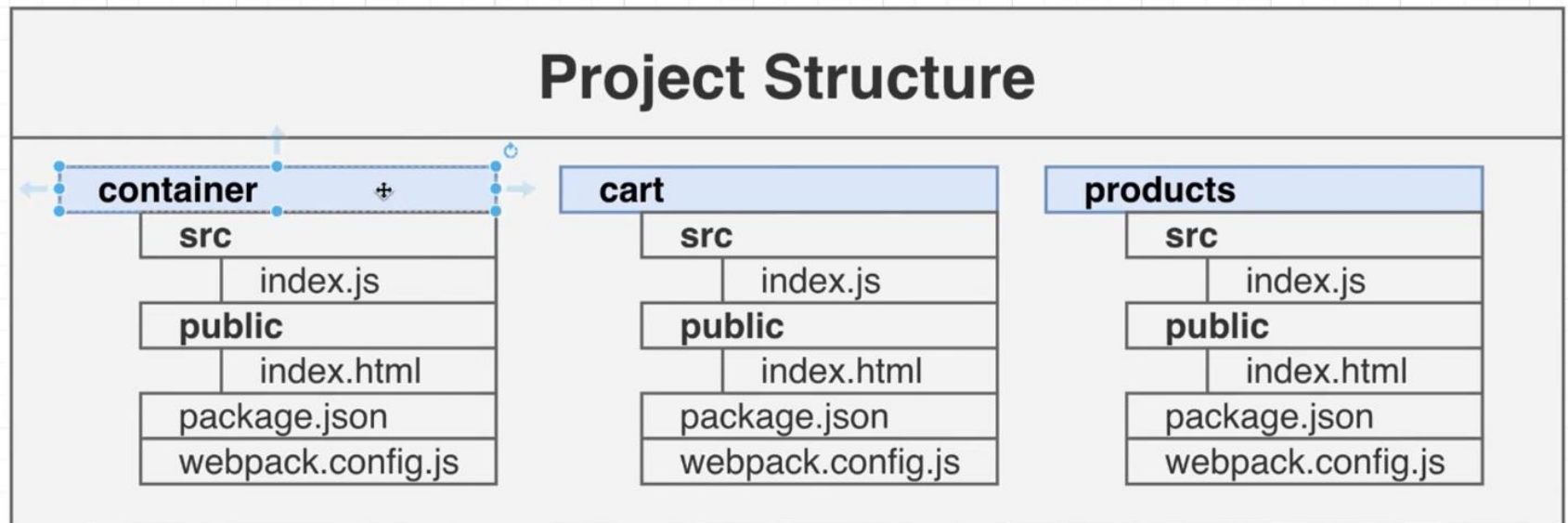
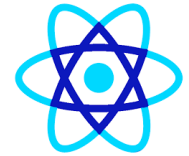
Server Side Composition & Rendering (SSR) (with Host, Product, Cart and Order Micro Frontends)

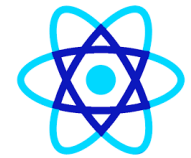


Run Time Integration



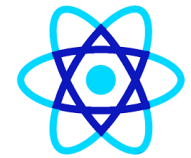
Run Time Integration





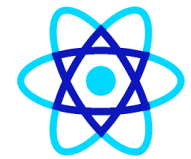
Run Time Integration

- **Runtime Integration** is a technique where micro frontends are **loaded and integrated while the application is running** in the browser.
- The **Shell (Host) application** dynamically loads the required micro frontends at runtime.
- Each team develops and deploys its **micro frontend independently**.
- Teams can work **in parallel** without affecting other teams.
- The shell application combines all micro frontends to provide a **single, seamless user experience**.
- No need to rebuild the entire application when one micro frontend changes.
- Supports **modular** and **scalable** frontend architecture.
- Micro frontends can be **updated independently** without redeploying the whole application.



Run Time Integration

- Runtime integration is commonly implemented using:
 - **JavaScript (Dynamic Imports / Module Federation)**
 - **Web Components**
 - **iFrames**
- The shell application is responsible for:
 - Loading micro frontends
 - Managing routing
 - Authentication
 - Communication between micro frontends
 - Shared state (if required)



Run Time Integration

Microfrontends are integrated at runtime in the browser.



1. RUNTIME VIA JAVASCRIPT

Dynamic loading and composition using JavaScript

Microfrontends are loaded and composed in the user's browser using JavaScript. This approach provides flexibility but can increase the initial page load time.

KEY POINTS

- ✓ Uses JavaScript (dynamic imports, Module Federation, etc.)
- ✓ Highly flexible and dynamic
- ✓ Shared dependencies possible
- ✓ May increase initial load time



2. RUNTIME VIA IFRAMES

Isolation using iFrames

Each microfrontend is encapsulated in an iFrame, which isolates their JavaScript and styles. Communication between microfrontends can be challenging but offers strong isolation.

KEY POINTS

- ✓ Each microfrontend runs in its own iFrame
- ✓ Strong isolation (JS, CSS, DOM)
- ✓ Technology agnostic
- ✓ Cross-domain integration possible
- ✓ Communication is more complex



3. RUNTIME VIA WEB COMPONENTS

Encapsulation using Web Components

Microfrontends are built as web components, which can be dynamically loaded and rendered within a container application. Web components allow for encapsulation and reusability.

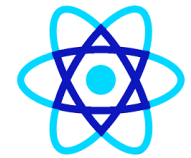
KEY POINTS

- ✓ Uses Web Components (Custom Elements)
- ✓ Encapsulation with Shadow DOM
- ✓ Reusable and framework independent
- ✓ Smaller bundle size
- ✓ Good balance of isolation and integration



NOTE

All these approaches enable independent development and deployment of microfrontends, while delivering a seamless user experience through the shell (host) application.



Build Time Integration

- **Build-Time Integration** is a technique where micro frontends are integrated **during the build/compile process**.
- Each micro frontend is **packaged as a library** and installed into the **Shell (Container) application**.
- The container application bundles all micro frontends into a **single deployable JavaScript bundle**.
- Integration happens **before the application is deployed**, not at runtime.
- Common dependencies (React, Angular, Vue, etc.) can be **shared and deduplicated**, reducing the overall bundle size.
- Frontend teams can develop micro frontends independently before they are integrated.
- The entire application is compiled into a **single production build**.
- If any micro frontend changes, the **container application must be rebuilt and redeployed**.



Build Time Integration

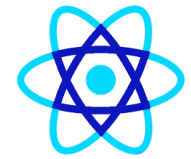
- Every new feature or component release requires:
 - Updating the micro frontend library
 - Rebuilding the container application
 - Redeploying the entire application
- Suitable for applications with **less frequent releases** and **stable components**.



Build Time Integration

- **Advantages**

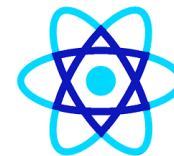
- Simple architecture
- Faster application startup
- Shared dependencies reduce bundle size
- Optimized performance
- Easier dependency management
- Single deployment artifact



Build Time Integration

- **Challenges**

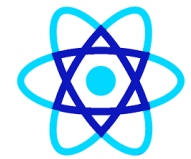
- Requires rebuilding the entire application for every change
- Slower release cycle
- Tightly coupled deployment process
- Less flexibility compared to runtime integration
- Independent deployment is not possible



Build Time Integration

- **Common Technologies**

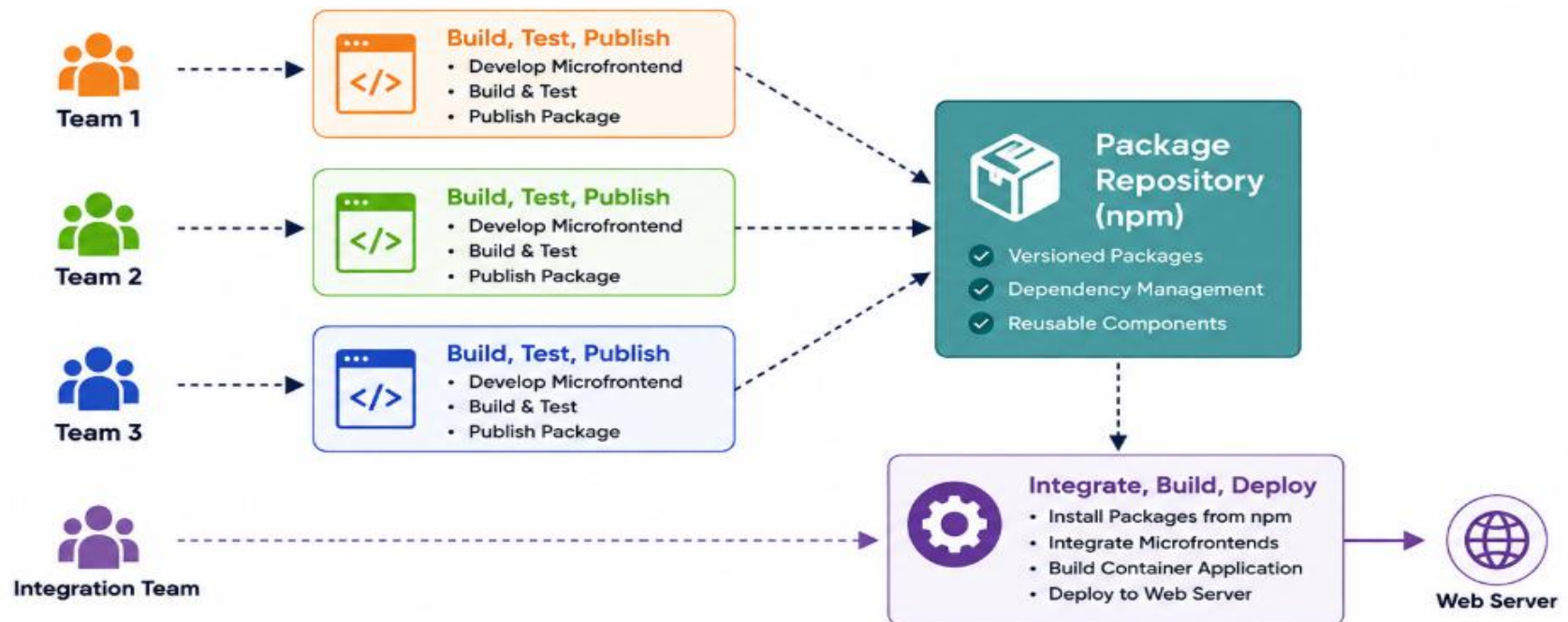
- npm Packages
- Yarn Workspaces
- Monorepo (Nx, Turborepo, Lerna)
- Webpack
- Vite



Build Time Integration

BUILD-TIME INTEGRATION WORKFLOW

Microfrontends are built, tested, published as packages, then integrated into the container application at build time.



KEY BENEFITS



Shared & Deduped Dependencies

Common dependencies are deduplicated in the final bundle.



Optimized Bundle Size

Smaller bundle size due to shared libraries.



Proven & Stable Approach

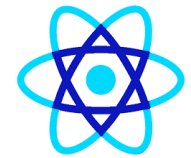
Simple architecture with predictable build process.



Rebuild Required for Changes

Any change in a microfrontend requires rebuilding & redeploying the container application.

Build Time Integration



Micro Frontends – Container & Micro Frontends

The container (shell) application decides when and where to load each micro frontend and composes the final page.



How it works?

1 User Opens App

User opens the e-commerce application

2 Container Decides

Container (Shell) decides which micro frontends to load and where to display them

3 Loads Micro Frontends

Container loads MFE #1 (ProductsList) and MFE #2 (Cart)

4 Composes Page

Micro frontends are rendered in their respective sections

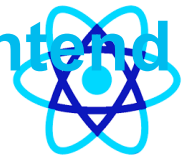
5 Final Page

Complete e-commerce page is shown to the user

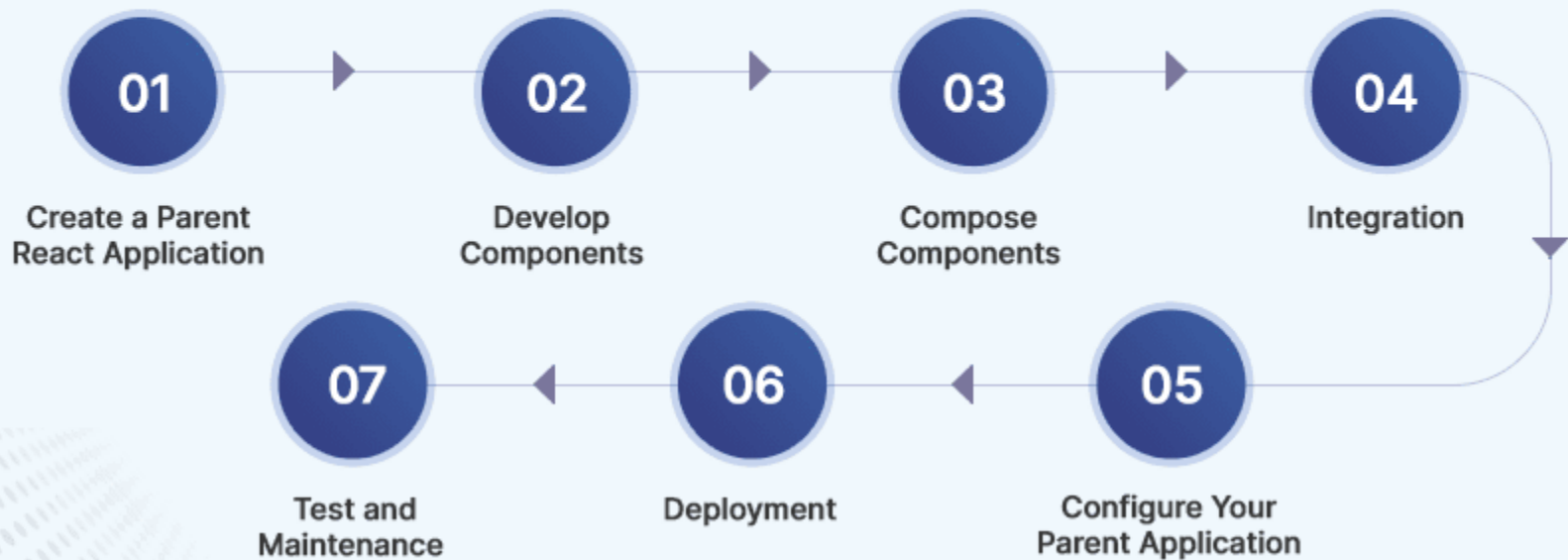
Key Benefits

- ✓ Independent development
- ✓ Independent deployment
- ✓ Modular and scalable
- ✓ Reusable micro frontends
- ✓ Better team productivity

Step-by-step guide: Building React Micro Frontend Architecture



7 Step Process of Micro Frontend Development with React

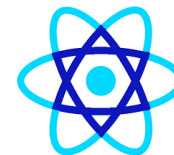




MFE Runtime Integration

Important! We must install slightly newer versions of these libraries to avoid errors. The updated command that should be run is:

```
npm install webpack@5.88.0 webpack-cli@4.10.0 webpack-dev-server@4.7.4  
faker@5.1.0 html-webpack-plugin@5.5.0 --save-exact
```



MFE Runtime Integration

```
{  
  "name": "products",  
  "version": "1.0.0",  
  "description": "",  
  "main": "index.js",  
  "scripts": {  
    "start": "webpack"  
  },  
  "keywords": [],  
  "author": "",  
  "license": "ISC",  
  "dependencies": {  
    "faker": "^5.1.0",  
    "html-webpack-plugin": "5.5.0",  
    "nodemon": "^2.0.15",  
    "webpack": "^5.88.0",  
    "webpack-cli": "4.10.0",  
    "webpack-dev-server": "4.7.4"  
  }  
}
```




MFE Runtime Integration

```
Run `npm audit` for details.
```

```
E:\mfejul2024\ecomm\products>npm run start
```

```
> products@1.0.0 start
```

```
> webpack serve
```

```
<i> [webpack-dev-server] Project is running at:
```

```
<i> [webpack-dev-server] Loopback: http://localhost:8081/
```

```
<i> [webpack-dev-server] On Your Network (IPv4): http://192.168.29.50:8081/
```

```
<i> [webpack-dev-server] On Your Network (IPv6): http://[fe80::3346:77d3:beb6:ac89]:8081/
```

```
<i> [webpack-dev-server] Content not from webpack is served from 'E:\mfejul2024\ecomm\products\public' directory
```

```
asset main.js 3.37 MiB [emitted] (name: main)
```

```
runtime modules 27.4 KiB 12 modules
```

```
modules by path ./node_modules/faker/lib/locales/ 1.93 MiB 1468 modules
```

```
modules by path ./node_modules/faker/lib/*.js 133 KiB 27 modules
```

```
modules by path ./node_modules/webpack-dev-server/client/ 56.8 KiB 12 modules
```

```
modules by path ./node_modules/webpack/hot/*.js 5.18 KiB 4 modules
```

```
modules by path ./node_modules/html-entities/lib/*.js 78.9 KiB
```

```
./node_modules/html-entities/lib/index.js 4.84 KiB [built] [code generated]
```

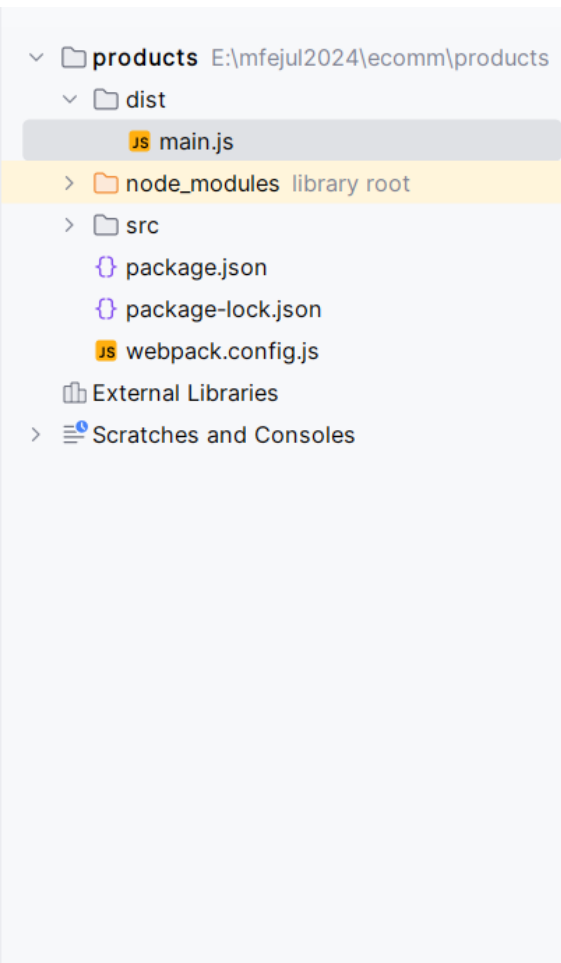
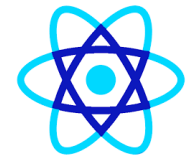
```
+ 3 modules
```



MFE Runtime Integration

```
ucts\public' directory
asset main.js 3.37 MiB [emitted] (name: main)
runtime modules 27.4 KiB 12 modules
modules by path ./node_modules/faker/lib/locales/ 1.93 MiB 1468 modules
modules by path ./node_modules/faker/lib/*.js 133 KiB 27 modules
modules by path ./node_modules/webpack-dev-server/client/ 56.8 KiB 12 modules
modules by path ./node_modules/webpack/hot/*.js 5.18 KiB 4 modules
modules by path ./node_modules/html-entities/lib/*.js 78.9 KiB
  ./node_modules/html-entities/lib/index.js 4.84 KiB [built] [code generated]
+ 3 modules
modules by path ./node_modules/faker/vendor/*.js 21.1 KiB
  ./node_modules/faker/vendor/unique.js 2.79 KiB [built] [code generated]
+ 2 modules
modules by path ./node_modules/faker/lib/image_providers/*.js 10.9 KiB
  ./node_modules/faker/lib/image_providers/lorempixel.js 5.11 KiB [built] [code generated]
+ 2 modules
+ 4 modules
webpack 5.93.0 compiled successfully in 3174 ms
```

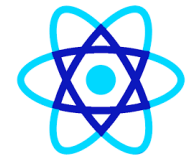
MFE Runtime Integration



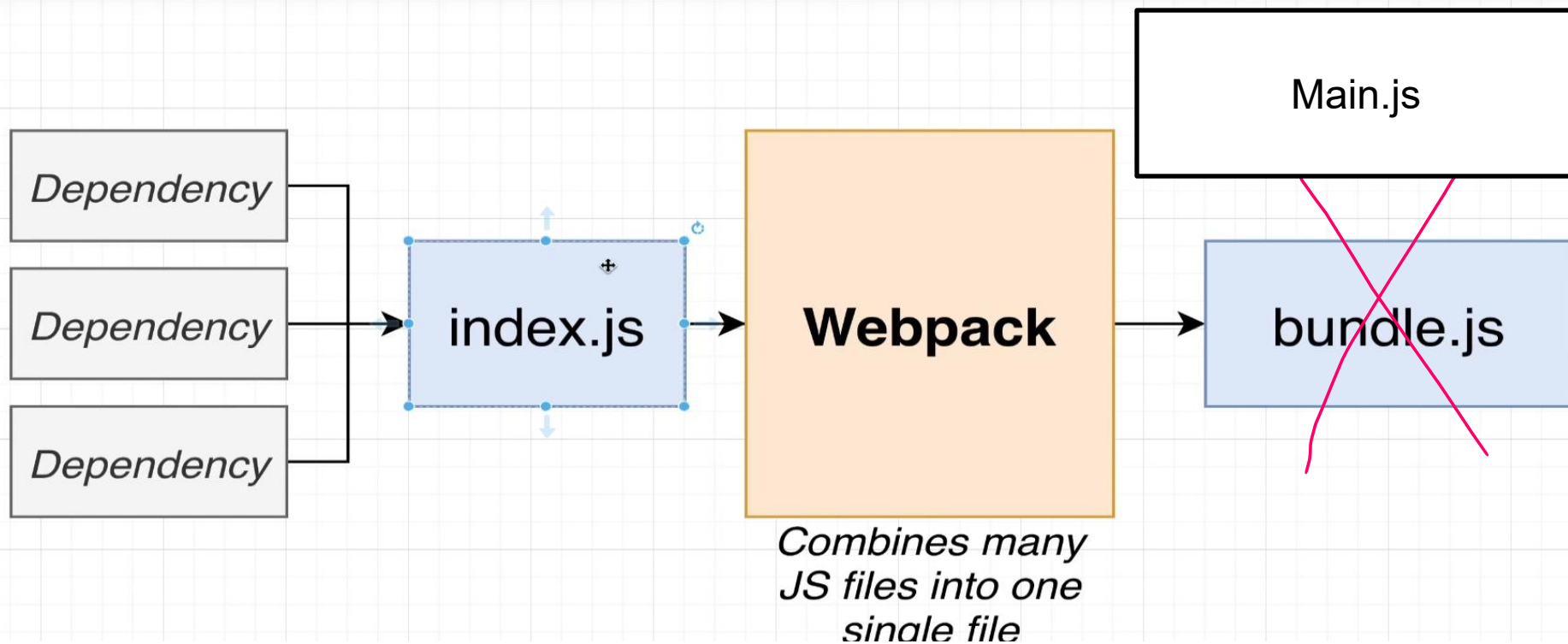
Visual layout of bidirectional text can depend on the base direction (View | Bidi Text Base Direction)

This document contains very long lines. Soft wraps were enabled to improve editor performance.

```
1  /*
2  ⚠ ATTENTION: The "eval" devtool has been used (maybe by default in mode: "development")
3  * This devtool is neither made for production nor for readable output files.
4  * It uses "eval()" calls to create a separate source file in the browser devtools.
5  * If you are trying to read the output file, select a different devtool (https://webpack.js.org/configuration/devtool/#mode)
6  * or disable the default devtool with "devtool: false".
7  * If you are looking for production-ready output files, see mode: "production" (https://webpack.js.org/configuration/devtool/#mode)
8  */
9  /*****/ (() => { // webpackBootstrap
10 /*****/   var __webpack_modules__ = ({
11
12   /**/   "./node_modules/faker/index.js":
13   /**/   /*!*****!*\
14     !*** ./node_modules/faker/index.js ***!
15     \*****/
16   /**/   ((module, __unused_webpack_exports, __webpack_require__) => {
17
18     eval("// since we are requiring the top level of faker, load all locales by default\n
    ./lib */ \"../node_modules/faker/lib/index.js\");\nvar faker = new Faker({ locales: [\n
    \"../node_modules/faker/lib/locales.js\"] });\nmodule['exports'] = faker;\n\n// source\n
    ./node_modules/faker/index.js?");
```

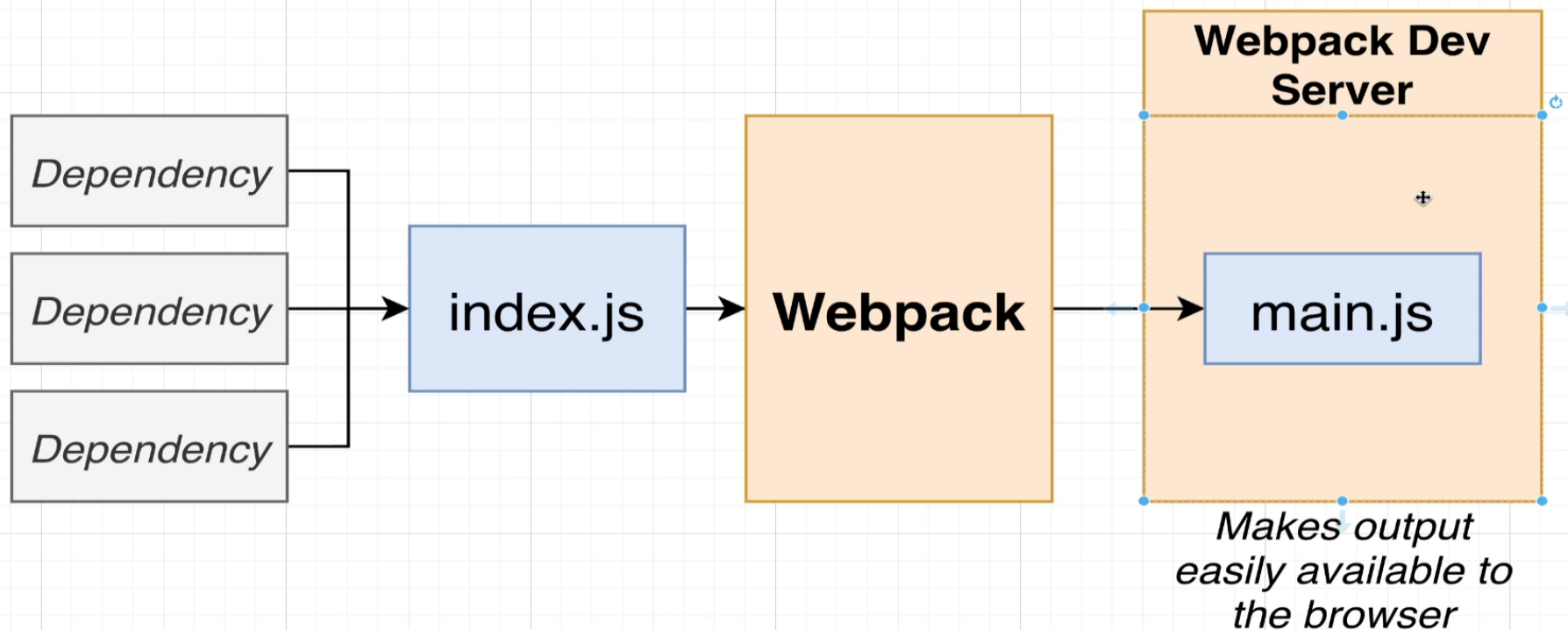


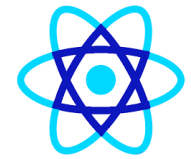
MFE Runtime Integration



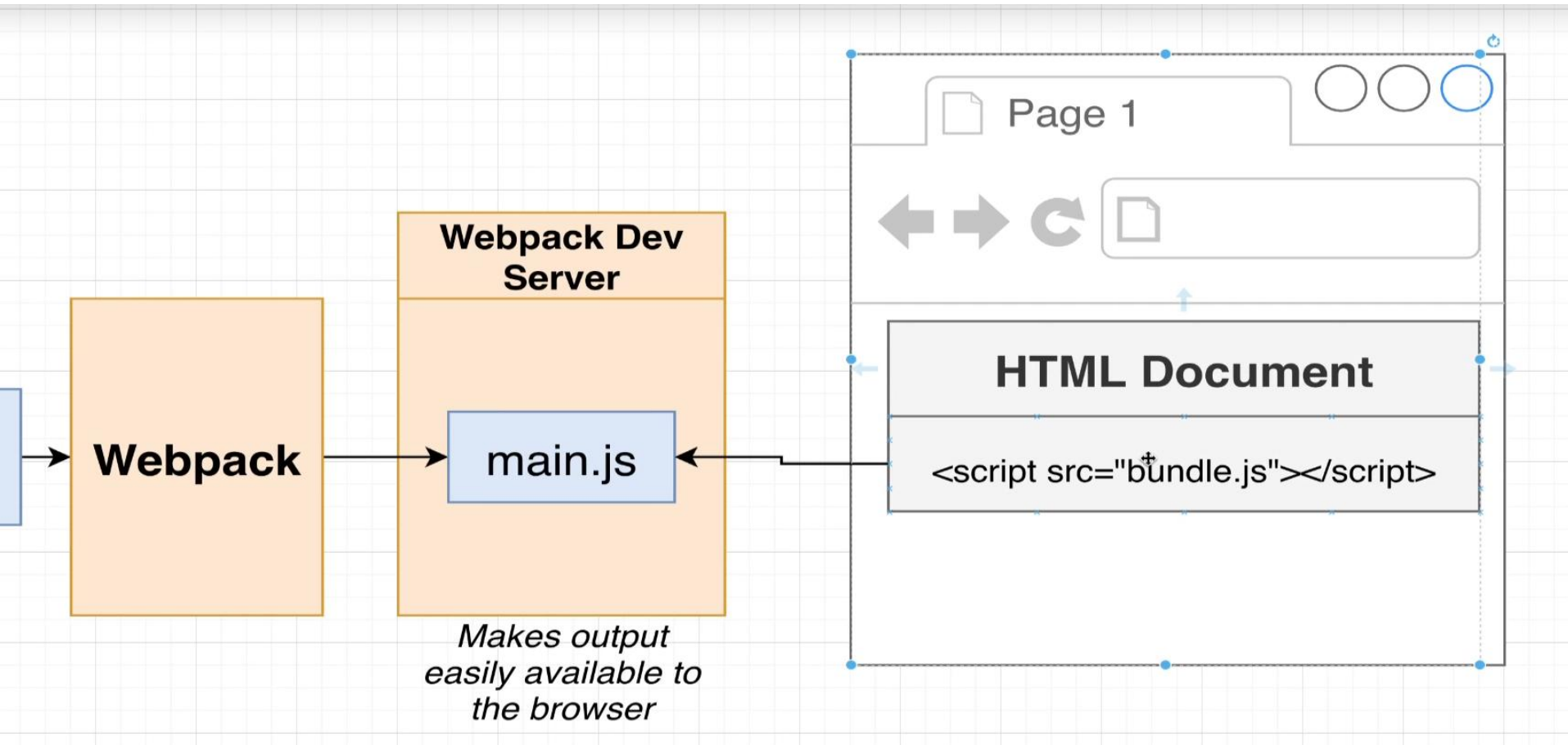


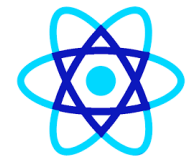
MFE Runtime Integration



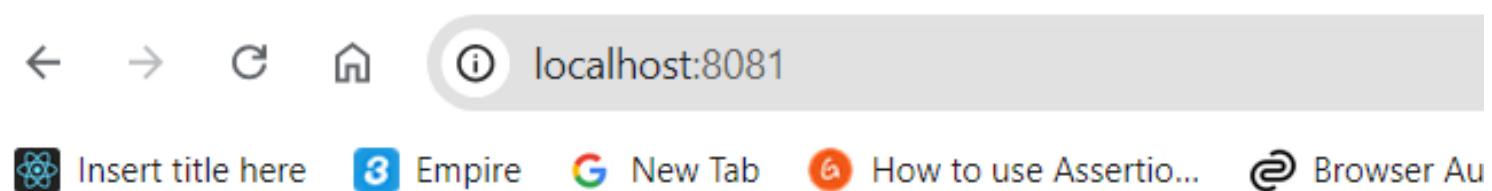


MFE Runtime Integration

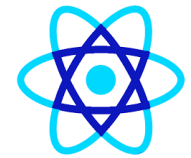




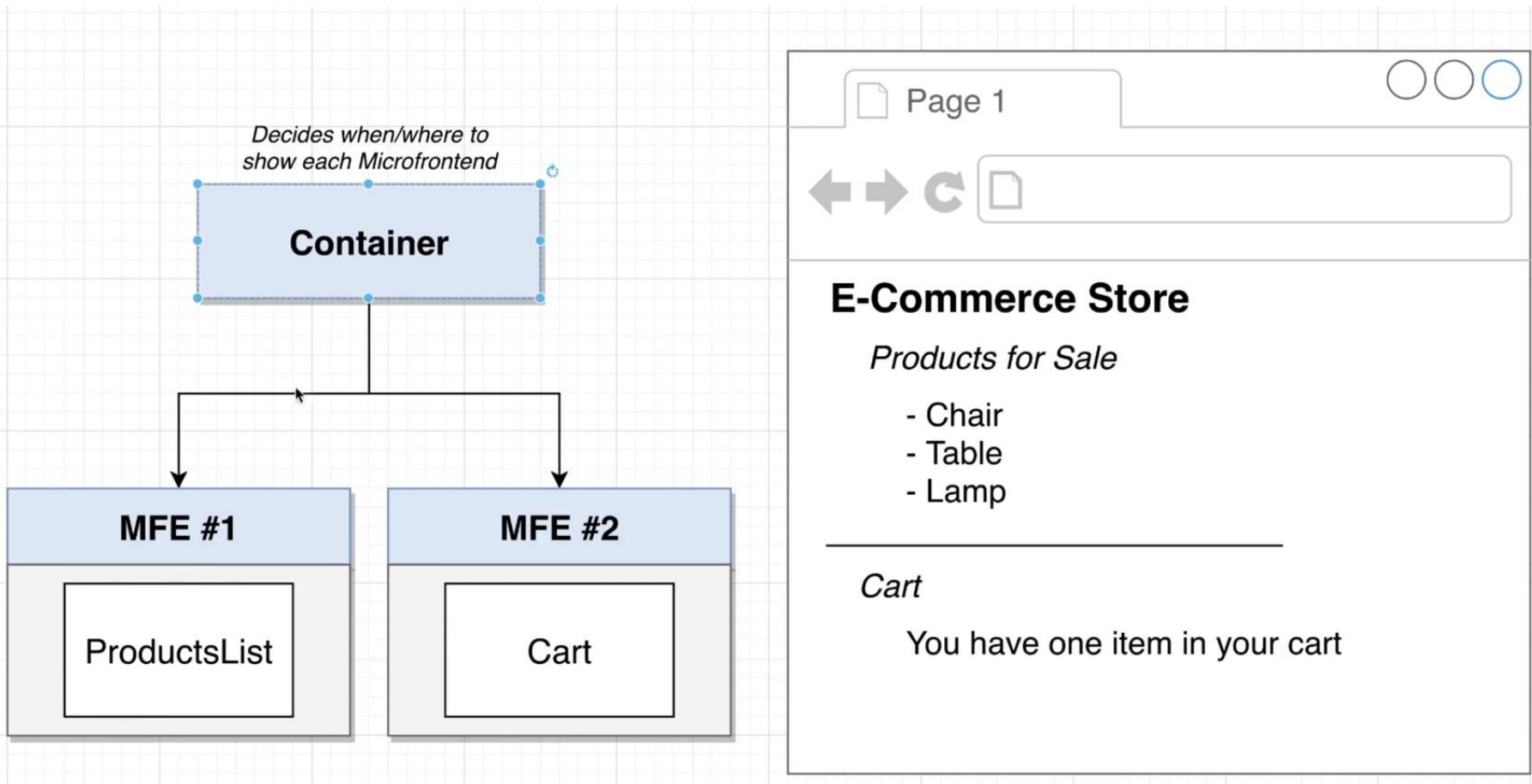
MFE Runtime Integration



Refined Steel Car
Generic Concrete Shoes
Incredible Steel Computer



MFE Runtime Integration

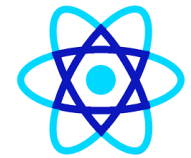




MFE Runtime Integration

Important! We must install slightly newer versions of these libraries to avoid errors. The updated command that should be run is:

```
npm install webpack@5.88.0 webpack-cli@4.10.0 webpack-dev-server@4.7.4  
html-webpack-plugin@5.5.0 nodemon --save-exact
```



MFE Runtime Integration

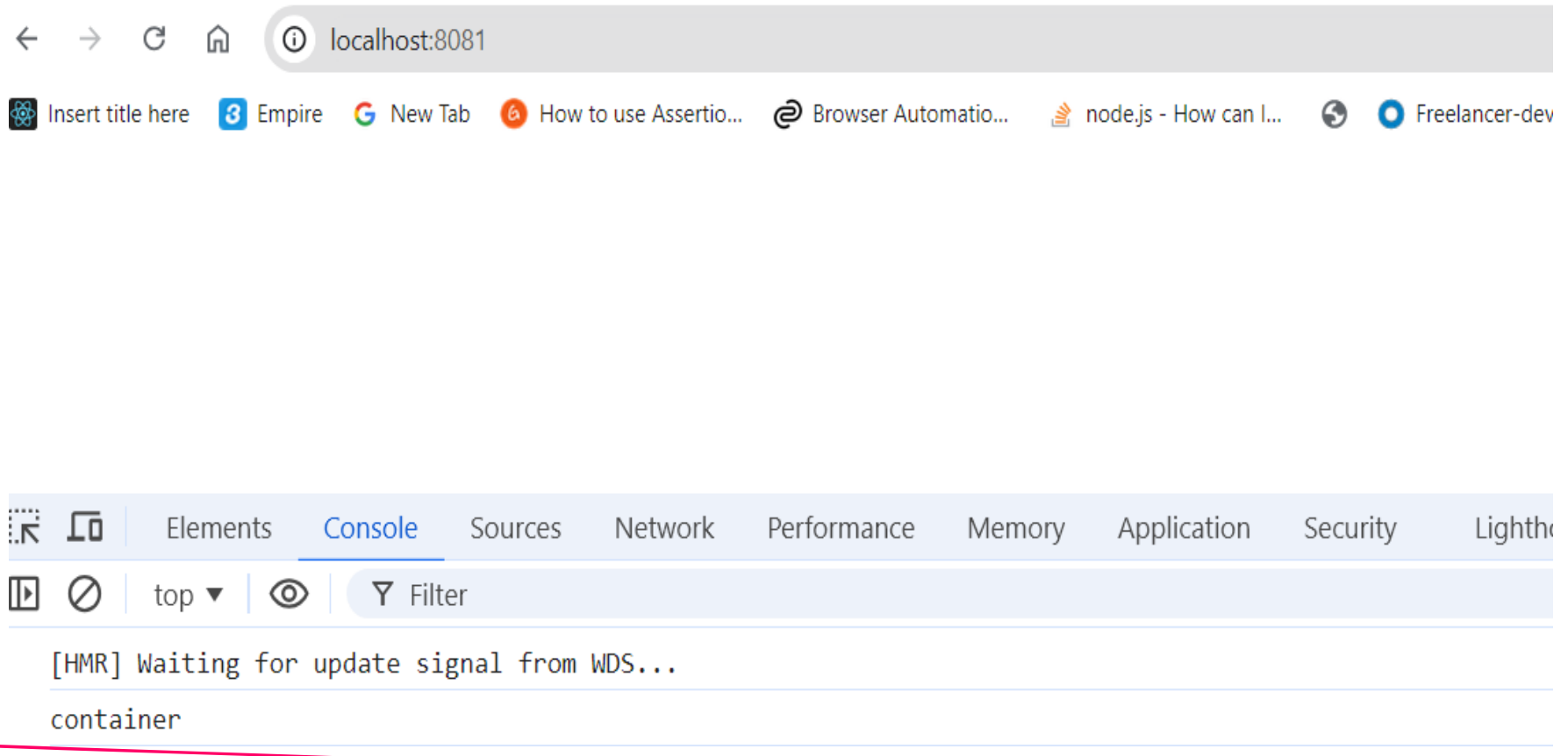
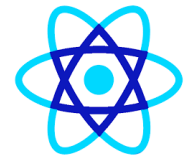
Project ▾

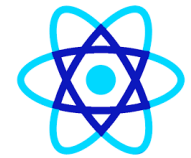
- ▾ ecomm E:\mfejul2024\ecomm
 - ▾ container
 - > node_modules library root
 - ▾ public
 - <> index.html
 - ▾ src
 - JS index.js
 - package.json
 - package-lock.json
 - JS webpack.config.js

JS index.js x <> index.html JS webpack.config.js package.json

```
1 console.log('container')
```

MFE Runtime Integration





Implementing Module Federation

Diagram ▾ Shapes Format Insert Delete ↶ ↷ [] 🔍 🔍 All changes saved

Designate one app as the Host and one as the Remote

In the Remote, decide which modules (files) you want to make available to other projects

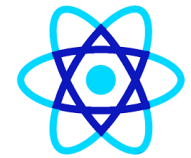
Set up Module Federation plugin to *expose* those files

In the Host, decide which files you want to get from the remote

Set up Module Federation plugin to fetch those files

In the Host, refactor the entry point to load asynchronously

In the Host, import whatever files you need from the remote



Implementing Module Federation

Designate one app as the Host (CONTAINER) and one as the Remote (PRODUCTS)

In the Remote, decide which modules (files) you want to make available to other projects

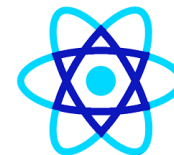
Set up Module Federation plugin to *expose* those files

In the Host, decide which files you want to get from the remote

Set up Module Federation plugin to fetch those files

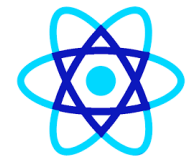
In the Host, refactor the entry point to load asynchronously

In the Host, import whatever files you need from the remote

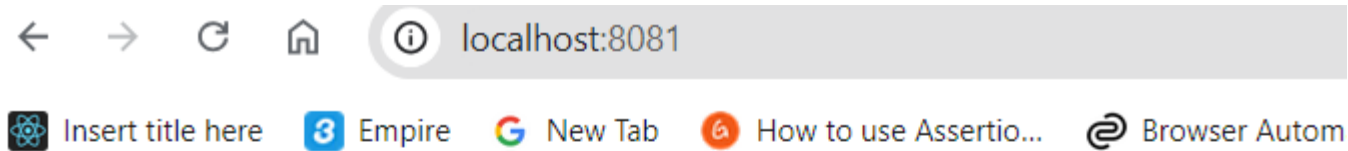


Implementing Module Federation

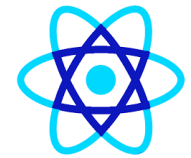
```
const HtmlWebpackPlugin = require("html-webpack-plugin");
const ModuleFederationPlugin=require('webpack/lib/container/ModuleFederationPlugin')
module.exports={
  mode:'development',
  devServer: {
    port:8082
  },
  plugins: [
    new ModuleFederationPlugin({
      name:'products',
      filename:'remoteEntry.js',
      exposes:{
        './ProductsIndex': './src/index',
      },
    }),
    new HtmlWebpackPlugin( options: {
      template: "./public/index.html"
    })
  ]
};
```



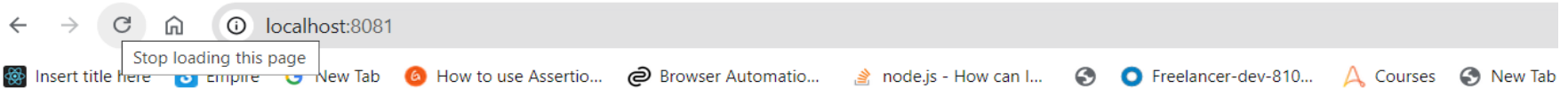
Implementing Module Federation



Awesome Soft Soap
Awesome Concrete Fish
Refined Granite Keyboard

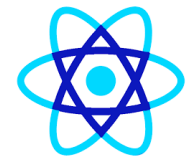


Implementing Module Federation



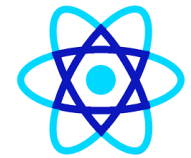
Small Steel Sausages
Handcrafted Soft Keyboard
Small Concrete Cheese

Elements Console Sources Network Performance Memory Application Security Lighthouse Recorder Perform			
⏏ ⌵ 🔍 ☐ Preserve log ☐ Disable cache No throttling ⌵ 📶 ⬆ ⬇			
🔍 Filter ☐ Invert ☐ Hide data URLs ☐ Hide extension URLs All Fetch/XHR Doc CSS JS Font Img Media M			
☐ Blocked requests ☐ 3rd-party requests			
100 ms 200 ms 300 ms 400 ms 500 ms 600 ms 700 ms			
Name	Status	Type	Initiator
main.js	304	script	(index):5
remoteEntry.js	200	script	main.js:445
src_bootstrap.js.js	304	script	main.js:445
vendors-node_modules_faker_index.js.js	200	script	remoteEntry.js:423
src_index.js.js	200	script	remoteEntry.js:423
injected-script.js	200	script	content_script.js:5
content_script_vite-148e3621.js	200	script	content_script_vite.js:1
prompt.js	200	script	playback.js:1

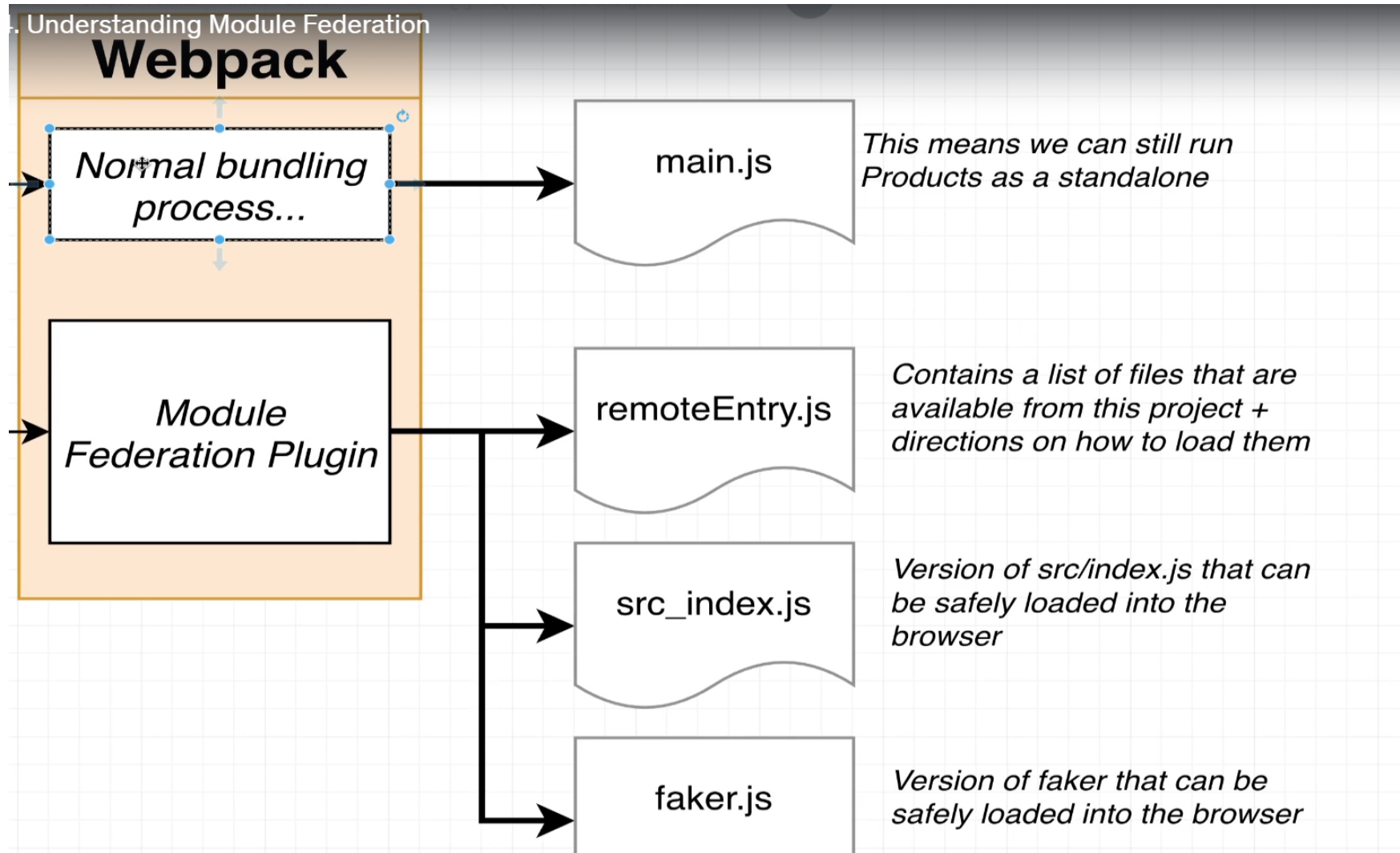


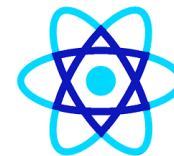
Implementing Module Federation

Name	Url	Status	Type
 main.js	http://localhost:8081/main.js	304	script
 remoteEntry.js	http://localhost:8082/remoteEntry.js	200	script
 src_bootstrap.js.js	http://localhost:8081/src_bootstrap_j...	304	script
 vendors-node_modules_faker_index_js.js	http://localhost:8082/vendors-node_...	200	script
 src_index_js.js	http://localhost:8082/src_index_js.js	200	script
 injected-script.js	chrome-extension://hmmohkncibpkl...	200	script
 content_script_vite-148e3621.js	chrome-extension://idgpnmonknjno...	200	script
 prompt.js	chrome-extension://mooikfkahbdckl...	200	script



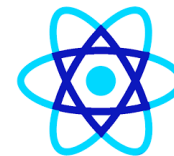
Implementing Module Federation





Webpack – Bootstrap.js

- Using bootstrap.js with Webpack involves bundling Bootstrap's JavaScript code with your project, allowing you to modularize and optimize the loading of Bootstrap's components.
- Webpack is a module bundler that helps manage dependencies and assets in modern front-end applications.
- When working with Bootstrap and Webpack, you can either load only the Bootstrap components you need or include the full bootstrap.js library.



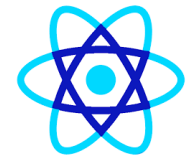
Webpack – Bootstrap.js

- Using bootstrap.js with Webpack involves bundling Bootstrap's JavaScript code with your project, allowing you to modularize and optimize the loading of Bootstrap's components.
- Webpack is a module bundler that helps manage dependencies and assets in modern front-end applications.
- When working with Bootstrap and Webpack, you can either load only the Bootstrap components you need or include the full bootstrap.js library.

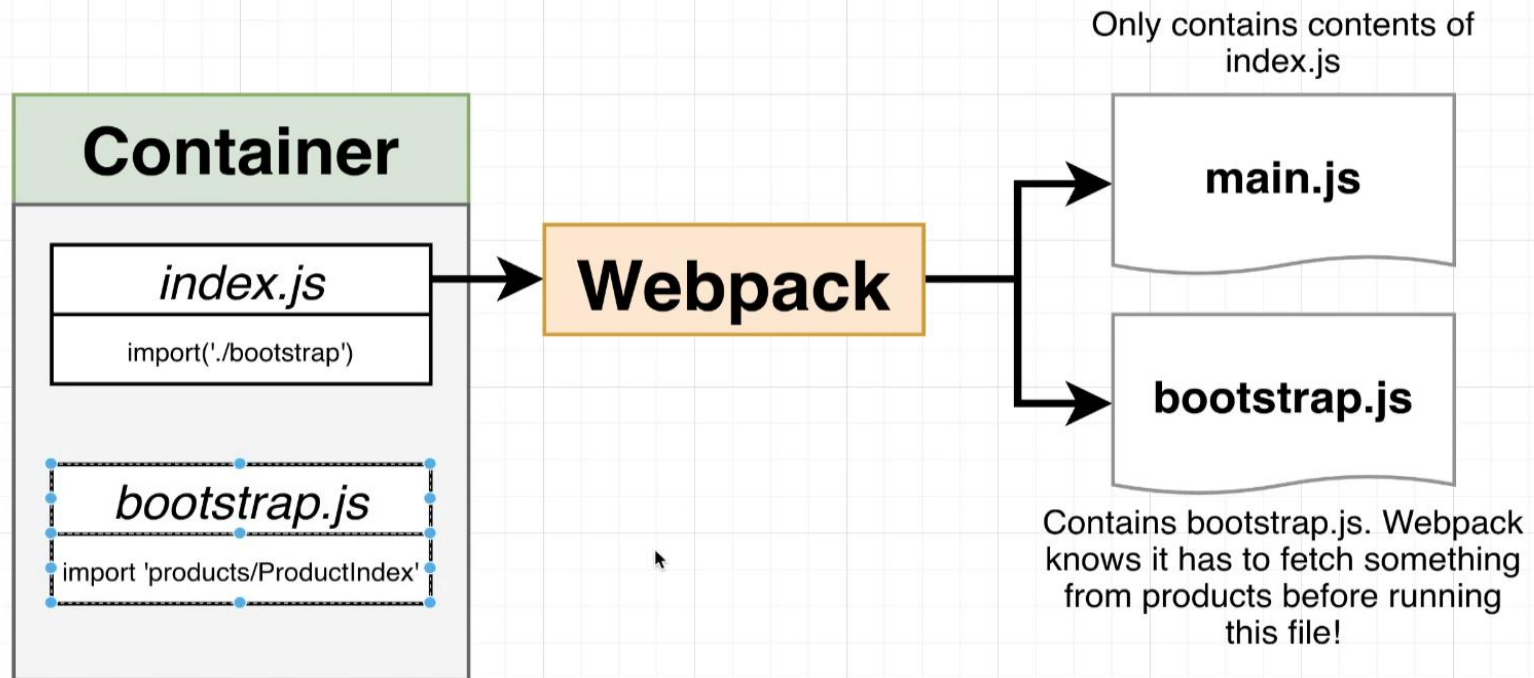


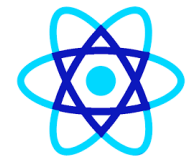
Webpack – Bootstrap.js

- Webpack Module Federation is a feature introduced in Webpack 5 that allows multiple, independently built and deployed JavaScript applications (or "modules") to share code and dependencies between them.
- When working with Bootstrap.js in a micro frontend architecture using Webpack Module Federation, you can share Bootstrap's JavaScript functionality across different applications (or micro frontends) without duplicating it in every build, thus optimizing load time and reducing redundancy.

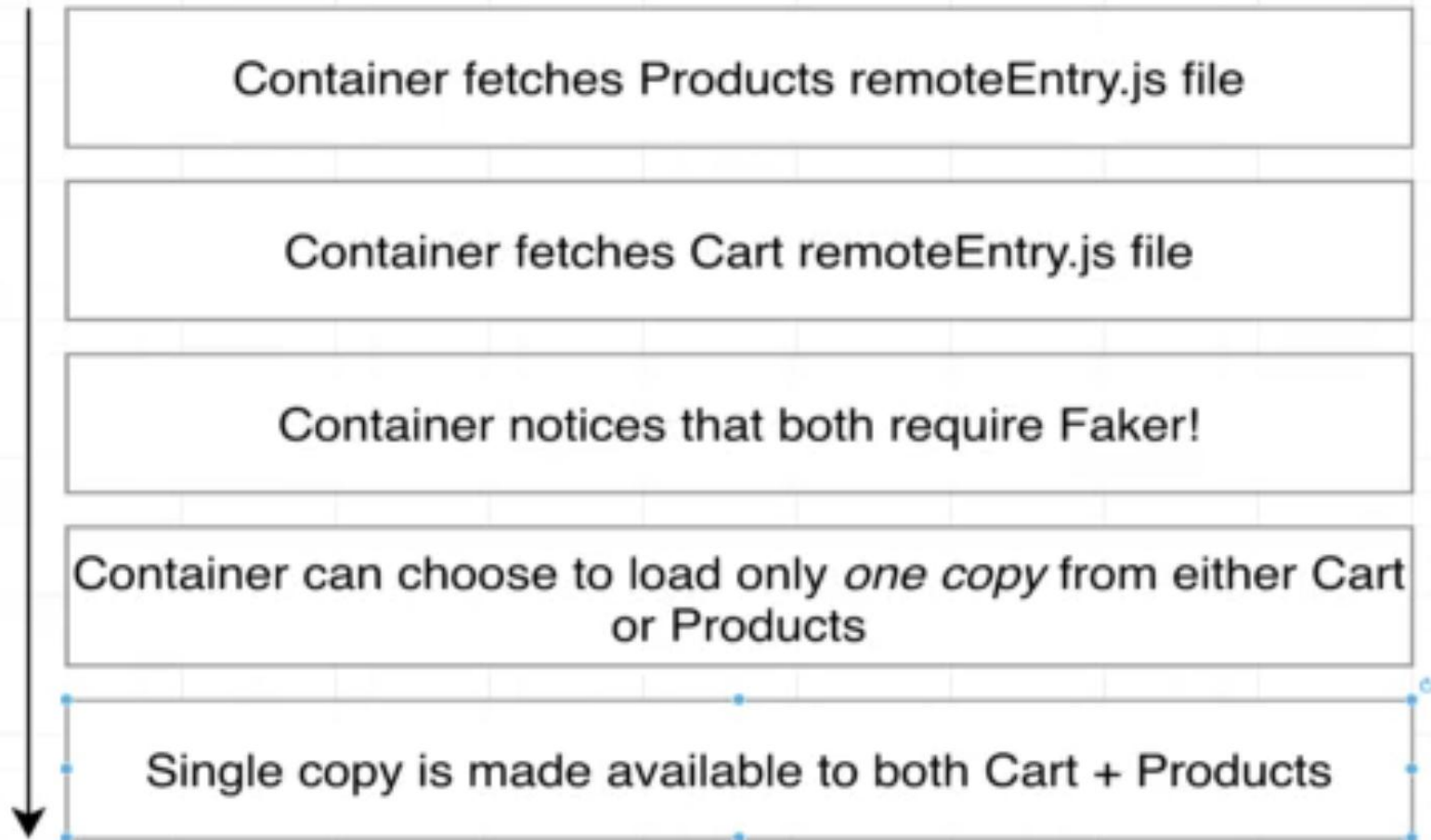


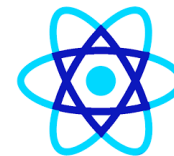
Implementing Module Federation





Shared Modules

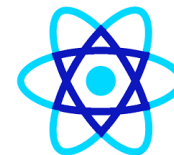




Shared Modules

```
const HtmlWebpackPlugin = require("html-webpack-plugin");
const ModuleFederationPlugin=require('webpack/lib/container/ModuleFederationPlugin')
module.exports={
  mode:'development',
  devServer: {
    port:8082
  },
  plugins: [
    new ModuleFederationPlugin({
      name:'products',
      filename:'remoteEntry.js',
      exposes:{
        './ProductsIndex': './src/index',
      },
      shared:["faker"]
    }),

    new HtmlWebpackPlugin( options: {
      template: "./public/index.html"
    })
  ]
};
```

Shared Modules

```
const HtmlWebpackPlugin = require("html-webpack-plugin");
const ModuleFederationPlugin=require('webpack/lib/container/ModuleFederationPlugin')
module.exports={
  mode:'development',
  devServer: {
    port:8083
  },
  plugins: [
    new ModuleFederationPlugin({
      name:'cart',
      filename:'remoteEntry.js',
      exposes:{
        './CartShow': './src/index',
      },
      shared:["faker"]
    }),
    new HtmlWebpackPlugin( options: {
      template: "./public/index.html"
    })
  ]
};
```

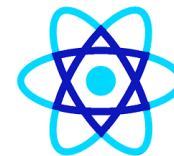


Shared Modules

[HMR] Waiting for update signal from WDS...

✖ ▶ Uncaught

```
Error: Shared module is not available for eager consumption: webpack/sharing/consume/default/faker/faker
  at __webpack_require__.m.<computed> (main.js:1036:54)
  at __webpack_require__ (main.js:282:32)
  at fn (main.js:549:21)
  at eval (index.js:2:63)
  at ./src/index.js (main.js:255:1)
  at __webpack_require__ (main.js:282:32)
  at main.js:1660:37
  at main.js:1662:12
```



Shared Modules

```
import('./bootstrap')
```

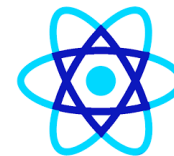


```
import faker from 'faker'
```

```
const cartText : string = `<div>You have ${faker.random.number()} items in the cart</div>`;
```



```
document.querySelector(selectors: "#dev-cart").innerHTML=cartText;
```



Shared Modules

🔍 | 🔇 | top ▼ | 👁 | Filter

[webpack-dev-server] Live Reloading enabled.

[webpack-dev-server] Hot Module Replacement enabled.

[webpack-dev-server] Live Reloading enabled.

chrome content script has been inserted

Start ajax tracing with delay of '0'.

start with href <http://localhost:8082/>

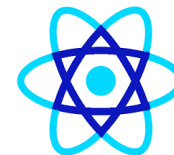
indicate that add on has been installed

check if opened window is for submitting to JIRA

window name = null

opened window is not for submitting to JIRA

> |



Shared Modules with different versions

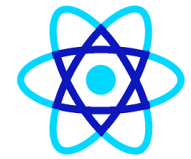
```
{
  "name": "carts",
  "version": "1.0.0",
  "description": "",
  "main": "index.js",
  "scripts": {
    "start": "webpack serve"
  },
  "keywords": [],
  "author": "",
  "license": "ISC",
  "dependencies": {
    "faker": "^4.1.0",
    "html-webpack-plugin": "5.5.0",
    "nodemon": "^2.0.15",
    "webpack": "^5.88.0",
    "webpack-cli": "4.10.0",
    "webpack-dev-server": "4.7.4"
  }
}
```

```
{
  "name": "products",
  "version": "1.0.0",
  "description": "",
  "main": "index.js",
  "scripts": {
    "start": "webpack serve"
  },
  "keywords": [],
  "author": "",
  "license": "ISC",
  "dependencies": {
    "faker": "^5.1.0",
    "html-webpack-plugin": "5.5.0",
    "nodemon": "^2.0.15",
    "webpack": "^5.88.0",
    "webpack-cli": "4.10.0",
    "webpack-dev-server": "4.7.4"
  }
}
```

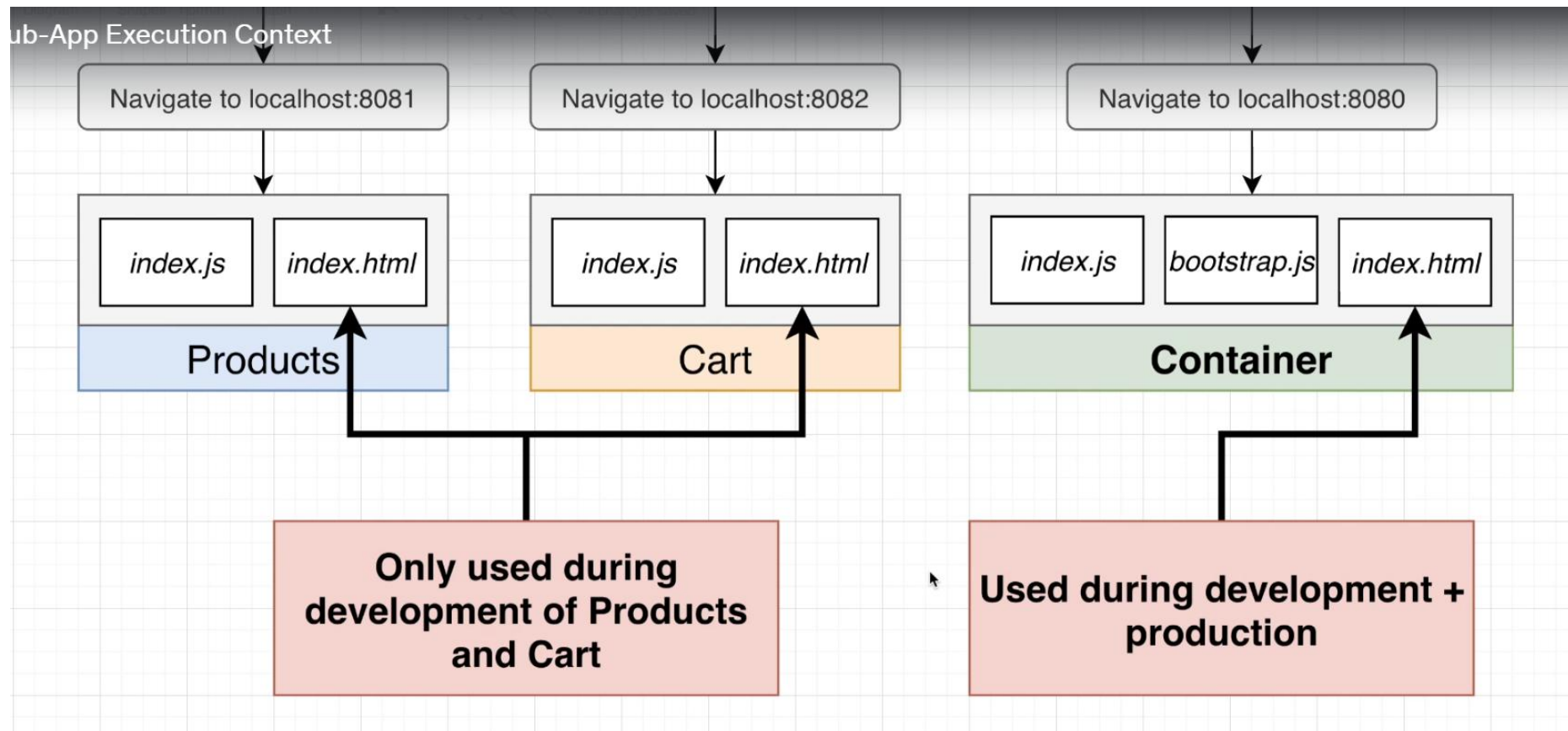


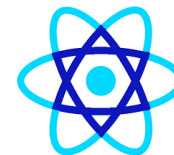
Singleton Loading

```
mode: 'development',
devServer: {
  port: 8083
},
plugins: [
  new ModuleFederationPlugin({
    name: 'cart',
    filename: 'remoteEntry.js',
    exposes: {
      './CartShow': './src/index',
    },
    // shared: ["faker"]
    shared: {
      faker: {
        singleton: true
      }
    }
  })
],
```



Sub App Execution





Remote Module

```
import faker from 'faker'
```

1+ usages

```
const mount=(el) :void =>{
  let products :string = '';
  for(let i :number =0;i<3;i++){
    const name= faker.commerce.productName();
    products+=`<div>${name}</div>`
  }
  el.innerHTML=products;
}

if(process.env.NODE_ENV === 'development'){
  const el :Element =document.querySelector(selectors: '#dev-products');
  if(el){
    mount(el);
  }
}

export {mount};
```

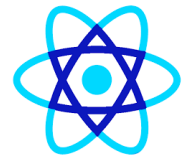


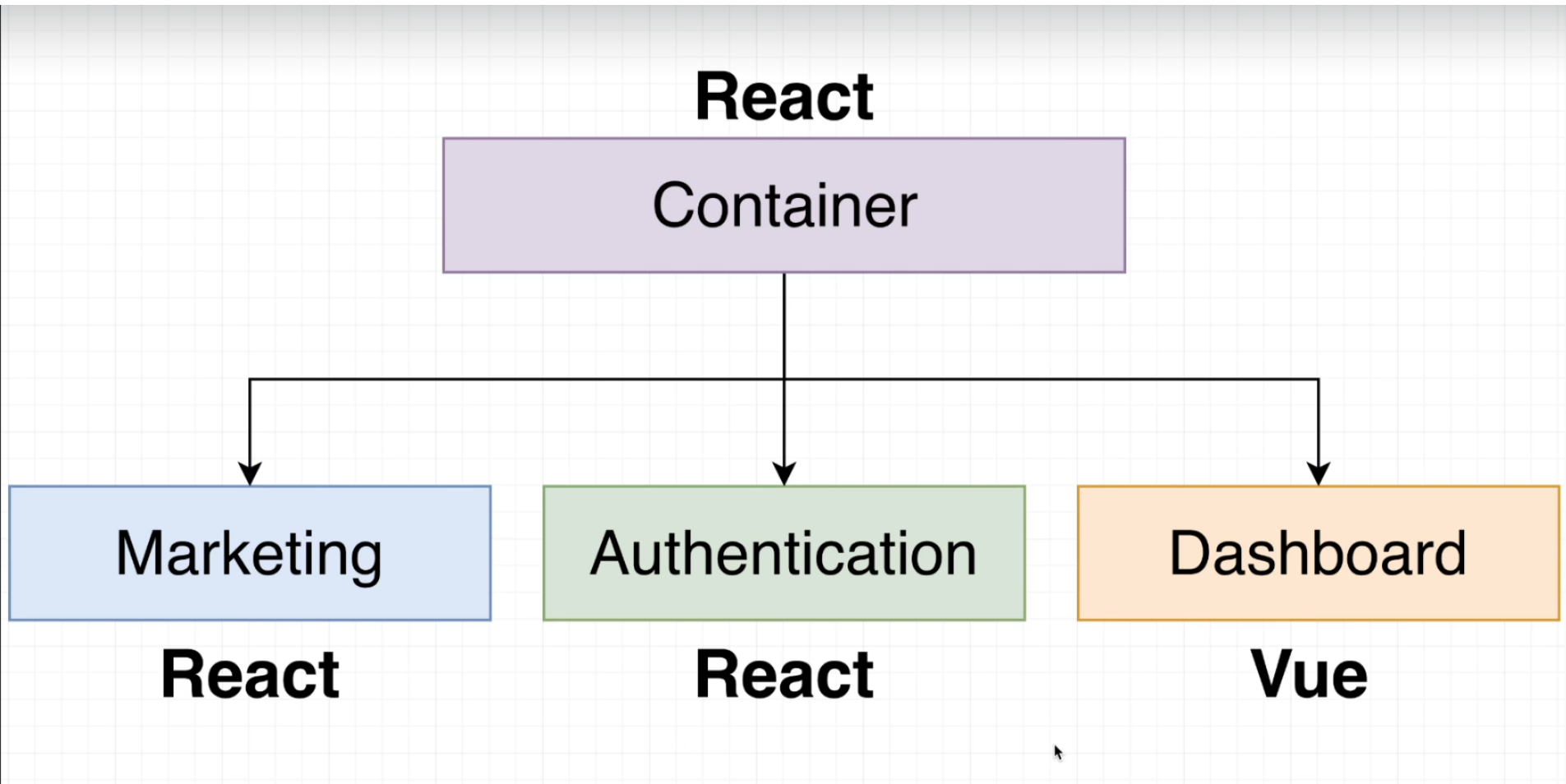
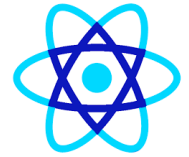

Remote Module

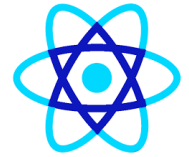
```
import {mount as productsMount} from 'products/ProductsIndex',  
import {mount as cartMount} from 'cart/CartShow';  
console.log('Container!');
```

```
productsMount(document.querySelector(selectors: '#my-products'));  
cartMount(document.querySelector(selectors: '#my-cart'))
```

Application Overview



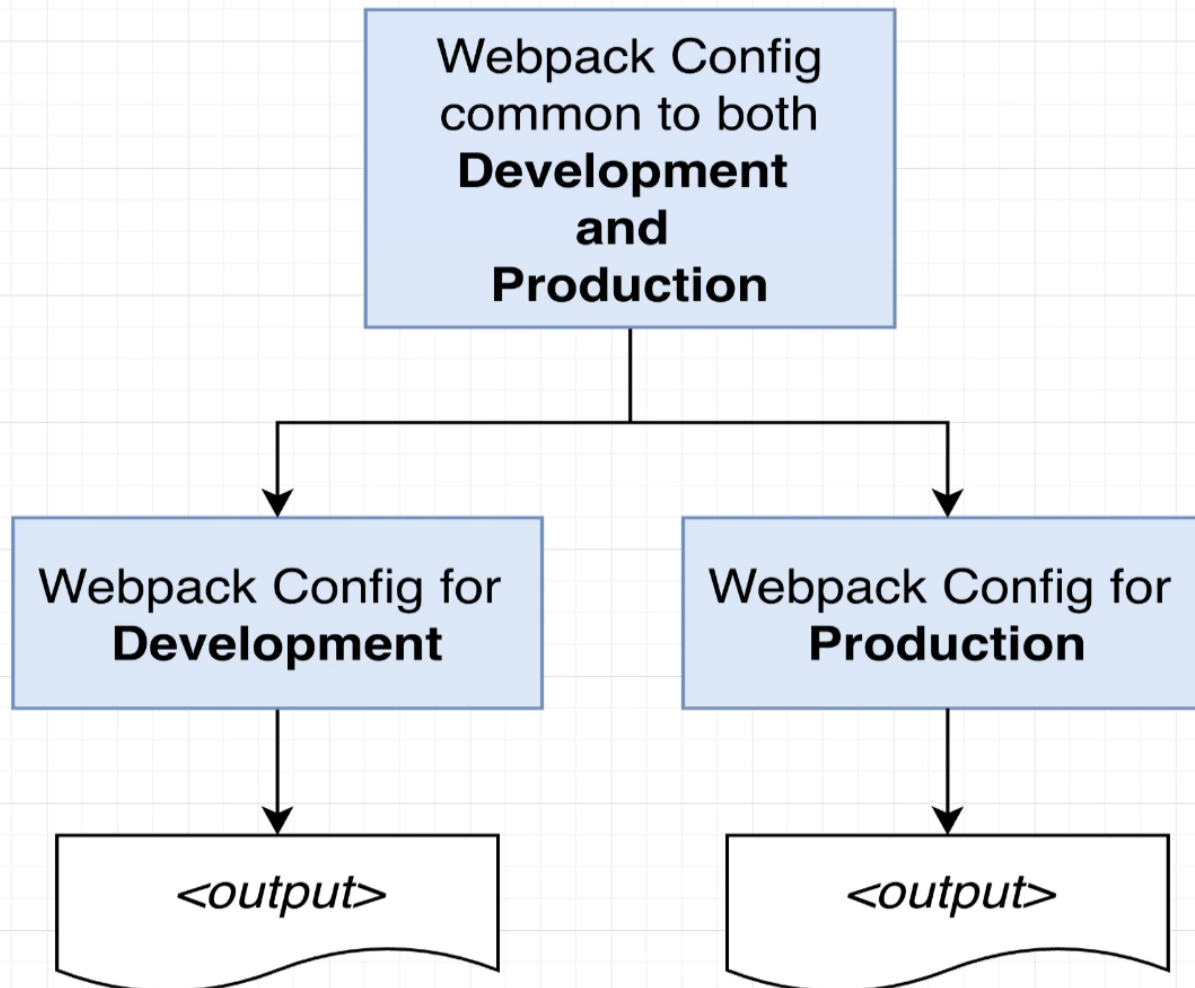
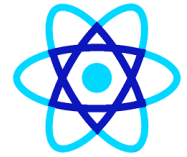




We are setting up all projects from scratch



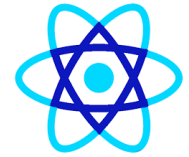
Why no Create-React-App or Vue CLI?





Reactjs Typescript

- `npx create-react-app customer-app --template typescript`



React Native

A Beginner's Guide for Freshers

Build native mobile apps for iOS and Android — with JavaScript

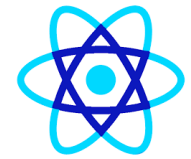


iOS



Android

< > One Codebase



What is React Native?



Created by Meta (Facebook) in 2015 · Open source

React Native is an open-source framework for building **truly native** mobile apps using **JavaScript and React**.



Real native apps

UI renders with real platform components — not a website in a wrapper.



One language, two platforms

Write JavaScript once; ship to both the App Store and Play Store.



Fast Refresh

See your code changes on the device in about a second while developing.



Huge community

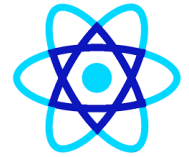
Backed by Meta and used by thousands of companies worldwide.



**“Learn once,
write anywhere.”**

*If you know React for the web, you already know most of
React Native.*

React Native



React Native is React, but the output is native mobile UI.

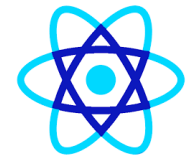




How does it work



Key takeaway: A `<Text>` in your code becomes a real `UILabel` on iOS and a real `TextView` on Android — that's why React Native apps feel native.



Why React Native

Pure Native

Swift / Kotlin

- Best possible performance
- Two separate codebases
- Two teams, two skill sets
- Slower to ship features

React Native

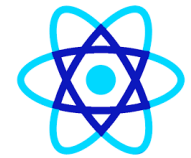
JavaScript + React

- ✓ Near-native performance
- ✓ One shared codebase (~90%)
- ✓ Web devs feel at home
- ✓ Fast Refresh = quick iteration

WebView Apps

Cordova / Ionic (classic)

- Easiest to start
- Runs a website in a shell
- UI can feel non-native
- Limited device performance



Why React Native

One React skillset can create mobile experiences across platforms.

Single codebase

Share most business logic and UI patterns across Android and iOS.

Fast learning

Familiar React concepts: components, props, state and hooks.

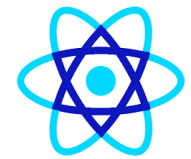
Native UI

React Native renders platform-native controls, not a WebView.

Large ecosystem

Use Expo, navigation, data libraries, native modules and SDKs.



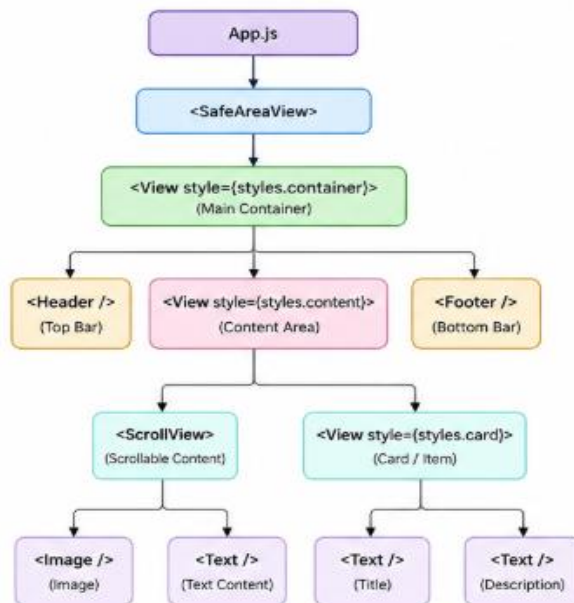


React Native Architecture

React Native Layout Design

Component Hierarchy & Layout Structure

1. App Component Tree



4. Key Points

- ✓ Use Flexbox (default in React Native) for layout.
- ✓ Design for different screen sizes.
- ✓ Keep UI components reusable.
- ✓ Use StyleSheet for consistent styling.

2. Layout Breakdown

Header (Top Bar)

- Usually contains Logo / Title / Icons
- Fixed at this top

Content Area

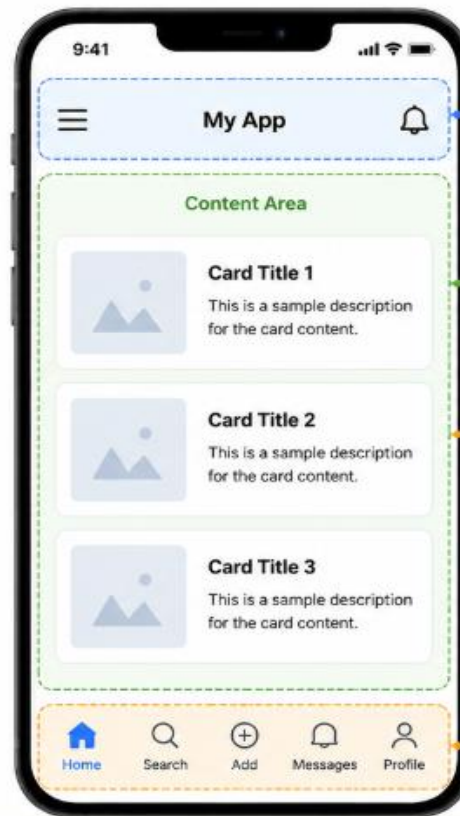
- Main screen content
- Uses ScrollView for long content
- Contains Cards, Images, Text, Buttons etc.

Footer / Tab Bar

- Navigation between screens
- Fixed at this bottom

3. Common Layout Components

	View Container for layout and styling
	Image To display images
	Text To display text content
	ScrollView For scrollable content
	TouchableOpacity / Pressable For touchable elements (buttons, cards)
	SafeAreaView To avoid notches & status bar overlap



React Native uses Flexbox for layout.
Everything is a component!



Core Concepts



Components

Reusable building blocks of your UI. Every screen is a tree of small components.



JSX

HTML-like syntax inside JavaScript that describes what the UI should look like.



Props

Data passed from a parent component to a child — read-only inputs.



State

Data owned by a component that changes over time.
Change state → UI re-renders.



Your First Component



A complete, working React Native counter app

```
import { useState } from 'react';
import { View, Text, Button } from 'react-native';

export default function Counter() {
  const [count, setCount] = useState(0);

  return (
    <View>
      <Text>You tapped {count} times</Text>
      <Button
        title="Tap me"
        onPress={() => setCount(count + 1)}
      />
    </View>
  );
}
```



Component

Counter is a function that returns UI. That's a component.



State

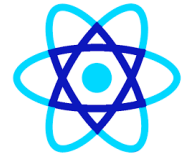
useState(0) stores the count. Calling setCount updates it.



Reactivity

When state changes, the UI re-renders automatically.

Styling & Layout



No CSS files — styles are JavaScript objects, layout is Flexbox

```
const styles = StyleSheet.create({
  card: {
    backgroundColor: '#20232A',
    padding: 16,
    borderRadius: 12,
    flexDirection: 'row',
    justifyContent: 'center',
    alignItems: 'center',
  },
});

<View style={styles.card}> ... </View>
```



camelCase properties

background-color becomes backgroundColor. Values are numbers or strings.



Flexbox everywhere

Every View is a flex container by default — flexDirection defaults to 'column'.



Density-independent units

Sizes like padding: 16 automatically scale across screen sizes.



Getting Started in 3 Steps



Expo is the official recommended way to begin

1 Create the project

```
npx create-expo-app my-app
```

Requires only Node.js installed — no Android Studio or Xcode needed to start.

2 Start the dev server

```
cd my-app && npx expo start
```

A QR code appears in your terminal.

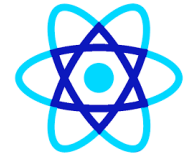
3 Scan with Expo Go

```
Install the free Expo Go app
```

Scan the QR code — your app runs on your own phone instantly.

Tip: You can even skip installation entirely — try React Native in the browser at snack.expo.dev

Create First App

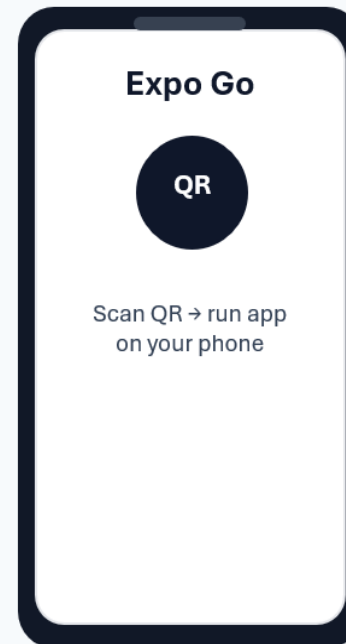


Expo is the easiest entry point for freshers.

```
# 1. Create project
npx create-expo-app@latest MyFirstApp

# 2. Go inside project
cd MyFirstApp

# 3. Start development server
npx expo start
```

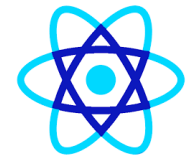


Freshers tip

Use a real Android phone first. Emulator setup can be done later.

When advanced

Use bare React Native CLI when you need more native control.



Project Structure

Learn where code lives before writing too much code.

```
MyFirstApp/  
├── app/ or src/  
│   ├── screens/  
│   ├── components/  
│   └── services/  
├── assets/  
├── package.json  
└── App.tsx / App.js
```

screens/

Full pages such as Home, ProductList, Cart and Login.

services/

API calls: REST, GraphQL, Firebase, auth.

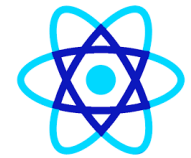
components/

Reusable UI: Button, Card, Input, Header.

assets/

Images, icons, fonts and static files.

Keep folders simple at first. Add architecture only when the app grows.



Core Components

React Native components map to native UI elements.

View

Layout container
div-like

Text

Display text
all text must be inside it

Image

Display image
local or remote

TextInput

Form input
keyboard aware

ScrollView

Small scroll content
simple lists

FlatList

Large list
recycles rows

```
import { View, Text, Image } from 'react-native';

export default function App() {
  return (
    <View>
      <Text>Hello Mobile!</Text>
    </View>
  );
}
```



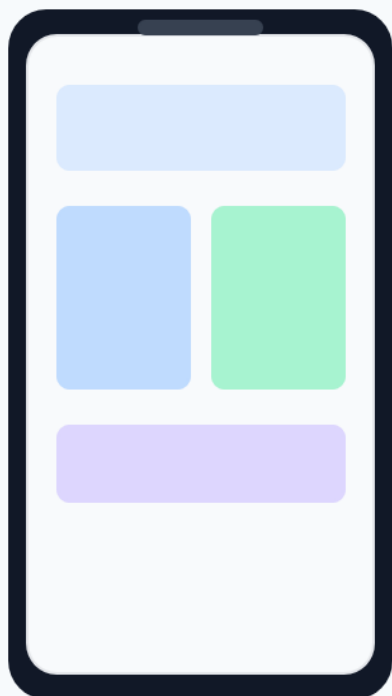
Core Components

Component	Web equivalent	What it does
 View	<code><div></code>	The basic container for layout — groups and positions children.
 Text	<code><p></code> , <code></code>	Displays text. All text must live inside a <code><Text></code> component.
 Image	<code></code>	Shows local or network images.
 TextInput	<code><input></code>	Lets the user type — forms, search boxes, chat fields.
 ScrollView / FlatList	scrolling <code><div></code> / <code>list</code>	Scrollable content; FlatList efficiently renders long lists.
 Pressable / Button	<code><button></code>	Handles taps and presses from the user.



Layout Styling

No CSS files are required to start. Use StyleSheet and Flexbox.



```
const styles = StyleSheet.create({
  container: {
    flex: 1,
    padding: 16,
    backgroundColor: '#fff'
  },
  row: {
    flexDirection: 'row',
    gap: 12
  }
});
```

Flexbox first

Most screens are built with flex, padding, margin and gap.

No px mindset

Design for multiple screen sizes, not only your emulator.

Use SafeArea

Avoid status bar and notch issues on phones.



Events and State

Every input changes state; every state change updates the UI.

```
const [name, setName] = useState('');

<TextInput
  placeholder="Enter name"
  value={name}
  onChangeText={setName}
/>

<Button
  title="Save"
  onPress={() => alert(name)}
/>
```

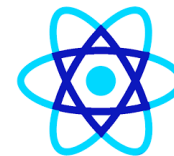
A mobile app mockup of a 'Register' screen. The screen has a white background with a dark blue border. At the top, the title 'Register' is centered in bold black text. Below the title is a light blue rounded rectangular input field with the placeholder text 'Enter name'. Underneath the input field is a solid blue rounded rectangular button with the text 'Save' in white, centered.

Event names

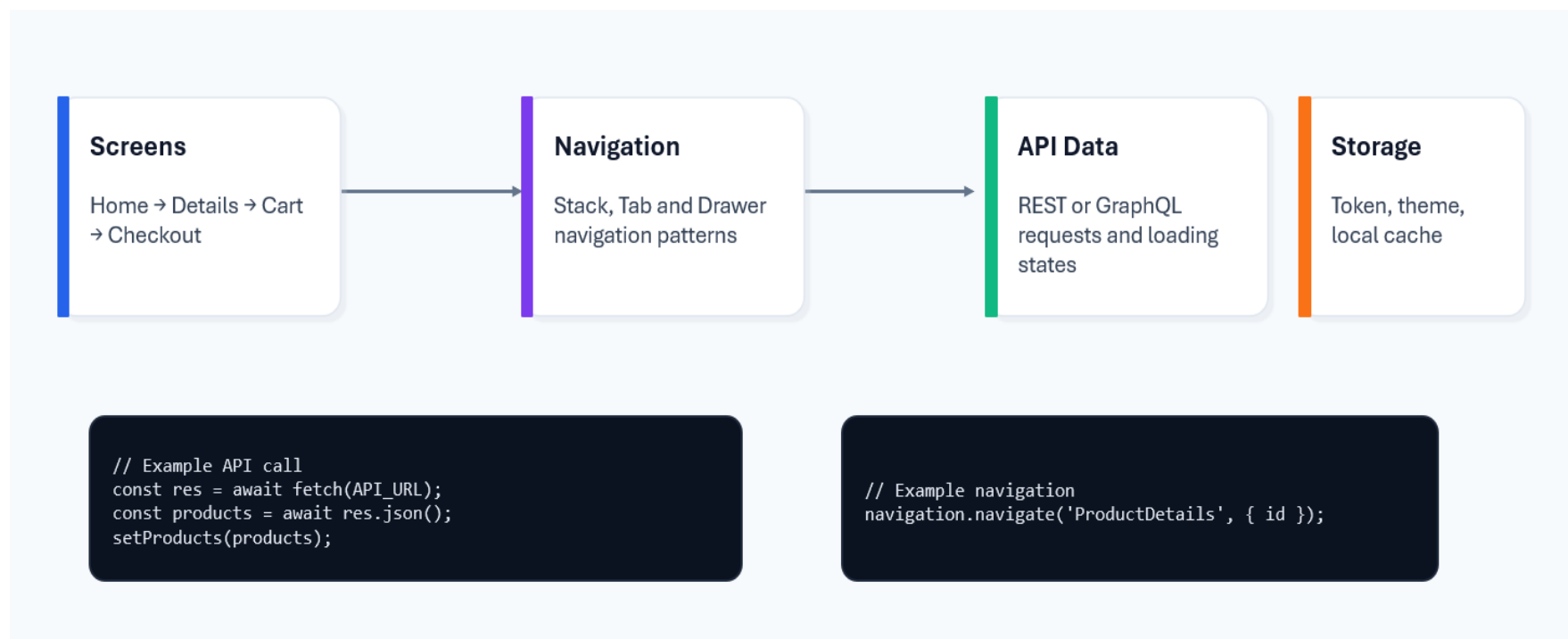
Use `onPress`,
`onChangeText`,
`onSubmitEditing`.

State choices

`useState` for local state,
Redux/Zustand for shared
state.



Navigation and Data



Questions





Module Summary

- Reactjs
- JSX, Props and State
- Events
- Component Interaction

